MATH 535 Mathematical Methods in Data Science
Supplemantary Lecture Notes

Hanbaek Lyu

Department of Mathematics, University of Wisconsin - Madison, WI 53709

Email address: hlyu@math.wisc.edu

www.hanbaeklyu.com

Supplementary Repository: https://github.com/HanbaekLyu/Math156_UCLA_SP21

Contents

Chapter 1. Introduction to Machine Learning 4
1. Prologue 4
2. Polynomial Regression 5
3. Decision Theory 13

Chapter 2. Algorithms for Classification 17
1. Introduction 17
2. Logistic Regression 18
3. Binary classification of MNIST by Logistic Regression 21
4. Classification of MNIST by Multiclass Logistic Regression 26
5. Probit Regression 28
6. Binary classification of MNIST by Probit Regression 30
7. Naive Bayes Classifier 32
8. Classification with the 20Newsgroups dataset 37
9. Gaussian Naive Bayes 41

Chapter 3. Neural Networks 43
1. Model formulation 43
2. Training Feedforward Neural Networks 44
3. Examples of 2-layer FFNN 50
4. Convolutional Neural Networks 52

Chapter 4. Matrix Factorization and Dictionary Learning 59
1. Introduction 59
2. Algorithms for Matrix Factorization 62
3. Applications of Matrix Factorization 63
4. Principal Component Analysis 67
5. Applications of PCA and Matrix Factorization 72
6. Probabilistic PCA and EM algorithm 74

Chapter 5. Markov chains 81
1. Definition and examples 81
2. Stationary distribution and examples 85
3. Markov property to the infinite future 90
4. Strong Markov Property 91
5. Recurrence and Transience 93
6. Existence and unieuqness of stationary distribution 99
7. Uniqueness of stationary distribution 101
8. Limit theorems for Markov chains 105
9. Convergence to the stationary distribution 107
10. Markov chain Monte Carlo 111

Bibliography 116

Appendix A. Review of basic probability theory 117
1. Probability measure and probability space 117
2. Random variables 118
3. Conditional and iterated expectations 120
4. Conditional variance 121
5. Joint probability distributions 123
6. Definition and Examples of MLE 124
7. Bayesian Inference 127

CHAPTER 1

Introduction to Machine Learning

1. Prologue

When do we say we learned something, say, how to ride bikes or how to multiply two fractions? Roughly speaking, this is to *recognizing patterns* between input and output so that we can use the recognized patterns to *predict* the output of fresh new input. *Machine Learning* (ML) consists of techniques that enable computers to this learning process. Below are some key components in ML.

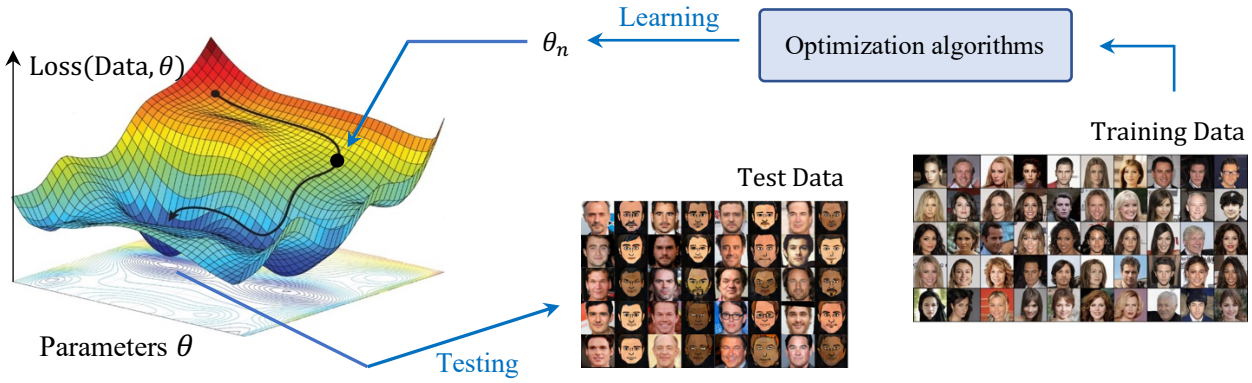


FIGURE 1. Basic scheme of Machine Learning: (1) Select a model to explain the dataset; (2) Fit the parameters to the training dataset using an optimization algorithm; (3) Test the performance of the fitted model to the test dataset.

1. Model Selection: The goal is to predict unknown output $\mathbf{y}$ from a new input $\mathbf{x}$. For this purpose, choose a model $\mathbf{y} \approx \hat{\mathbf{y}}(\mathbf{x}, \boldsymbol{\theta})$ with parameters $\boldsymbol{\theta}$[1]. Given input $\mathbf{x}$ and parameters $\boldsymbol{\theta}$, the model will output the value $\hat{\mathbf{y}}$ that should be close to the true and unkonwn $\mathbf{y}$.

e.g., Linear regression, Logistic regression, PCA, NMF, feedforward neural networks, CNN, RNN, etc.

2. Traing Data: We need a large number of known input and output pairs $(\mathbf{x}_1, \mathbf{y}_1), (\mathbf{x}_2, \mathbf{y}_2), \ldots, (\mathbf{x}_n, \mathbf{y}_N)$, from which we want to recognize patterns between the variables $\mathbf{x}$ and $\mathbf{y}$.

e.g., vectors, matrices, tensors, images, videos, texts, networks, etc.

3. Training the model: We need to find the optimal parameter $\hat{\boldsymbol{\theta}}$ for which the model $\mathbf{x} \mapsto \hat{\mathbf{y}}(\mathbf{x}, \hat{\boldsymbol{\theta}})$ gives a good fit for the training data. More precisely, this is done by solving following optimization problem

$$\hat{\boldsymbol{\theta}} = \underset{\boldsymbol{\theta} \in \boldsymbol{\Theta}}{\arg\min}\, \ell(\text{training data}, \boldsymbol{\theta}), \tag{1}$$

where ℓ is a *loss function* that measures the error between the true output y and the predicted output $\hat{\mathbf{y}}$, and $\boldsymbol{\Theta}$ is a set of admissible parameter values. A typical choise of such loss function is

[1]Model parameters are usually denoted by θ in statistics and by w (for 'weights') in machine learning. We will use both interchangably.

the *mean squared error* (MSE)

$$\ell(\text{training data}, \boldsymbol{\theta}) = \frac{1}{N}\sum_{i=1}^{N} \|\mathbf{y}_i - \hat{\mathbf{y}}(\mathbf{x}_i, \boldsymbol{\theta})\|^2, \tag{2}$$

where 'training data' consists of training examples $(\mathbf{x}_1, \mathbf{y}_1), \ldots, (\mathbf{x}_N, \mathbf{y}_N)$. Then use *optimizaiton algorithms* to find the optimial parameter $\hat{\boldsymbol{\theta}}$ in (1).

e.g., gradient descent, stochastic gradient descent, expectation-maximization, majorization-minimization, etc.

4. Testing: After an optimial parameter $\hat{\boldsymbol{\theta}}$ is learned, test the 'recognized pattern' $\mathbf{x} \mapsto \hat{\mathbf{y}}(\mathbf{x}, \hat{\boldsymbol{\theta}})$ on *test data* $(\mathbf{x}'_1, \mathbf{y}'_1), \ldots, (\mathbf{x}'_m, \mathbf{y}'_m)$, that were kept hidden during the training step. If the loss $\ell(\text{test data}, \hat{\boldsymbol{\theta}})$ is small, then we have learned the correct pattern succesfully.

2. Polynomial Regression

2.1. Basic theory. Suppose we have a training dataset of pairs of real numbers $(x_1, y_1), \cdots, (x_N, y_N)$. Think of y_k is the price of the stock you are holding and x_k is NASDAQ composite at time k. You want to obtain a simple functional explaining the *dependent variable* y_k in terms of the *independent variable* x_k. While there could be a general relationship between the variables x and y, here we will first study a simple model of *polynomial regression*. Namely, consider the following polynomial function $x \mapsto \hat{y}(x; \mathbf{w})$,

$$\hat{\mathbf{y}}(x; \mathbf{w}) := w_0 + w_1 x + w_2 x^2 + \cdots + w_M x^M = \sum_{k=0}^{M} w_k x^M = \boldsymbol{\phi}(x)^T \mathbf{w}, \tag{3}$$

where

$$\boldsymbol{\phi}(x) = \begin{bmatrix} 1 & x & \cdots & x^M \end{bmatrix}^T, \qquad \mathbf{w} = \begin{bmatrix} w_0 & w_1 & \cdots & w_M \end{bmatrix}^T \in \mathbb{R}^{M+1}. \tag{4}$$

Here $\mathbf{w} \in \mathbb{R}^{M+1}$ denotes the $(M+1)$-dimensional vector of parameters. Here $\hat{\mathbf{y}}(x; \mathbf{w})$ is a degree M polynomial in x and the components of $\mathbf{w}$ give the coefficients of monomials of x.

Suppose we have training data samples $(x_1, y_1), \cdots, (x_N, y_N)$. Let $\mathbf{Y}_{\text{train}} = [y_1, \cdots, y_N]^T$. We would like to find the optimial parameter $\mathbf{w}$ so that the corresponding polynomial regression (3) gives a good fit for the training data. In order to do so, we first write the vector of n predicted outcomes as the following matrix form

$$\begin{bmatrix} \hat{\mathbf{y}}(x_1, \mathbf{w}) \\ \hat{\mathbf{y}}(x_2, \mathbf{w}) \\ \vdots \\ \hat{\mathbf{y}}(x_N, \mathbf{w}) \end{bmatrix} = \begin{bmatrix} 1 & x_1 & \cdots & x_1^M \\ 1 & x_2 & \cdots & x_2^M \\ & \vdots & & \\ 1 & x_2 & \cdots & x_N^M \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_M \end{bmatrix} \quad \Longleftrightarrow \quad \hat{\mathbf{Y}}_{\text{train}} = \mathbf{X}_{\text{train}} \mathbf{w}. \tag{5}$$

Since we want to fit the polynomial regression model in (3) to the training data, we want to choose the parameter $\mathbf{w}$ so that

$$\mathbf{Y}_{\text{train}} \approx \hat{\mathbf{Y}} = \mathbf{X}_{\text{train}} \mathbf{w}. \tag{6}$$

In order to do so, we will use the *method of least squares*. Namely, we seek to choose $\mathbf{w} = \hat{\mathbf{w}}$ where

$$\hat{\mathbf{w}} = \underset{\mathbf{w} \in \mathbb{R}^{M+1}}{\arg\min} \|\mathbf{Y}_{\text{train}} - \mathbf{X}_{\text{train}} \mathbf{w}\|_F^2, \tag{7}$$

where the *matrix Frobenius norm* $\|\cdot\|_F$ is defined by $\|A\|_F = \text{tr}(A^T A)$. When $A = [a_1, \ldots, a_d]^T$ is a column vector, then $A^T A$ is a scalar so that

$$\|A\|_F^2 = \text{tr}(A^T A) = A^T A = a_1^2 + a_2^2 + \cdots + a_d^2, \tag{8}$$

so the Frobenius norm agrees with the Euclidean norm in this case. In Exercise (1.2.1) (with $\mathbf{Y} = \mathbf{Y}_{\text{train}}$, $\mathbf{X} = \mathbf{X}_{\text{train}}$, $\mathbf{B} = \mathbf{w}$, and $\lambda = 0$), we will show that the solution of (7) is given by

$$\hat{\mathbf{w}} = (\mathbf{X}_{\text{train}}^T \mathbf{X}_{\text{train}})^{-1} \mathbf{X}_{\text{train}}^T \mathbf{Y}_{\text{train}}. \tag{9}$$

Hence for any new input x, the optimal polynomial regreesion model will tell us to predict the corresponding ouput y as

$$y \approx \hat{\mathbf{y}}(x;\hat{\mathbf{w}}) = \begin{bmatrix} 1 & x & x^2 & \cdots & x^M \end{bmatrix} \hat{\mathbf{w}}. \tag{10}$$

Exercise 1.2.1 (Method of Least Squares)**.** Suppose we have matrices $\mathbf{Y} \in \mathbb{R}^{d\times n}$ and $\mathbf{X} \in \mathbb{R}^{d\times r}$. We seek to find a matrix $\hat{\mathbf{B}} \in \mathbb{R}^{r\times n}$ where

$$\hat{\mathbf{B}} = \underset{\mathbf{B}\in\mathbb{R}^{r\times n}}{\operatorname{argmin}} \|\mathbf{Y}-\mathbf{X}\mathbf{B}\|_F^2 + \lambda\|\mathbf{B}\|_F^2. \tag{11}$$

Here $\lambda \geq 0$ is called the L_2-regularization parameter. (This is an instance of unconstrained quadratic optimization problem.)

(i) Show that

$$\|\mathbf{Y}-\mathbf{X}\mathbf{B}\|_F^2 + \lambda\|\mathbf{B}\|_F^2 = \operatorname{tr}(\mathbf{Y}-\mathbf{X}\mathbf{B})^T(\mathbf{Y}-\mathbf{X}\mathbf{B}) + \lambda\operatorname{tr}(\mathbf{B}^T\mathbf{B}) \tag{12}$$

$$= \operatorname{tr}(\mathbf{Y}^T\mathbf{Y}) - 2\operatorname{tr}(\mathbf{Y}^T\mathbf{X}\mathbf{B}) + \operatorname{tr}(\mathbf{B}^T\mathbf{X}^T\mathbf{X}\mathbf{B}) + \lambda\operatorname{tr}(\mathbf{B}^T\mathbf{B}). \tag{13}$$

(ii) Show that (use Exercise 1.2.2)

$$\frac{\partial}{\partial \mathbf{B}}\left(\|\mathbf{Y}-\mathbf{X}\mathbf{B}\|_F^2 + \lambda\|\mathbf{B}\|_F^2\right) = 2(\mathbf{X}^T\mathbf{X}+\lambda\mathbf{I})\mathbf{B} - 2\mathbf{X}^T\mathbf{Y}. \tag{14}$$

(iii) For each $\mathbf{x} = [x_1,\ldots,x_n] \in \mathbb{R}^{a\times b}$, let $\mathbf{x}^{\text{vec}} = [x_1^T,\ldots,x_n^T]^T \in \mathbb{R}^{ab}$ denote its vectorization. Show that $(\mathbf{X}\mathbf{B})^{\text{vec}} = \mathbf{X}^{n\times n}\mathbf{B}^{\text{vec}}$, where

$$\mathbf{X}^{n\times n} := \overbrace{\begin{bmatrix} \mathbf{X} & O & O & \cdots & O \\ O & \mathbf{X} & O & \cdots & O \\ \vdots & \vdots & & & \vdots \\ O & \cdots & O & \mathbf{X} & O \\ O & O & \cdots & O & \mathbf{X} \end{bmatrix}}^{n\times n \text{ block matrix}} \in \mathbb{R}^{dn\times rn}. \tag{15}$$

Also show that

$$\nabla_{\mathbf{B}^{\text{vec}}}\nabla_{(\mathbf{B}^{\text{vec}})^T}\left(\|\mathbf{Y}^{\text{vec}} - \mathbf{X}^{n\times n}\mathbf{B}^{\text{vec}}\|_F^2 + \lambda\|\mathbf{B}^{\text{vec}}\|_F^2\right) \tag{16}$$

$$= 2\overbrace{\begin{bmatrix} (\mathbf{X}^T\mathbf{X}+\lambda\mathbf{I}) & O & O & \cdots & O \\ O & (\mathbf{X}^T\mathbf{X}+\lambda\mathbf{I}) & O & \cdots & O \\ \vdots & \vdots & & & \vdots \\ O & \cdots & O & (\mathbf{X}^T\mathbf{X}+\lambda\mathbf{I}) & O \\ O & O & \cdots & O & (\mathbf{X}^T\mathbf{X}+\lambda\mathbf{I}) \end{bmatrix}}^{n\times n \text{ block matrix}} \in \mathbb{R}^{rn\times rn}. \tag{17}$$

(iv) From **(ii)**-**(iii)**, conclude that the quadratic function $\mathbf{B} \mapsto \|\mathbf{Y}-\mathbf{X}\mathbf{B}\|_F^2 + \lambda\|\mathbf{B}\|_F^2$ is convex, and and hence its every critical point is a local minimum. (See Ref)

(v) Suppose $\lambda = 0$ and $\mathbf{X}^T\mathbf{X}$ is invertible[2]. Then from **(ii)**-**(iv)**, conclude that the quadratic function $\mathbf{B} \mapsto \|\mathbf{Y}-\mathbf{X}\mathbf{B}\|_F^2 + \lambda\|\mathbf{B}\|_F^2$ has a unique global minimum $\hat{\mathbf{B}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{Y}$ [3]

(vi) Suppose $\lambda > 0$. Then argue that $\mathbf{X}^T\mathbf{X}+\lambda\mathbf{I}$ is always invertible[4], and the quadratic function $\mathbf{B} \mapsto \|\mathbf{Y}-\mathbf{X}\mathbf{B}\|_F^2 + \lambda\|\mathbf{B}\|_F^2$ has a unique global minimum $\hat{\mathbf{B}} = (\mathbf{X}^T\mathbf{X}+\lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{Y}$.

---

[2] $\mathbf{X}^T\mathbf{X}$ is symmetric and positive semidefinite, and it is invertible iff the singular values of $\mathbf{X}$ are all nonzero.

[3] The matrix $\mathbf{X}^\dagger := (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T$ is called the *Moore-Penrose pseudo-inverse* of $\mathbf{X}$. If $\mathbf{X}$ is square and invertible, then $\mathbf{X}^\dagger = \mathbf{X}^{-1}(\mathbf{X}^T)^{-1}\mathbf{X}^T = \mathbf{X}^{-1}$. So the the psuedo-inverse can be regarded as a generalization of matrix inverse for non-square matrices.

[4] Hint: Show that $\mathbf{X}^T\mathbf{X}+\lambda\mathbf{I}$ is positive definite if $\lambda > 0$. Use the fact that the eigenvalues of a positive definite matrix $\mathbf{A}$ has to be positive (why?), so $\mathbf{A}\mathbf{y} \neq \mathbf{0}$ for any $\mathbf{y}$ (why?) so $\mathbf{A}$ is invertible (why?).

Exercise 1.2.2 (Matrix derivatives)**.** Show the following matrix derivatives: (Ref: The Matrix Cookbook)

(i) (First order) $\dfrac{\partial}{\partial \mathbf{X}}\mathrm{tr}(\mathbf{BXC}) = \mathbf{B}^T\mathbf{C}^T, \qquad \dfrac{\partial}{\partial \mathbf{X}}\mathrm{tr}(\mathbf{BX}^T\mathbf{C}) = \mathbf{CB}.$

(ii) (Second order) $\dfrac{\partial}{\partial \mathbf{X}}\mathrm{tr}(\mathbf{X}^T\mathbf{BX}) = \mathbf{BX}+\mathbf{B}^T\mathbf{X}, \qquad \dfrac{\partial}{\partial \mathbf{X}}\mathrm{tr}(\mathbf{B}^T\mathbf{X}^T\mathbf{CXB}) = \mathbf{C}^T\mathbf{XBB}^T+\mathbf{CXBB}^T.$

2.2. Examples and further discussions. In this subsection, we give an example of polynomial regression on a synthetic data. We will sample various instances of the following random variable

$$Y = \sin(2\pi X) + \varepsilon, \tag{18}$$

where $X \sim \mathrm{Uniform}([0,1])$ and $\varepsilon \sim N(0,\sigma^2)$ with $\sigma = 0.35$, where X and ε are drawn independently. ε represents white noise, where the true signal should lie on the sine curve $y = \sin(2\pi x)$, which is unknown. We can control the signal-noise ratio through the parameter σ, which is the standard deviation of the Gaussian noise ε. For each $N \in \{8, 100\}$, we generate N i.i.d. samples of Y to form the train set, and another N i.i.d. samples of Y forms the test set. The goal is to fit the degree-M polynomial regression model (3) to the train set and learn the optimial coefficients $\hat{\mathbf{w}} \in \mathbb{R}^{M+1}$ by solving the least squares in (7), and use it to predict the test set via (10). Recall the closed-form solution for the optimial parameter $\hat{\mathbf{w}}$ given in (9).

First consider $N = 8$ samples with $M = 1$ degree polynomial regression as shown in Figure 2 left. We are only given the $N = 8$ training points represented by the blue dots. The red line represents the fitted polynomial, which is a linear function being a degree $M = 1$ polynomial. Increasing the degree M of the polynomial model (and hence the degree of freedom of our model) to $M = 4$, we see the quartic polynomial in Figure 2 middle gives a much better fit to the sine curve. However, further increasing M to 9, the polynomial shown in Figure 2 right oscillates more wildly around the sine curve, giving a less favorable explanation of the true signal. While this may seem somewhat counterintuitive as the $M = 9$ polynomial model entails the $M = 4$ polynomial model, what happens here is an issue known as *overfitting*. Indeed, notice that the $M = 9$ polynomial in Figure 2 right passes through every single train data point, while the $M = 4$ polynomial in Figure 2 middle does not. Since the train set contains noise, our model should not be fitted too much to the observed data in the train set as it may learn 'false patterns' misled by the noise.

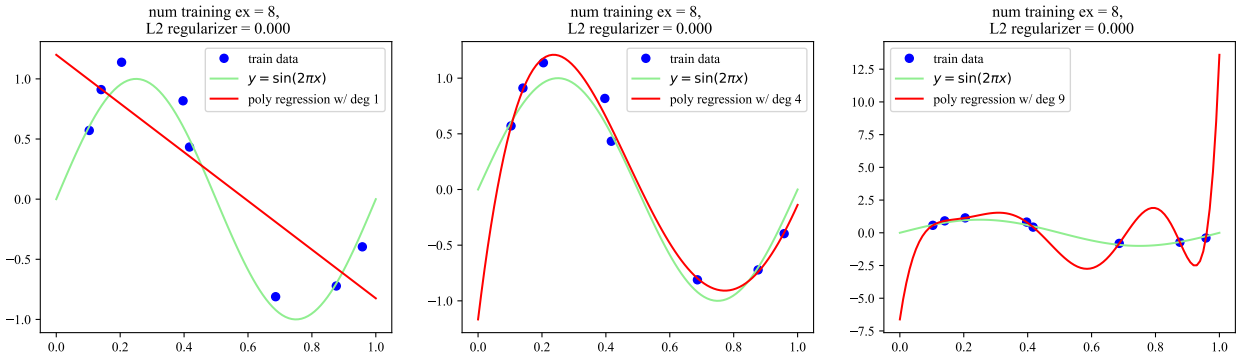


FIGURE 2. Example of polynomial regression fitting. $N = 8$ samples for the train set and $\lambda = 0$ L_2-regularization parameter.

One way to overcome the issue of overfitting is to collect more training data. Indeed, if we use $N = 100$ data points for the train set, the $M = 9$ polynomial regression in Figure 3 right becomes more accurate to the true sine curve than the $M = 4$ case in Figure 3 middle. Since the Gaussian noise in $N = 100$ train samples are all independent, it will be extremely unlikely that many of the points lie on any given $M = 9$ degree polynomial. Even though our synthetic data was built this way, this intuition is often useful in

real-world data analysis tasks – noise in nature is close to Gaussian, and fitting to large train set learns 'average patterns', which should be close to the patterns in the original signal.

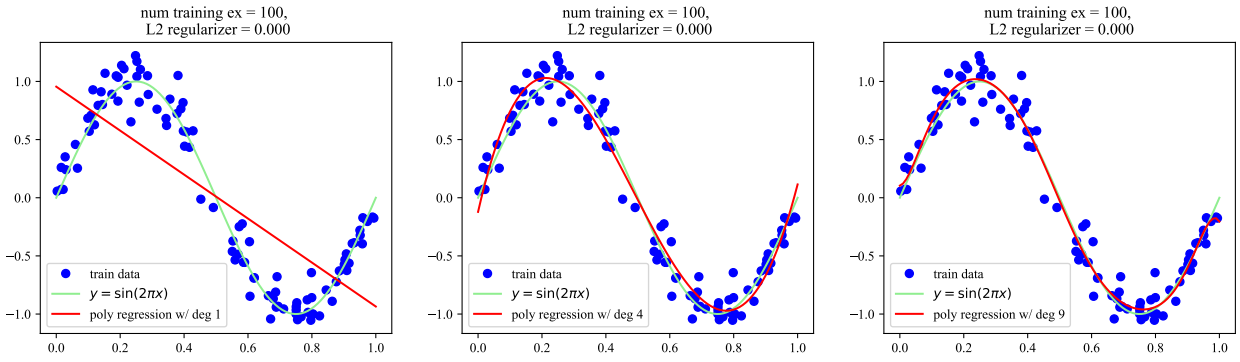


FIGURE 3. Example of polynomial regression fitting. $N = 100$ samples for the train set and $\lambda = 0$ L_2-regularization parameter.

Another way to reduce overfitting is to use *regularization*. Namely, this is to put some penalization on the parameters of the model so that they cannot be too large. For example, consider the following optimization problem

$$\hat{\mathbf{w}} = \underset{\mathbf{w}\in\mathbb{R}^{M+1}}{\arg\min} \|\mathbf{Y}_{\text{train}} - \mathbf{X}_{\text{train}}\mathbf{w}\|_F^2 + \lambda\|\mathbf{w}\|_F^2, \tag{19}$$

where we have added the 'L_2-regularization term' $\lambda\|\mathbf{w}\|_F^2$ to the squared loss $\|\mathbf{Y}_{\text{train}} - \mathbf{X}_{\text{train}}\mathbf{w}\|_F^2$. Here $\lambda \geq 0$ is called the L_2*-regularizer*, which is a new hyperparameter that we introduced to the model (previously M was the only hyperparameter). Due to this additional penalization term, the coefficients of $\hat{\mathbf{w}}$ in (19) cannot be too large. This technique is also called *shrinkage methods* in statistics as it shrinks the magnitude of the coefficients, and *weight decay* in the context of neural networks. The particular case of a quadratic regularizer as in (19) is called *ridge regression* [HK70]. If we instead use the L_1-regularization $\lambda\|\mathbf{w}\|_1$, then this is known as the (Lagrangian form of) the LASSO regression [Tib96], which better promotes 'sparsity' of $\hat{\mathbf{w}}$. An advantage of using the L_2-regularization is that we still have a closed-form solution for (19) by Exercise 1.2.1:

$$\hat{\mathbf{w}} = (\mathbf{X}_{\text{train}}^T\mathbf{X}_{\text{train}} + \lambda\mathbf{I})^{-1}\mathbf{X}_{\text{train}}^T\mathbf{Y}_{\text{train}}. \tag{20}$$

Indeed, using the L_2-regularizer $\lambda = 0.001$ in the $M = 9$ polynomial regression improves the performance for $N = 8$ training data points, as shown in Figure 4 right.

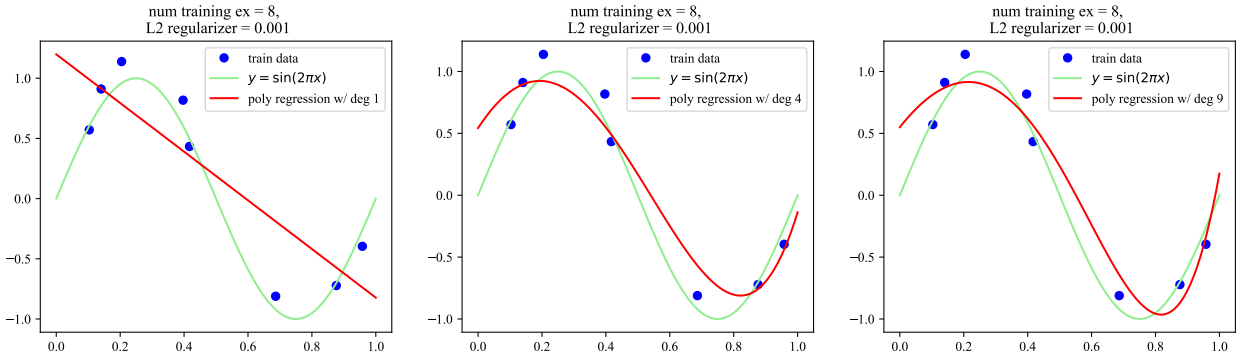


FIGURE 4. Example of polynomial regression fitting. $N = 8$ samples for the train set and $\lambda = 0.001$ L_2-regularization parameter.

Lastly, we evalute the performance of our learned polynomial model on the N test data points. We use the *mean squared error* (MSE)

$$\mathrm{MSE}(\hat{\mathbf{w}}) := \frac{1}{N}\|\mathbf{Y}_{\text{test}} - \hat{\mathbf{Y}}_{\text{test}}\|_F^2, \tag{21}$$

where $\hat{\mathbf{Y}}_{\text{test}} = \mathbf{X}_{\text{test}}\hat{\mathbf{w}}$ and $\hat{\mathbf{w}}$ is given by (20). We use a similar MSE for the train set.

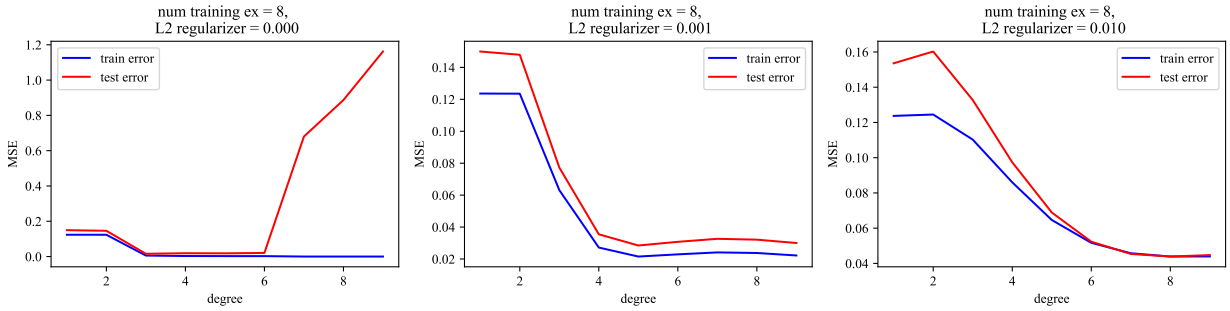


FIGURE 5. Plot of test and train error with respect to the degree M for three L_2-regularization parameters. $N = 8$ points in both train and test sets.

In Figure 5, we show the MSE for train and test sets against the degree M for three L_2-regularization parameters $\lambda \in \{0, 0.001, 0.01\}$ for $N \in \{8, 100\}$ data points both for the test and train sets. While the train error (in blue) mostly decreases as we increase the degree M in all cases, the test error start to increase rapidly at $M = 6$ with no regularization $\lambda = 0$ (Figure 5 left). This indicates severe overfitting occurs past $M = 6$. The gap between the test and train error is also known as the *generalization error*[5]. In Figures 5 middle and right, we see the use of moderate L_2-regularizer $\lambda = 0.001$ could significantly reduce overfitting (and hence the generalization error), but larger values of relgularizer $\lambda = 0.01$ hinders both the test and train errors. In general, the right value of the regularization parameter can be estimated by *cross-valiation*. However, the performance of our model on hyperparameter becomes less sensitive when we have a large traning set ($N = 100$), as we see from Figure 6.

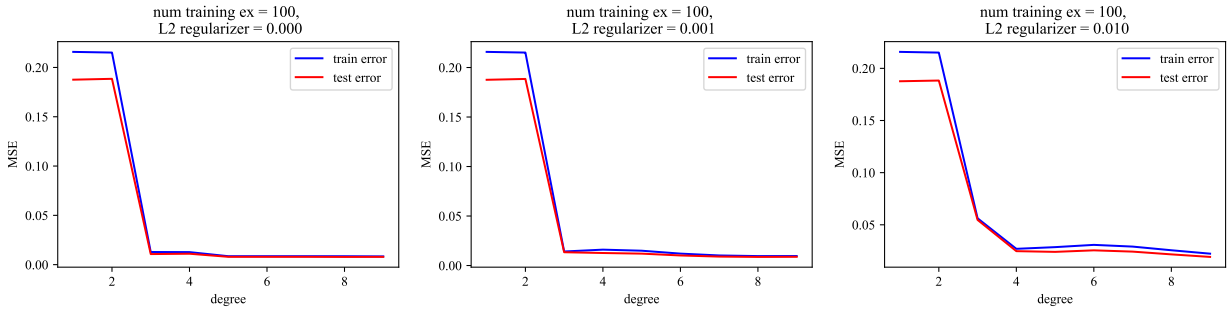


FIGURE 6. Plot of test and train error with respect to the degree M for three L_2-regularization parameters. $N = 100$ points in both train and test sets.

[5]We learn patterns from the observed test set. If the learned patterns capture the generic features of the underlying signal, then they should apply to the unobserved test set and yield a small test error; otherwise, then the pattern we learn does not 'generalize' beyond the test set.

Exercise 1.2.3. Using the Jupyter notebook provided in the course repository, reproduce the Figures 2 - 6, where the data are independently generated from the following random variable

$$Y = \cos(2\pi X) + \varepsilon, \tag{22}$$

where $X \sim \text{Uniform}([0,1])$ and $\varepsilon \sim N(0, 0.16)$ are independent.

2.3. Polynomial Regression by Maximum Likelihood. In this subsection, we consider the polynomial regression with *Gaussian noise*. Namely, given an input x, the model predicts the ouput as a *random variable* (RV) $\hat{\mathbf{y}}(x;\mathbf{w},\sigma) = \hat{\mathbf{y}}(x;\mathbf{w}) + \varepsilon$, where $\hat{\mathbf{y}}(x;\mathbf{w})$ is the polynomial function given in (3) and $\varepsilon \in N(0,\sigma^2)$ is an independent Gaussian noise with mean 0 and variance σ^2. Note that the predicted output $\hat{\mathbf{y}}(x;\mathbf{w},\sigma)$ itself is a Gaussian RV with mean $\hat{\mathbf{y}}(x;\mathbf{w})$ and variance σ^2 (see (10)):

$$\hat{\mathbf{y}}(x;\mathbf{w},\sigma) \sim N(\hat{\mathbf{y}}(x;\mathbf{w}),\sigma^2), \qquad \hat{\mathbf{y}}(x;\mathbf{w}) = \sum_{k=0}^{M} w_k x^k = \boldsymbol{\phi}(x)^T \mathbf{w}. \tag{23}$$

This is a probabilistic model of polynomial regression with parameters $\mathbf{w} = [w_0,\ldots,w_M]^T$ for the coefficients of the unknown polynomial and σ for the standard deviation of the Gaussian noise.

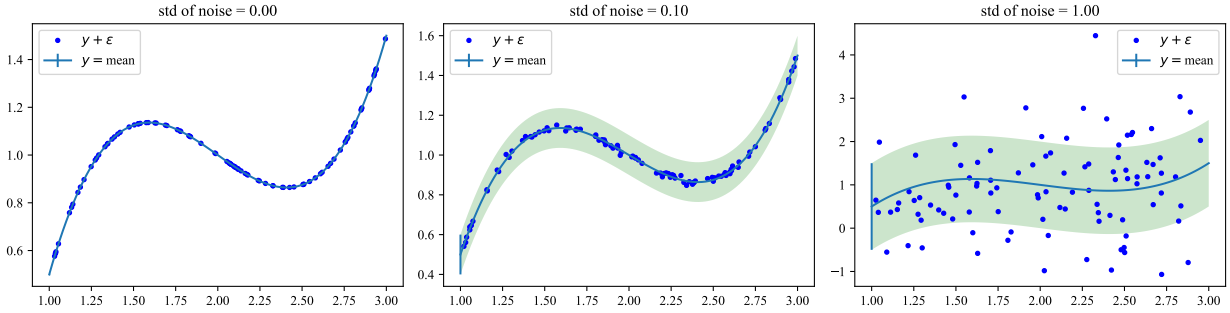


FIGURE 7. Plot of sample points of $Y = -6 + (23/2)x - 6x^2 + x^3 + \varepsilon$, $\varepsilon \sim N(0,\sigma^2)$. Shaded regions represents $\pm$ 1 standard deviation of the noise.

Suppose we are given with a sequence of training examples $[(x_1,y_1),\ldots,(x_N,y_N)]$. In order to fit the probabilistic polynomial regression (23) to the training dataset, we assume that the observed outputs $y_1,\ldots,y_N$ are realizations of the independent RVs $\hat{\mathbf{y}}_1(x_1;\mathbf{w},\sigma),\ldots,\hat{\mathbf{y}}_N(x_N;\mathbf{w},\sigma)$ satisfying (23). According to the method of *maximum likelihood* (see Section 6), we will have to find values of the parameters $\mathbf{w}$ and σ for which the joint likelihood $L(y_1,\ldots,y_N;\mathbf{w},\sigma)$ of observing the tuple $(y_1,\ldots,y_N)$ from the tuple of independent RVs $\hat{\mathbf{y}}_1(x_1;\mathbf{w},\sigma),\ldots,\hat{\mathbf{y}}_N(x_N;\mathbf{w},\sigma)$ is maximized. Namely, the joint likelihood is given by

$$L(y_1,\ldots,y_N;\mathbf{w},\sigma) = \prod_{i=1}^{n} f_{\hat{\mathbf{y}}(x_i;\mathbf{w},\sigma)}(y_i) \tag{24}$$

$$= \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{(y_i - \hat{\mathbf{y}}(x_i;\mathbf{w}))^2}{2\sigma^2}\right] \tag{25}$$

$$= (2\pi\sigma^2)^{-N/2} \exp\left[-\frac{1}{2\sigma^2}\sum_{k=1}^{N}(y_i - \hat{\mathbf{y}}(x_i;\mathbf{w}))^2\right]. \tag{26}$$

A key observation to make is the following: Denoting $\mathbf{Y}_{\text{train}} := [y_1,\ldots,y_N]^T$,

$$\sum_{k=1}^{N}(y_i - \hat{\mathbf{y}}(x_i;\mathbf{w}))^2 = \|\mathbf{Y}_{\text{train}} - \mathbf{X}_{\text{train}}\mathbf{w}\|_F^2, \tag{27}$$

where $\mathbf{X}_{\text{train}}$ is defined in (5) (see Exercise 1.2.4). Thus we can write the log likelihood function as

$$\log L(y_1,\dots,y_N;\mathbf{w},\sigma) = -\frac{N}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\|\mathbf{Y}_{\text{train}} - \mathbf{X}_{\text{train}}\mathbf{w}\|_F^2. \tag{28}$$

Noting that log is strictly increasing function, we can find the *maximum likelihood estimate* (MLE) of the parameters $\mathbf{w}$,σ by the following optimization problem

$$\begin{aligned}(\hat{\mathbf{w}},\hat{\sigma}) &:= \operatorname*{argmax}_{\mathbf{w}\in\mathbb{R}^{M+1},\sigma\geq 0} L(y_1,\dots,y_N;\mathbf{w},\sigma) &(29)\\ &= \operatorname*{argmax}_{\mathbf{w}\in\mathbb{R}^{M+1},\sigma\geq 0} \log L(y_1,\dots,y_N;\mathbf{w},\sigma) &(30)\\ &= \operatorname*{argmax}_{\mathbf{w}\in\mathbb{R}^{M+1},\sigma\geq 0} \left(-\frac{N}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\|\mathbf{Y}_{\text{train}} - \mathbf{X}_{\text{train}}\mathbf{w}\|_F^2\right). &(31)\end{aligned}$$

Since the first term in the log likelihood function in (28) does not depend on $\mathbf{w}$, we see that $\hat{\mathbf{w}}$ is the maximizer of $-\frac{1}{2\sigma^2}\|\mathbf{Y}_{\text{train}} - \mathbf{X}_{\text{train}}\mathbf{w}\|_F^2$. It follows that

$$\hat{\mathbf{w}} = \operatorname*{argmin}_{\mathbf{w}\in\mathbb{R}^{M+1}} \|\mathbf{Y}_{\text{train}} - \mathbf{X}_{\text{train}}\mathbf{w}\|_F^2 = (\mathbf{X}_{\text{train}}^T\mathbf{X}_{\text{train}})^{-1}\mathbf{X}_{\text{train}}^T\mathbf{Y}_{\text{train}}, \tag{32}$$

where the second equality follows from Exercise 1.2.1. Note that this agrees with the optimal parameter $\hat{\mathbf{w}}$ in (10) for the polynomial regression without noise.

On the other hand, in order to find $\hat{\sigma}$, we take the first and the second partial derivatives of (28) in σ:

$$\frac{\partial}{\partial\sigma}\log L(y_1,\dots,y_N;\mathbf{w},\sigma) = -\frac{N}{\sigma} + \frac{1}{\sigma^3}\|\mathbf{Y}_{\text{train}} - \mathbf{X}_{\text{train}}\mathbf{w}\|_F^2, \tag{33}$$

$$\frac{\partial^2}{\partial\sigma^2}\log L(y_1,\dots,y_N;\mathbf{w},\sigma) = -\frac{N}{\sigma^2} - \frac{3}{\sigma^4}\|\mathbf{Y}_{\text{train}} - \mathbf{X}_{\text{train}}\mathbf{w}\|_F^2 \leq 0. \tag{34}$$

According to the second derivative, the log likelihood is a concave function in σ. According to the first derivative, the log likelihood has a unique critical point. Since $(\hat{w},\hat{\sigma})$ jointly maximizes the log likelihood, we deduce

$$\hat{\sigma}^2 = \frac{1}{N}\|\mathbf{Y}_{\text{train}} - \mathbf{X}_{\text{train}}\hat{\mathbf{w}}\|_F^2, \tag{35}$$

where $\hat{\mathbf{w}}$ is given in (32).

Hence for any new input x, the optimal polynomial regression model will tell us to predict the corresponding ouput y as the following RV

$$\hat{Y}(x;\hat{\mathbf{w}},\hat{\sigma}) \sim N(\boldsymbol{\phi}(x)^T\hat{\mathbf{w}},\hat{\sigma}^2), \tag{36}$$

where the optimal parameters $\hat{\mathbf{w}}$ and $\hat{\sigma}$ are given in (32) and (35), respectively.

Exercise 1.2.4. Fix $\mathbf{w} = [w_0, w_1,\dots,w_M] \in \mathbb{R}^{M+1}$ and $\sigma \geq 0$. Let $\hat{Y}_1,\dots,\hat{Y}_N$ be independent Gaussian RVs where $\hat{Y}(x_i;\mathbf{w},\sigma) \sim N(\boldsymbol{\phi}(\mathbf{x})^T\mathbf{w},\sigma^2)$ for $i \in \{1,\dots,N\}$, where $\boldsymbol{\phi}(x) = [1,x,\dots,x^M]^T$. Show that the joint likelihood function for observing the values $y_1,\dots,y_N$ is given by

$$L(y_1,\dots,y_N;\mathbf{w},\sigma) = (2\pi\sigma^2)^{-N/2}\exp\left[-\frac{1}{2\sigma^2}\|\mathbf{Y}-\mathbf{X}\mathbf{w}\|_F^2\right], \tag{37}$$

where

$$\mathbf{Y} = [y_1,\dots,y_N]^T \in \mathbb{R}^N, \qquad \mathbf{X} = \begin{bmatrix} 1 & x_1 & \cdots & x_1^M \\ & \vdots & & \\ 1 & x_2 & \cdots & x_N^M \end{bmatrix} \in \mathbb{R}^{N\times M}. \tag{38}$$

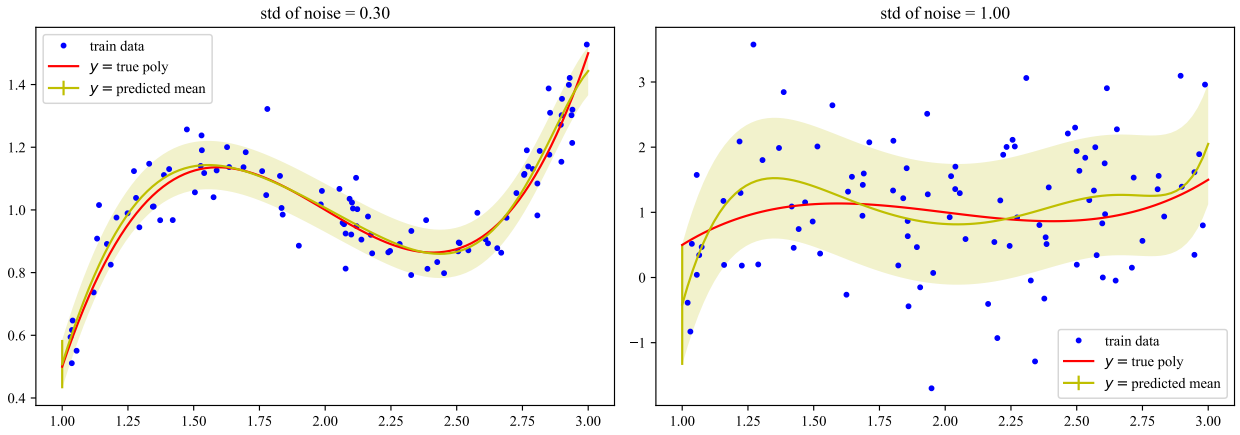


FIGURE 8. Plot of sample points of $Y = -6 + (23/2)x - 6x^2 + x^3 + \varepsilon$, $\varepsilon \sim N(0, \sigma^2)$ in blue for $\sigma = 0.3$ (left) and $\sigma = 1$ (right). Degree $M = 9$ polynomial regression with Gaussian noise is fitted by maximum likelihood. The predicted mean is shown in the yellow curve with ± 1 predicted standard deviation ($\hat{\sigma}$) shown in yellow shade.

2.4. Bayesian Polynomial Regression. In this subsection, we take a Bayesian approach of the polynomial regression by regarding the $\mathbf{w} = [w_0, \dots, w_M]^T \in \mathbb{R}^{M+1}$ as a (vector-valued) RV. For simplicity, we assume σ in (23) is known.

We assume that the components of $\mathbf{w}$ are i.i.d. Gaussian RV with distribution $N(0, \tau^2)$ for some $\tau \geq 0$. Equivalently, $\mathbf{w}$ follows the multivariate Gaussian distribution $N(\mathbf{0}, \tau\mathbf{I})$ with mean vector $\mathbf{0} = [0, \dots, 0]^T \in \mathbb{R}^{M+1}$ and covariance matrix τI, where $\mathbf{I}$ is the $(M+1) \times (M+1)$ identity matrix. Its probability distribution function (PDF) is given by

$$p(\mathbf{x}; \mathbf{0}, \tau\mathbf{I}) = (2\pi\tau^2)^{-(M+1)/2} \exp\left[-\frac{1}{2\tau^2}\mathbf{x}^T\mathbf{x}\right]. \tag{39}$$

See Example 3.2.2 for the general multivariate Gaussian distribution.

As brefiely introduced in Section 7, Bayesian approach assumes parameters are random and continuously updates a hypothetical probability distribution on the parameters (a.k.a. *prior*) to the new distribution (a.k.a. *posterior*) with respect to the newly observed data according to Bayes' Theorem A.7.1. Namely, as in Subsection 2.3, suppose we are given with training examples $[(x_1, y_1), \dots, (x_N, y_N)]$. Accodring to Bayes' Theorem, Exericse 1.2.4, and (39), the posterior distribution of the coefficients $\mathbf{w}$ is given by the following proportionality relation

$$\overbrace{p(\mathbf{w}|\text{Data})}^{\text{posterior}} \propto \overbrace{L(\text{Data}|\mathbf{w})}^{\text{likelihood}} \overbrace{p(\mathbf{w})}^{\text{prior}} \tag{40}$$

$$= (2\pi\sigma^2)^{-N/2} \exp\left[-\frac{1}{2\sigma^2}\|\mathbf{Y} - \mathbf{X}\mathbf{w}\|_F^2\right] (2\pi\tau^2)^{-(M+1)/2} \exp\left[-\frac{1}{2\tau^2}\mathbf{w}^T\mathbf{w}\right] \tag{41}$$

$$= (2\pi\sigma^2)^{-N/2}(2\pi\tau^2)^{-(M+1)/2} \exp\left[-\frac{1}{2\sigma^2}\|\mathbf{Y} - \mathbf{X}\mathbf{w}\|_F^2 - \frac{1}{2\tau^2}\|\mathbf{w}\|_F^2\right]. \tag{42}$$

Later, we will show that the posterior distribution is in fact a multivarite Gaussian

$$p(\mathbf{w}|\text{Data}) = N(\mathbf{m}, \mathbf{S}), \tag{43}$$

where

$$\mathbf{m} := \sigma^{-2}\mathbf{S}\mathbf{X}_{\text{train}}^T\mathbf{Y}_{\text{train}} \qquad \in \mathbb{R}^{M+1}, \tag{44}$$

$$\mathbf{S}^{-1} := \tau^{-2}\mathbf{I} + \sigma^{-2}\mathbf{X}_{\text{train}}^T\mathbf{X}_{\text{train}} \qquad \in \mathbb{R}^{(M+1)\times(M+1)}. \tag{45}$$

Also, notice that the above posterior distribution is maximized when the exponent is maximized. Hence

$$\hat{\mathbf{w}}_{\text{MAP}} := \underset{\mathbf{w}\in\mathbb{R}^{M+1}}{\arg\max}\, p(\mathbf{w};\text{Data}) = \underset{\mathbf{w}\in\mathbb{R}^{M+1}}{\arg\min} \left(\|\mathbf{Y}-\mathbf{X}\mathbf{w}\|_F^2 + \frac{\sigma^2}{\tau^2}\|\mathbf{w}\|_F^2 \right). \tag{46}$$

The quantity in the left hand side is called *maximum a posteriori* (MAP) estimate of $\mathbf{w}$. Note that the minimization problem in the right hand side is equivalent to (19) with the L_2-regularization parameter $\lambda = (\sigma/\tau)^2$.

Finally, for any new input x, the Bayesian polynomial regression model above will tell us to predict the corresponding ouput y as the following RV

$$\hat{Y}(x;\text{Data},\sigma) \sim N(\boldsymbol{\phi}(x)^T \mathbf{w}_{\text{pos}}, \sigma^2), \tag{47}$$

where $\mathbf{w}_{\text{pos}}$ is a *random* parameter following the posterior distribution given in (42). In fact, it can be shown that the distribution of $\hat{Y}(x;\text{Data},\sigma)$ is a (univariate) Gaussian:

$$\hat{Y}(x;\text{Data},\sigma) \sim N(\boldsymbol{\phi}(x)^T \mathbf{m}, \sigma^2 + \boldsymbol{\phi}(x)^T \mathbf{S}\boldsymbol{\phi}(x)). \tag{48}$$

These claims will be justified later.

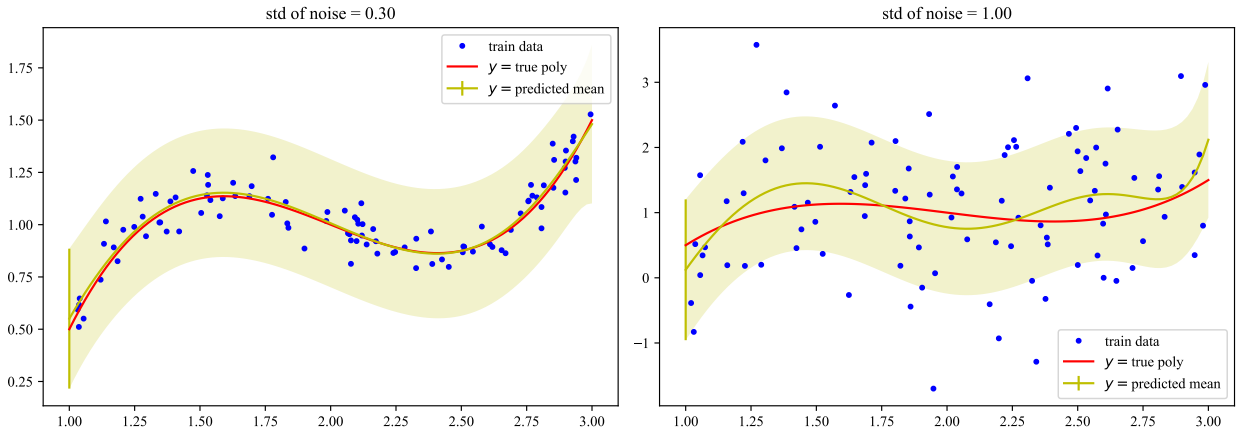


FIGURE 9. Plot of sample points of $Y = -6 + (23/2)x - 6x^2 + x^3 + \varepsilon$, $\varepsilon \sim N(0,\sigma^2)$ in blue for $\sigma = 0.3$ (left) and $\sigma = 1$ (right). Degree $M = 9$ Bayesian polynomial regression with prior $\mathbf{w} \sim N(\mathbf{0}, \tau^2\mathbf{I})$, $\tau = 10$ is fitted. The predicted mean is shown in the yellow curve with ± 1 predicted standard deviation ($\hat{\sigma}$) shown in yellow shade.

Exercise 1.2.5. Using the Jupyter notebook provided in the course repository, reproduce the Figures 8 - 9, where the data are independently generated from the following random variable

$$Y = \cos(2\pi X) + \varepsilon, \tag{49}$$

where $X \sim \text{Uniform}([0,1])$ and $\varepsilon \sim N(0, 0.16)$ are independent. Compare the results of the maximum likelihood and the Bayesian polynomial regression.

3. Decision Theory

Suppose we have an input vector $\mathbf{x} \in \mathscr{X}$ together with a corresponding output vector $\mathbf{y} \in \mathscr{Y}$. Our goal is to predict the output $\mathbf{y}$ given a new value for $\mathbf{x}$, which will take a continuous value for regression problems and discrete value (class label) for classification problems. More formally, a *decision algorithm* is a function $\mathscr{A} : \mathscr{X} \to \mathscr{Y}$ that maps a new input vector $\mathbf{x} \in \mathscr{X}$ to a predicted output $\mathbf{y} \in \mathscr{Y}$.

Method of learning decision algorithms can be roughly divided into 'discriminative' and 'generative' ones. *Discriminative* methods learn decision algorithms directly from training dataset without any probabilistic modling of the relationship between the input $\mathbf{x}$ and the output $\mathbf{y}$. For instance, the polynomial regession model $\mathbf{x} \mapsto \hat{y}(\mathbf{x};\mathbf{w}) = \boldsymbol{\phi}(x)^T\mathbf{w}$ in (10) is a classical example of discriminative decision algorithms for the case of regression problems. On the other hand, a classical example of discriminative binary classification algorithm is the following *perceptron algorithm*:

$$\mathbf{x} \longmapsto f(\boldsymbol{\phi}(\mathbf{x})^T\mathbf{w}) \in \{+1,-1\}, \tag{50}$$

where $\boldsymbol{\phi}(\mathbf{x})$ is a *feature vector* of the input $\mathbf{x}$ (e.g., $\boldsymbol{\phi}(x) = [1, x, \dots, x^M]^T$ in the case of polynomial regression) and f is the *activation function* of the form

$$f(x) = \mathbf{1}(x \geq \theta) - \mathbf{1}(x < \theta) = \begin{cases} +1 & x \geq \theta \\ 1 & x < \theta \end{cases}, \tag{51}$$

where $\theta \geq 0$ is a threshold parameter.

In the rest of this section, we take a closer look at the generative methods of learning decision algorithms. This is roughly a two-step process involving the 'inference problem' and the 'decision problem'.

1. Inference problem: We model the unknown input and output as the random vector $(\mathbf{X},\mathbf{Y})$ following the joint distribution $(\mathbf{x},\mathbf{y}) \mapsto p(\mathbf{x},\mathbf{y})$:

$$\mathbb{P}(\mathbf{X} = \mathbf{x},\, \mathbf{Y} = \mathbf{y}) = p(\mathbf{x},\mathbf{y}). \tag{52}$$

This describes our complete understanding on the pattern between $\mathbf{x}$ and $\mathbf{y}$, where randomness represents the uncertainty associated to these variables. Conditioning on the input value $\mathbf{x}$, we can also deduce that the output $\mathbf{Y}$ given the observed input $\mathbf{x}$ has the following conditional distribution

$$p(\mathbf{y}\,|\,\mathbf{x}) := \mathbb{P}(\mathbf{Y} = \mathbf{y}\,|\,\mathbf{X} = \mathbf{x}) = \frac{p(\mathbf{x},\mathbf{y})}{p(\mathbf{x})}, \tag{53}$$

where the marginal distribution of $\mathbf{X}$ is given by

$$p(\mathbf{x}) = \int_{\mathscr{Y}} p(\mathbf{x},\mathbf{y})\, d\mathbf{y}. \tag{54}$$

It is possible that one only compute the conditional distribution $p(\mathbf{y}\,|\,\mathbf{x})$ in the inference step and proceed to the decision problem (e.g., logistic regression).

2. Decision problem: Given a conditional probability distribution $\mathbf{y} \mapsto p(\mathbf{y}\,|\,\mathbf{x})$ and given a new value of input $\mathbf{x}$, decide (predict) the value of corresponding output $\mathbf{y}$. A general principle to find an optimal decision algorithm using the conditional distribution is the following *(conditional) expected loss minmization* (ELM):

$$\mathbf{x} \longmapsto \mathscr{A}(\mathbf{x}) := \operatorname*{arg\,min}_{\hat{\mathbf{y}} \in \mathscr{Y}} \mathbb{E}\left[\ell(\mathbf{Y},\hat{\mathbf{y}})\,|\,\mathbf{X} = \mathbf{x}\right], \tag{55}$$

where $(\mathbf{y},\hat{\mathbf{y}}) \mapsto \ell(\mathbf{y},\hat{\mathbf{y}}) \in \mathbb{R}$ is a chosen *loss function* that measures the 'goodness' of our prediction $\hat{\mathbf{y}}$ with respect to the true output $\mathbf{y}$.

Below are some instances of decision algorithms found by ELM:

Classicifaion: The prediction $\hat{\mathbf{y}}$ is the maximizer of the conditional distribution $p(\mathbf{y}\,|\,\mathbf{x})$:

$$\hat{\mathbf{y}} = \operatorname*{arg\,max}_{\mathbf{y} \in \mathscr{Y}} p(\mathbf{y}\,|\,\mathbf{x}) = \text{'most probable output given } \mathbf{x}\text{'}. \tag{56}$$

Regression: The prediction $\hat{\mathbf{y}}$ is the conditional expectation of the random output $\mathbf{Y}$ given $\mathbf{x}$:

$$\hat{\mathbf{y}} = \mathbb{E}[\mathbf{Y}\,|\,\mathbf{X} = \mathbf{x}] = \text{'expected output given } \mathbf{x}\text{'}. \tag{57}$$

The inference problem is more demanding both theoretically and computationally than solving the decision problem. In practical applications, we must often make a specific prediction for the value of $\mathbf{y}$ in order to take a suitable action based on our current best knowledge of the problem.

Example 1.3.1 (Detecting cancer from x-ray images)**.** Consider the situation where the input is an x-ray image of a new patient, and we would like to determine the output $\mathbf{y}$ between two classes `normal` and `cancer`. Our decision algorithm $\mathscr{A}$ maps the patient's x-ray image $\mathbf{x}$ to a class label $\mathbf{y} \in \{\texttt{normal}, \texttt{cancer}\}$. There are four possible outcomes depending on whether the patient has cancer or not and how she has been diagnosed by our algorithm. Since misclassifying a patient who has cancer may be far more detrimental than misclassifying a patient who does not have cancer, it would be reasonable to consider a loss function ℓ of the following form:

$$\ell(\text{true}, \text{prediction}) = \begin{array}{lcc} \texttt{true\textbackslash prediction} & \texttt{normal} & \texttt{cancer} \\ \texttt{normal} & 0 & 1 \\ \texttt{cancer} & 1000 & 0 \end{array}. \tag{58}$$

Then the decision algorithm $\mathscr{A}$ that minimizes the corresponding expected loss would be more aggressive in looking for cues of cancer and classifying as `cancer` in order to avoid the huge loss incurred when missing patient with cancer. ▲

Example 1.3.2 (Maxizing conditional probabilities for classification)**.** Consider the case of classification problem, where the output variable $\mathbf{Y}$ is a discrete random variable (taking values of class labels). Consider the following 'indicator loss function'

$$\ell(\mathbf{y}, \hat{\mathbf{y}}) = \mathbf{1}(\mathbf{y} \neq \hat{\mathbf{y}}). \tag{59}$$

Namely, we gain a unit loss whenever our prediction $\hat{y}$ differs from the true output $\mathbf{y}$. Then note that

$$\mathbb{E}\left[\ell(\mathbf{Y}, \hat{\mathbf{y}}) \,|\, \mathbf{X} = \mathbf{x}\right] = \mathbb{E}\left[\mathbf{1}(\mathbf{y} \neq \hat{\mathbf{y}}) \,|\, \mathbf{X} = \mathbf{x}\right] = \mathbb{P}(\mathbf{Y} \neq \hat{\mathbf{y}} \,|\, \mathbf{X} = \mathbf{x}) = 1 - \mathbb{P}(\mathbf{Y} = \hat{\mathbf{y}} \,|\, \mathbf{X} = \mathbf{x}) = 1 - p(\hat{\mathbf{y}} \,|\, \mathbf{x}). \tag{60}$$

Thus, the decision algorithm $\mathscr{A}$ given by the ELM (55) in this case reduces to

$$\mathbf{x} \longmapsto \mathscr{A}(\mathbf{x}) = \underset{\hat{\mathbf{y}} \in \mathscr{Y}}{\arg\min}\ \mathbb{E}\left[\ell(\mathbf{Y}, \hat{\mathbf{y}}) \,|\, \mathbf{X} = \mathbf{x}\right] = \underset{\hat{\mathbf{y}} \in \mathscr{Y}}{\arg\max}\ p(\hat{\mathbf{y}} \,|\, \mathbf{x}). \tag{61}$$

In words, we predict the true class label $\mathbf{Y}$ given $\mathbf{x}$ as the most probable class label given $\mathbf{x}$. ▲

Exercise 1.3.3 (Minimizing squared loss for regression)**.** Consider the case of regression problem, where the output variable $\mathbf{Y} \in \mathbb{R}^d$ is a continuous d-dimensional random vector (e.g., stock prices in a portfolio tomorrow). Consider the squared loss:

$$\ell(\mathbf{y}, \hat{\mathbf{y}}) = \|\mathbf{y} - \hat{\mathbf{y}}\|_F^2 = (\mathbf{y} - \hat{\mathbf{y}})^T (\mathbf{y} - \hat{\mathbf{y}}). \tag{62}$$

In this exercise, we will show that the decision algorithm $\mathscr{A}$ given by the ELM (55) in this case reduces to

$$\mathbf{x} \longmapsto \mathscr{A}(\mathbf{x}) = \underset{\hat{\mathbf{y}} \in \mathscr{Y}}{\arg\min}\ \mathbb{E}\left[\ell(\mathbf{Y}, \hat{\mathbf{y}}) \,|\, \mathbf{X} = \mathbf{x}\right] = \mathbb{E}\left[\mathbf{Y} \,|\, \mathbf{X} = \mathbf{x}\right]. \tag{63}$$

(i) Write $\mathbf{Y} = [Y_1, Y_2, \ldots, Y_d]^T$. Show that

$$\mathbb{E}\left[\|\mathbf{Y}\|_F^2 \,\middle|\, \mathbf{X} = \mathbf{x}\right] - \|\mathbb{E}\left[\mathbf{Y} \,|\, \mathbf{X} = \mathbf{x}\right]\|_F^2 = \sum_{i=1}^{d} \mathrm{Var}(Y_i \,|\, \mathbf{X} = \mathbf{x}). \tag{64}$$

(ii) Justify each equality in the following 'completing squares' computation:

$$\mathbb{E}\left[\|\mathbf{Y} - \mathscr{A}(\mathbf{x})\|_F^2 \,\middle|\, \mathbf{X} = \mathbf{x}\right] = \mathbb{E}\left[\mathbf{Y}^T\mathbf{Y} - 2\mathscr{A}(\mathbf{x})^T\mathbf{Y} + \mathscr{A}(\mathbf{x})^T\mathscr{A}(\mathbf{x}) \,\middle|\, \mathbf{X} = \mathbf{x}\right] \tag{65}$$

$$= \mathbb{E}\left[\|\mathbf{Y}\|_F^2 \,\middle|\, \mathbf{X} = \mathbf{x}\right] - 2\mathscr{A}(\mathbf{x})^T\mathbb{E}\left[\mathbf{Y} \,|\, \mathbf{X} = \mathbf{x}\right] + \|\mathscr{A}(\mathbf{x})\|_F^2 \tag{66}$$

$$= \|\mathscr{A}(\mathbf{x}) - \mathbb{E}\left[\mathbf{Y} \,|\, \mathbf{X} = \mathbf{x}\right]\|_F^2 + \sum_{i=1}^{d} \mathrm{Var}(Y_i \,|\, \mathbf{X} = \mathbf{x}). \tag{67}$$

(iii) From **(ii)**, deduce that the expected squared loss $\mathbb{E}\left[\ell(\mathbf{Y}, \hat{\mathbf{y}}) \,|\, \mathbf{X} = \mathbf{x}\right]$ of the decision algorithm $\mathbf{x} \mapsto \mathscr{A}(\mathbf{x})$ is minimized when $\mathscr{A}(\mathbf{x}) = \mathbb{E}\left[\mathbf{Y} \,|\, \mathbf{X} = \mathbf{x}\right]$ and the global minimum equals $\sum_{i=1}^{d} \mathrm{Var}(Y_i \,|\, \mathbf{X} = \mathbf{x})$.

In the following proposition, we show that minimizing 'total expected loss' and 'conditional expected loss' give the same decision algorithm.

Proposition 1.3.4 (Total expected loss minimization)**.** *Consider the following decision algorithm* $\mathscr{B} : \mathscr{X} \to \mathscr{Y}$ *given by the following total expected loss minimization:*

$$\mathscr{B} = \underset{f:\mathscr{X}\to\mathscr{Y}}{\arg\min}\, \mathbb{E}\left[\ell(\mathbf{Y}, f(\mathbf{X}))\right], \tag{68}$$

where $(\mathbf{y}, \hat{\mathbf{y}}) \mapsto \ell(\mathbf{y}, \hat{\mathbf{y}})$ *is a chosen loss function. Let* $\mathscr{A}$ *denote the decision algorithm given by the (conditional) expected loss minimization:*

$$\mathbf{x} \longmapsto \mathscr{A}(\mathbf{x}) = \underset{\hat{\mathbf{y}}\in\mathscr{Y}}{\arg\min}\, \mathbb{E}\left[\ell(\mathbf{Y}, \hat{\mathbf{y}}) \,|\, \mathbf{X} = \mathbf{x}\right]. \tag{69}$$

Then it holds that $\mathscr{B} = \mathscr{A}$.

PROOF. It suffices to show that $\mathscr{A}$ achieves the minimum total expected loss:

$$\mathbb{E}\left[\ell(\mathbf{Y}, \mathscr{A}(\mathbf{X}))\right] = \min_{f:\mathscr{X}\to\mathscr{Y}} \mathbb{E}\left[\ell(\mathbf{Y}, f(\mathbf{X}))\right] = \mathbb{E}\left[\ell(\mathbf{Y}, \mathscr{B}(\mathbf{X}))\right], \tag{70}$$

where the second equality follows from the definition of $\mathscr{B}$. First, since $\mathscr{B}$ minimizes the total expected loss and $\mathscr{A}$ is one of the possible decision algorithms,

$$\mathbb{E}\left[\ell(\mathbf{Y}, \mathscr{A}(\mathbf{X}))\right] \geq \mathbb{E}\left[\ell(\mathbf{Y}, \mathscr{B}(\mathbf{X}))\right]. \tag{71}$$

On the other hand, by the minimality of $\mathscr{A}$, we have for each $\mathbf{x} \in \mathscr{X}$,

$$\mathbb{E}_{\mathbf{Y}}\left[\ell(\mathbf{Y}, \mathscr{B}(\mathbf{X})) \,|\, \mathbf{X} = \mathbf{x}\right] \geq \mathbb{E}\left[\ell(\mathbf{Y}, \mathscr{A}(\mathbf{x})) \,|\, \mathbf{X} = \mathbf{x}\right] \tag{72}$$

By taking expectation over $\mathbf{X}$ and using iterated expectation,

$$\mathbb{E}\left[\ell(\mathbf{Y}, \mathscr{B}(\mathbf{X}))\right] = \mathbb{E}_{\mathbf{X}}\left[\mathbb{E}_{\mathbf{Y}}\left[\ell(\mathbf{Y}, \mathscr{B}(\mathbf{X})) \,|\, \mathbf{X}\right]\right] \geq \mathbb{E}_{\mathbf{X}}\left[\mathbb{E}_{\mathbf{Y}}\left[\ell(\mathbf{Y}, \mathscr{A}(\mathbf{X})) \,|\, \mathbf{X}\right]\right] = \mathbb{E}\left[\ell(\mathbf{Y}, \mathscr{A}(\mathbf{X}))\right]. \tag{73}$$

Hence by combining (71) and (73), we conclude (70), as desired. □

CHAPTER 2

Algorithms for Classification

1. Introduction

A particular type of pattern reconition problem that machine learning methods have made significant progress over the past decades is *classiciation problems*. Here are some motivating examples:

1. Classify patients' chest x-ray images into classes '`normal`' and '`pneumonia`'.
2. Classify dog and cat images into classes '`dog`' and '`cat`'.
3. Classify an image of handwritten digit into the correct digit. (see Figure 10)
4. Classify incoming emails into '`spam`' and '`not spam`'.
5. Classify given user behavior on a social network into '`churn`' or '`human`'

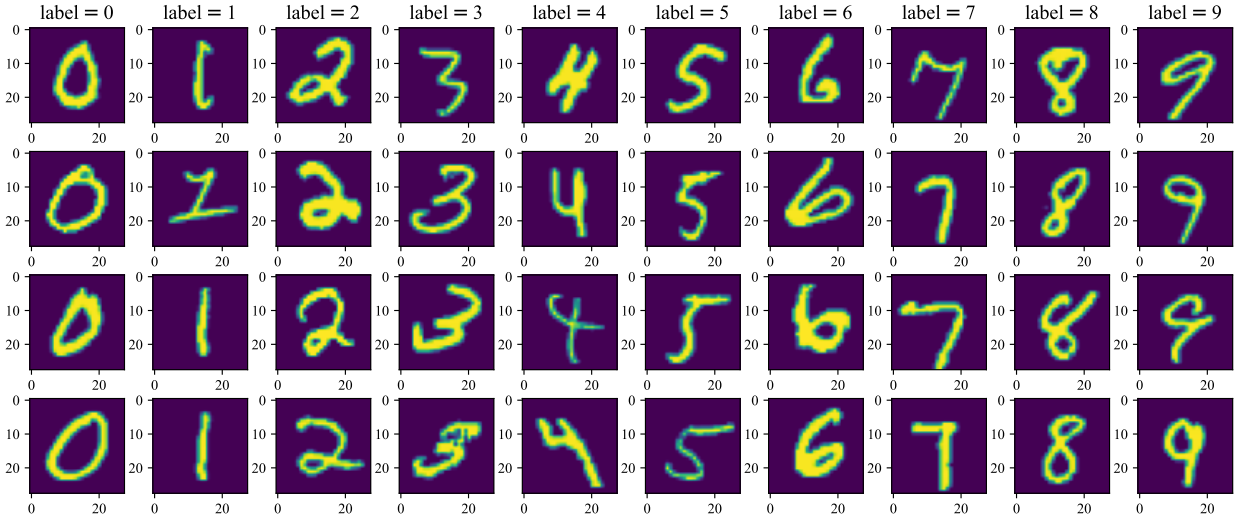


FIGURE 10. Examples of MNIST dataset of handwritten digits by Yann LeCun, Corinna Cortes, and Christopher J.C. Burges. There are total 70000 images of size 28×28 pixels.

In general, for classification problem, we seek to learn a decision algorithm $\mathscr{A} : \mathscr{X} \to \mathscr{Y} = \{\mathscr{C}_1, \ldots, \mathscr{C}_\kappa\}$, which maps each object $\mathbf{x} \in \mathscr{X}$ into one of κ possible classes $\mathscr{C}_1, \ldots, \mathscr{C}_\kappa$. This problem is called a *binary classification* if $\kappa = 2$ and a *multi-class classification* if $\kappa \geq 3$. For instance, the MNST dataset consists of total 70000 black-and-white images of size 28×28 pixels. Viewing each image as a matrix in $\mathbb{R}^{28\times 28}$, the classification problem for the MNST dataset $\mathscr{X}_{\text{MNIST}}$ can be thought of as a multi-class classification problem where one wants to learn a decision algorithm $\mathscr{A} : \mathbb{R}^{28\times 28} \supseteq \mathscr{X}_{\text{MNIST}} \to \{0, 1, 2, \ldots, 9\}$. Equivalently, by vectorizing each image into a $784 = 28^2$ dimensional vector, this is also to learn a decision algorithm $\mathscr{A} :\subseteq \mathbb{R}^{784} \supseteq \mathscr{X}_{\text{MNIST}} \to \{0, 1, 2, \ldots, 9\}$.

Some of the popular classification algorithms are: Logistic Regression, k-Nearest Neighbors, Decision Trees, Support Vector Machine, and Naive Bayes. We will take a closer look at some of the algorithms in this chapter.

2. Logistic Regression

2.1. The model formulation. We begin our discussion for the simplest case of binary classification. Without loss of generality, we can assume that the two classes are 0 and 1. Say we have training examples $(\boldsymbol{\phi}(\mathbf{x}_1), y_1), \ldots, (\boldsymbol{\phi}(\mathbf{x}_N), y_N)$, where

- $\mathbf{x}_1, \ldots, \mathbf{x}_N$: Input data (e.g., collection of all medical records of each patient)
- $\boldsymbol{\phi}(\mathbf{x}_1), \ldots, \boldsymbol{\phi}(\mathbf{x}_N) \in \mathbb{R}^p$: Features (e.g., some useful (derived) information for each patient)
- $y_1, \ldots, y_n \in \{0, 1\}$: Binary class labels (e.g., '`normal`' or '`pneumonia`').

The basic idea of logistic regression is to model the output y as a Bernoulli (0-1) random variable $Y \sim \text{Bernoulli}(p)$ with success probability p that depends on the observed feature $\boldsymbol{\phi}(\mathbf{x})$. More specifically,

$$Y \mid \boldsymbol{\phi}(\mathbf{x}) \sim \text{Bernoulli}(p), \qquad p = \sigma(\boldsymbol{\phi}(\mathbf{x})^T \mathbf{w}) := \frac{\exp(\boldsymbol{\phi}(\mathbf{x})^T \mathbf{w})}{1 + \exp(\boldsymbol{\phi}(\mathbf{x})^T \mathbf{w})} \tag{74}$$

Here, $\sigma : x \mapsto e^x/(1+e^x) = 1/(1+e^{-x})$ is called a *sigmoid* (or *logistic*) function, and $\mathbf{w} \in \mathbb{R}^p$ is a p-dimensional parameter vector. Notice that the sigmoid function maps the real line onto the interval $[0, 1]$ (see Figure 11). Hence one can think of it as a function that 'links' real scalar with an associated probability. So in logistic regression, we model the 'success probability' p given the feature $\boldsymbol{\phi}(\mathbf{x})$ as the dot product $\boldsymbol{\phi}(\mathbf{x})^T \mathbf{w} \in \mathbb{R}$ fed into the sigmoid function σ in order to make it a probability $\in [0, 1]$.

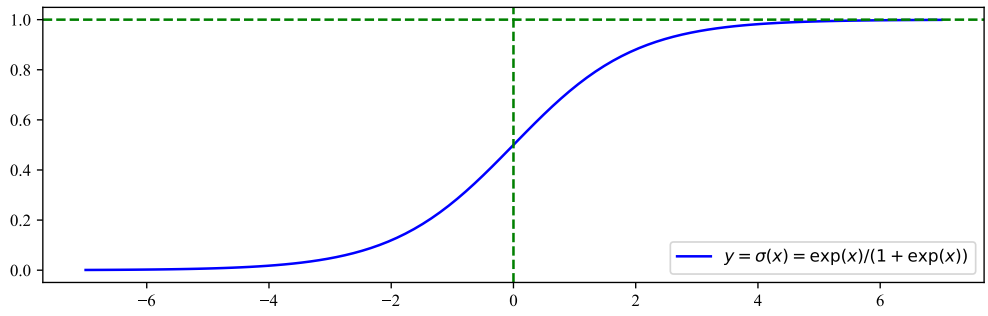


FIGURE 11. Plot of sigmoid function.

Next, let's discuss how to train logistic regression model. Since it is a probabilistic model, we will use the method of maximum likelihood. For that, we compute the joint likelihood of observing the output values $(y_1, \ldots, y_N)$ from the tuple of independent Bernoulli variables $(Y_1, \ldots, Y_N)$ with success probabilities $p_1 := \sigma(\boldsymbol{\phi}(\mathbf{x}_1)^T \mathbf{w}), \ldots, p_N := \sigma(\boldsymbol{\phi}(\mathbf{x}_N)^T \mathbf{w})$:

$$L(y_1, \ldots, y_N; \mathbf{w}) = \mathbb{P}(Y_1 = y_1, \ldots, Y_N = y_N; \mathbf{w}) = \prod_{i=1}^{N} p_i^{y_i} (1 - p_i)^{1 - y_i}. \tag{75}$$

Hence the optimial parameter $\hat{\mathbf{w}} \in \mathbb{R}^p$ is given by

$$\hat{\mathbf{w}} = \operatorname*{arg\,max}_{\mathbf{w} \in \mathbb{R}^p} \log L(y_1, \ldots, y_N; \mathbf{w}) \tag{76}$$

$$= \operatorname*{arg\,max}_{\mathbf{w} \in \mathbb{R}^p} \sum_{i=1}^{N} y_i \log p_i + (1 - y_i) \log(1 - p_i) \tag{77}$$

$$= \operatorname*{arg\,min}_{\mathbf{w} \in \mathbb{R}^p} \sum_{i=1}^{N} y_i \log(p_i^{-1} - 1) - \log(1 - p_i) \tag{78}$$

$$= \operatorname*{arg\,min}_{\mathbf{w} \in \mathbb{R}^p} \sum_{i=1}^{N} -y_i \boldsymbol{\phi}(\mathbf{x}_i)^T \mathbf{w} + \log\left(1 + \exp(\boldsymbol{\phi}(\mathbf{x}_i)^T \mathbf{w})\right), \tag{79}$$

where we have used the following observations

$$p_i^{-1} - 1 = \exp(-\boldsymbol{\phi}(\mathbf{x}_i)^T \mathbf{w}), \qquad (1-p_i)^{-1} = 1 + \exp(\boldsymbol{\phi}(\mathbf{x}_i)^T \mathbf{w}). \tag{80}$$

Using a matrix formulation, we have

$$\hat{\mathbf{w}} = \underset{\mathbf{w}\in\mathbb{R}^p}{\arg\min} \left[\ell_{LR}(\mathbf{w}) := \left(\sum_{i=1}^{N} \log\left(1 + \exp(\boldsymbol{\phi}(\mathbf{x}_i)^T \mathbf{w})\right) \right) - \mathbf{Y}^T \boldsymbol{\Phi}^T \mathbf{w} \right], \tag{81}$$

where

$$\mathbf{Y} = \begin{bmatrix} y_1 \\ \vdots \\ y_N \end{bmatrix} \in \{0,1\}^N, \qquad \boldsymbol{\Phi} = \begin{bmatrix} \uparrow & & \uparrow \\ \boldsymbol{\phi}(\mathbf{x}_1) & \cdots & \boldsymbol{\phi}(\mathbf{x}_N) \\ \downarrow & & \downarrow \end{bmatrix} \in \mathbb{R}^{p\times N}. \tag{82}$$

Here $\boldsymbol{\Phi}$ is also called the *design matrix*. The loss function $\ell_{LR}(\mathbf{w})$ in (81) is called the *logistic regression loss*.

2.2. Minimizing the logistic regression loss. In the previous section, we have formulated the logistic regression model and derived an optimization problem (81) for its training. Namely, we need to minimize the logistic regression loss ℓ_{LR} defined in (81) in order to find the optimal (maximum likelihood) parameter $\hat{\mathbf{w}}$. Recall that in the case of polynomial regression, the quadratic optimization problem (7) had a closed-form solution for the optimal $\hat{\mathbf{w}}$. In the case of the logistic regression, we will observe that there is no closed-form solution for the minimizer, but it is still a *convex optimization problem*, which can be solved effectively by an iterative algorithm (e.g., gradient descent, Newton-Ralphson).

The following exercise provides the gradient, the Hessian, and convexity of the logistic regression loss ℓ_{LR}.

Exercise 2.2.1 (Convexity of the logistic regression loss)**.** Let $\ell_{LR}(\mathbf{w})$ denote the logistic regression loss defined in (81). Let $\mathbf{Q} = \mathbf{Q}(\mathbf{w}) = [p_1, \ldots, p_N]^T \in \mathbb{R}^N$ where $p_i = \sigma(\boldsymbol{\phi}(\mathbf{x}_i)^T \mathbf{w})$ with $\sigma(x) = e^x/(1+e^x)$.

(i) Show that $\frac{\partial \sigma(x)}{x} = \sigma(x)(1-\sigma(x))$.

(ii) Show that

$$\frac{\partial \mathbf{Q}^T}{\partial \mathbf{w}} = \begin{bmatrix} \frac{\partial}{\partial w_1} \\ \vdots \\ \frac{\partial}{\partial w_p} \end{bmatrix} \begin{bmatrix} p_1 & \cdots & p_N \end{bmatrix} = \begin{bmatrix} \uparrow & & \uparrow \\ p_1(1-p_1)\boldsymbol{\phi}(\mathbf{x}_1) & \cdots & p_N(1-p_N)\boldsymbol{\phi}(\mathbf{x}_N) \\ \downarrow & & \downarrow \end{bmatrix} = \boldsymbol{\Phi}\mathbf{D}, \tag{83}$$

where $\mathbf{D}$ is the $N \times N$ diagonal matrix with diagonal entries $\mathbf{D}_{ii} = p_i(1-p_i)$.

(iii) Show that

$$\nabla_{\mathbf{w}} \ell_{LR}(\mathbf{w}) = \boldsymbol{\Phi}(\mathbf{Q} - \mathbf{Y}) \in \mathbb{R}^p, \qquad \mathbf{H} := \frac{\partial^2 \ell_{LR}(\mathbf{w})}{\partial \mathbf{w} \partial \mathbf{w}^T} = \nabla_{\mathbf{w}} \mathbf{Q}^T \boldsymbol{\Phi}^T = \boldsymbol{\Phi}\mathbf{D}\boldsymbol{\Phi}^T \in \mathbb{R}^{p\times p}. \tag{84}$$

(iv) Argue that the Hessian $\mathbf{H}$ in **(iii)** is positive semi-definite. Conclude that the logistic regression loss ℓ_{LR} is convex. (Hint: $\mathbf{D}$ is a diagonal matrix with positive diagonal entries.)

Now, we will introduce two convex optimization algorithms for logistic regression. The algorithms are iterative in nature, meaning that they apply the same update rule until a certain convergence criterion is satisfied. The first algorithm belongs to the class of *gradient descent*:

$$\mathbf{w}_{t+1} \longleftarrow \mathbf{w}_t - \eta_t \nabla_{\mathbf{w}} \ell_{LR}(\mathbf{w}_t) = \mathbf{w}_t - \eta_t \boldsymbol{\Phi}(\mathbf{Q}(\mathbf{w}_t) - \mathbf{Y}), \tag{85}$$

where $(\eta_t)_{t\geq 0}$ is a sequence of positive numbers called *step sizes*. Recall that the gradient $\nabla_{\mathbf{w}} \ell_{LR}$ is the direction of change in $\mathbf{w}$ that increases the loss function ℓ_{LR} most rapidly. Since we want to minimize the loss function, we would want to incrementally change the current iterate $\mathbf{w}_t$ to the opposite direction of the gradient. The remaining degree of freedom is the step size that tells us how far we follow such

direction $-\nabla_{\mathbf{w}}\ell_{LR}(\mathbf{w}_t)$ of the steepest descent[1]. A particular choice that is guaranteed to converge to a global minimum for convex optimization problems is $\eta_t = \gamma/((\log t)\sqrt{t})$, where $\gamma > 0$ is a constant[2].

The second algorithm also belongs to the class of gradient descent but uses the Hessian $\mathbf{H} = \nabla^2\ell_{LR}(\mathbf{w}_t)$, which is called the *Newton-Ralphson method*. It takes the following form:

$$\mathbf{w}_{t+1} \longleftarrow \mathbf{w}_t - \mathbf{H}^{-1}\nabla_{\mathbf{w}}\ell_{LR}(\mathbf{w}_t). \tag{86}$$

One can think of the above algorithm as a vresion of gradient descent where we used the Hessian inverse $\mathbf{H}^{-1}$ as a 'matrix-valued step size'[3]. The use of second-order derivatives often accelerates the speed of convergence but suffers from higher computational cost per iteration. In our current case of logistic regression, the Newton-Ralphson algorithm in (86) takes the following simple form

$$\mathbf{w}_{t+1} \longleftarrow \mathbf{w}_t - (\mathbf{\Phi}\mathbf{D}\mathbf{\Phi}^T)^{-1}\mathbf{\Phi}(\mathbf{Q}-\mathbf{Y}) \tag{87}$$

$$= (\mathbf{\Phi}\mathbf{D}\mathbf{\Phi}^T)^{-1}\big(\mathbf{\Phi}\mathbf{D}\mathbf{\Phi}^T\mathbf{w}_t - \mathbf{\Phi}(\mathbf{Q}-\mathbf{Y})\big) \tag{88}$$

$$= (\mathbf{\Phi}\mathbf{D}\mathbf{\Phi}^T)^{-1}\mathbf{\Phi}\mathbf{D}\mathbf{z}, \tag{89}$$

where $\mathbf{D} \in \mathbb{R}^{N\times N}$ is the diagonal matrix defined in Exercise 2.6.1 and we have denoted $\mathbf{z} := \mathbf{\Phi}^T\mathbf{w}_t - \mathbf{D}^{-1}(\mathbf{Q}-\mathbf{Y}) \in \mathbb{R}^N$. From the analysis in Exercise 1.2.1, one can observe that the last expression is the solution of the following 'weighted least sqaures problem'

$$\tilde{\mathbf{w}} := \underset{\mathbf{w}\in\mathbb{R}^p}{\operatorname{argmin}} \|\mathbf{z} - \sqrt{\mathbf{D}}\mathbf{\Phi}^T\mathbf{w}\|_F^2. \tag{90}$$

However, both the target vector $\mathbf{z}$ and the weight matrix $\sqrt{\mathbf{D}}$ change when we update $\mathbf{w}_t$. Hence one has to apply the above formula iteratively. For these reasons, the Newton-Ralphson algorithm (86) for logistic regression is also called *iterative reweighted least squares* (IRWS).

Exercise 2.2.2. The gradient descent (GD) algorithm for logistic regression training in (85) is implemented as the python function 'fit_LR_GD' in the Jupyter notebook provided in the course repository.

```python
def fit_LR_GD(Y, H, W0=None, sub_iter=100, stopping_diff=0.01):
        '''
        Convex optimization algorithm for Logistic Regression using Gradient Descent
        Y = (n x 1), H = (p x n) (\Phi in lecture note), W = (p x 1)
        Logistic Regression: Y ~ Bernoulli(Q), Q = sigmoid(H.T @ W)
        MLE -->
        Find \hat{W} = argmin_W ( sum_j ( log(1+exp(H_j.T @ W) ) - Y.T @ H.T @ W ) )
        '''
        if W0 is None:
            W0 = np.random.rand(H.shape[0],1) #If initial coefficients W0 is None, randomly initialize

        W1 = W0.copy()
        i = 0
        dist = 1
        grad = np.ones(W0.shape)
        while (i < sub_iter) and (np.linalg.norm(grad) > stopping_diff):
            Q = 1/(1+np.exp(-H.T @ W1))  # probability matrix, same shape as Y
            # grad = H @ (Q - Y).T + alpha * np.ones(W0.shape[1])
            grad = H @ (Q - Y)
            W1 = W1 - (np.log(i+1) / (((i + 1) ** (0.5)))) * grad
            i = i + 1
            # print('iter %i, grad_norm %f' %(i, np.linalg.norm(grad)))
        return W1
```

FIGURE 12. A python implementation of the gradient descent algorithm for logistic regression training (85).

[1] See Figure 1: The trajectory on the graph of the loss function can be viewed as that of a gradient descent algorithm.

[2] This is an instance of dinimishing step sizes, where the step sizes are required to be non-summable $\sum_{t=1}^{\infty}\eta_t = \infty$ but square-summable $\sum_{t=1}^{\infty}\eta_t^2 < \infty$. See [SSY18].

[3] The logic behind this algorithm is to minimize the second-order Taylor approximation of the objective ℓ_{LR}. Namely, $\ell_{LR}(\mathbf{w}) = \ell_{LR}(\mathbf{w}_t) + \nabla\ell_{LR}(\mathbf{w}_t)^T(\mathbf{w}-\mathbf{w}_t) + \frac{1}{2}(\mathbf{w}-\mathbf{w}_t)^T\nabla^2\ell_{LR}(\mathbf{w}_t)(\mathbf{w}-\mathbf{w}_t) + O(\|\mathbf{w}-\mathbf{w}_t\|_F^3)$, and the minimizer of the quadratic function here is exactly (86).

(i) Modify the code (or make your own function) so that each gradient descent step (85), the current value of objective function ℓ_{LR} in (81) is printed. Test it on the MNIST example with digits ['0','1']. Does the loss monotonically decrease?

(ii) The implemented version of GD for (85) uses the step size $\eta_t = \gamma \log(t+1)/\sqrt{(t+1)}$ with $\gamma = 1$. Test the same step sizes for $\gamma = 10$ and also $\eta_t = 1$ and $\eta_t = 1/t$. Test it on the MNIST example with digits ['0','1']. Is there any difference in the way the loss function decrease?

(iii) If we use the step size of the form $\eta_t = \gamma t^{-\delta}$ for $0 \le \delta \le 1$, what would be your optimal choice of the hyperparameters γ and δ? (There is no answer and trying out a few choices and make your observation.)

Exercise 2.2.3. See the Jupyter notebook provided in the course repository.

```
def fit_LR_NR(Y, H, W0=None, sub_iter=100, stopping_diff=0.01):
    '''
    Convex optimization algorithm for Logistic Regression using Newton-Ralphson algorithm.
    Y = (n x 1), H = (p x n) (\Phi in lecture note), W = (p x 1)
    Logistic Regression: Y ~ Bernoulli(Q), Q = sigmoid(H.T @ W)
    MLE -->
    Find \hat{W} = argmin_W ( sum_j ( log(1+exp(H_j.T @ W) ) - Y.T @ H.T @ W ) )
    '''
    ### Implement by yourself.
```

FIGURE 13. A python implementation of the Newton-Ralphson algorithm for logistic regression training (86) to be completed.

(i) Implement the Newton-Ralphson algorithm for logistic regression training in (86) in the same notebook, and reproduce Figure 23. How does the result compare with two optimization algorithms?

(ii) Compare with the gradient descent algorithm. Which one do you think is faster? Which one do you think is more efficient? (Do some experiments make your own argument. There is no single answer.)

3. Binary classification of MNIST by Logistic Regression

3.1. Training. Recall that the MNIST dataset (see Figure 10) consists of total 70000 black-and-white images of size 28×28 pixels, corresponding to one of the 10 digits from $\{0, 1, \ldots, 9\}$. In this subsection, we will apply logistic regression to classify the MNIST images of chosen two digits.

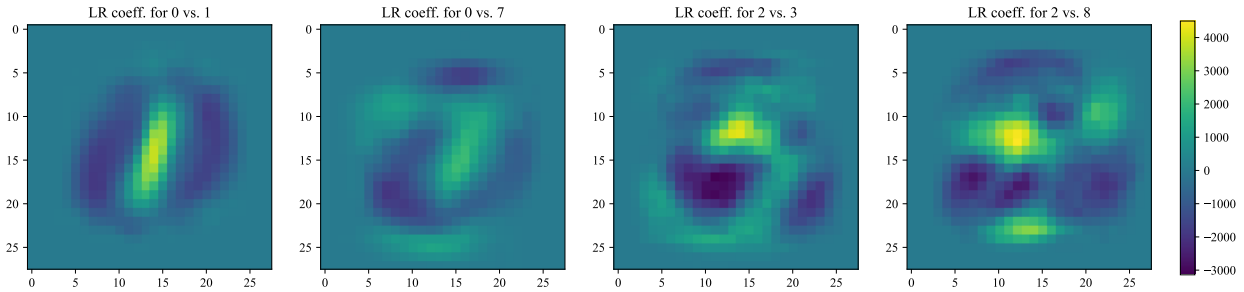


FIGURE 14. Examples of the fitted logistic regression coefficients to the MNIST dataset with chosen two digits. Gradient descent algorithm is used for training.

For instance, suppose we choose the images for digits '0' and '1'. There are total 6903 and 7877 images of digits '0' and '1', respectively. Viewing each image as a matrix in $\mathbb{R}^{28\times 28}$, the binary classification problem now is to learn a decision algorithm $\mathscr{A} : \mathbb{R}^{28\times 28} \supseteq \{\text{Images of '0' or '1'}\} \to \{0, 1\}$, where class label 1 means the image is classified as digit '1' and label 0 for digit '0'. The goal is to learn the optimal logistic regression parameter $\hat{\mathbf{w}} \in \mathbb{R}^{28\times 28}$ by solving the optimization problem (81). The meaning of the (i, j) entry $\hat{\mathbf{w}}[i, j]$ is the positive association of the (i, j) pixel of all 28×28 images in the train set with class label 1.

The first subplot in Figure 23 displays $\hat{\mathbf{w}}$ for digits '0′ and '1′, learned by the gradient descent algorithm in (85) with $\gamma = 1$. One can see that the pixels lying on the overall shape of digit '1' have large regression coefficients, meaning that they contribute to the image being classified as '1' with a large probability. On the other hand, one can also see that the pixels lying on the overall shape of digit '0' have small regression coefficients, meaning that they discount the probability of the image being classified as '1' and hence encourage it to be classified as '0'. The results for other digit pairs ['0′, '7′], ['2′, '3′], and ['2′, '8′] are also shown in Figure 23.

Notice such a pixel-based regression for image classification would make sense only if the images are well-aligned. For example, if the images for digit '1' show up in all different locations and orientations and even with various scaling, then it won't be possible to attribute any pixel (location in the image) as an important feature of being class 1.

Exercise 2.3.1. See the Jupyter notebook provided in the course repository.

(i) Reproduce the experiments in Figure 23 for all pairs of digits. (There will be $\binom{10}{2} = 55$ pairs. Use either convex optimization algorithms.)

(ii) By looking at the plot of the optimal regression coefficients, for each digit i, make a guess on digits j that would be the easiest and the hardest for the logistic regression model would distinguish.

3.2. Testing. Now let's discuss testing our trained logistic regression model for the MNIST dataset. We first need to discuss how we measure the accuracy of binary classification. Suppose we have target and predicted binary vectors $y_{\text{test}}, y_{\text{pred}} \in \{0,1\}^N$. For each test example i, there are four possibilities: $(y_{\text{test}}[i], y_{\text{pred}}[i]) \in \{(T,T),(T,F),(F,T),(F,F)\}$, where T stands for 'true' or 'positive' and F stands for 'false' or 'negative'. These four cases have the following specific names:

- *True Positive*: $(y_{\text{test}}[i], y_{\text{pred}}[i]) = (T,T)$
- *False Positive* $(y_{\text{test}}[i], y_{\text{pred}}[i]) = (F,T)$ (e.g., 'patient diagnosed to have cancer but does not')
- *False Negative* $(y_{\text{test}}[i], y_{\text{pred}}[i]) = (T,F)$ (e.g., 'patient diagnosed healthy but has cancer')
- *True Negative*$(y_{\text{test}}[i], y_{\text{pred}}[i]) = (F,F)$

One of the above instances hold for each test example i. The table in Figure 15 summaries various statistics associated to predicting y_{test} as y_{pred}.

		CONDITION determined by "Gold Standard"			
	TOTAL POPULATION	CONDITION POS	CONDITION NEG	PREVALENCE = CONDITION POS / TOTAL POPULATION	
TEST OUTCOME	TEST POS	True Pos TP	*Type I Error* False Pos FP	*Precision* Pos Predictive Value PPV = TP / TEST P	False Discovery Rate FDR = FP / TEST P
	TEST NEG	*Type II Error* False Neg FN	True Neg TN	False Omission Rate FOR = FN / TEST N	Neg Predictive Value NPV = TN / TEST N
	ACCURACY ACC = (TP + TN) / TOT POP	*Sensitivity (SN), Recall* Total Pos Rate TPR = TP / CONDITION POS	*Fall-Out* False Pos Rate FPR = FP / CONDITION NEG	Pos Likelihood Ratio LR+ = TPR / FPR	Diagnostic Odds Ratio DOR = LR+ / LR-
		Miss Rate False Neg Rate FNR = FN / CONDITION POS	*Specificity (SPC)* True Neg Rate TNR = TN / CONDITION NEG	Neg Likelihood Ratio LR- = TNR / FNR	

FIGURE 15. Various statistics of binary classification testing. Figure excerpted from link.

Here is a list of a few commonly used statistics:

- *Accuracy* $= \frac{\#\text{ predicted correctly}}{\text{size of the test set}}$ (e.g. $= \frac{\#\text{ predicted correctly as cancer or healthy}}{\#\text{ total subjects}}$)
- *Sensitivity* $= \frac{\#\text{ predicted positive correctly}}{\#\text{ positive test examples}}$ (e.g. $= \frac{\#\text{ predicted correctly as cancer}}{\#\text{ cancer patients}}$)
- *Specificity* $= \frac{\#\text{ predicted negative correctly}}{\#\text{ negative test examples}}$ (e.g. $= \frac{\#\text{ predicted correctly as healthy}}{\#\text{ healthy subjects}}$)
- *Precision* $= \frac{\#\text{ predicted positive correctly}}{\#\text{ predicted positive}}$ (e.g. $= \frac{\#\text{ predicted correctly as cancer}}{\#\text{ predicted as cancer}}$)

Next, before we proceed to the running example of LR on MNIST, we need to say a bit more on how we get the binary vector of predictions y_{pred}. Namely, recall that LR is a probabilistic model, where one models the output as a Bernoulli RV Y with success probability $p = \sigma(\boldsymbol{\phi}(\mathbf{x})^T\mathbf{w})$ as in (74). The result of fitting LR to the training set is the optimal parameter $\hat{\mathbf{w}}$ defined in (81), so we can compute the predictive probability $p = p(\mathbf{y}|\boldsymbol{\phi}(\mathbf{x}))$ for the unknown output. Suppose we have $p = 0.8$. This means that our prediction on the output has probability 0.8 to be 1 and 0.2 to be 0, so it would be reasonable to predict the output as 1. More precisely, for the current example, we use the following decision algorithm

$$\mathbf{x} \longmapsto \hat{\mathbf{y}} = \mathscr{A}(\mathbf{x}) = \begin{cases} 1 & \text{if } p = \sigma(\boldsymbol{\phi}(\mathbf{x})^T\hat{\mathbf{w}}) \geq 0.5 \\ 0 & \text{if } p = \sigma(\boldsymbol{\phi}(\mathbf{x})^T\hat{\mathbf{w}}) < 0.5. \end{cases} \tag{91}$$

Simply speaking, we are predicting $\hat{\mathbf{y}}$ to be more likely class label predicted by the predicted probability, with the classification threshold being 0.5[4]. This is an instance of the more general decision algorithm discussed in Exercise 1.3.2.

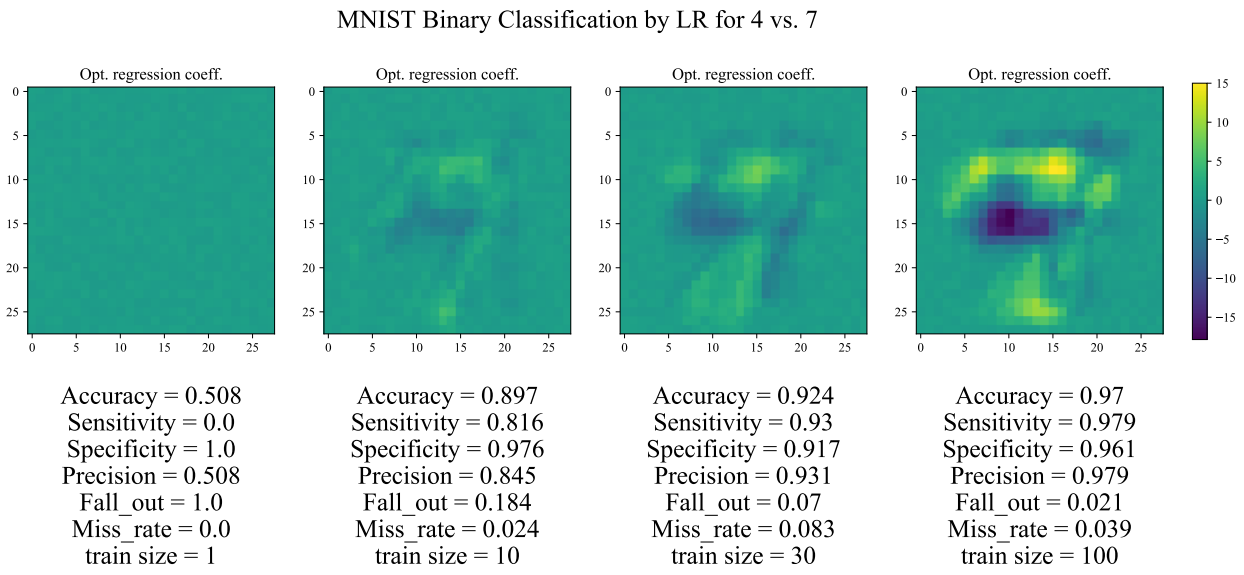


FIGURE 16. Examples of the training and testing logistic regression coefficients to the MNIST dataset with chosen two digits. Gradient descent algorithm is used for training. Training datasets of size in $\{1, 10, 30, 100\}$ are subsampled and used for fitting the model. Various statistics for binary classification are also shown.

Now we discuss testing LR on the MNIST dataset. MNIST is such a clean dataset so that we only need a few training examples to achieve very good classification accuracies. We test our LR classifier trained on randomly subsampled test sets of sizes 1, 10, 30, and 100 for classifying digits '4' vs. '7'. The test set contains about 20% of examples in the dataset, which is about 14,000 images of digits '4' and '7'. In Figure

[4]The use of threshold 0.5 is not necessary. For highligy imbalanced training datasets, the predicted probabilities could be skewed either to 0 or 1, in which case we would need to use some other threshold $\theta \in [0,1]$. The 'overall accuracy' of binary classification using arbitrary threshold θ is measured by other means, such as the *area-under-curve* (AUC) score for the corresponding *receiver operation characteristic* (ROC)

16, we have examples of trained regression coefficients (shown as a heat map on 28×28 image) as well as various statistics of binary classification we discussed before. Observe that as we increase the size of the train set, the optimal regression coefficients become more aware of the shapes of '7' as positive (yellow) and '4' as negative (blue), and the classification metrics improve in general. See Figure 17 for a similar example with digits '3' and '8' instead, which seems to be a bit more difficult case than distinguishing between '4' and '7'.

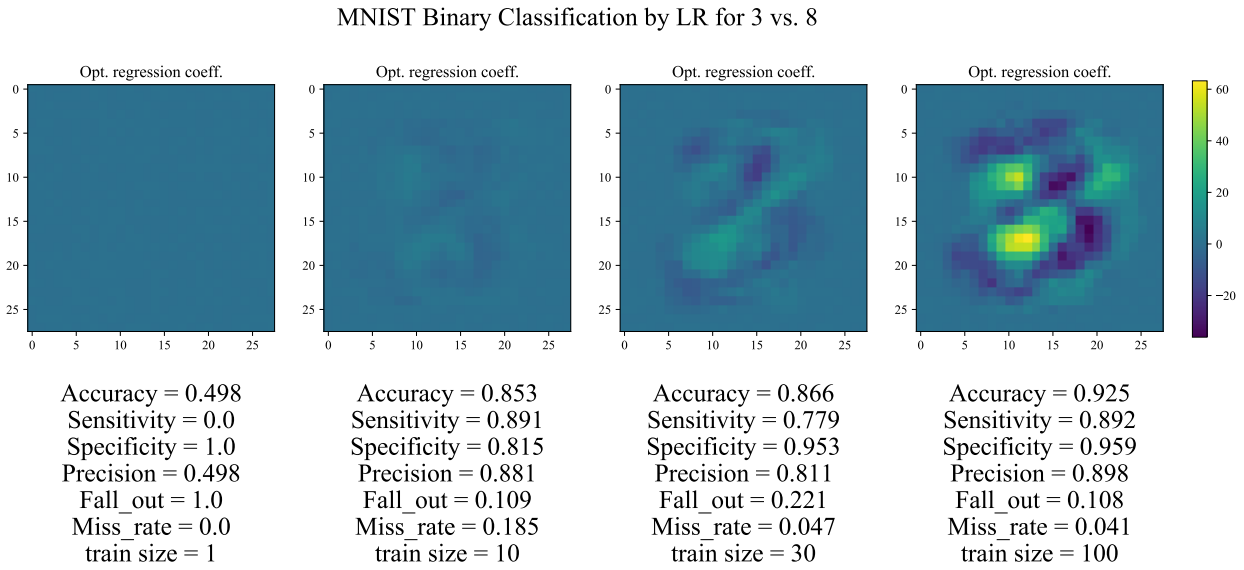


FIGURE 17. Examples of the training and testing logistic regression coefficients to the MNIST dataset with chosen two digits. Gradient descent algorithm is used for training. Training datasets of size in $\{1, 10, 30, 100\}$ are subsampled and used for fitting the model. Various statistics for binary classification are also shown.

3.3. Multiclass Logistic Regression. Next, we extend our logistic regression model to the multiclass classification, where the class labels now assume $\kappa \geq 2$ distinct values. This model is called the *multiclass logistic regression* (MLR). Without loss of generality, we can assume that the κ classes are the integers in $\{1, 2, \ldots, \kappa\}$. Say we have training examples $(\boldsymbol{\phi}(\mathbf{x}_1), y_1), \ldots, (\boldsymbol{\phi}(\mathbf{x}_N), y_N)$, where

- $\mathbf{x}_1, \ldots, \mathbf{x}_N$: Input data (e.g., collection of all medical records of each patient)
- $\boldsymbol{\phi}(\mathbf{x}_1), \ldots, \boldsymbol{\phi}(\mathbf{x}_N) \in \mathbb{R}^p$: Features (e.g., some useful (derived) information for each patient)
- $y_1, \ldots, y_n \in \{1, 2, \ldots, \kappa\}$: κ class labels (e.g., digits from 0 to 9).

The basic idea of multiclass logistic regression is to model the output y as a discrete random variable Y with probability mass function (PMF) $p = [p_1, \ldots, p_\kappa]$ that depends on the observed feature $\boldsymbol{\phi}(\mathbf{x})$. More specifically,

$$\mathbb{P}\left(Y = i \,|\, \boldsymbol{\phi}(\mathbf{x})\right) = p_i, \qquad p_i \propto \exp(\boldsymbol{\phi}(\mathbf{x})^T \mathbf{w}_i) \quad \text{for } i = 1, \ldots, \kappa. \tag{92}$$

More precisely, the predictive probability distribution $p = [p_1, \ldots, p_\kappa]$ can be written as

$$p_i = \frac{\exp(\boldsymbol{\phi}(\mathbf{x})^T \mathbf{w}_i)}{\sum_{j=1}^{\kappa} \exp(\boldsymbol{\phi}(\mathbf{x})^T \mathbf{w}_j)}, \qquad \text{for } i = 1, \ldots, \kappa. \tag{93}$$

A probability distribution of the form $p_i \propto \exp(a_i)$ is called the *softmax distribution* with *activation* $\mathbf{a} = [a_1, \ldots, a_\kappa]$. Hence in the case of multinomial LR, the predictive probability distribution $p = [p_1, \ldots, p_\kappa]$

is the softmax distribution with 'linear activation' $\boldsymbol{\phi}(\mathbf{x})^T\mathbf{w}_1 \ldots, \boldsymbol{\phi}(\mathbf{x})^T\mathbf{w}_\kappa$ of the feature vector $\boldsymbol{\phi}(\mathbf{x})$. Notice that this model has parameter vectors $\mathbf{w}_1, \ldots, \mathbf{w}_\kappa$, one for each of the κ class labels[5].

Next, let's discuss how to train MLR. We will use the following matrix notations

$$\mathbf{Y} = \begin{bmatrix} \mathbf{1}(y_1 = 1) & \cdots & \mathbf{1}(y_1 = \kappa) \\ \vdots & & \vdots \\ \mathbf{1}(y_N = 1) & \cdots & \mathbf{1}(y_N = \kappa) \end{bmatrix} \in \{0,1\}^{N\times\kappa}, \qquad \boldsymbol{\Phi} = \begin{bmatrix} \uparrow & & \uparrow \\ \boldsymbol{\phi}(\mathbf{x}_1) & \cdots & \boldsymbol{\phi}(\mathbf{x}_N) \\ \downarrow & & \downarrow \end{bmatrix} \in \mathbb{R}^{p\times N}, \tag{95}$$

$$\mathbf{W} = \begin{bmatrix} \uparrow & & \uparrow \\ \mathbf{w}_1 & \cdots & \mathbf{w}_\kappa \\ \downarrow & & \downarrow \end{bmatrix} \in \mathbb{R}^{p\times\kappa}. \tag{96}$$

Again since it is a probabilistic model, we use the method of maximum likelihood. For that, we compute the joint likelihood of observing the output values $(y_1, \ldots, y_N)$ from the tuple of independent RVs $(Y_1, \ldots, Y_N)$ with PMFs $p_{ij} \propto \exp(\boldsymbol{\phi}(\mathbf{x}_j)^T\mathbf{w}_j)$ where $p_{ij} = \mathbb{P}(Y_i = j)$:

$$L(y_1, \ldots, y_N; \mathbf{W}) = \mathbb{P}(Y_1 = y_1, \ldots, Y_N = y_N; \mathbf{W}) = \prod_{i=1}^{N}\prod_{j=1}^{\kappa}(p_{ij})^{\mathbf{1}(y_i=j)} \tag{97}$$

Hence the optimial parameter matrix $\hat{\mathbf{W}} = [\hat{\mathbf{w}}_1, \ldots, \hat{\mathbf{w}}_\kappa] \in \mathbb{R}^{p\times\kappa}$ by

$$\hat{\mathbf{W}} = \underset{\mathbf{W}\in\mathbb{R}^{p\times\kappa}}{\arg\max} \log L(y_1, \ldots, y_N; \mathbf{W}) \tag{98}$$

$$= \underset{\mathbf{W}\in\mathbb{R}^{p\times\kappa}}{\arg\max} \sum_{i=1}^{N}\sum_{j=1}^{\kappa}\mathbf{1}(y_i = j)\log p_{ij} \tag{99}$$

$$= \underset{\mathbf{W}\in\mathbb{R}^{p\times\kappa}}{\arg\min} \sum_{i=1}^{N}\sum_{j=1}^{\kappa}\mathbf{1}(y_i = j)\left[-\boldsymbol{\phi}(\mathbf{x}_i)^T\mathbf{w}_j + \log\left(\sum_{q=1}^{\kappa}\exp(\boldsymbol{\phi}(\mathbf{x}_i)^T\mathbf{w}_q)\right)\right] \tag{100}$$

$$= \underset{\mathbf{W}\in\mathbb{R}^{p\times\kappa}}{\arg\min} \left[-\sum_{i=1}^{N}\sum_{j=1}^{\kappa}\mathbf{1}(y_i = j)\boldsymbol{\phi}(\mathbf{x}_i)^T\mathbf{w}_j + \sum_{i=1}^{N}\log\left(\sum_{q=1}^{\kappa}\exp(\boldsymbol{\phi}(\mathbf{x}_i)^T\mathbf{w}_q)\right)\right], \tag{101}$$

where we have used the following observation

$$1 = \sum_{j=1}^{\kappa}\mathbf{1}(y_i = j) \quad \text{for all } i = 1, \ldots, N. \tag{102}$$

Using a matrix formulation, we have

$$\hat{\mathbf{W}} = \underset{\mathbf{W}\in\mathbb{R}^{p\times\kappa}}{\arg\min} \left[\ell_{MLR}(\mathbf{W}) := \left(\sum_{i=1}^{N}\log\left(\sum_{q=1}^{\kappa}\exp(\boldsymbol{\phi}(\mathbf{x}_i)^T\mathbf{w}_q)\right)\right) - \operatorname{tr}\left(\mathbf{Y}^T\boldsymbol{\Phi}^T\mathbf{W}\right)\right], \tag{103}$$

The loss function $\ell_{MLR}(\mathbf{w})$ in (81) is called the *multiclass logistic regression loss.*

In Exercise 2.3.2, we show that the above is a convex optimization problem and also compute the gradient and Hessian. As in the logistic regression case, we can solve Exercise 2.3.2 iteratively either by gradient descent or Newton-Ralphson. For gradient descent algorithm, we use

$$\mathbf{W}_{t+1} \longleftarrow \mathbf{W}_t - \eta_t\nabla_{\mathbf{W}}\ell_{MLR}(\mathbf{W}_t) = \mathbf{W}_t - \eta_t\boldsymbol{\Phi}(\mathbf{Q}(\mathbf{W}_t) - \mathbf{Y}) \in \mathbb{R}^{p\times\kappa}, \tag{104}$$

where we can use the same step sizes $\eta_t = \gamma/((\log t)\sqrt{t})$ for some $\gamma > 0$. The Newton-Ralphson algorithm (86) can also be implemented for the case of MLR by using the Hessian computed in Exercise 2.3.2.

[5] In fact, it is enough to determine $\kappa - 1$ p-dimensional vectors $\mathbf{w}_2 - \mathbf{w}_1, \ldots, \mathbf{w}_\kappa - \mathbf{w}_1$ since by dividing both the numerator and the denominator by $\exp(\boldsymbol{\phi}(\mathbf{x})^T\mathbf{w}_1)$, we can write the MLR predictive distribution in (93) as

$$p_i = \frac{\exp(\boldsymbol{\phi}(\mathbf{x})^T(\mathbf{w}_i - \mathbf{w}_1))}{\sum_{j=1}^{\kappa}\exp(\boldsymbol{\phi}(\mathbf{x})^T(\mathbf{w}_j - \mathbf{w}_1))}, \qquad \text{for } i = 1, \ldots, \kappa \tag{94}$$

Exercise 2.3.2 (Convexity of the multiclass logistic regression loss)**.** Let $\ell_{MLR}(\mathbf{W})$ denote the multiclass logistic regression loss defined in (103). Let $\mathbf{Q} = \mathbf{Q}(\mathbf{W}) = (p_{ij})_{i,j} \in \mathbb{R}^{N\times\kappa}$ where $p_{ij} \propto \exp(\boldsymbol{\phi}(\mathbf{x}_i)^T \mathbf{w}_j)$.

(i) Show that

$$\nabla_{\mathbf{W}} \ell_{MLR}(\mathbf{W}) = \boldsymbol{\Phi}(\mathbf{Q} - \mathbf{Y}) \in \mathbb{R}^{p\times\kappa}. \tag{105}$$

(ii) For each $\mathbf{W} = [\mathbf{w}_1, \ldots, \mathbf{w}_\kappa] \in \mathbb{R}^{p\times\kappa}$, let $\mathbf{W}^{\text{vec}} := [\mathbf{w}_1^T, \ldots, \mathbf{w}_\kappa^T]^T \in \mathbb{R}^{p\kappa}$ denote its vectorization. Identify $\mathbf{W}$ with $\mathbf{W}^{\text{vec}}$ so that $\ell_{MLR} : \mathbb{R}^{p\kappa} \to \mathbb{R}$ is a function that maps a vector to a scalar. Then its Hessian $\mathbf{H} := \frac{\partial^2 \ell_{MLR}(\mathbf{W})}{\partial \mathbf{W} \partial \mathbf{W}^T}$ is a $p\kappa \times p\kappa$ matrix. Show that

$$\nabla_{\mathbf{W}} \ell_{MLR}(\mathbf{W}) = (\boldsymbol{\Phi}(\mathbf{Q} - \mathbf{Y}))^{\text{vec}} = \overbrace{\begin{bmatrix} \boldsymbol{\Phi} & O & O & \cdots & O \\ O & \boldsymbol{\Phi} & O & \cdots & O \\ \vdots & \vdots & & & \vdots \\ O & \cdots & O & \boldsymbol{\Phi} & O \\ O & O & \cdots & O & \boldsymbol{\Phi} \end{bmatrix}}^{\kappa \times \kappa \text{ block matrix}} (\mathbf{Q}^{\text{vec}} - \mathbf{Y}^{\text{vec}}) \in \mathbb{R}^{p\kappa}, \tag{106}$$

$$\mathbf{H} = \frac{\partial^2 \ell_{MLR}(\mathbf{W})}{\partial \mathbf{W} \partial \mathbf{W}^T} = \nabla_{\mathbf{W}} \mathbf{Q}^T \boldsymbol{\Phi}^T = \begin{bmatrix} \boldsymbol{\Phi}\mathbf{D}_1\boldsymbol{\Phi}^T & O & \cdots & O \\ O & \boldsymbol{\Phi}\mathbf{D}_2\boldsymbol{\Phi}^T & \cdots & O \\ \vdots & \vdots & & \vdots \\ O & \cdots & O & \boldsymbol{\Phi}\mathbf{D}_\kappa\boldsymbol{\Phi}^T \end{bmatrix} \in \mathbb{R}^{p\kappa\times p\kappa}, \tag{107}$$

where for $j = 1, \ldots, \kappa$, $\mathbf{D}_j$ is the $N \times N$ diagonal matrix with diagonal entries $\mathbf{D}_{ii} = p_{ij}(1 - p_{ij})$.

(iii) Argue that the Hessian $\mathbf{H}$ in **(ii)** is positive semi-definite. Conclude that the logistic regression loss ℓ_{MLR} is convex. (Hint: Each $\mathbf{D}_j$ is a diagonal matrix with positive diagonal entries.)

4. Classification of MNIST by Multiclass Logistic Regression

4.1. Training. Recall that the MNIST dataset (see Figure 10) consists of total 70000 black-and-white images of size 28×28 pixels, corresponding to one of the 10 digits from $\{0, 1, \ldots, 9\}$. In this subsection, we will apply multiclass logistic regression to classify the MNIST images of for more then two digits.

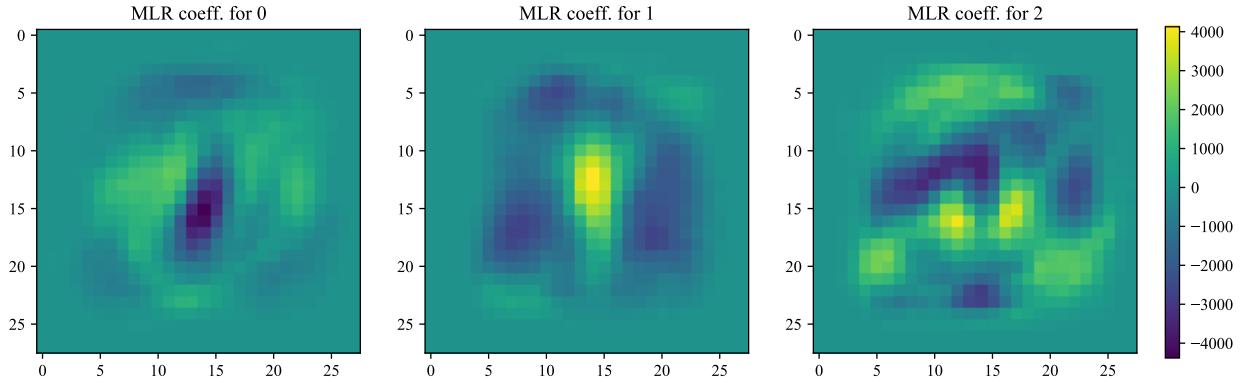


FIGURE 18. Examples of the fitted logistic regression coefficients to the MNIST dataset with digits in $['0', '1', '2']$. Gradient descent algorithm is used for training.

For instance, suppose we choose the images for digits '0', '1', and '2'. There are total 6903 , 7877, and 6990 images of digits '0', '1', and '2', respectively. Viewing each image as a matrix in $\mathbb{R}^{28\times28}$, the 3-class classification problem now is to learn a decision algorithm $\mathscr{A} : \mathbb{R}^{28\times28} \supseteq \{\text{Images of '0' or '1' or '2'}\} \to \{0, 1, 2\}$, where class label i means the image is classified as digit 'i' for $i \in \{0, 1, 2\}$. The goal is to learn the optimal logistic regression parameter $\hat{\mathbf{W}} \in \mathbb{R}^{28\times28\times3}$ by solving the optimization problem (103). The

meaning of the (i,j,k) entry $\hat{\mathbf{W}}[i,j,k]$ is the positive association of the (i,j) pixel of all 28×28 images in the train set with class label k.

The first subplot in Figure 18 displays $\hat{\mathbf{W}}[:,:,0]$ learned by the gradient descent algorithm in (104) with $\gamma = 1$. One can see that the pixels lying on the overall shape of digit '0' have large (yellow) regression coefficients, meaning that they contribute to the image being classified as '0' with a large probability. On the other hand, one can also see that the pixels lying on the overall shape of digits '1' or '2' have small regression coefficients, meaning that they discount the probability of the image being classified as '0'. The other subplots show $\hat{\mathbf{W}}[:,:,1]$ and $\hat{\mathbf{W}}[:,:,2]$.

4.2. Testing. Here we discuss testing our trained multiclass logistic regression model for the MNIST dataset. Suppose we have target and predicted vectors $y_{\text{test}}, y_{\text{pred}} \in \{1,2,\dots,\kappa\}^N$. We measure the overall accuracy of this prediction by the *confusion matrix*, which is a $\kappa\times\kappa$ matrix whose (i,j) entry means the number of instances of true class i classified as class j. Note that in the ideal situation of perfect prediction $y_{\text{pred}} = y_{\text{test}}$, the confusion matrix will be N times the $\kappa\times\kappa$ identity matrix.

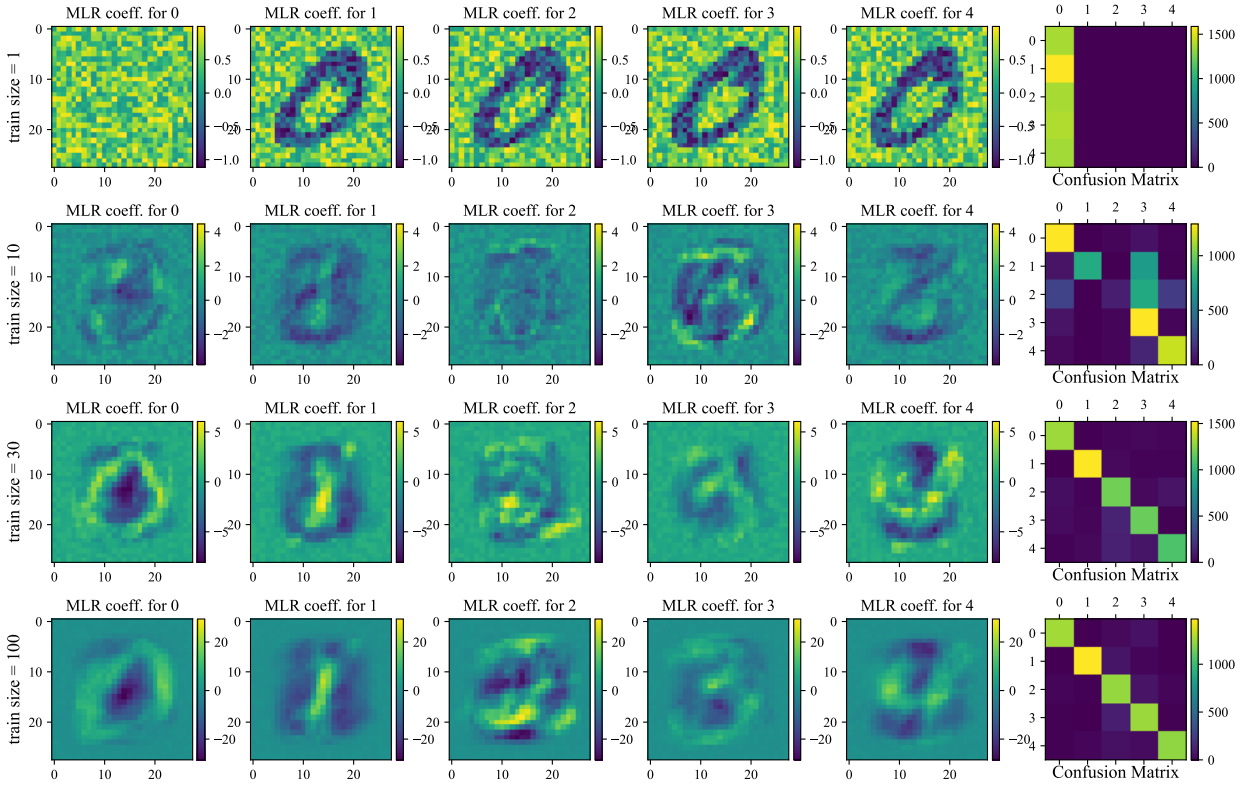


FIGURE 19. Examples of the fitted logistic regression coefficients to the MNIST dataset with digits in $['0','1','2','3','4']$. Gradient descent algorithm is used for training.

Next, recall that MLR is a probabilistic model, where one models the output as a discrete RV Y with PMF $p = [p_1,\dots,p_\kappa]$, $p_i \propto \exp(\boldsymbol{\phi}(\mathbf{x})^T\mathbf{w}_i)$ as in (92). The result of fitting MLR to the training set is the optimal parameter $\hat{\mathbf{W}} = [\hat{\mathbf{w}}_1,\dots,\hat{\mathbf{w}}_\kappa]$ defined in (103), so we can compute the predictive PMF $p = p(\mathbf{y}|\boldsymbol{\phi}(\mathbf{x}))$ for the unknown output. We will use the following *maximum a posteriori* decision algorithm

$$\mathbf{x} \longmapsto \hat{\mathbf{y}} = \mathscr{A}(\mathbf{x}) = \operatorname*{argmax}_{i\in\{1,2,\dots,\kappa\}} p(i\,|\,\boldsymbol{\phi}(\mathbf{x})) \tag{108}$$

$$= \operatorname*{argmax}_{i\in\{1,2,\dots,\kappa\}} \exp(\boldsymbol{\phi}(\mathbf{x})^T\hat{\mathbf{w}}_i) \tag{109}$$

$$= \underset{i\in\{1,2,\ldots,\kappa\}}{\operatorname{argmax}} \boldsymbol{\phi}(\mathbf{x})^T \hat{\mathbf{w}}_i. \tag{110}$$

Simply speaking, we are predicting $\hat{\mathbf{y}}$ to be more likely class label predicted by the predicted PMF.

In Figure 19, we test our MLR classifier trained on randomly subsampled test sets of sizes 1, 10, 30, and 100 for classifying digits between '0', '1', ..., '4'. The test set contains about 20% of examples in the dataset, which is about 35,000 images of the five digits. In Figure 19, we have examples of trained regression coefficients (shown as a heat map on 28×28 image) as well as the confusion matrices. Observe that as we increase the size of the train set, the optimal regression coefficients become more aware of the shapes of the correct digits, and the confusion matrix becomes closer to a diagonal matrix, meaning the the algorithm gets less 'confused'.

5. Probit Regression

5.1. Model Formulation. In this section, we discuss a probabilistic regression (PR) model bor binary classification similar to logistic regression. The setting is identical, but the 'link function' is not the sigmoid function but something called 'probit' (probability + unit) function. Probit regression is widely used in Economics.

As before, we assume that the two classes are 0 and 1. Let $(\boldsymbol{\phi}(\mathbf{x}_1), y_1), \ldots, (\boldsymbol{\phi}(\mathbf{x}_N), y_N)$ be the training examples, where

- $\mathbf{x}_1, \ldots, \mathbf{x}_N$: Input data (e.g., collection of all medical records of each patient)
- $\boldsymbol{\phi}(\mathbf{x}_1), \ldots, \boldsymbol{\phi}(\mathbf{x}_N) \in \mathbb{R}^p$: Features (e.g., some useful (derived) information for each patient)
- $y_1, \ldots, y_n \in \{0, 1\}$: Binary class labels (e.g., '`normal`' or '`pneumonia`').

The basic idea of probit regression is to model the output y as a Bernoulli (0-1) random variable Y that takes value 1 iff the linear activation $\boldsymbol{\phi}(x)^T\mathbf{w}$ is at leaset a threshold θ, where $\mathbf{w} \in \mathbb{R}^p$ is a p-dimensional parameter vector. The point here is to randomize the θ as an independent Gaussian RV. More specifically,

$$Y \mid \boldsymbol{\phi}(\mathbf{x}) \sim \text{Bernoulli}(p), \qquad p = \mathbb{P}\left(\boldsymbol{\phi}(\mathbf{x})^T\mathbf{w} \geq Z\right) =: \Psi(\boldsymbol{\phi}(\mathbf{x})^T\mathbf{w}), \tag{111}$$

where $Z \sim N(0,1)$ is an independent standard Gaussian RV. We can also write the probit model more compactly as

$$Y \mid \boldsymbol{\phi}(\mathbf{x}) \sim \mathbf{1}\left(\boldsymbol{\phi}(\mathbf{x})^T\mathbf{w} \geq Z\right). \tag{112}$$

Here, the function $\Psi : x \mapsto \mathbb{P}(x \geq Z) = \mathbb{P}(Z \leq x)$ is called the *probit* function, which is exactly the CDF of standard Gaussian distribution. Simmilarly as sigmoid function does, the probit function also maps the real line onto the interval $[0,1]$. As in Figure 20, we see the probit link maps real numbers to the extreme probabilities 0 and 1 more aggressively than the sigmoid link.

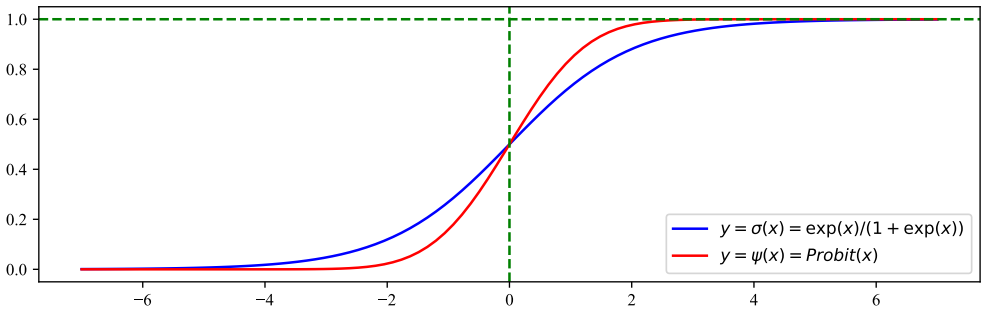


FIGURE 20. Plot of probit and sigmoid function.

Next, let's discuss how to train probit regression model. As before, we will use the method of maximum likelihood. For that, we compute the joint likelihood of observing the output values $(y_1, \ldots, y_N)$

from the tuple of independent Bernoulli variables $(Y_1,\dots,Y_N)$ with success probabilities $p_1 := \Psi(\boldsymbol{\phi}(\mathbf{x}_1)^T\mathbf{w})$, $\dots$, $p_N := \Psi(\boldsymbol{\phi}(\mathbf{x}_N)^T\mathbf{w})$:

$$L(y_1,\dots,y_N;\mathbf{w}) = \mathbb{P}(Y_1 = y_1,\dots,Y_N = y_N;\mathbf{w}) = \prod_{i=1}^N p_i^{y_i}(1-p_i)^{1-y_i}. \tag{113}$$

Hence the optimial parameter $\hat{\mathbf{w}} \in \mathbb{R}^p$ is given by

$$\begin{aligned}
\hat{\mathbf{w}} &= \operatorname*{arg\,max}_{\mathbf{w}\in\mathbb{R}^p} \log L(y_1,\dots,y_N;\mathbf{w}) && (114)\\
&= \operatorname*{arg\,max}_{\mathbf{w}\in\mathbb{R}^p} \sum_{i=1}^N y_i \log p_i + (1-y_i)\log(1-p_i) && (115)\\
&= \operatorname*{arg\,max}_{\mathbf{w}\in\mathbb{R}^p} \left[\sum_{i=1}^N y_i \log \Psi(\boldsymbol{\phi}(\mathbf{x}_i)^T\mathbf{w}) + (1-y_i)\log(1-\Psi(\boldsymbol{\phi}(\mathbf{x}_i)^T\mathbf{w}))\right] && (116)\\
&= \operatorname*{arg\,min}_{\mathbf{w}\in\mathbb{R}^p} \left[\ell_{PR}(\mathbf{w}) := -\sum_{i=1}^N y_i \log \Psi(\boldsymbol{\phi}(\mathbf{x}_i)^T\mathbf{w}) + (1-y_i)\log \Psi(-\boldsymbol{\phi}(\mathbf{x}_i)^T\mathbf{w})\right], && (117)
\end{aligned}$$

where we have use the property $\Psi(-x) = 1-\Psi(x)$ of standard normal CDF. The loss function $\ell_{PR}(\mathbf{w})$ in (114) is called the *probit regression loss*.

In Exercise 2.5.1, we show that the above is a convex optimization problem and also compute the gradient and Hessian. As in the logistic regression case, we can solve Exercise 2.5.1 iteratively either by gradient descent or Newton-Ralphson. For gradient descent algorithm, we use

$$\mathbf{w}_{t+1} \longleftarrow \mathbf{w}_t - \eta_t \nabla_{\mathbf{w}} \ell_{PR}(\mathbf{w}_t) \in \mathbb{R}^p, \tag{118}$$

where we can use the same step sizes $\eta_t = \gamma/((\log t)\sqrt{t})$ for some $\gamma > 0$. The gradient $\nabla_{\mathbf{w}}\ell_{PR}(\mathbf{w}_t)$ of the probit loss is computed in Exercise 2.5.1. The Newton-Ralphson algorithm can also be implemented for the case of probit regression similarly by using the Hessian computed in Exercise 2.5.1.

Exercise 2.5.1 (Convexity of the probit regression loss)**.** Let $\ell_{PR}(\mathbf{w})$ denote the logistic regression loss defined in (114). Let $\psi(x) = (2\pi)^{-1/2}\exp(-x^2/2)$ denote the PDF of standard normal distribution and let $\Psi(x) = \int_{-\infty}^x \psi(t)\,dt$ denote the corresponding CDF. Let $\boldsymbol{\Phi} \in \mathbb{R}^{p\times N}$ is the design matrix defined in (82).

(i) Show that

$$\begin{aligned}
\nabla_{\mathbf{w}}\ell_{PR}(\mathbf{w}) &= \sum_{i=1}^N \psi(\boldsymbol{\phi}(\mathbf{x}_i)^T\mathbf{w})\left[\frac{1-y_i}{\Psi(-\boldsymbol{\phi}(\mathbf{x}_i)^T\mathbf{w})} - \frac{y_i}{\Psi(\boldsymbol{\phi}(\mathbf{x}_i)^T\mathbf{w})}\right]\boldsymbol{\phi}(\mathbf{x}_i) && (119)\\
&= \boldsymbol{\Phi}(\mathbf{Q}_1 - \mathbf{Q}_2), && (120)
\end{aligned}$$

where

$$\mathbf{Q}_1 = \left[\frac{(1-y_1)\psi(\boldsymbol{\phi}(\mathbf{x}_1)^T\mathbf{w})}{\Psi(-\boldsymbol{\phi}(\mathbf{x}_1)^T\mathbf{w})} \quad \cdots \quad \frac{(1-y_N)\psi(\boldsymbol{\phi}(\mathbf{x}_N)^T\mathbf{w})}{\Psi(-\boldsymbol{\phi}(\mathbf{x}_N)^T\mathbf{w})}\right]^T, \quad \mathbf{Q}_2 = \left[\frac{y_1\psi(\boldsymbol{\phi}(\mathbf{x}_1)^T\mathbf{w})}{\Psi(\boldsymbol{\phi}(\mathbf{x}_1)^T\mathbf{w})} \quad \cdots \quad \frac{y_N\psi(\boldsymbol{\phi}(\mathbf{x}_N)^T\mathbf{w})}{\Psi(\boldsymbol{\phi}(\mathbf{x}_N)^T\mathbf{w})}\right]^T \in \mathbb{R}^N. \tag{121}$$

(ii) Show the following properties of PDF and CDF of standard normal distribution:

$$\begin{aligned}
&\psi(x) = \psi(-x), \qquad \Psi(-x) = 1-\Psi(x), \qquad \psi'(x) = -x\psi(x), && (122)\\
&\frac{\partial}{\partial x}\frac{\psi(x)}{\Psi(x)} = \frac{\psi(x)\left[-x\Psi(x)-\psi(x)\right]}{\Psi(x)^2}, \qquad \frac{\partial}{\partial x}\frac{\psi(x)}{\Psi(-x)} = \frac{\psi(x)\left[-x\Psi(-x)+\psi(x)\right]}{\Psi(-x)^2}. && (123)
\end{aligned}$$

(iii) Show that

$$\frac{\partial(\mathbf{Q}_1^T - \mathbf{Q}_2^T)}{\partial \mathbf{w}} = \begin{bmatrix} \uparrow & & \uparrow \\ G_1\boldsymbol{\phi}(\mathbf{x}_1) & \cdots & G_N\boldsymbol{\phi}(\mathbf{x}_N) \\ \downarrow & & \downarrow \end{bmatrix} = \boldsymbol{\Phi}\mathbf{D}, \tag{124}$$

where for $z_i := \boldsymbol{\phi}(\mathbf{x}_i)^T\mathbf{w}$, we denote

$$G_i := (1-y_i)\frac{\psi(z_i)\left[-z_i\Psi(-z_i)+\psi(z_i)\right]}{\Psi(-z_i)^2} + y_i\frac{\psi(z_i)\left[z_i\Psi(z_i)+\psi(z_i)\right]}{\Psi(z_i)^2}. \tag{125}$$

and $\mathbf{D}$ is the $N\times N$ diagonal matrix with diagonal entries $\mathbf{D}_{ii} = G_i$.

(iv) Show that

$$\mathbf{H} := \frac{\partial^2 \ell_{PR}(\mathbf{w})}{\partial\mathbf{w}\partial\mathbf{w}^T} = \nabla_{\mathbf{w}}(\mathbf{Q}_1^T - \mathbf{Q}_2^T)\boldsymbol{\Phi}^T = \boldsymbol{\Phi}\mathbf{D}\boldsymbol{\Phi}^T \in \mathbb{R}^{p\times p}. \tag{126}$$

(v) (Optional*) Argue that the Hessian $\mathbf{H}$ is positive semi-definite. Conclude that the probit regression loss ℓ_{PR} is convex. (Hint: $\mathbf{D}$ is a diagonal matrix with positive diagonal entries. This was clear for logistic regression. Here, one can show the funtions $-z\Psi(-z)+\psi(z)$ and $-z\Psi(z)+\psi(z)$ always assume positive values by some calculus argument. See [ZL12, Lem. 1])

6. Binary classification of MNIST by Probit Regression

We revisit the MNIST data with probit regression. In this subsection, we will apply probit regression to classify the MNIST images of chosen two digits. The first subplot in Figure 21 displays $\hat{\mathbf{w}}$ for digits '0′ and '1′, learned by the gradient descent algorithm in (118) with $\gamma = 1$. A similar discussion for LR we made below Figure 23 holds as well in the PR case.

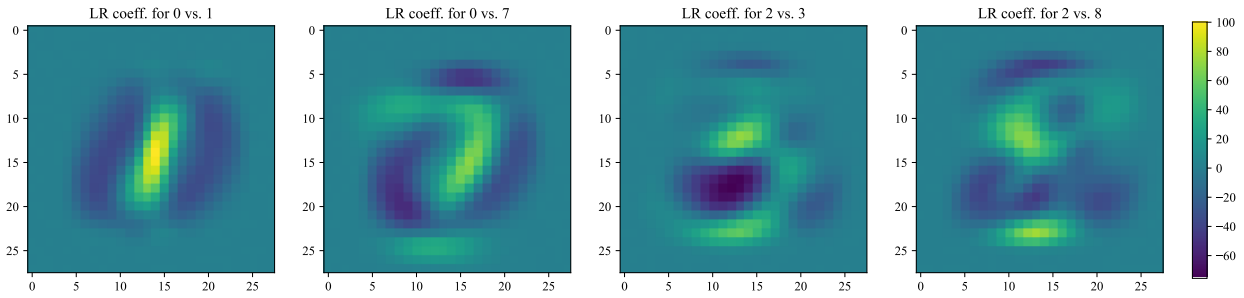


FIGURE 21. Examples of the fitted probit regression coefficients to the MNIST dataset with chosen two digits. Gradient descent algorithm is used for training.

Next, we discuss testing probit regression for binarty classification of MNIST dataset. Recall that probit regression is a probabilistic model, where one models the output as a Bernoulli RV Y with success probability $p = \Psi(\boldsymbol{\phi}(\mathbf{x})^T\mathbf{w})$ as in (111). The result of fitting PR to the training set is the optimal parameter $\hat{\mathbf{w}}$ defined in (114), so we can compute the predictive probability $p = p(\mathbf{y}|\boldsymbol{\phi}(\mathbf{x}))$ for the unknown output. For the current example, we use the following decision algorithm

$$\mathbf{x} \longmapsto \hat{\mathbf{y}} = \mathscr{A}(\mathbf{x}) = \begin{cases} 1 & \text{if } p = \Psi(\boldsymbol{\phi}(\mathbf{x})^T\hat{\mathbf{w}}) \geq 0.5 \\ 0 & \text{if } p = \Psi(\boldsymbol{\phi}(\mathbf{x})^T\hat{\mathbf{w}}) < 0.5. \end{cases} \tag{127}$$

Namely, we are predicting $\hat{\mathbf{y}}$ to be more likely class label predicted by the predicted probability, with the classification threshold being 0.5.

In Figure 16, we show experiments on teseting PR on the MNIST dataset similar to LR in Figure 16. As berfore, we see that as we increase the size of the train set, the optimal regression coefficients become more aware of the shapes of '7' as positive (yellow) and '4' as negative (blue), and the classification metrics improve in general.

Below, we investigate the gradient descent algorithm (118) in a bit more detail as we did for LR in Exercise 2.6.1. We also compare the performance of LR and PR on MNIST dataset with varying degree of noise in the data.

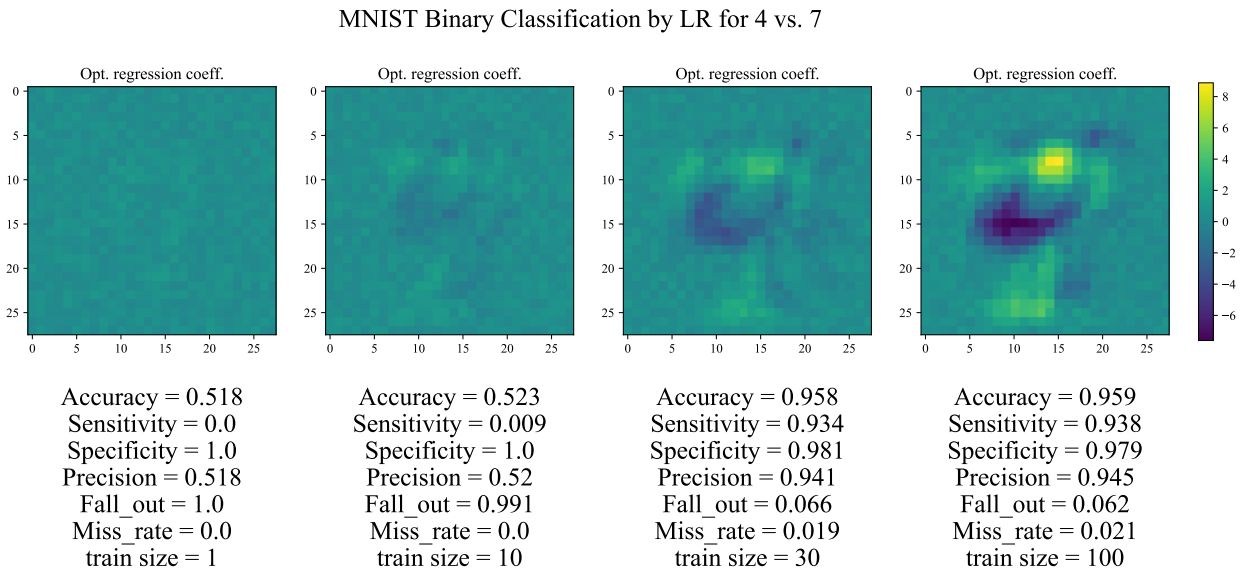


FIGURE 22. Examples of the training and testing probit regression coefficients to the MNIST dataset with chosen two digits. Gradient descent algorithm is used for training. Training datasets of size in $\{1, 10, 30, 100\}$ are subsampled and used for fitting the model. Various statistics for binary classification are also shown.

Exercise 2.6.1. The gradient descent (GD) algorithm for probit regression training in (85) is implemented as the python function 'fit_PR_GD' in the Jupyter notebook provided in the course repository.

```
def fit_PR_GD(Y, H, W0=None, sub_iter=100, stopping_diff=0.01):
        '''
        Convex optimization algorithm for Probit Regression using Gradient Descent
        Y = (n x 1), H = (p x n) (\Phi in lecture note), W = (p x 1)
        Logistic Regression: Y ~ Bernoulli(Q), Q = Probit(H.T @ W)
        '''
        if W0 is None:
            W0 = np.random.rand(H.shape[0],1) #If initial coefficients W0 is None, randomly initialize

        W1 = W0.copy()
        i = 0
        grad = np.ones(W0.shape)
        while (i < sub_iter) and (np.linalg.norm(grad) > stopping_diff):
            Q = norm.pdf(H.T @ W1) * ( (1-Y)/norm.cdf(-H.T @ W1) - Y/norm.cdf(H.T @ W1) )
            grad = H @ Q
            W1 = W1 - (np.log(i+1) / (((i + 1) ** (0.5)))) * grad
            i = i + 1
            # print('iter %i, grad_norm %f' %(i, np.linalg.norm(grad)))
        return W1
```

FIGURE 23. A python implementation of the gradient descent algorithm for logistic regression training (118).

(i) Modify the code (or make your own function) so that each gradient descent step (118), the current value of objective function ℓ_{PR} in (114) is printed. Test it on the MNIST example with digits $['0','1']$. Does the loss monotonically decrease?

(ii) The implemented version of GD for (118) uses the step size $\eta_t = \gamma/(\log(t+1)\sqrt{(t+1)})$ with $\gamma = 1$. Test the same step sizes for $\gamma = 10$ and also $\eta_t = 1$ and $\eta_t = 1/(t+1)$. Test it on the MNIST example with digits $['0','1']$. Is there any difference in the way the loss function decrease?

(iii) If we use the step size of the form $\eta_t = \gamma t^{-\delta}$ for $0 \le \delta \le 1$, what would be your optimal choice of the hyperparameters γ and δ? (There is no answer and trying out a few choices and make your observation.)

Exercise 2.6.2. In this exercise, we investigate the 'robustness' of the logistic and the probit regression for binary classification.

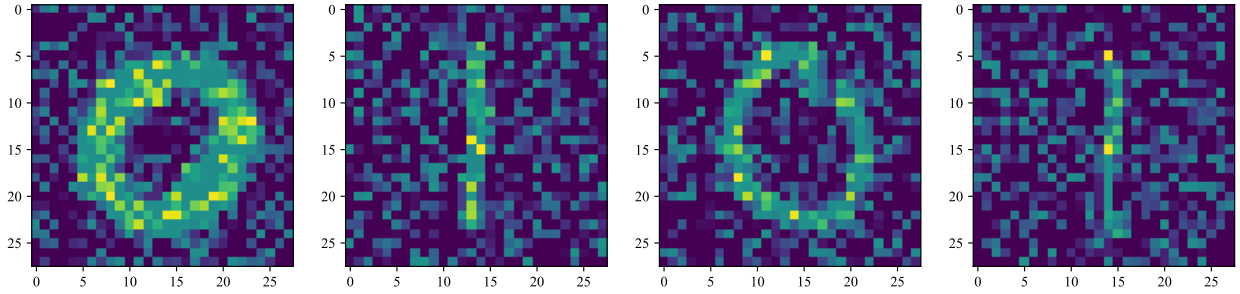


FIGURE 24. Corrupted MNIST images where Uniform([0,1]) is added to 50% of randomly chosen pixels.

(i) Modify the data preprocessing function "sample_binary_MNIST" so that it has the option to add noise with given ratio. Namely, if 'noise_ratio' is 0.5, then it will add independent Uniform([0,1]) variables to 50% of randomly chosen pixels. (See Figure 24)

(ii) For logistic regression, re-generate figures similar to Figure 16 with varying noise ratios in $\{0.1, 0.5, 0.9\}$.

(iii) For probit regression, re-generate figures similar to Figure 22 with varying noise ratios in $\{0.1, 0.5, 0.9\}$.

(iv) From the previous experiments, compare the robustness of LR and PR against random noise you gave. Can you conclude that one is significantly more robust than the other?

7. Naive Bayes Classifier

In this section, we discuss a class of classification algorithms called *Naive Bayes classifier*. Recall that in logistic (or probit) regression, we modeled the output class Y given the feature vector $\boldsymbol{\phi}(\mathbf{x})$ directly as a RV. In other words, we modeled the *predictive probability* $p(\text{class}\,|\,\text{features})$. However, Naive Bayes approach, we instead model the likelihood $p(\text{features}\,|\,\text{class})$, which is also called the *class-conditional probability* in the current situation. This change of perspective is possible by using Bayes' Theorem, and the 'Naive Bayes' assumption is used to simplify the computation of class-conditional probabilities.

7.1. The model formulation. We discuss the basic formulation of Naive Bayes Classifier in the setting of multiclass classification. We assume that there are κ class labels in $\{1,2,\dots,\kappa\}$. Say we have training examples $(\boldsymbol{\phi}(\mathbf{x}_1), y_1), \dots, (\boldsymbol{\phi}(\mathbf{x}_N), y_N)$, where

- $\mathbf{x}_1, \dots, \mathbf{x}_N$: Input data (e.g., collection of all medical records of each patient)
- $\boldsymbol{\phi}(\mathbf{x}_1), \dots, \boldsymbol{\phi}(\mathbf{x}_N) \in \mathbb{R}^p$: Features (e.g., some useful (derived) information for each patient)
- $y_1, \dots, y_n \in \{1,2,\dots,\kappa\}$: κ class labels (e.g., digits from 0 to 9).

As in the multiclass logistic regression, we model the output y as a discrete random variable Y with probability mass function (PMF) $[p_1, \dots, p_\kappa]$ that depends on the observed feature $\boldsymbol{\phi}(\mathbf{x})$:

$$\mathbb{P}\left(Y = i \,|\, \boldsymbol{\phi}(\mathbf{X})\right) = p_i \qquad \text{for } i = 1, \dots, \kappa. \tag{128}$$

Instead of modeling the predictive PMF p directly, we use Bayes' Theorem to rewrite the above as

$$\overbrace{\mathbb{P}\left(Y = i \,|\, \boldsymbol{\phi}(\mathbf{X}) = [\phi_1, \dots, \phi_m]\right)}^{\text{predictive prob.}} \propto \overbrace{\mathbb{P}\left(\boldsymbol{\phi}(\mathbf{X}) = [\phi_1, \dots, \phi_p] \,|\, Y = i\right)}^{\text{class-conditional prob.}} \overbrace{\mathbb{P}(Y = i)}^{\text{prior}}. \tag{129}$$

Now the main focus is to model the class-conditional probability. Namely, what is the probability that, given the class label Y is i, the observed feature vector $\phi(\mathbf{X}) = [\phi_1(\mathbf{X}), \dots, \phi_p(\mathbf{X})]$ equals $[\phi_1, \dots, \phi_p]$?

In order to properly model the p-dimensional random feature vector $\boldsymbol{\phi}(\mathbf{X}) \in \mathbb{R}^p$, we need to specify its p-dimensional class-conditional joint distribution. When the features $\phi_1, \dots, \phi_p$ assume discrete values,

then the joint distribution can be represented as a $p \times p$ contingency table, which has $(p-1)^2$ degrees of freedom to be specified. When p is large, such as quadratic dependence of model complexity is not desirable. In order to simplify the situation, we make the following 'naive Bayes assumption':

Naive Bayes Assumption: Given the class label $Y = i$, the features $\boldsymbol{\phi}(\mathbf{X}) = [\phi_1(\mathbf{X}), \ldots, \phi_p(\mathbf{X})]$ are *independent*.

Clearly, such an assumption is not realistic and hence 'naive'. For instance, the events that a patient has weight > 75kg and age > 65 are not independent conditionally on having cancer. However, the naive Bayes assumption simplifies our model to linear complexity:

$$\mathbb{P}\left([\phi_1(\mathbf{X}), \ldots, \phi_p(\mathbf{X})] = [\phi_1, \ldots, \phi_p] \,\middle|\, Y = i\right) = \prod_{j=1}^{p} \mathbb{P}\left(\phi_j(\mathbf{X}) = \phi_j \mid Y = i\right). \tag{130}$$

Hence, we only need to model the marginal class-conditional probabilities $\mathbb{P}\left(\phi_j(\mathbf{X}) = \phi_j \mid Y = i\right)$ individually and then multiply all to get the joint class-conditional probability.

Suppose we use parameter $\mathbf{w}_{ij}$ for the marginal class-conditional probability $\mathbb{P}\left(\phi_j(\mathbf{X}) = \phi_j(\mathbf{x}_i) \mid Y = y_i\right)$ of jth feature given ith class. Then the joint likelihood of observing the class labels $y_1, \ldots, y_N$ given the observed features $\boldsymbol{\phi}(\mathbf{x}_1), \ldots, \boldsymbol{\phi}(\mathbf{x}_N)$ can be written as

$$L(y_1, \ldots, y_N; \mathbf{W}) = \prod_{s=1}^{N} \overbrace{\mathbb{P}\left(Y_s = y_s \mid \boldsymbol{\phi}(\mathbf{X}) = \boldsymbol{\phi}(\mathbf{x}_s)\right)}^{\text{predictive prob.}} \tag{131}$$

$$\propto \prod_{s=1}^{N} \overbrace{\mathbb{P}\left(\boldsymbol{\phi}(\mathbf{X}) = \boldsymbol{\phi}(\mathbf{x}_s) \mid Y_s = y_s\right)}^{\text{class-conditional prob.}} \overbrace{\mathbb{P}(Y_s = y_s)}^{\text{prior}} \tag{132}$$

$$= \prod_{s=1}^{N} \prod_{i=1}^{\kappa} \left(\prod_{j=1}^{p} \mathbb{P}\left(\phi_j(\mathbf{X}) = \phi_j(\mathbf{x}_s) \,\middle|\, Y_s = i\right) \mathbb{P}(Y_s = i) \right)^{\mathbf{1}(y_s = i)}. \tag{133}$$

Hence the optimal paramter $\hat{\mathbf{W}} = (\hat{\mathbf{w}}_{ij})$ can be found by the standard maximum likelihood method:[6]

$$\hat{\mathbf{W}} = \underset{\mathbf{W}}{\arg\max} \left[\sum_{s=1}^{N} \sum_{i=1}^{\kappa} p\mathbf{1}(y_s = i) \log \mathbb{P}(Y_s = i) + \sum_{s=1}^{N} \sum_{j=1}^{p} \sum_{i=1}^{\kappa} \mathbf{1}(y_s = i) \log \underbrace{\mathbb{P}\left(\phi_j(\mathbf{X}) = \phi_j(\mathbf{x}_s) \,\middle|\, Y_s = i\right)}_{\text{parameterized by } \mathbf{w}_{i,j}} \right]. \tag{134}$$

In the following sections, we will solve the above optimization problem for some well-known cases of naive Bayes classifiers.

Lastly, given a new input $\mathbf{x}$ and the the optimal paramter $\hat{\mathbf{W}}$ for naive Bayes classifier found by (134) using the training set, we can make the prediction of the class label using the computed predictive probabilities. As before for the multiclass logistic regression (108), we can use the maximum a posteriori (MAP) decision rule:

$$\mathbf{x} \longmapsto \mathscr{A}(\mathbf{x}) = \underset{i \in \{1,2,\ldots,\kappa\}}{\arg\max} \overbrace{\mathbb{P}\left(Y = i \mid \boldsymbol{\phi}(\mathbf{X}) = \boldsymbol{\phi}(\mathbf{x}), \hat{\mathbf{W}}\right)}^{\text{predictive prob.}} \tag{135}$$

$$= \underset{i \in \{1,2,\ldots,\kappa\}}{\arg\max} \left[\sum_{i=1}^{N} \log \mathbb{P}(Y = i) + \sum_{i=1}^{N} \sum_{j=1}^{p} \log \underbrace{\mathbb{P}\left(\phi_j(\mathbf{X}) = \phi_j(\mathbf{x}_i) \mid Y = y_i\right)}_{\text{parameterized by } \hat{\mathbf{w}}_{y_i,j}} \right]. \tag{136}$$

[6] Here we treat prior as is given, but we also regard it as a parameter to be optimized.

7.2. Multinomial Naive Bayes. In this subsection, we discuss the case naive Bayes classifier with discrete features in the context of document classification. We will make the following version of naive Bayes assumption:

Naive Bayes Model for Document Classification:

- A set of 'vocabulary' $\mathcal{V} = \{a_1, \dots, a_M\}$ is given. (e.g., $\mathcal{V} = \{\text{Dear, Sir, Money, Friend, Thanks}\}$)
- Class lables $\mathscr{Y} = \{1, 2, \dots, \kappa\}$ (e.g., 0 ='normal', 1 ='spam')
- For each data point (document) $\mathbf{x}$, assume its feature $\boldsymbol{\phi}(\mathbf{x})$ is given by a frequency vector (histogram) $\boldsymbol{\phi}(\mathbf{x}) = [c_1, \dots, c_M]$, where c_i means the count of word a_i in $\mathbf{x}$.

$$\text{(e.g., } \mathbf{x} = \text{"Dear Sir, Money Money Money. Thanks."}, \boldsymbol{\phi}(\mathbf{x}) = [1,1,3,0,1]) \tag{137}$$

- Given the class label $Y = i$, each word in document $\mathbf{x}$ is sampled independently from the set $\mathcal{V}$ of vocabulary with probability q_{ij}.
- Use q_i for the prior probability of a random document being of class i.

$$\text{(e.g., } \quad \mathbb{P}\big(\boldsymbol{\phi}(\mathbf{X}) = [1,1,3,0,1] \,|\, Y = \text{spam}\big) \propto q_{11} \cdot q_{12} \cdot q_{13}^3 \cdot q_{14}^0 \cdot q_{15}). \tag{138}$$

- Let $\mathbf{W} = (q_i, q_{ij})$ denote the parameter vector.

Suppose we have training examples $(\boldsymbol{\phi}(\mathbf{x}_1), y_1), \dots, (\boldsymbol{\phi}(\mathbf{x}_N), y_N)$. For each $s \in \{1, \dots, N\}$ (data index), $i \in \{1, \dots, \kappa\}$ (class label), and $j \in \{1, \dots, M\}$ (word index), using the naive Bayes assumption above, we can write

$$\mathbb{P}\Big(\phi_j(\mathbf{X}) = \phi_j(\mathbf{x}_s) \,\Big|\, Y = i\Big) = \mathbb{P}(\text{seeing the } j\text{th word } a_j \in \mathcal{V} \ \phi(\mathbf{x}_s) \text{ times in } \mathbf{x}_s \,|\, Y = i) \tag{139}$$

$$\propto (q_{i,j})^{\phi_j(\mathbf{x}_s)}. \tag{140}$$

Hence we have

$$\hat{\mathbf{W}} = \underset{\mathbf{W}=(q_i, q_{ij})}{\operatorname{argmax}} \left[\sum_{s=1}^{N} \sum_{i=1}^{\kappa} M \mathbf{1}(y_s = i) \log q_i + \sum_{s=1}^{N} \sum_{j=1}^{M} \sum_{i=1}^{\kappa} \mathbf{1}(y_s = i) \phi_j(\mathbf{x}_s) \log(q_{i,j}) \right] \tag{141}$$

$$= \underset{\mathbf{W}=(q_i, q_{ij})}{\operatorname{argmax}} \left[M \sum_{i=1}^{\kappa} \underbrace{\left(\sum_{s=1}^{N} \mathbf{1}(y_s = i) \right)}_{c_i := \#\text{ of class } i \text{ examples}} \log q_i + \sum_{i=1}^{\kappa} \sum_{j=1}^{M} \underbrace{\left(\sum_{s=1}^{N} \mathbf{1}(y_s = i) \phi_j(\mathbf{x}_s) \right)}_{c_{i,j} := \#\text{ of } j\text{th word in class } i \text{ examples}} \log(q_{i,j}) \right], \tag{142}$$

where the parameters $\mathbf{W} = (q_i, q_{i,j})$ are subject to the constraint

$$0 \le q_1, \dots, q_\kappa \le 1, \qquad q_1 + \cdots + q_\kappa = 1 \tag{143}$$

$$0 \le q_{i,1}, \dots, q_{i,M} \le 1, \qquad q_{i,1} + \cdots + q_{i,M} = 1 \qquad \text{for } i = 0, 1, \dots, \kappa. \tag{144}$$

In the following proposition, we show that the optimal parameters that solve the above maximum likelihood formulation are given by simple formulas involding empirical frequencies of each class and of each words given each class.

Proposition 2.7.1. *The solution $\hat{\mathbf{W}} = (\hat{q}_i, \hat{q}_{i,j})$ to the optimization problem (141) is given by*

$$\hat{q}_i = \frac{1}{N} \sum_{s=1}^{N} \mathbf{1}(y_s = i), \qquad \hat{q}_{i,j} \propto \sum_{s=1}^{N} \mathbf{1}(y_s = i) \phi_j(\mathbf{x}_s) \quad \textit{for } j \in \{1, \dots, M\}. \tag{145}$$

PROOF. First, note that the second term in the objective function in (141) does not depend on the parameters $q_1, \dots, q_\kappa$. Hence $[\hat{q}_1, \dots, \hat{q}_\kappa]$ is a PMF on $\{1, \dots, \kappa\}$ that maximizes $\sum_{i=1}^{\kappa} c_i \log q_i$, where $c_i := \sum_{s=1}^{N} \mathbf{1}(y_s = i) \ge 0$. Hence the formula for $\hat{q}_i$'s follow from Exercise 2.7.2.

Next, fix a class label $i \in \{1, \dots, \kappa\}$. Observe that the only terms in the objective function in (141) that depends on $q_{i,1}, \dots, q_{i,M}$ is $\sum_{j=1}^{M} \big(\sum_{s=1}^{N} \mathbf{1}(y_s = i) \phi_j(\mathbf{x}_s) \big) \log(q_{i,j})$. Since $[q_{i,1}, \dots, q_{i,M}]$ is a PMF on $\{a_1, \dots, a_M\}$, again the formula for $\hat{q}_{i,j}$'s for $j = 1, \dots, M$ follow from Exercise 2.7.2. $\square$

Exercise 2.7.2. In this exercise, we solve a simple optimization problem related to the multinomial naive Bayes classifier. Fix a finite set $\mathscr{X} = \{x_1, \ldots, x_M\}$ and real numbers $c_1, \cdots, c_M \geq 0$. Consider the following optimization problem

$$\hat{\mathbf{w}} = \underset{\mathbf{w}=[w_1,\ldots,w_M]^T \in \mathbb{R}^M}{\arg\max} \left(L(\mathbf{w}) := \sum_{i=1}^{M} c_i \log w_i \right) \quad \text{subject to: } \mathbf{w} \text{ is a PMF on } \mathscr{X}. \tag{146}$$

We will show that the solution is given by

$$\hat{\mathbf{w}} = \left[\frac{c_1}{\sum_{i=1}^M c_i}, \ldots, \frac{c_M}{\sum_{i=1}^M c_i} \right]^T. \tag{147}$$

(i)) The constraint set of (146) is the 'simplex' $\mathscr{C} = \{\mathbf{w} \in \mathbb{R}^M \,|\, 0 \leq w_1, \cdots, w_M \leq 1, \sum_{i=1}^M w_i = 1\}$. Denote the larger constraint set $\overline{\mathscr{C}} = \{\mathbf{w} \in \mathbb{R}^M \,|\, \sum_{i=1}^M w_i = 1\}$. Let $\lambda \in \mathbb{R}$ be a Lagrange multiplier for $\overline{\mathscr{C}}$. Then the Lagrangian is

$$g(\lambda, \mathbf{w}) = L(\mathbf{w}) - \lambda \left(\sum_{i=1}^{M} w_i - 1 \right). \tag{148}$$

Show that λ needs to satisfy

$$\frac{\partial g(\lambda, \mathbf{w})}{\partial \mathbf{w}} = \left[\frac{c_1}{w_1}, \ldots, \frac{c_M}{w_M} \right]^T - \lambda \, [1, \ldots, 1]^T = 0. \tag{149}$$

Conclude that (147) is the global maximum $L(\mathbf{w})$ over $\overline{\mathscr{C}}$.

(ii) From **(i)**, conclude that (147) is the global maximum $L(\mathbf{w})$ over $\mathscr{C}$.

Example 2.7.3 (Email Classification)**.** Suppose we have training examples $(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_{100}, y_{100})$ of normal and spam emails over the set of vocabulary $\mathscr{V} = \{\text{Dear}, \text{Sir}, \text{Money}, \text{Friend}, \text{Thanks}\}$. The class labels are from $\{0, 1\}$, where 0 is for normal and 1 is for spam emails. Let $c_{i,j}$ denote the total number of jth word in $\mathscr{V}$ that appear in all emails in the training set of class i. Suppose the training set gives the following statistics:

$$c_0 := \sum_{s=1}^{100} \mathbf{1}(y_s = \text{'0'}) = 70, \quad c_1 := \sum_{s=1}^{100} \mathbf{1}(y_s = \text{'1'}) = 30, \tag{150}$$

$$[c_{0,1}, c_{0,2}, c_{0,3}, c_{0,4}, c_{0,5}] = [70, 20, 15, 40, 90], \tag{151}$$

$$[c_{1,1}, c_{1,2}, c_{1,3}, c_{1,4}, c_{1,5}] = [73, 65, 98, 20, 120]. \tag{152}$$

Then according to Proposition 2.7.1, the optimal prior and class-conditional probabilities for naive Bayes classifier are given by

$$\text{prior PMF} = [\hat{q}_0, \hat{q}_1] = [0.7, 0.3], \tag{153}$$

$$\text{class-conditional PMF for normal emails} = [\hat{q}_{0,1}, \ldots, \hat{q}_{0,5}] = \frac{1}{235} [70, 20, 15, 40, 90], \tag{154}$$

$$\text{class-conditional PMF for spam emails} = [\hat{q}_{1,1}, \ldots, \hat{q}_{1,5}] = \frac{1}{376} [73, 65, 98, 20, 120]. \tag{155}$$

Now suppose we have a new email $\mathbf{x}$ = "Dear Sir, Money Money Money. Thanks.", so its bag-of-word feature vector is $\boldsymbol{\phi}(\mathbf{x}) = [1, 1, 3, 0, 1]$. In order to classify this email, we compute the predictive probabilities using the trained naive Bayes classifier as follow:

$$\mathbb{P}(Y = 0 \,|\, \boldsymbol{\phi}(\mathbf{X}) = [1, 1, 3, 0, 1]) \propto \frac{70 \cdot 20 \cdot 15^3 \cdot 40^0 \cdot 90}{235^6} \cdot (0.7)] = 1.7674 \times 10^{-6}, \tag{156}$$

$$\mathbb{P}(Y = 1 \,|\, \boldsymbol{\phi}(\mathbf{X}) = [1, 1, 3, 0, 1]) \propto \frac{73 \cdot 65 \cdot 98^3 \cdot 20^0 \cdot 120}{376^6} \cdot (0.3) = 5.6897 \times 10^{-5}. \tag{157}$$

It follows that

$$\mathbb{P}(Y=0\,|\,\boldsymbol{\phi}(\mathbf{X})=[1,1,3,0,1])=0.0301, \qquad \mathbb{P}(Y=1\,|\,\boldsymbol{\phi}(\mathbf{X})=[1,1,3,0,1])=0.9698. \tag{158}$$

According to the MAP decision rule in (135), we classify $\mathbf{x}$ as a spam email. ▲

Exercise 2.7.4. Consider the following training data: The goal is to predict whether new subjects with

age	income	student	credit_rating	buys_computer
≤ 30	high	no	fair	no
≤ 30	high	no	excellent	no
$[31,40]$	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	no	fair	yes
>40	low	yes	excellent	no
$[31,40]$	low	yes	excellent	yes
≤ 30	medium	yes	fair	yes
≤ 30	low	no	fair	yes
>40	medium	yes	fair	yes
≤ 30	medium	yes	excellent	yes
$[31,40]$	medium	yes	excellent	yes
$[31,40]$	high	no	fair	yes
>40	medium	no	excellent	no

given features (age, income, student, credit rating) would buy a computer, using naive Bayes classifier.

(i) Compute the maximum likelihood prior distribution on the label $\{0,1\}$, where $0=$'does not buy computer' and $1=$'buys computer'.

(ii) We will model the feature space as the following 10-dimensional product space of binary values $\{0,1\}$:[7]

$$\boldsymbol{\phi}(\mathbf{x})=\big[\mathbf{1}(\text{age}\leq 30),\,\mathbf{1}(\text{age}\in[31,40]),\,\mathbf{1}(\text{age}>40),\ldots,\mathbf{1}(\text{credit}=\text{ecellent})\big]\in\{0,1\}^{10}. \tag{159}$$

Compute the maximum likelihood class-conditional probabilities of each feature.

(iii) Suppose we have the following testing examples:

$$\mathbf{x}_1=\text{“ age}\leq 30\text{, medium income, student, fair credit rating”} \tag{160}$$

$$\mathbf{x}_2=\text{“ age}\in[31,40]\text{, low income, not student, fair credit rating”} \tag{161}$$

$$\mathbf{x}_3=\text{“ age}>40\text{, high income, not student, excellent credit rating”} \tag{162}$$

Compute the predictive probabilities (posterior distribution of class labels) of each testing examples. Make the prediction. For each example, can you tell which factor affected the most for the classification result?

(iv) Write a python script (in Jupyter notebook) that implements your computation automatically. Test your code on a similar but larger (≥ 100 training examples and ≥ 20 testing examples) dataset of your choice.

Exercise 2.7.5 (Multinomial Naive Bayes for nonnegative features)**.** In this exercise, we extend the multinomial naive Bayes classifier from discrete features to continuous nonnegative features.

1. Consider a κ-class classification problem for class labels $\mathcal{Y}=\{1,\ldots,\kappa\}$ and the feature vector $\boldsymbol{\phi}(\mathbf{x})\in\mathbb{R}^M$ of each data point $\mathbf{x}$ assume *nonnegative real* values in each coordinate.

[7] The resulting naive Bayes classifier is a special instance of the multinomial naive Bayes called *Bernoulli naive Bayes.*

2. We assume the following probabilitic model for (data, class) = $(\mathbf{X}, Y)$:

$$\text{(prior)} \quad \mathbb{P}(Y=i) =: q_i \qquad \text{for } i \in \{1,\dots,\kappa\} \tag{163}$$

$$\text{(class-conditional PMF)} \quad \mathbb{P}(\boldsymbol{\phi}(\mathbf{X}) = [c_1,\dots,c_M] \,|\, Y=i) \propto \prod_{j=1}^{M} (q_{ij})^{c_j} \qquad \text{for } i \in \{1,\dots,\kappa\} \tag{164}$$

$$\text{(constraints)} \quad \begin{cases} 0 \le q_1,\dots,q_\kappa \le 1, \qquad q_1 + \cdots + q_\kappa = 1 \\ 0 \le q_{i,1},\dots,q_{i,M} \le 1, \qquad q_{i,1} + \cdots + q_{i,M} = 1 \qquad \text{for } i = 0,1,\dots,\kappa. \end{cases} \tag{165}$$

We denote $\mathbf{W} = (q_i, q_{i,j})$ for the vector of parameters of our model above.

Now show the following:

(i) Suppose training examples are given: $(\boldsymbol{\phi}(\mathbf{x}_s), y_s)$, $s = 1,\dots,N$, and write $\boldsymbol{\phi}(\mathbf{x}_s) = [\phi_1(\mathbf{x}_s),\dots,\phi_N(\mathbf{x}_s)]$ for each s. Show that the maximum likelihood estimate of $\mathbf{W}$, $\hat{\mathbf{W}}$, is given by

$$\hat{\mathbf{W}} = \underset{\mathbf{W}=(q_i,q_{ij})}{\operatorname{argmax}} \left[M \sum_{i=1}^{\kappa} \left(\sum_{s=1}^{N} \mathbf{1}(y_s = i) \right) \log q_i + \sum_{i=1}^{\kappa} \sum_{j=1}^{M} \left(\sum_{s=1}^{N} \mathbf{1}(y_s = i)\phi_j(\mathbf{x}_s) \right) \log(q_{i,j}) \right]. \tag{166}$$

(ii) Show that the solution to (166) is given by

$$\hat{q}_i = \frac{1}{N} \sum_{s=1}^{N} \mathbf{1}(y_s = i), \qquad \hat{q}_{i,j} \propto \sum_{s=1}^{N} \mathbf{1}(y_s = i)\phi_j(\mathbf{x}_s) \quad \text{for } j \in \{1,\dots,M\}. \tag{167}$$

(Hint: Notice that the conclusion of Exercise 2.7.2 holds for nonnegative constants $c_1,\dots,c_N \ge 0$, not necessarily only for integers. Then argue similarly as in the proof of Proposition 2.7.1.)

(iii) Does the same conclusion hold if we allow negative feature values?

8. Classification with the 20Newsgroups dataset

All experiements in this section can be found in the Jupyter notebook Math156_20NewsGroups.ipynb.

8.1. Preliminaries. The 20 Newsgroups data set is a collection of approximately 18,000 newsgroup documents, partitioned (nearly) evenly across 20 different newsgroups. The 20 newsgroups collection has become a popular data set for experiments in text applications of machine learning techniques, such as text classification and text clustering[8]. We will use the convenient 20 Newgroups loader by sklearn for our experiments.

Of the 20 categories, we will use the following 10 categories:

$$\text{categories} = \begin{bmatrix} \text{'comp.graphics', 'comp.sys.mac.hardware', 'misc.forsale',} \\ \text{'rec.motorcycles', 'rec.sport.baseball', 'sci.med', 'sci.space',} \\ \text{'talk.politics.guns', 'talk.politics.mideast', 'talk.religion.misc'} \end{bmatrix} \tag{168}$$

Here is an example of raw text in the 'macs.hardware' category:

```
»» 4257 th doc:  Anyone know what would cause my IIcx to not turn on when I hit
the keyboard switch?  The one in the back of the machine doesn't work either...
The only way I can turn it on is to unplug the machine for a few minutes, then
plug it back in and hit the power switch in the back immediately...  Sometimes
this doesn't even work for a long time...
   I remember hearing about this problem a long time ago, and that a logic
board failure was mentioned as the source of the problem...is this true?
```

After we load the raw documents, we need to extract features from them and represent in vector form. There are two widely used vectorization method, the *bag-of-words* (BOW) and the *tf-idf* (term-frequency-inverse-document-frequency). We use a standard vocabulary $\mathcal{V}$ of 45534 words provided in sklearn. For a given document $\mathbf{x}$, its BOW and tf-idf feature vectors are defined by

$$\boldsymbol{\phi}_{\text{BOW}}(\mathbf{x}) = \left[\#\text{ of } j\text{th word in } \mathbf{x};\ 1 \le j \le |\mathcal{V}| \right], \tag{169}$$

[8] See http://qwone.com/~jason/20Newsgroups/

$$\boldsymbol{\phi}_{\text{tf-idf}}(\mathbf{x}) = \left[\frac{\#\text{ of } j\text{th word in } \mathbf{x}}{\#\text{ of documents that contains the } j\text{th word}} ; 1 \le j \le |\mathcal{V}| \right]. \tag{170}$$

Below are the BOW and tf-idf feature vectors of the sample document (4257th) above.

	Coordinate	Bag-of-words	tf-idf
ago	1286	1	0.115122
back	3503	3	0.288116
board	4850	1	0.138195
cause	6355	1	0.125130
either	12551	1	0.108417
failure	14215	1	0.170555
hearing	17842	1	0.170555
hit	18278	2	0.261832
iicx	19200	1	0.217706
immediately	19354	1	0.155037
keyboard	21813	1	0.160377
know	22116	1	0.075488
logic	23500	1	0.158818
long	23525	2	0.203652
machine	23918	2	0.266555
mentioned	25060	1	0.135408
minutes	25592	1	0.145699
plug	30501	1	0.171330
power	30919	1	0.111667
problem	31391	2	0.203382
remember	33429	1	0.119059
sometimes	37293	1	0.135854
source	37380	1	0.127372
switch	39267	2	0.321834
time	40488	2	0.164800
true	41292	1	0.114800
turn	41456	2	0.269503
unplug	42226	1	0.233554
way	43812	1	0.089654
work	44510	2	0.197874

FIGURE 25. Bag-of-words and tf-idf feature vectors of a sample document from 20Newsgroups.

One can think of the tf-idf representation as an inverse-document-frequency (idf) correction to the BOW representation. Namely, even though some words appear many times in a document (e.g., 'The'), if it appears in many other documents, we discount its frequency by the idf. On the other hand, if some words appear only a few times in a document (e.g., 'OECD'), if it does not appear in many other documents, we boost its frequency (relatively to the more widely used words).

8.2. Classification by multiclass Logistic Regression. In this subsection, we use the multiclass logistic regression (MLR) in Section 3.3 for the task of document classification for the 20Newsgroups dataset we discussed in Subsection 8.1. In Figure (26), we show the optimal MLR coefficients (for two classes) fitted to the 20Newsgroups dataset using the gradient descent algorithm. We show the coefficients for both choice of vectorizers, BOW and tf-idf. By comparing these plots, we see that the coefficients with BOW vectoerizer has larger variance than the one with tf-idf vectorizer (see the reported standard deviation in legends). This is because the inverse-document-frequency acts as a regularizer in feature extraction step and the resulting feature vector has smaller variance (check this numerically).

In Figure 29, we show the confusion matrix of testing the fitted MLR on the 20Newsgroups dataset. As before for the MNIST dataset (see Sec. 4), we use the MAP decision algorithm 108 in order to turn the computed predictive probabilities into final predicted class labels. Overall MLR perfoms well in the 10-class classification task regardless of the choice of the vectorizer, as the confusion matrices are close to being diagonal.

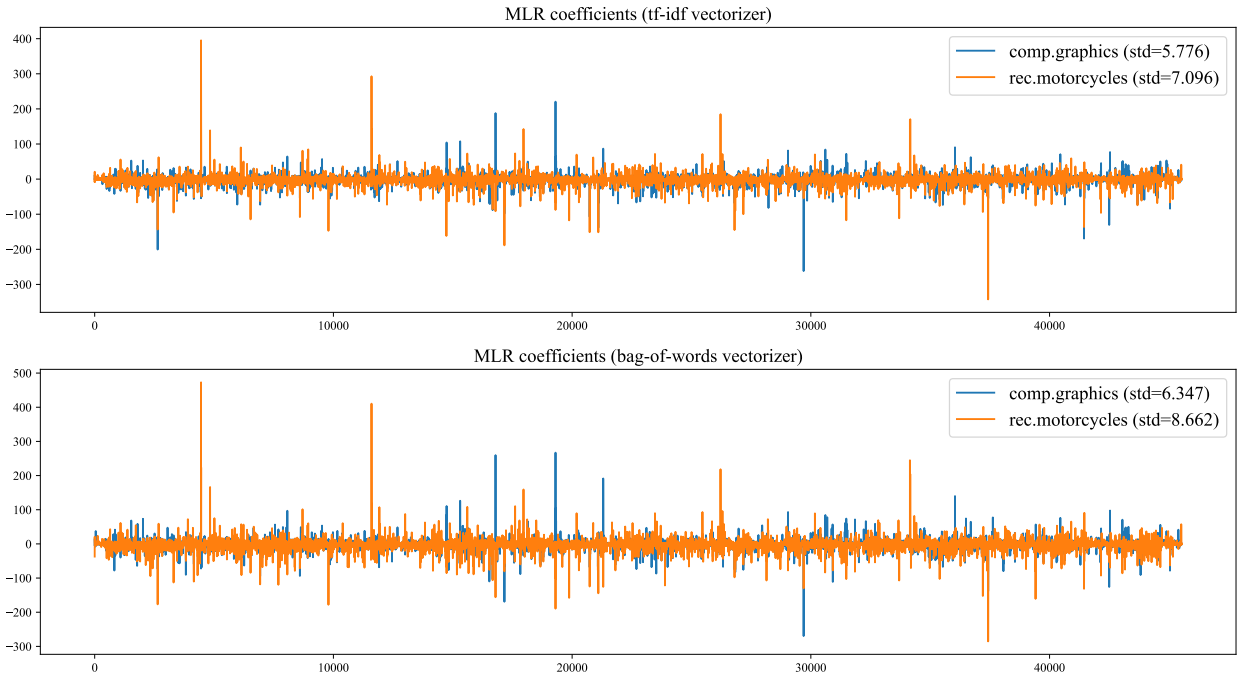


FIGURE 26. Examples of the fitted logistic regression coefficients to the 20Newsgroups dataset with digits. Gradient descent algorithm is used for training. Regression coefficients for the two classes 'comp.graphics' and 'rec.motorcycles' are shown.

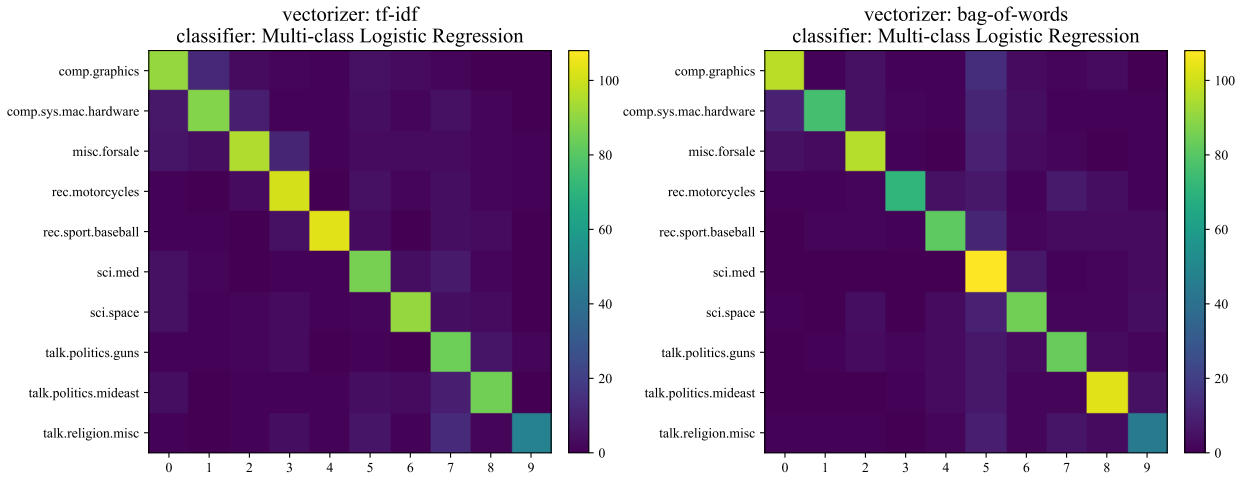


FIGURE 27. Confusion matrices for the 10-class document classficiation for the 20Newsgroups datatset with multiclass logistic regression. Vectorizers of bag-of-words and tf-idf are both used.

8.3. Classification by Multinomial Naive Bayes. In this subsection, we use the multinomial naive Bayesian classifier (MNB) in Section 7.2 for the task of document classification for the 20Newsgroups dataset in Subsection 8.1. In Figure (26), we show the optimal (maximum likelihood) class-conditional probabilities (see Prop. 2.7.1) for two classes. We plot the class-conditional PMFs on the vocabulary for both choice of vectorizers, BOW and tf-idf. Note that even though tf-idf feature vectors are not discrete, according to Exercise 2.7.5, we can still apply MNR to tf-idf feature vectors.

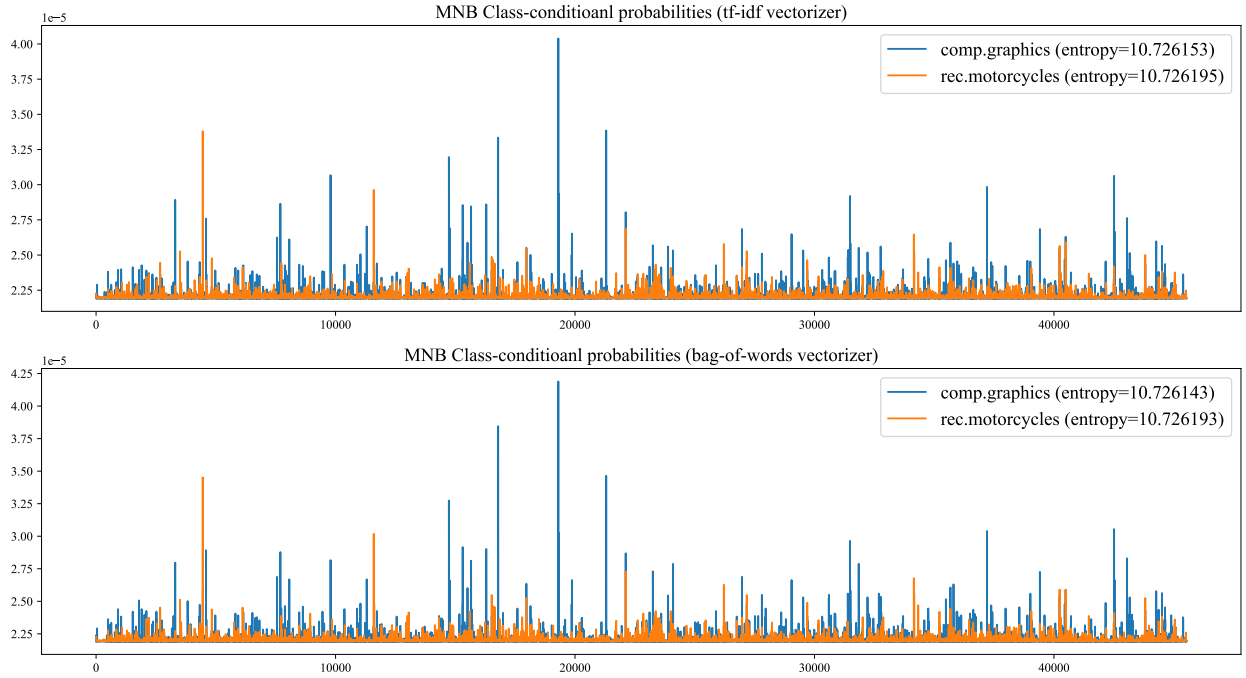


FIGURE 28. Examples of the fitted class-conditional probabilities of the 20Newsgroups dataset with digits. Two classes 'comp.graphics' and 'rec.motorcycles' are shown.

For each PMF, we also show its (Shannon) *entropy*, which is a measure of the 'information' or 'surprise' in a probability distribution. For a PMF $\mathbf{p} = [p_1, \ldots, p_n]$, its entropy $H(\mathbf{p})$ is defined as

$$H(\mathbf{p}) := -\sum_{i=1}^{n} p_i \log p_i \geq 0. \tag{171}$$

For any finite sample space, the PMF of maximum entropy on it is the uniform distribution, which should be the 'most disordered' among all possible PMFs, so in order to get to know it, we need large samples from it (hence more information content). On the other extreme, the PMFs supported on one single outcome have the lowest possible entropy of zero, and such PMFs contain 'no surprise'. In general, we regard PMFs with large entropy to be more disordered and hence contain more information. In Figure 28, we see that the class-conditional PMFs for the two classes have almost the same entropy regardless of the choice of the vectorizer. However, the ones with tf-idf vectorizer have slightly larger entropy for both classes. As before, this can be understood as the inverse-document-frequency being a regularizer in feature extraction, so that the feature values are more uniform on the coordinates. Since the (normalied) features have larger entropy, so are the fitted class-conditional probabilities.

In Figure 29, we show the confusion matrix of testing the fitted MNB on the 20Newsgroups dataset. As before for the MNIST dataset (see Sec. 4), we use the MAP decision algorithm 108 in order to turn the computed predictive probabilities into final predicted class labels. Overall MLR perfoms well in the 10-class classification task regardless of the choice of the vectorizer, as the confusion matrices are close to being diagonal. But less accuratly than multiclass logistic regression (see Figure 28). In particular, it seems that MNB suffers classifiying documents in the category 'sci.med'. (Why?)

Exercise 2.8.1. According to Exercise 2.7.5, we can apply multinomial naive Bayes for the image classification of MNIST dataset, since the features there (pixel values) are nonnegative.
(Ref: Math156_classification_MNIST.ipynb and Math156_20NewsGroups.ipynb)

(i) Reproduce Figure 16 for classifying digits '4' and '7' using multinomial naive Bayes. Compare the performance with logistic regression and probit regression.

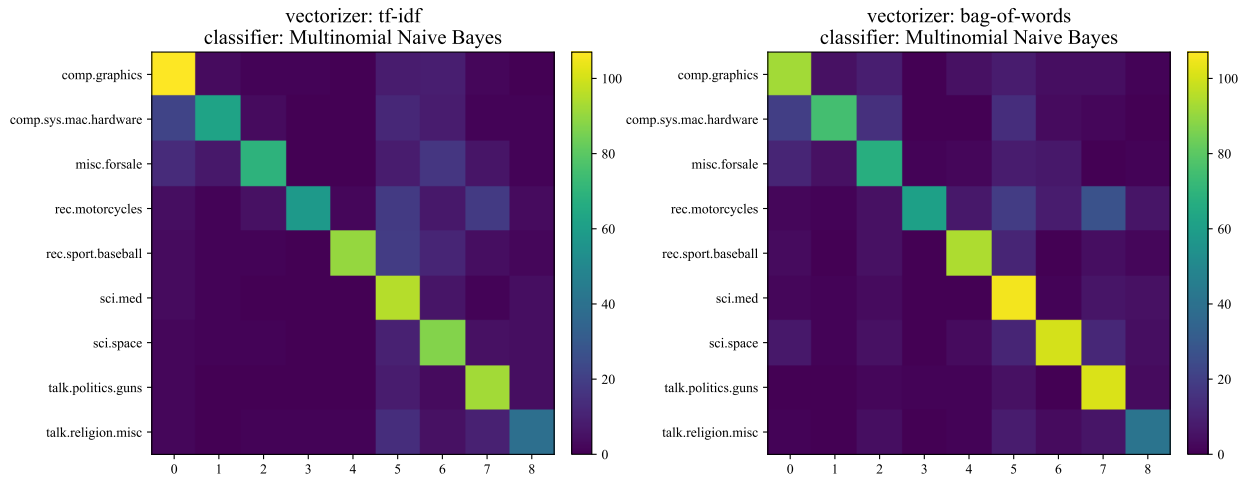


FIGURE 29. Confusion matrices for the 10-class document classficiation for the 20Newsgroups datatset with multinomial naive Bayes. Vectorizers of bag-of-words and tf-idf are both used.

(ii) Reproduce Figure 19 for classifying digites 0 through 4 using multinomial naive Bayes. Compare the performance with multiclass logistic regression.

Exercise 2.8.2. In this exercise, we investigate the 'robustness' of multinomial naive Bayes (MNB) classifier on the 20Newsgroups dataset. (Ref: Math156_20NewsGroups.ipynb)

(i) Modify the data preprocessing function "sample_multiclass_20NEWS" in the Jupyter notebook so that it has additional parameters 'noise_ratio' and 'noise_intensity': If 'noise_ratio' is $\varepsilon \in [0,1]$, and 'noise_intensity' is $\delta > 0$, then then it will add independent Uniform($[0,1]$) variables times δ to ε density of randomly chosen coordinates.

(ii) Choose two categories in the 20Newsgroups dataset and compare the performance of binary classification using MNB with varying noise parameters $(\varepsilon, \delta) \in \{0.1, 0.5, 0.9\} \times \{0.001, 0.01, 0.1\}$. (Use tf-idf vectorizer.) Do you think if MNB is robust against noise on the 20Newsgroups dataset? If so, is it more robust against noise ratio (ε) or noise intensity (δ)? Otherwise, can you say why?

(iii) Perform similar experiments as in **(ii)** for multiclass logistic regression.

9. Gaussian Naive Bayes

Consider the case of classification problem where the features are contiuous (e.g., height, weight, pixel values, tf-idf frequency) and may also assume negative values (e.g., bitcoin price change, EEG signal). In fact, in Exercise 2.7.5, we did discuss a naive Bayes model for continuous and nonnegative featrues, which was a straightforward generalization of the multinomial naive Bayes for nonnegative integer valued features. In this section, we introduce another widely used naive Bayes model called the *Gaussian naive Bayes* (GNB) model. The gist of it is the following: Assume each feature coordinate is a Gaussian RV with its own mean and variance, independently conditional on the class.

1. Consider a κ-class classification problem for class labels $\mathcal{Y} = \{1, \dots, \kappa\}$ and the feature vector $\boldsymbol{\phi}(\mathbf{x}) \in \mathbb{R}^M$ of each data point $\mathbf{x}$ assume general *real* values in each coordinate (including negative values).

2. We assume the following probabilitic model for (data, class) = $(\mathbf{X}, Y)$:

$$\text{(prior)} \quad \mathbb{P}(Y = i) =: q_i \qquad \text{for } i \in \{1, \dots, \kappa\} \tag{172}$$

$$\text{(class-conditional PDF)} \quad p(\boldsymbol{\phi}(\mathbf{X}) = [c_1, \dots, c_M] \,|\, Y = i) \propto \prod_{j=1}^{M} f_{\mu_{ij}, \sigma_{ij}}(c_j) \qquad \text{for } i \in \{1, \dots, \kappa\}, \tag{173}$$

$$\text{(constraints)} \quad \begin{cases} 0 \le q_1, \dots, q_\kappa \le 1, & q_1 + \cdots + q_\kappa = 1 \\ \sigma_{i,j} \ge 0 & \text{for all } i \in \{1, \dots, \kappa\},\ j \in \{1, \dots, M\}, \end{cases} \tag{174}$$

where $f_{\mu,\sigma}(x)$ is the PDF of $N(\mu, \sigma)$:

$$f_{\mu,\sigma}(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right). \tag{175}$$

We denote $\mathbf{W} = (q_i, \mu_{ij}, \sigma_{ij})$ for the vector of parameters of our model above.

Suppose training examples are given: $(\boldsymbol{\phi}(\mathbf{x}_s), y_s)$, $s = 1, \dots, N$, and write $\boldsymbol{\phi}(\mathbf{x}_s) = [\phi_1(\mathbf{x}_s), \dots, \phi_N(\mathbf{x}_s)]$ for each s. We appeal to the general maximum likelihood formulation of naive Bayes models in (134) and obtain the following:

$$\hat{\mathbf{W}} = \operatorname*{arg\,max}_{\mathbf{W}} \left[\sum_{s=1}^{N} \sum_{i=1}^{\kappa} p\mathbf{1}(y_s = i) \log q_i + \sum_{s=1}^{N} \sum_{j=1}^{p} \sum_{i=1}^{\kappa} \mathbf{1}(y_s = i) \log \frac{1}{\sqrt{2\pi\sigma_{ij}^2}} \exp\left(-\frac{(\phi_j(\mathbf{x}_s) - \mu_{ij})^2}{2\sigma_{ij}^2}\right) \right] \tag{176}$$

$$= \operatorname*{arg\,max}_{\mathbf{W}} \left[\ell_{GNB}(\mathbf{W}) := \begin{aligned} & p \sum_{i=1}^{\kappa} \left(\sum_{s=1}^{N} \mathbf{1}(y_s = i) \right) \log q_i - \frac{1}{2} \sum_{j=1}^{p} \sum_{i=1}^{\kappa} \left(\sum_{s=1}^{N} \mathbf{1}(y_s = i) \right) \log(2\pi\sigma_{ij}^2) \\ & - \sum_{s=1}^{N} \sum_{j=1}^{p} \sum_{i=1}^{\kappa} \mathbf{1}(y_s = i) \frac{(\phi_j(\mathbf{x}_s) - \mu_{ij})^2}{2\sigma_{ij}^2} \end{aligned} \right]. \tag{177}$$

The optimization problem (176) can be solved in closed form using standard arguments. The details are given in (2.9.1).

Exercise 2.9.1. Let ℓ_{GNB} denote the Gaussian naive Bayes loss function defined in (176). Denote the solution of (176) as $\hat{\mathbf{W}} = (\hat{\mathbf{q}}_i, \hat{\mu}_{ij}, \hat{\sigma}_{ij})$.

(i) Argue that the optimal prior PMF $\hat{\mathbf{q}} := [\hat{q}_1, \dots, \hat{q}_\kappa]$ solves the following optimization problem

$$\hat{\mathbf{q}} = \operatorname*{arg\,max}_{\substack{\mathbf{q}=[q_1,\dots,q_\kappa] \\ \text{PMF on } \{1,\dots,\kappa\}}} \sum_{i=1}^{\kappa} \left(\sum_{s=1}^{N} \mathbf{1}(y_s = i) \right) \log q_i. \tag{178}$$

Conclude that $\hat{q}_i = \frac{1}{N} \sum_{s=1}^{N} \mathbf{1}(y_s = i)$ (Hint: Use Exercise 2.7.2).

(ii) Show that

$$\frac{\partial \ell_{GNB}(\mathbf{W})}{\partial \mu_{ij}} = \sum_{s=1}^{N} \mathbf{1}(y_s = i) \frac{(\phi_j(\mathbf{x}_s) - \mu_{ij})}{\sigma_{ij}^2}. \tag{179}$$

Deduce that $\hat{\mu}_{ij}$ equals the following 'sample mean of the jth feature in class i':

$$\hat{\mu}_{ij} = \frac{\sum_{s=1}^{N} \mathbf{1}(y_s = i) \phi_j(\mathbf{x}_s)}{\sum_{s=1}^{N} \mathbf{1}(y_s = i)}. \tag{180}$$

(Compare this with $\hat{q}_{ij}$ in Prop. 2.7.1.)

(iii) Show that

$$\frac{\partial \ell_{GNB}(\mathbf{W})}{\partial \sigma_{ij}} = -\sigma_{ij}^{-1} \left(\sum_{s=1}^{N} \mathbf{1}(y_s = i) \right) + \sigma_{ij}^{-3} \sum_{s=1}^{N} \mathbf{1}(y_s = i)(\phi_j(\mathbf{x}_s) - \mu_{ij})^2. \tag{181}$$

Deduce that $\hat{\sigma}_{ij}$ equals the following 'variance of the class-i empirical distribution':

$$\hat{\sigma}_{ij}^2 = \frac{\sum_{s=1}^{N} \mathbf{1}(y_s = i)(\phi_j(\mathbf{x}_s) - \hat{\mu}_{ij})^2}{\sum_{s=1}^{N} \mathbf{1}(y_s = i)}, \tag{182}$$

where $\hat{\mu}_{ij}$ is given in **(ii)**.

CHAPTER 3

Neural Networks

In this chapter, we discuss a simple form of multilayer feedforward neural network. The model is simple enough to discuss without too much technical details but also is sophisticated enough to convey essential ideas in modern neural netowrk models.

1. Model formulation

Very simpliy put, one can think of a feedword neural network as a composition of multiple general linear models (e.g., logistic regression, probit regression). To motivate this construction, recall that in generalized linear models, we model the predictive probabilities as

$$\hat{\mathbf{y}} := \underbrace{\begin{bmatrix} p(1 \mid \boldsymbol{\phi}(\mathbf{x})) \\ \vdots \\ p(\kappa \mid \boldsymbol{\phi}(\mathbf{x})) \end{bmatrix}}_{\text{predictive prob.}} = \sigma(\underbrace{\boldsymbol{\phi}(\mathbf{x})}_{???}{}^T \mathbf{W}), \tag{183}$$

where σ is an activation function (e.g., signomid, logit, ReLu, tanh. See Figure 31.), $\boldsymbol{\phi} \in \mathbb{R}^M$ is a M-dimensional feature vector of data $\mathbf{x}$, $\mathbf{W} \in \mathbb{R}^{M \times \kappa}$ is a parameter matrix to be learned. So far, we have chosen a particular feature vector $\boldsymbol{\phi}(\mathbf{x})$ before we fit the model to the training data (e.g., pixel values for MNIST images, bag-of-words or tf-idf vectors for 20Newgroups texts). But how do we know what feature should we choose? Could we somehow also *learn* what feature to use during the training process?[1]

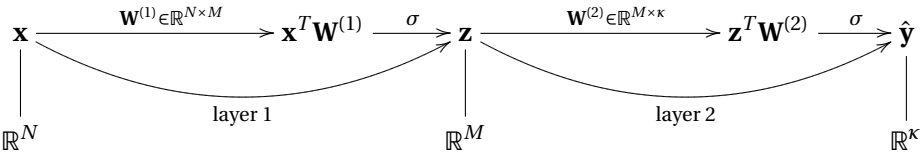
A simple idea is to *recycle the same generalized linear model in order to model the feature vector*. Namely, we consider

$$\mathbf{z} := \boldsymbol{\phi}(\mathbf{x}) = \sigma(\mathbf{x}^T \mathbf{W}'), \tag{184}$$

where we view $\mathbf{x} \in \mathbb{R}^N$ and $\mathbf{W}' \in \mathbb{R}^{N \times M}$ is another parameter matrix. Rewriting the parameter vectors as $\mathbf{W}^{(1)} = \mathbf{W}' \in \mathbb{R}^N$ and $\mathbf{W}^{(2)} = \mathbf{W} \in \mathbb{R}^M$, the whole model reads as

$$\hat{\mathbf{y}} = \sigma\left(\sigma(\mathbf{x}^T \mathbf{W}^{(1)})^T \mathbf{W}^{(2)}\right). \tag{185}$$

As a diagram, it can also be described as (see also Figure 30)



(186)

This model is called a *two-layer neural network* (a.k.a. *multilayer perceptron*). Since the information only flows across layers in one direction, it is also called a *feedforward* neural network[2]. It is also called

[1] In the language of linear algebra, the feature vectors are coordinates on a chosen basis. We have worked with a fixed basis, but here we are attempting to also learn the basis from the training data. We will revisit this point later in dimensionality reduction (e.g., principal component analysis, nonnegative matrix factorization)

[2] Recurrent Neural Networks, which is a current state-of-the-art model for natural languate processing, use feedback loop.

a *fully connected* neural network, since each coordinate in a given layer can get information from all coordinates in the previous layer[3].

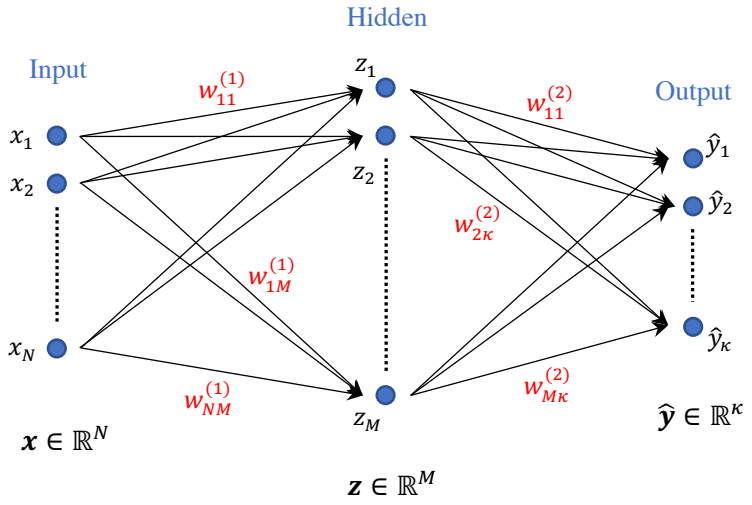


FIGURE 30. Diagrammatic representation of a 2-layer neural network.

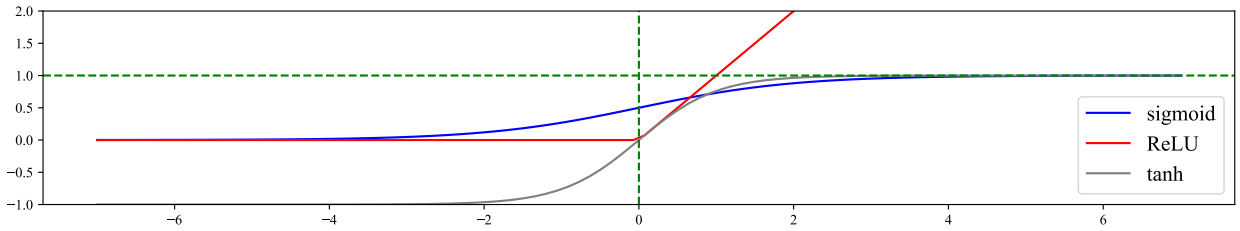


FIGURE 31. Common activation functions used in neural networks. Sigmoid: $x \mapsto 1/(1+e^{-x})$, ReLU: $x \mapsto \max(0,x)$, and tanh: $x \mapsto (e^{2x}-1)/(e^{2x}+1)$.

2. Training Feedforward Neural Networks

In this section, we discuss how we train feedforward neural networks.

2.1. Optimization framework. As always, the key step is to formulate a suitable optimization problem for training the model. Suppose we are given training examples $(\mathbf{x}_1,\mathbf{y}_1),\dots,(\mathbf{x}_n,\mathbf{y}_n)$ in $\mathbb{R}^n \times \mathbb{R}^\kappa$. Denote the predicted output for each input $\mathbf{x}_s$ using parameter $\mathbf{w}$ by $\hat{\mathbf{y}}(\mathbf{x}_s;\mathbf{w}) = [\hat{y}_1(\mathbf{x}_s;\mathbf{w}),\dots,\hat{y}_\kappa(\mathbf{x}_s;\mathbf{w})]$. We think of them as the output computed by the two-layer neural network (186), where $\mathbf{w}$ denotes the collection of all paramter matrices $[\mathbf{W}^{(1)},\mathbf{W}^{(2)}]$[4]. We seek to find an optimal parameter $\hat{\mathbf{w}}$ that minimizes the overall trainign loss:

$$\hat{\mathbf{w}} := \arg\min_{\mathbf{w}} \left[\ell_{NN}(\mathbf{w}) := \sum_{s=1}^{n} \ell(\mathbf{y}_s, \hat{\mathbf{y}}(\mathbf{x}_s;\mathbf{w})) \right], \tag{187}$$

where ℓ is a loss function that measures the discrepancy between the true output $\mathbf{y}_i$ and the predicted output $\hat{\mathbf{y}}(\mathbf{x}_i;\mathbf{w})$, which may depend on whether we are solving a regression or a classification problem[5].

[3]Convolutional Neural Network, wich is a current state-of-the-art for image classification, uses locally connected convolutional layers, mimicing the human visual cortex.

[4]However, our discussion in this section is general and not necessarily limited to the 2-layer neural networks.

[5]For classification problems, we may represent each class i by the indicator vector (one-hot encoding) $[0,\dots,0,1,0,\dots,0] \in \mathbb{R}^\kappa$, where the 1 is at the ith coordinate.

The standard choices are

$$\texttt{Regression} \qquad \text{(square loss):} \quad \ell(\mathbf{y}_s, \hat{\mathbf{y}}(\mathbf{x}_s;\mathbf{w})) = \frac{1}{2}\|\mathbf{y}_s - \hat{\mathbf{y}}(\mathbf{x}_s;\mathbf{w})\|_F^2, \tag{188}$$

$$\texttt{Classification} \quad \text{(cross-entropy loss):} \quad \ell(\mathbf{y}_s, \hat{\mathbf{y}}(\mathbf{x}_s;\mathbf{w})) = -\sum_{i=1}^{\kappa} \mathbf{1}(\mathbf{y}_s = \text{class } i) \log \hat{y}_i(\mathbf{x}_s;\mathbf{w}), \tag{189}$$

$$\text{(softmax-cross-entropy loss):} \quad \ell(\mathbf{y}_s, \hat{\mathbf{y}}(\mathbf{x}_s;\mathbf{w})) = -\sum_{i=1}^{\kappa} \mathbf{1}(\mathbf{y}_s = \text{class } i) \log \operatorname{softmax}_i\left(\hat{\mathbf{y}}(\mathbf{x}_s;\mathbf{w})\right). \tag{190}$$

The square loss for the regression case is natural and we have seen it for polynomial regression models in Section 2. On the other hand, for classification problems, recall that we usually model the predictive probabilities given features so that $\hat{\mathbf{y}}(\mathbf{x}_s;\mathbf{w}) \in \mathbb{R}^\kappa$ can be viewed as a predictive PMF of the class of $\mathbf{x}_s$. One can think of the 'cross-entropy' as a similarity meature between two probability distributions. Namely, if we have two PMFs $\mathbf{p} = [p_1, \ldots, p_\kappa]$ and $\mathbf{q} = [q_1, \ldots, q_\kappa]$, then the *cross-entropy* of $\mathbf{q}$ relative to $\mathbf{p}$ is defined by

$$H(\mathbf{p}, \mathbf{q}) = -\sum_{i=1}^{\kappa} p_i \log q_i. \tag{191}$$

Note that the following class-indicator vector

$$\left[\mathbf{1}(\mathbf{y}_s = \text{class } 1), \ldots, \mathbf{1}(\mathbf{y}_s = \text{class } \kappa)\right] \tag{192}$$

is a PMF on the κ classes. Hence the cross-entropy loss in (189) is in fact the cross-entropy between the class-indicator vector above with the predictive PMF $\hat{\mathbf{y}}(\mathbf{x}_s;\mathbf{w}) \in \mathbb{R}^\kappa$. Lastly, the softmax-cross-entropy loss in (190) is frequently used in classification problems where the target output $\mathbf{y}_s$ is a PMF but the predicted output $\hat{y}(\mathbf{x}_s;\mathbf{w})$ is not and is yet to be converted into a PMF. In this case, one can use the following softmax transform

$$\mathbf{x} = [x_1, \ldots, x_\kappa]^T \mapsto \operatorname{softmax}(\mathbf{x}) = \left[\frac{\exp(x_1)}{\sum_{i=1}^{\kappa} \exp(x_i)}, \ldots, \frac{\exp(x_n)}{\sum_{i=1}^{\kappa} \exp(x_i)}\right]^T \in \mathbb{R}^\kappa \tag{193}$$

first and then compute the cross-entropy loss, which results in the combined loss of the form in (190).

One of the recurreing theme in all of regression and classification models we studied so far is to take the *loss function as the negative log likelihood*. Namely,

$$\ell_{\text{total}}(\mathbf{w}) := -\log L(y_1, \ldots, y_n; \mathbf{w}). \tag{194}$$

In fact, the square loss and cross-entropy loss above can be derived from this maximum likelihood formulation. For instance, consider the case of κ-class classification. Recall that $\hat{\mathbf{y}}(\mathbf{x}_s;\mathbf{w})$ is the predictive PMF so that its ith coordinate, $\hat{\mathbf{y}}_i(\mathbf{x}_s;\mathbf{w})$, is the probability that the class of $\mathbf{x}_s$ is i according to our model. Hence the joint likelihood is

$$L(y_1, \ldots, y_N; \mathbf{w}) = \mathbb{P}(Y_1 = y_1, \ldots, Y_N = y_N; \mathbf{w}) = \prod_{s=1}^{n} \prod_{i=1}^{\kappa} (\hat{y}_i(\mathbf{x}_s;\mathbf{w}))^{\mathbf{1}(\mathbf{y}_s = \text{class } i)} \tag{195}$$

Hence we have

$$-\log L(y_1, \ldots, y_N; \mathbf{w}) = -\sum_{s=1}^{n} \sum_{i=1}^{\kappa} \mathbf{1}(\mathbf{y}_s = \text{class } i) \log(\hat{y}_i(\mathbf{x}_s;\mathbf{w})), \tag{196}$$

which is exactly the total loss function in (187) using the cross-entropy loss in (189)[6].

Exercise 3.2.1. Derive the total loss function in (187) using the square loss in (188) by using the maximum likelihood framework in (194). You may assume that the true output $\mathbf{y} \in \mathbb{R}^\kappa$ is generated by a k-diemnsional multivariate Normal distribution $N(\hat{\mathbf{y}}(\mathbf{x};\mathbf{w}), \sigma^2 I)$, where I is the $\kappa \times \kappa$ identity matrix. (See Example 3.2.2.)

[6]Compare this computation with the one for the multiclass logistic regression in Section 3.3.

Example 3.2.2 (Multivariate Gaussian distribution)**.** The a p-dimensional *multivariate Gaussian distribution* is a function $\mathbb{R}^p \to \mathbb{R}$ given by

$$p(\mathbf{x};\boldsymbol{\mu},\boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{p/2}|\boldsymbol{\Sigma}|^{1/2}} \exp\left[-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right], \tag{197}$$

where $\boldsymbol{\mu} \in \mathbb{R}^p$ and $\boldsymbol{\Sigma} \in \mathbb{R}^{p\times p}$ is a symmetric positive semi-definite matrix[7] and $|\boldsymbol{\Sigma}|$ denotes the determinant of $\boldsymbol{\Sigma}$. If a random vector $\mathbf{X} \in \mathbb{R}^p$ has distribution given by (197), then we denote $\mathbf{X} \sim N(\boldsymbol{\mu},\boldsymbol{\Sigma})$. Then it has the following properties

$$\boldsymbol{\mu} = \mathbb{E}[X], \qquad \boldsymbol{\Sigma} = \mathrm{Cov}(X) = \mathbb{E}\left[(\mathbf{X}-\boldsymbol{\mu})(\mathbf{X}-\boldsymbol{\mu})^T\right]. \tag{198}$$

For these reasons, we call $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$ the *mean* and the *covariance matrix* of $\mathbf{X}$, respectively. ▲

2.2. Complexity of the optimization problem. The first thing to note for solving the optimization problem (187) for the two-layer neural networks is that it is a *non-convex* optimization problem. Namely, in both cases of square and cross-entropy loss, the total loss $\ell_{\text{total}}(\mathbf{w})$ depends on the model output $\hat{\mathbf{y}}(\mathbf{x}_s;\mathbf{w})$ in a convex manner but the model output depends on the weight matrices $\mathbf{w} = [\mathbf{W}^{(1)},\mathbf{W}^{(2)}]$ in a non-convex manner. On the contrary, recall that the cross-entropy loss for logistic regression, or a single-layer neural network, is a convex function on the parameters (see Exercise 2.3.2). So, unfortunately, it is going to be very difficult (practically impossible) to find the global minimum of the total loss for training a 2-layer neural network.

Another nice way to see the complexity of the current optimization problem is using model symmetry. Observe that the model output of a 2-layer neural network (see Figure 30) is invariant under permuting the order of hidden units along with the corresponding in- and out-edge weights. For instance, if we swap z_1 and z_N, then the weights $w_{i1}^{(1)}$ from input x_i to z_1 should be renamed to $w_{iN}^{(1)}$ and also the weights $w_{N1}^{(1)}$ from x_i to z_N should be renamed to $w_{i1}^{(1)}$, and so on; also the out-weights $w_{1j}^{(2)}$ and $w_{Nj}^{(2)}$ should be swapped. The model output $\mathbf{y}$ should be unaware of this permutation since the values of the two linear transforms in the network is unchanged. Note that there are $M!$ ways to permute the hidden units and their neural weights in this way. A consequence of this symmetry observation is that there are at least $M!$ 'equivalent local minima' of the total loss function (187)[8]. For instance, if we use $M = 10$ hidden units, then there are at least $10! = 3,628,800$ global minima to look for.

Since the training of the 2-layer neural network has a complex 'optimization landscape', we change our goal from the ambitious one of finding a global minimum to the more modest one of finding a local minimum. It is a consistent experience in the literature that a local minimum found by some standard non-convex optimization algorithm (e.g., stochastic gradient descent, block coordinate descent) will perform as well as any other local minima, and this phenomenon belongs to an active area of machine learning research.

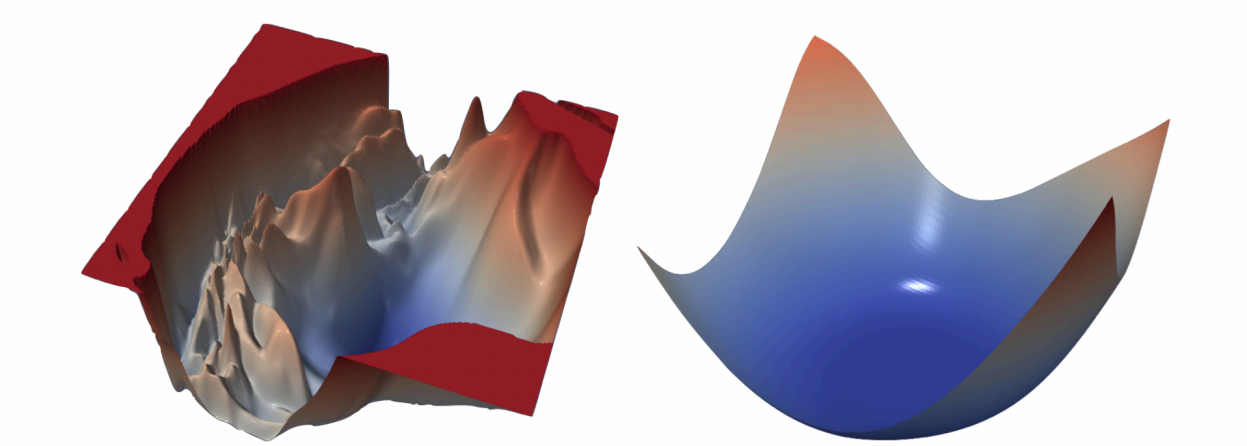
2.3. Stochastic Gradient Descent and Backpropagation. In order to find a local minimum of of (186), consider the familar optimizatoin algorithm of gradient descent, which we introduced in discussing training of logistic regression in (85):

$$\mathbf{w}_{t+1} \longleftarrow \mathbf{w}_t - \eta_t \nabla_{\mathbf{w}} \ell_{NN}(\mathbf{w}_t) \tag{199}$$

One of the main bottleneck in the development of neural network models in machine learning literature was the huge computational cost in computing the full gradient $\ell_{NN}(\mathbf{w}_t)$ for using the above gradient descent algorithm. We will discuss two ways to overcome this issue: 1) Stochastic Gradient Descent (SGD) and 2) Backpropagation.

[7] A matrix $\mathbf{A} \in \mathbb{R}^{p\times p}$ is *symmetric* if $\mathbf{A} = \mathbf{A}^T$ and *positive semi-definite* if $\mathbf{x}^T\mathbf{A}\mathbf{x} \geq 0$ for all $\mathbf{x} \in \mathbb{R}^p$. Moreover, $\mathbf{A}$ is *positive definite* if it is positive semi-definite and furthermore, $\mathbf{x}^T\mathbf{A}\mathbf{x} > 0$ if $\mathbf{x} \neq \mathbf{0}$.

[8] For the hyperbolic tangent activation function $\sigma(x) = \tanh(x) = (e^{2x}-1)/(e^{2x}+1)$, the model output is also invariant under sign flips of the neural weights, so there are at least $M!2^M$ equivalent local minimia.



(a) ResNet-110, no skip connections (b) DenseNet, 121 layers

FIGURE 32. Optimization landscape for ResNet-110-noshort (left, nonconvex) and DenseNet for CIFAR-10 (right, convex). Figure excerpted from [LXT+17].

2.3.1. *Stochastic Gradient Descent.* The idea of SGD is not limited only to the context of neural networks, but is applicable for general nonconvex optimization problems. Simply speaking it is to use only a 'partial gradient' in each iteration of the gradient descent, which typically requires much less computational cost. In our case, note that we can decompose the gradient $\nabla \ell_{NN}(\mathbf{w}_t)$ as the sum of gradients per sample:

$$\nabla \ell_{NN}(\mathbf{w}_t) = \sum_{s=1}^{n} \nabla_{\mathbf{w}} \ell(\mathbf{y}_s, \hat{\mathbf{y}}(\mathbf{x}_s; \mathbf{w}_t)), \tag{200}$$

by using the definition of total loss function ℓ_{NN} and the linearity of differentiation. Hence, instead of computing the full gradient $\nabla \ell_{\mathbf{w}} \ell(\mathbf{w}_t)$, why don't we choose a small set $I \subseteq \{1, \ldots, n\}$ of training data samples and compute the corresponding partial gradient

$$\ell_{NN;I} \ell(\mathbf{w}_t) := \sum_{i \in I} \nabla_{\mathbf{w}} \ell(\mathbf{y}_s, \hat{\mathbf{y}}(\mathbf{x}_s; \mathbf{w}_t)). \tag{201}$$

The stochastic component of SGD lies in choosing the index set I randomly. Hence the corresponding SGD algorithm takes the following form:

Algorithm 1 Stochastic Gradient Descent for Neural Networks

1: **Input:** $\mathbf{w}_0$ (initial parameter); T (number of iterations); $(\eta_t)_{t \geq 1}$, (non-increasing weights in $(0, 1]$); $n_0 \in \{1, \ldots, n\}$ (size of subsample)
2: **for** $t = 1, \ldots, T$ **do**:
3: Sample $I \subseteq \{1, \cdots, n\}$ with $|I| = n_0$ over all subsets of size n_0;
4: $\mathbf{w}_t \longleftarrow \mathbf{w}_{t-1} - \eta_t \nabla_{\mathbf{w}} \ell_{NN;I}(\mathbf{w}_{t-1})$
5: **end for**
6: **output:** $\mathbf{w}_T$

An extreme case of the SGD algorithm in Algorithm 1 is when the size n_0 of subsample is 1. In that case, in each iteration, we only use a single randomly chosen training example to update the current parameter $\mathbf{w}_t$. In general, the chosen set of training examples $\{\mathbf{x}_i ; i \in I\}$ is called a *minibatch*, and SGD amounts to process only a randomly selected minibatch in each iteration of gradient descent, instead of the full *batch* $\{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$. Processing minibatches instead of the full batch is what reduces the computational cost of SGD. Moreover, the randomized choice of minibatches is what helps the SGD not to overfit to some particular training examples and to escape from undesirable local minima.

It is a classical result in the theory of nonconvex optimization that the parameters $(\mathbf{w}_t)_{t\geq 0}$ produced by SGD converges to a stationary point of the loss function (186) if a 'diminishing step size' is used: $\sum_{t=1}^{\infty}\eta_t=\infty$ and $\sum_{t=1}^{\infty}\eta_t^2<\infty$ when the loss function is nice enough (see, e.g., [SSY18]). As in the case of logistic regression, we will use the schedule $\eta_t=\gamma(\log t)/\sqrt{t}$, where $\gamma>0$ is a constant.

2.3.2. *Backpropagation. Error backpropagation* is an effective method to *evaluate* the gradient of a feedforward neural network at the current parameters, first developed by Rumelhart, Hinton, and Williams [RHW86].

In order to simplify the notation, we only consider the loss at sth training sample $(\mathbf{x}_s,\mathbf{y}_s)$:

$$\ell_s(\mathbf{w}):=\ell(\mathbf{y}_s,\hat{\mathbf{y}}(\mathbf{x}_s;\mathbf{w})). \tag{202}$$

In order to execute the SGD algorithm for training neural networks, we need to compute the following derivatives: For each $s\in\{1,\dots,n\}$,

$$\frac{\partial\ell_s}{\partial w_{ij}^{(1)}},\qquad \frac{\partial\ell_s}{\partial w_{jq}^{(2)}}. \tag{203}$$

Namely, we want to differentiate the loss function with respect to the edge weights. However, it would be extremely helpful for us to compute the derivative of the loss function with respect to *node weights* z_j and $\hat{y}_q$. Recall the following relations:

$$z_j=\sigma\left(\mathbf{x}^T\mathbf{W}^{(1)}[:,j]\right)=\sigma\left(\sum_{i=1}^{N}x_i w_{ij}^{(1)}\right),\qquad \hat{y}_q=\sigma\left(\mathbf{z}^T\mathbf{W}^{(2)}[:,q]\right)=\sigma\left(\sum_{j=1}^{M}z_j w_{jq}^{(2)}\right). \tag{204}$$

Denoting $\mathbf{y}_s=[(\mathbf{y}_s)_1,\dots,(\mathbf{y}_s)_\kappa]$, we first differentiate the loss function with respect to $\hat{y}_q$:

$$\text{(square loss):}\qquad \frac{\partial\ell_s}{\partial\hat{y}_q}=\frac{1}{2}\frac{\partial}{\partial\hat{y}_q}\|\mathbf{y}_s-\hat{\mathbf{y}}(\mathbf{x}_s;\mathbf{w})\|_F^2=\left(\hat{y}_q-(\mathbf{y}_s)_q\right), \tag{205}$$

$$\text{(cross-entropy loss):}\qquad \frac{\partial\ell_s}{\partial\hat{y}_q}=-\frac{\partial}{\partial\hat{y}_q}\sum_{i=1}^{\kappa}\mathbf{1}(\mathbf{y}_s=\text{class }i)\log\hat{y}_i(\mathbf{x}_s;\mathbf{w})=-\frac{\mathbf{1}(\mathbf{y}_s=\text{class }q)}{\hat{y}_q}. \tag{206}$$

$$\text{(softmax-cross-entropy loss):}\qquad \frac{\partial\ell_s}{\partial\hat{y}_q}=-\frac{\partial}{\partial\hat{y}_q}\sum_{i=1}^{\kappa}\mathbf{1}(\mathbf{y}_s=\text{class }i)\log\operatorname{softmax}_i\left(\hat{\mathbf{y}}(\mathbf{x}_s;\mathbf{w})\right) \tag{207}$$

$$=\operatorname{softmax}_q\left(\hat{\mathbf{y}}(\mathbf{x}_s;\mathbf{w})\right)-(\mathbf{y}_s)_q. \tag{208}$$

For (208), see Exercise 3.2.3.

Next, we compute the derivatives of the loss function with respect to the hidden state z_j. By using chain rule:

$$\underbrace{\frac{\partial\ell_s}{\partial z_j}}_{\text{'error' at node } z_j}=\sum_{q=1}^{\kappa}\frac{\partial\ell_s}{\partial\hat{y}_q}\frac{\partial\hat{y}_q}{\partial z_j}=\sum_{q=1}^{\kappa}\underbrace{\frac{\partial\ell_s}{\partial\hat{y}_q}}_{\text{'error' at node }\hat{y}_q}\sigma'\left(\underbrace{\mathbf{z}^T\mathbf{W}^{(2)}[:,q]}_{\text{activation for }\hat{y}_q}\right)w_{jq}^{(2)}, \tag{209}$$

where σ' denotes the derivative of the activation function σ. The equation in (209) is an instance of error backpropagation (See Figure 35).

In order to explain the mechanics of error backpropagation, we introduce the notion of 'errors at nodes' of a neural network to be the loss function with respect to the state of the node[9]. For instance, $\frac{\partial\ell_s}{\partial\hat{y}_q}$ is the error at node $\hat{y}_q$ and $\frac{\partial\ell_s}{\partial z_j}$ is the error at node z_j, and so on[10]. With this notion at hand, in (205) and (206), we computed the errors at the output nodes $\hat{y}_j$ directly. Also, we can view (209) as a *recursive relation* that computes the error at node z_j in terms of the errors at nodes $\hat{y}_q$ in the output layer. Below is a verbal description of the error backpropagation for 2-layer neural networks:

[9]We are making a slight abuse of notation of identiying a node in a neural network with its state.

[10]Why such a derivative is called an 'error'? One can make sense of it at least for the square loss at the output layer. See (205).

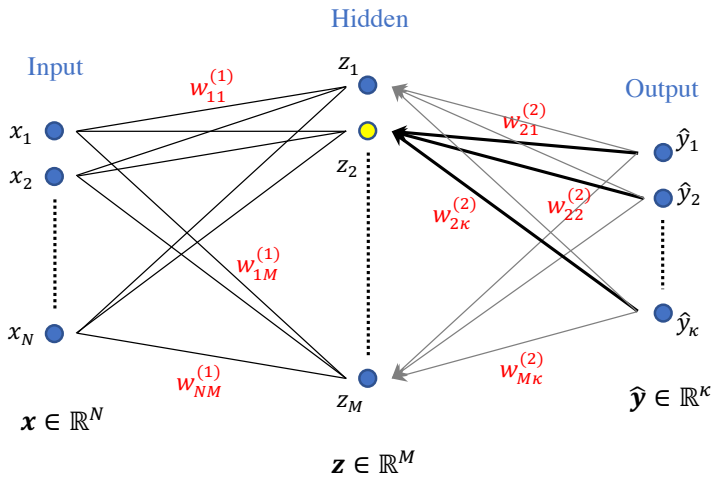


FIGURE 33. Error backpropatation in a 2-layer neural network. The 'errors' computed at the output layer are backpropagated to the second hidden node through the edges, where the corresponding edge weight is multiplied. The total sum of such weighted backpropagated errors at node z_2 is then the 'error' at node z_2.

Computation of gradients for training a 2-layer FFNN

1. Forward propagate the input all the way to the output layer and compute the activation and state of every node.
2. Compute the errors at the output nodes using (205)-(208).
3. Backprogagate the errors in the output layer to the hidden layer and compute the errors at hidden nodes, using (209).

In order to see that such recursion can be used to compute the errors in deeper layers, it is instrumental to compute the errors in the input layer, even though we do not use them for training the 2-layer neural network. One can easily verify

$$\underbrace{\frac{\partial \ell_s}{\partial x_i}}_{\text{'error at node } x_i\text{'}} = \sum_{j=1}^{M} \frac{\partial \ell_s}{\partial z_j}\frac{\partial z_j}{\partial x_i} = \sum_{j=1}^{M} \underbrace{\frac{\partial \ell_s}{\partial z_j}}_{\text{'error' at } z_j} \sigma'\left(\underbrace{\mathbf{x}^T\mathbf{W}^{(1)}[:,j]}_{\text{activation for } z_j}\right) w_{ij}^{(1)}. \tag{210}$$

Hence, *if we had a deep neural network with 2 or more layers, then we could use the same backpropagation recursion layer by layer to compute the errors at each node in all layers.*

Now that we can compute the errors at each node, it remains to see how we use them to compute the desired derivatives of the loss function with respect to the neural weights in (203). This can be easily done by the relation (204) and chain rule:

$$\frac{\partial \ell_s}{\partial w_{jq}^{(2)}} = \frac{\partial \ell_s}{\partial \hat{y}_q}\frac{\partial \hat{y}_q}{\partial w_{jq}^{(2)}} = \frac{\partial \ell_s}{\partial \hat{y}_q}\sigma'(\mathbf{z}^T\mathbf{W}^{(2)}[:,q])z_j, \tag{211}$$

$$\frac{\partial \ell_s}{\partial w_{ij}^{(1)}} = \frac{\partial \ell_s}{\partial z_j}\frac{\partial z_j}{\partial w_{ij}^{(1)}} = \frac{\partial \ell_s}{\partial z_j}\sigma'(\mathbf{x}^T\mathbf{W}^{(1)}[:,j])x_i. \tag{212}$$

Lastly, with all the derivatives as above, we can now execute the SGD algorithm (Algorithm 1) by aggregating the above per-sample derivatives to get the minibatch gradient $\nabla_{\mathbf{w}}\ell_{NN;I}(\mathbf{w}_{t-1})$ by using (201).

Exercise 3.2.3 (Derivative of softmax-cross-entropy)**.** The softmax function $\sigma : \mathbb{R}^n \to \mathbb{R}^n$, is defined by

$$\mathbf{x} = [x_1, \ldots, x_n]^T \mapsto \sigma(\mathbf{x}) = [\sigma_1(\mathbf{x}), \ldots, \sigma(\mathbf{x}_n)] = \left[\frac{\exp(x_1)}{\sum_{i=1}^n \exp(x_i)}, \ldots, \frac{\exp(x_n)}{\sum_{i=1}^n \exp(x_i)} \right]^T. \tag{213}$$

Define the following softmax-cross-entropy loss

$$\ell(\mathbf{y}, \hat{\mathbf{y}}) := -\sum_{i=1}^{\kappa} \mathbf{1}(\mathbf{y} = \text{class } i) \log \sigma_i(\hat{\mathbf{y}}) \tag{214}$$

for each $\mathbf{y}, \hat{\mathbf{y}} \in \mathbb{R}^n$.

(i) Show that

$$\frac{\partial}{\partial x_j} \sigma_i(\mathbf{x}) = \sigma_i(\mathbf{x}) \left(\mathbf{1}(i = j) - \sigma_j(\mathbf{x}) \right). \tag{215}$$

(ii) Show that

$$\frac{\partial}{\partial \hat{\mathbf{y}}} \ell(\mathbf{y}, \hat{\mathbf{y}}) = \sigma(\hat{\mathbf{y}}) - \mathbf{y}. \tag{216}$$

Exercise 3.2.4. Formulate an algorithm that computes the gradient of a general L-layer neural network. You may need to extend the error backpropagation algorithm for 2-layer NN we discussed above to the general L-layer case and show its correctness.

3. Examples of 2-layer FFNN

Example 3.3.1. Consider a two-layer neural network with the following choices:

Loss function: $\ell_s(\mathbf{y}_s, \hat{\mathbf{y}}(\mathbf{x}_s; \mathbf{w})) = (1/2)\|\mathbf{y}_s - \hat{\mathbf{y}}(\mathbf{x}_s; \mathbf{w})\|_F^2$ (Square loss)

Activation functions: $\tanh(x) = (e^x - e^{-x})/(e^x + e^{-x})$ for the hidden layer and sigmoid $\sigma(x)$ for the output layer.

Then the total loss function is given by

$$\nabla \ell_{NN}(\mathbf{w}) = \frac{1}{2} \sum_{s=1}^{n} \|\mathbf{y}_s - \hat{\mathbf{y}}(\mathbf{x}_s; \mathbf{w})\|_F^2. \tag{217}$$

After forward-propagating an input $\mathbf{x}_s \in \mathbb{R}^N$, the hidden states $\mathbf{z}_s \in \mathbb{R}^M$ and the output $\hat{\mathbf{y}}_s = \hat{\mathbf{y}}(\mathbf{x}_s; \mathbf{w}) \in \mathbb{R}^\kappa$ are given by

$$\mathbf{z}_s = \tanh(\mathbf{x}_s^T \mathbf{W}^{(1)}), \qquad \hat{\mathbf{y}}_s = \sigma(\mathbf{z}_s^T \mathbf{W}^{(2)}). \tag{218}$$

For backpropagation, the errors at the output nodes are given by

$$\frac{\partial \ell_s}{\partial (\hat{\mathbf{y}}_s)_q} = \left((\hat{\mathbf{y}}_s)_q - (\mathbf{y}_s)_q \right), \tag{219}$$

Hence, from (209), the errors at the hidden nodes are given by

$$\frac{\partial \ell_s}{\partial (\mathbf{z}_s)_j} = \sum_{q=1}^{\kappa} \frac{\partial \ell_s}{\partial (\hat{\mathbf{y}}_s)_q} \left((\hat{\mathbf{y}}_s)_q (1 - (\hat{\mathbf{y}}_s)_q) \right) w_{jq}^{(2)}, \tag{220}$$

where we have used that $\sigma'(x) = \sigma(x)(1 - \sigma(x))$.

Now from (211), we get

$$\frac{\partial \ell_s}{\partial w_{jq}^{(2)}} = \frac{\partial \ell_s}{\partial (\hat{\mathbf{y}}_s)_q} \left((\hat{\mathbf{y}}_s)_q (1 - (\hat{\mathbf{y}}_s)_q) \right) (\mathbf{z}_s)_j. \tag{221}$$

On the other hand, note that $\tanh'(x) = 1 - \tanh^2(x)$ so

$$\tanh'(\mathbf{x}_s^T \mathbf{W}^{(1)}[:, j]) = 1 - [(\mathbf{z}_s)_j]^2. \tag{222}$$

Then (212) gives

$$\frac{\partial \ell_s}{\partial w_{ij}^{(1)}} = \frac{\partial \ell_s}{\partial (\mathbf{z}_s)_j} \left(1 - [(\mathbf{z}_s)_j]^2\right) (\mathbf{x}_s)_i. \tag{223}$$

Lastly, the derivatives of the total loss ℓ_{NN} with repsect to the neural weights are given by summing up the per-sample derivatives above over s in the selected minibatch $I \subseteq \{1, \ldots, n\}$. ▲

Exercise 3.3.2 (2-layer FFNN on MNIST)**.** Below we show the train and test accuracy of classifying MNIST dataset (digits 0 through 4) using the 2-layer FFNN discussed in Example 3.3.1. Here the accuracy for multiclass classification is computed by the total number of correctly classified examples divided by the total number of examples. (Ref: Jupyter Notebook)

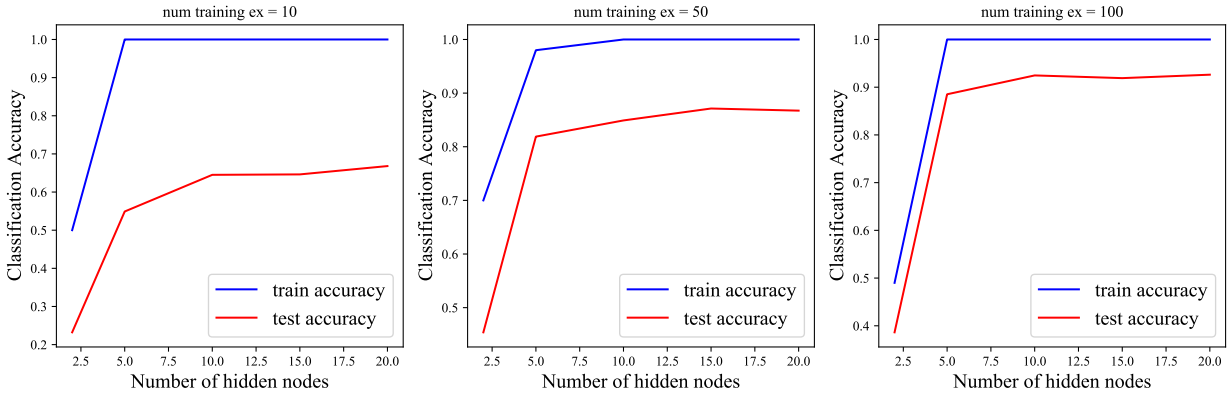


FIGURE 34. Classification accuracy of 2-layer FFNN for MNIST data with digits from 0 up to 4. The number of hidden nodes are varied within $\{2, 5, 10, 15, 20\}$ and the number of training examples are varied within $\{10, 50, 100\}$.

(i) What can you conclude from Figure 34? Optimal number of hidden nodes for each size of training set? Overfitting in terms of the number of hidden nodes and the size of training set?

(ii) Make similar plots for (1) digits 5 through 9 and (2) all digits while keeping other parameters the same. (You may increase the number of iterations for training for classifing all digits.) Do your observations in (ii) still hold in these experiments?

Exercise 3.3.3. Consider the following L_2-regularized 2-layer neural network training problem:

$$\hat{\mathbf{w}} := \underset{\mathbf{w}=[\mathbf{W}^{(1)}, \mathbf{W}^{(2)}]}{\operatorname{argmin}} \left[\ell_{NN}(\mathbf{w}; \lambda) := \left(\sum_{s=1}^{n} \ell(\mathbf{y}_s, \hat{\mathbf{y}}(\mathbf{x}_s; \mathbf{w})) \right) + \lambda \left(\|\mathbf{W}^{(1)}\|_F^2 + \|\mathbf{W}^{(2)}\|_F^2 \right) \right], \tag{224}$$

where $\lambda \geq 0$ is a L_2-regularization constant. This will penalize the weight matrices $\mathbf{W}^{(1)}, \mathbf{W}^{(2)}$ to have large L_2-norm.

(i) How does this change the training algorithm? Derive the gradient of the above loss function and formulate the modified training algorithm.

(ii) Modify the Multilayer Perceptron implementation `MLP` in Jupyter Notebook so that it has the option to use L_2-regularization. Reproduce Figure 34 with varing levels of regularization. Report a reaonable value of the regularization constant for the experiment in Figure 34.

Exercise 3.3.4. Implement the two-layer neural network discussed in Example 3.3.1 on your own, where the activation function in the output layer should allow the identity function in order for applying it on regression problems. Using your code, reproduce a figure similar to Figure 35 below. For each $n = 6, 10$ data points in the sinusoidal data set (see (18)), what do you think is the best value of M?

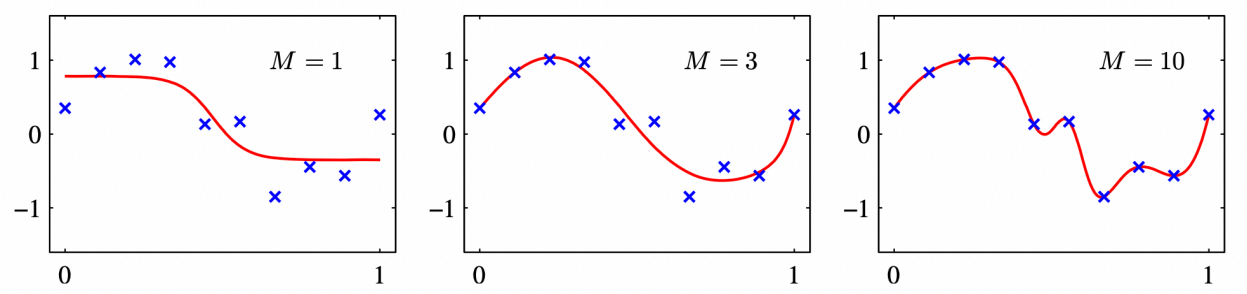


FIGURE 35. Examples of two-layer networks trained on 10 data points drawn from the sinusoidal data set (see Sec. 2.2). The graphs show the result of fitting networks having $M = 1$, 3 and 10 hidden units, respectively, by minimizing a sum-of-squares error function using a SGD algorithm. Figure excerpted from [Bis06].

4. Convolutional Neural Networks

4.1. Introduction. In this section, we discuss one of the most successful and architecture of feedforward deep neural networks for image classification, namely, the *convolutional neural networks* (CNN, or ConvNet). A few highlighing features of CNN are as follow:

1. CNN is a 'localized' version of the fully connected deep feedforward neural network (inspired by how animal visual cortex works).
2. CNN learns patterns in images that are 'shift-invariant', meaning that the location of a certain geometric pattern in the image matters less[11].
3. CNN not only to recognize patterns from images, but also to abstract such patterns. (e.g., first layer learns edges or colors, next layer learns larger curves and shapes, and so on.)

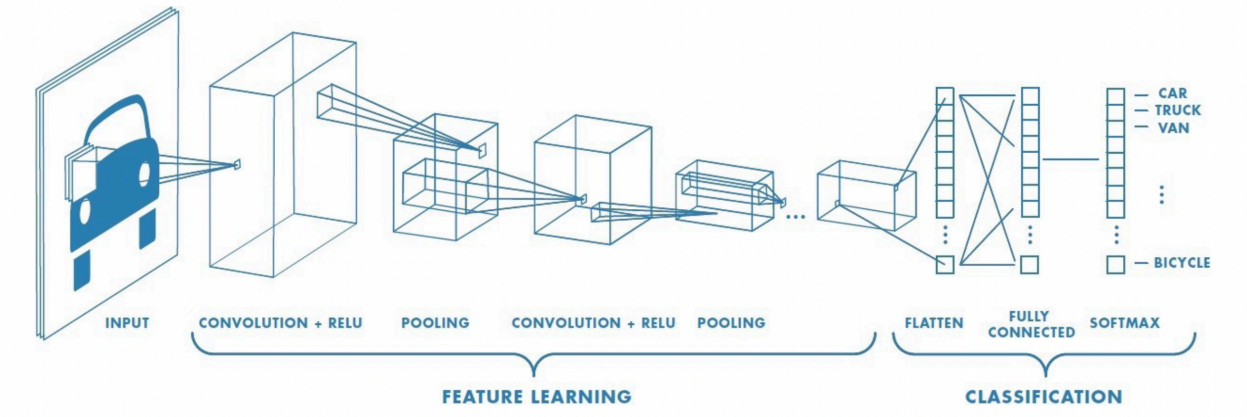


FIGURE 36. Architecture of a Convolutional Neural Network for image classification. Figure by S. Saha link

We first discuss a motivating example. Consider the now-familar MNIST handwritten digits image dataset. We have seen that regardless of what algorithm we use for classifying the digits, we usually get very high accuracy (e.g., see Figure 16, 17, 22 for binary classification with LR and PR, Figure 19 for multiclass classification with MLR). Since the images in the MNIST dataset are so well alinged within 28×28 frame, it only means that we can classify the images to the correct classes against variation within each class under such a good alignment. So what if the images are not perfectly alinged within the same frame, as in Figure 37? There, we 'padded' additional zeros aound the orignal image to purturb the

[11] Think about whether the usual FFNN or multiclass logistic regression have such abilities

alignment. In other words, we embedded each 28×28 image inside 48×48 zero matrices at uniformly randomly chosen locations. Do you think the classification algorithms we have developed so far would still work well after this perturbation?[12]

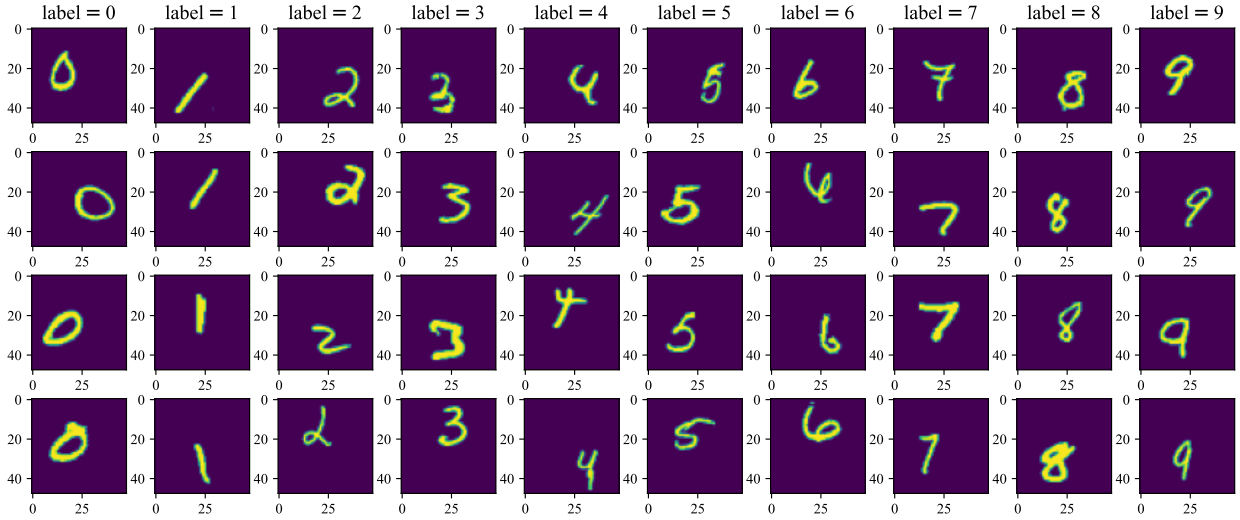


FIGURE 37. Examples of MNIST dataset of handwritten digits by Yann LeCun, Corinna Cortes, and Christopher J.C. Burges with random padding. Each 28×28 image is superimposed within a 48×48 zero matrix at uniformly chosen random location.

In Figure 38, we show the training and testing accuracies of using 2-layer neural network with 10 hidden nodes for classifing MNIST (five digits from 0 through 4) with random padding. We vary the 'thickness' of the padding, namely, when the thickness is r, it means that each 28×28 image is randomly embedded inside a zero matrx of shape $(28+r) \times (28+r)$. As we can see in Figure 38, the training accuracies are close to 1 but the testing accurcies drop quickly as we increase the padding thickness. This means our model overfits to some non-shift-invariant features. Can we overcome this shortcoming by increasing the number of layers in our model? Not quite, as we see in Figure 39 for 3-layer FFNN.

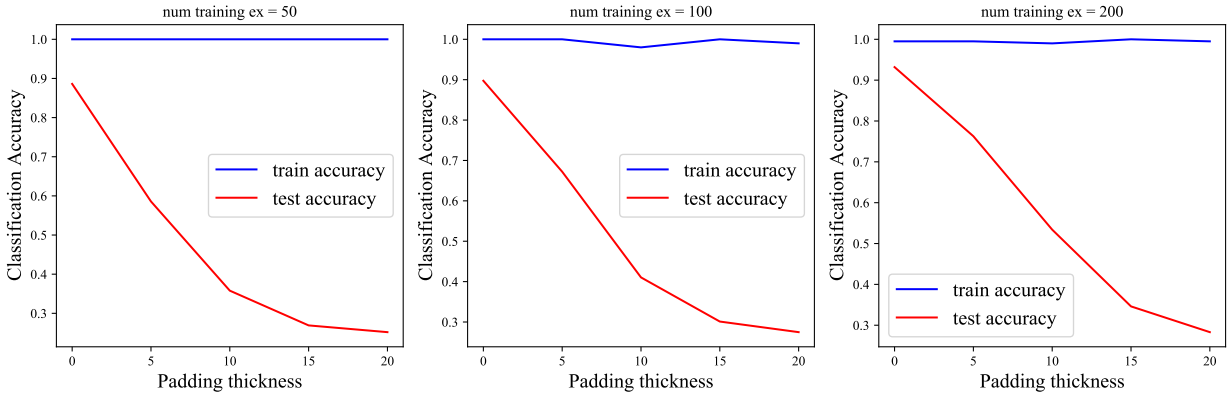


FIGURE 38. Classification accuracy of 2-layer FFNN for MNIST data with digits from 0 up to 4 with random padding with varying thickness. The number of hidden nodes is 10. When the thickness is r, it means that each 28×28 image is randomly embedded inside a zero matrx of shape $(28+r) \times (28+r)$.

[12] Certainly our brain doesn't care the location of the digits within a larger image.

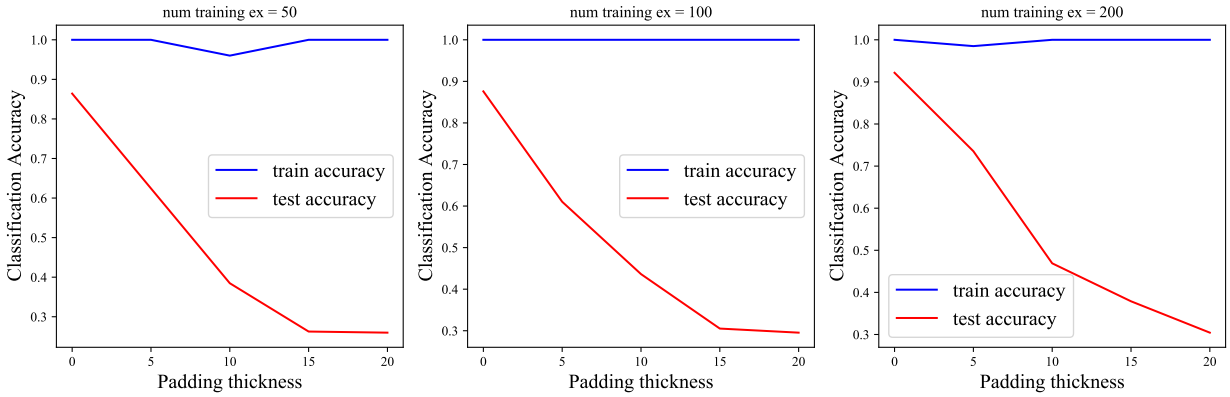


FIGURE 39. Classification accuracy of 3-layer FFNN for MNIST data with digits from 0 up to 4 with random padding with varying thickness. The number of hidden nodes are 10 for each hidden layer. When the thickness is r, it means that each 28×28 image is randomly embedded inside a zero matrx of shape $(28+r) \times (28+r)$.

Exercise 3.4.1. Reproduce figures similar to Figure 38 using (1) multiclass logistic regression; and (2) naive Bayes classifier. Conclude that whether these algorithms are able to learn shift-invariant features. (Ref: Jupyter Notebook – 'Classifying non-aligned MNIST images')

4.2. Model formulation. In order to formulate CNN, we need to introduce (1) convolutional layer and (2) pooling layer and how they work. Each 'layer' in a CNN is the following composition: convolutional with set of filters + ReLU + pool. Every time we complete such procedure, we learn more abstract and shift-invariant features from the previous layer. (See Figure 40 for an illustration[13])

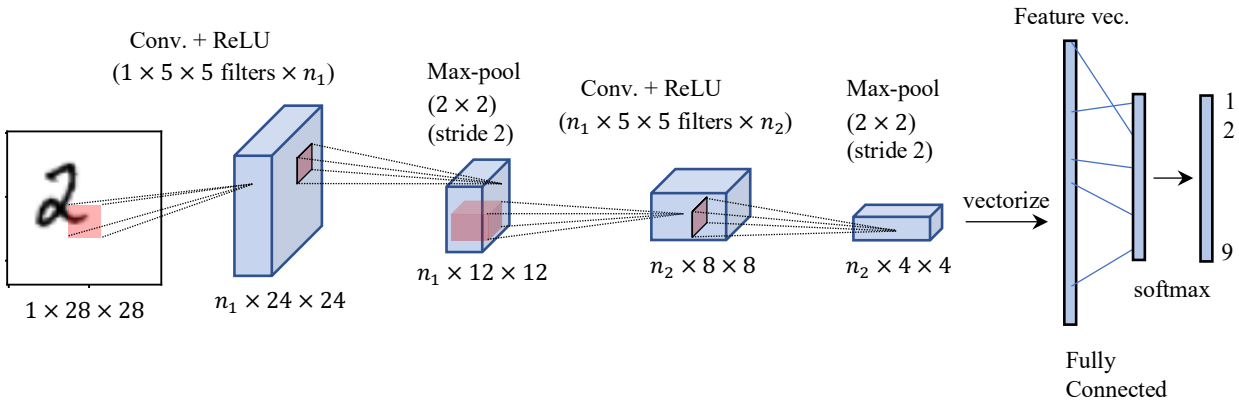


FIGURE 40. An example of CNN architecture for MNIST image classification. Convolution + ReLU activation followed by a max-pull layers are applied twice with to extract shift-invariant feature vector. Pass this into a multiclass logistic regression (fully connected layer + softmax) for classification.

4.2.1. *Convolutional layer.* First we explain how a convolutional layer works in CNN. Recall that in a fully connected neural network, the previous layer's activation (node state), say $\mathbf{z}$, is dot producted with each colums of the following weight matrix to produce the activation of each nodes in the following layer. In CNN, it is useful to note that

$$\text{Weight matrix in a CNN layer} = \text{Set of tensors (filters/kernels) } F_1, \dots, F_m \in \mathbb{R}^{c \times r \times r}, \tag{225}$$

[13]See also 'A Comprehensive Guide to Convolutional Neural Networks' by S. Saha

where the 'tensor depth' c should match the depth of the previous 'image'. For example, color images have shape $3 \times x \times y$ and monochrome images have shape $1 \times x \times y$. Each filter is supposed to capture a particular pattern *everywhere* in the image. We explain this in steps as follow:

1. Say we have a *filter* F of shape $1 \times 3 \times 3$. For an input X of shape $1 \times 3 \times 3$, the convolution $X * F$ of X with filter F is the sum of entrywise product of X with F.

$$\underbrace{\begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \end{bmatrix}}_{X} * \underbrace{\begin{bmatrix} f_{11} & f_{12} & f_{13} \\ f_{21} & f_{22} & f_{23} \\ f_{31} & f_{32} & f_{33} \end{bmatrix}}_{F} = \operatorname{sum}\left(\begin{bmatrix} x_{11}f_{11} & x_{12}f_{12} & x_{13}f_{13} \\ x_{21}f_{21} & x_{22}f_{22} & x_{23}f_{23} \\ x_{31}f_{31} & x_{32}f_{32} & x_{33}f_{33} \end{bmatrix} \right) = \sum_{i,j} x_{ij} f_{ij}. \tag{226}$$

For general depth c, the convolution of a filter F of shape $c \times r \times r$ on an input of shape $1 \times r \times r$ is a scalar given similarly as follow:

$$X * F = \sum_{i,j,k} X[i,j,k]\, F[i,j,k], \tag{227}$$

where the sum is over all entries (i,j,k).

2. If the input X is $1 \times 4 \times 4$, then we can choose four $1 \times 3 \times 3$ *patches*[14] inside X by *sliding*[15] a window (frame) of $1 \times 3 \times 3$. We can convolve each patch with F to get a scalar. So output of $X * F$ is a $1 \times 2 \times 2$ tensor.

$$\underbrace{\begin{bmatrix} x_{11} & x_{12} & x_{13} & x_{14} \\ x_{21} & x_{22} & x_{23} & x_{24} \\ x_{31} & x_{32} & x_{33} & x_{34} \\ x_{41} & x_{42} & x_{43} & x_{44} \end{bmatrix}}_{X} * \underbrace{\begin{bmatrix} f_{11} & f_{12} & f_{13} \\ f_{21} & f_{22} & f_{23} \\ f_{31} & f_{32} & f_{33} \end{bmatrix}}_{F} = \underbrace{\begin{bmatrix} X[0:3,0:3] * F & X[0:3,1:] * F \\ X[1:,0:3] * F & X[1:,1:] * F \end{bmatrix}}_{\text{In python slicing}}. \tag{228}$$

3. If we have a $c \times r \times r$ filter F and $c \times a \times b$ input X, then the result of convolution $X * F$ with *stride* 1 is a $1 \times (a-r+1) \times (b-r+1)$ tensor:

$$(X * F)[0,j,k] = X[:, i:i+r, j:j+r] * F. \tag{229}$$

Namely, the (i,j) entry of $X * F$ is convolution of the (i,j) patch $X[:, i:i+r, j:j+r] \in \mathbb{R}^{c \times r \times r}$ with F.

4. If we have a $c \times r \times r$ filter F and $c \times a \times b$ input X, then the result of convolution $X * F$ with *stride* s is a $1 \times (\frac{a-r}{s}+1) \times (\frac{b-r}{s}+1)$ tensor:

$$(X * F)[0,j,k] = X[:, is:is+r, js:js+r] * F. \tag{230}$$

Namely, the (i,j) entry of $X * F$ is convolution of the (i,j) patch $X[:, is:is+r, js:js+r] \in \mathbb{R}^{c \times r \times r}$ with F.

4. The result of the convolutional layer with input $X \in \mathbb{R}^{c \times a \times b}$ and filters $F_1, \ldots, F_m \in \mathbb{R}^{c \times r \times r}$ with stride s is

$$X * (\text{filters } F_1, \ldots, F_m) = [X * F_1, \ldots, X * F_m] \in \mathbb{R}^{m \times (\frac{a-r}{s}+1) \times (\frac{b-r}{s}+1)}, \tag{231}$$

which we think of as a 'depth' m image of size $(\frac{a-r}{s}+1) \times (\frac{b-r}{s}+1)$.

See Figure 40 for an illustration.

[14] Square sub-matrices of consecutive rows and colums

[15] We may want to skip every some rows and columns in extracting patches from X to avoid too much overlap between patches and/or reduce the output of convolution more rapidly. The number of rows/columns we slide over is called *stride*. For stride= 1, we sample the maximum number of patches.

4.2.2. *Pooling layer.* In CNN, a *pooling* is used to reduce the size of the previous layer's output by either keeping the most dominant cell value in each patch (called 'max-pool') or averaging every patch into a single scalar (called 'avg-pool'). We introduce these concepts with the following examples.

1. 2×2 pooling on $1\times 2\times 2$ input X:

$$\text{pool}\left(X=\begin{bmatrix}x_{11} & x_{12}\\ x_{21} & x_{22}\end{bmatrix}\right)=\begin{cases}\max(X)=\max\{x_{ij}\,|\,1\le i,j\le 2\} & \text{for max-pool}\\ \text{avg}(X)=(1/4)\sum_{1\le i,j\le 2}x_{ij} & \text{for avg-pool}\end{cases}. \tag{232}$$

2. 2×2 pooling on $1\times 4\times 4$ input X with stride 2:

$$\text{pool}\left(X=\begin{bmatrix}x_{11} & x_{12} & x_{13} & x_{14}\\ x_{21} & x_{22} & x_{23} & x_{24}\\ x_{31} & x_{32} & x_{33} & x_{34}\\ x_{41} & x_{42} & x_{43} & x_{44}\end{bmatrix}\right)=\begin{bmatrix}\text{pool}\begin{pmatrix}x_{11} & x_{12}\\ x_{21} & x_{22}\end{pmatrix} & \text{pool}\begin{pmatrix}x_{13} & x_{14}\\ x_{23} & x_{24}\end{pmatrix}\\ \text{pool}\begin{pmatrix}x_{31} & x_{32}\\ x_{41} & x_{42}\end{pmatrix} & \text{pool}\begin{pmatrix}x_{33} & x_{34}\\ x_{43} & x_{44}\end{pmatrix}\end{bmatrix}\in\mathbb{R}^{1\times 2\times 2}. \tag{233}$$

3. 2×2 pooling on $1\times 4\times 4$ input X with stride 1:

$$\text{pool}\left(X=\begin{bmatrix}x_{11} & x_{12} & x_{13} & x_{14}\\ x_{21} & x_{22} & x_{23} & x_{24}\\ x_{31} & x_{32} & x_{33} & x_{34}\\ x_{41} & x_{42} & x_{43} & x_{44}\end{bmatrix}\right)=\left[\text{pool}\left(X[i:i+2,j:j+2];0\le i,j\le 2\right)\right]\in\mathbb{R}^{1\times 3\times 3}. \tag{234}$$

4. 2×2 pooling on $c\times a\times b$ input X with stride 1:

$$\text{pool}(X)=\left[\text{pool}\left(X[d,i:i+2,j:j+2];0\le d<c,0\le i\le a-2,0\le j\le b-2\right)\right] \tag{235}$$

$$\in\mathbb{R}^{c\times(a-1)\times(b-1)}. \tag{236}$$

5. $r\times r$ pooling on $c\times a\times b$ input X with stride s:

$$\text{pool}(X)=\left[\text{pool}\left(X[d,is:is+r,j:j+r];0\le d<c,0\le i\le\tfrac{a-r}{s},0\le j\le\tfrac{b-r}{s}\right)\right] \tag{237}$$

$$\in\mathbb{R}^{c\times(\frac{a-r}{s}+1)\times(\frac{b-r}{s}+1)}. \tag{238}$$

Exercise 3.4.2. Consider the CNN architecture shown in Figure 40. Suppose we use the following general choices of convolutional and pooling layers:

Conv1: n_1 filters of shape $1\times r\times r$ with stride s_f
Conv2: n_2 filters of shape $n_1\times r\times r$ with stride s_f
maxpool (both): $d\times d$ maxpool with stride s_p

Also assume that input image has dimension $1\times a\times b$. Then compute the dimension of the feature vector that goes into the first dense layer as a function of n_1,n_2,r,d,s_f,s_p,a,b.

4.3. Training. Training a CNN is similar to that for the feedforward neural networks. Namely, we use stochastic gradient descent in order to find the optimal parameters (that is, all filters, weight matrices, and biases). In order to compute the gradients of the parameters, we first backpropagate a chosen training image, compute the error at the output layer, and then backpropagate it all the way down to the input layer. This will give the gradients of the loss with respect to node states, which we can use to compute the gradients of filters and weight matrices by chain rule.

In order to execute backpropagation for CNN, we need to perform the followng four kinds of backpropagations:

1. Backprop through a dense layer — Same as in FFNN
2. Backprop through a maxpool layer — Assign errors to the 'winning pixels'
3. Backprop through a ReLU layer — Set the errors zero at coordinates where the input activation is negative
4. Backprop through a convolutional layer –Details omited. (Ref: Backprop. in CNN)

Computation of gradients for training a CNN

1. Forward propagate the input all the way to the output layer and compute the activation and state of every node.
2. Compute the errors at the output nodes using (205)-(208).
3. Backprogagate the errors in the output layer to the hidden layer and compute the errors at hidden nodes, using (209).

4.4. Example: MNIST + padding. Next, we give some examples of using CNN for MNIST image classification. We are going to use MNIST with random padding as shown in Figure 37. The architecture of CNN we use here is as follow:

$$\text{Input} \to \underbrace{\text{Conv1}}_{\text{filter1}\in\mathbb{R}^{n_1\times 1\times 5\times 5}} + \text{ReLU} + \text{maxpool} \to \underbrace{\text{Conv2}}_{\text{filter1}\in\mathbb{R}^{n_2\times n_1\times 5\times 5}} + \text{ReLU} + \text{maxpool} \tag{239}$$

$$\to \underbrace{\text{Dense1}}_{W1} + \text{ReLU} \to \underbrace{\text{Dense2}}_{W2} + \text{Softmax} \tag{240}$$

$$\to \text{Output} \tag{241}$$

The paramters we need to optimize are n_1 filters of shape 1×5 in the first convolutional layer, n_2 filters of shape $n_1\times 5\times 5$ in the second convolutional layer, and the two weight matrices for the dense layers. For all pooling layer, we use 2×2 maxpool with stride 2. All convolutions use stride 1.

In Figure 41, we show the training and testing accuracies of using the above CNN in (241) for classifing MNIST (five digits from 0 through 4) with random padding. Again we vary the thickness of the padding, namely, when the thickness is r, it means that each 28×28 image is randomly embedded inside a zero matrx of shape $(28+r)\times(28+r)$. The overall trend is similar as for FFNN in Figure 38: The training accuracies are close to 1 but the testing accurcies drop quickly as we increase the padding thickness. However, the test accuracy is about 10% higher with 100 training examples and 20% higher with 200 training examples than those for FFNN. This means that the above CNN is able to learn shift-invariant features to some extent, at least much better than FFNN.

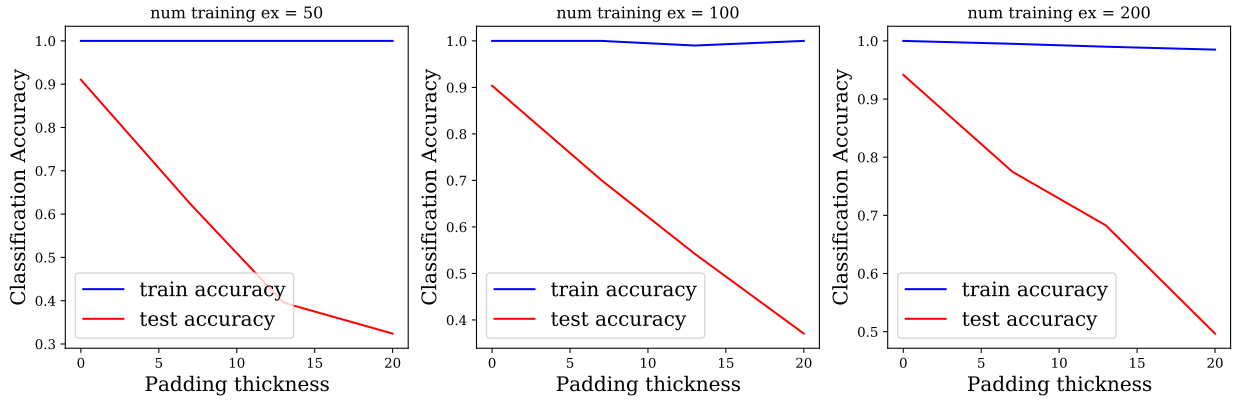


FIGURE 41. Classification accuracy of 3-layer FFNN for MNIST data with digits from 0 up to 4 with random padding with varying thickness. The number of hidden nodes are 10 for each hidden layer. When the thickness is r, it means that each 28×28 image is randomly embedded inside a zero matrx of shape $(28+r)\times(28+r)$.

Exercise 3.4.3. Reproduce Figures 39 and 41 with larger number of training examples: 300, 500, 1000. Compare the performance of FFNN and CNN using your plots. (Ref: Jupyter notebook on FFNN and on CNN)

Exercise 3.4.4. Produce a plot similar to Figure 34 for the CNN model in (241), where the horizontal axis correspond to the shared number $n_1 = n_2 \in \{2,5,10,20\}$ of filters in each convolutional layer. Use

padding thickness 10 and keep all the other settings the same as in Figure 41. What do you think the optimal number of filters to use is? (Ref: Jupyter notebook on CNN)

In Figure 42, we show 8 filters of shape $1 \times 5 \times 5$ in the first convolutional layer that are learned for a binary MNIST classification (digits 0 and 1). Can you imagine that kind of pattern that each filter is suppose to pick up from the image?

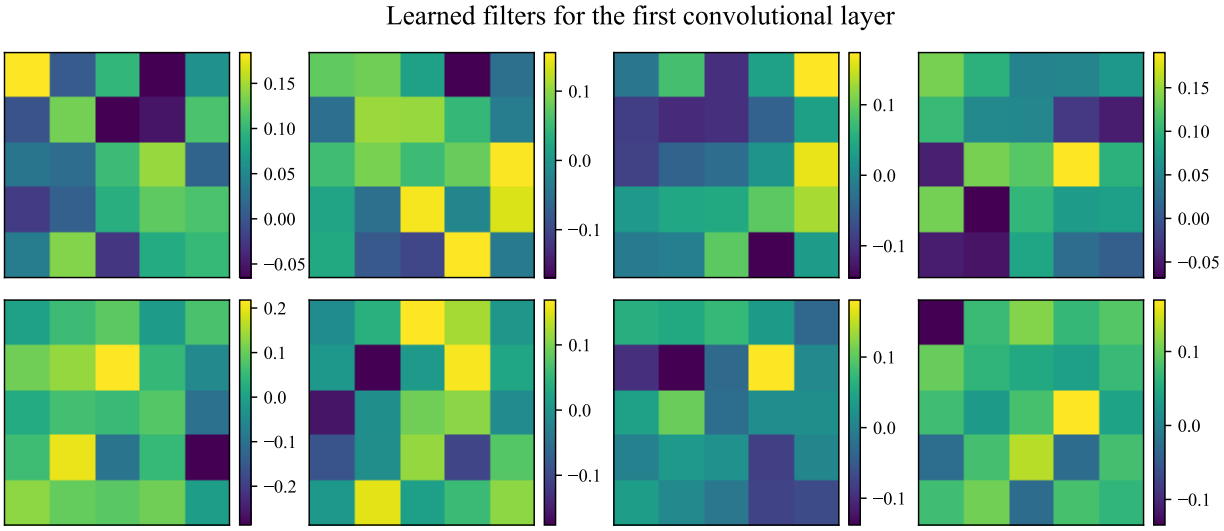


FIGURE 42. 8 filters learned for a binary MNIST classification (digits 0 and 1) using the CNN model in (241).

Exercise 3.4.5. Train the CNN in (241) for MNIST classification with digits 0 and 1. Pick two images of each digit. Using the trained model, plot the image after the first and the second convolutional layers for each of the four chosen images. (Ref: Jupyter notebook on CNN) (Hint: Modify the 'predict' function in src/CNN.py so that it also returns outputs of each convolutional layer.)

CHAPTER 4

Matrix Factorization and Dictionary Learning

In this chaper, we introduce the problem of matrix factorization, which is a powerful unifying framework for a number of unsupervised machine learning problems such as dimension reduction or dictionary learning. Two main techniques we will be learning are *nonnegative matrix factorization* (NMF) and *principal component analysis* (PCA).

1. Introduction

1.1. Dictionary Learning. *Dictionary-learning* algorithms are machine-learning techniques that are able to learn interpretable latent structures of complex data sets and are applied regularly in the data analysis of text and images. Such algorithms usually consist of two steps. First, one samples a large number of structured subsets of a data set (e.g., square patches of an image or collections of a few sentences of a text); Second, one finds a set of basis elements such that taking a *nonnegative* linear combination of them can successfully approximate each of the sampled mesoscale patches. Such a set of basis elements is called a *dictionary*, and one can interpret each basis element as a latent structure of the data set. As an example, consider the image of the artwork CYCLE by M. C. Escher in Figure 43**a**. We first sample 10,000 square patches of 21×21 pixels, and we then use a nonnegative matrix factorization (NMF) [LS99] algorithm to find a dictionary with $r = 25$ square patches (see Figure 43**a**). Each element of the learned dictionary describes a latent shape in the image.

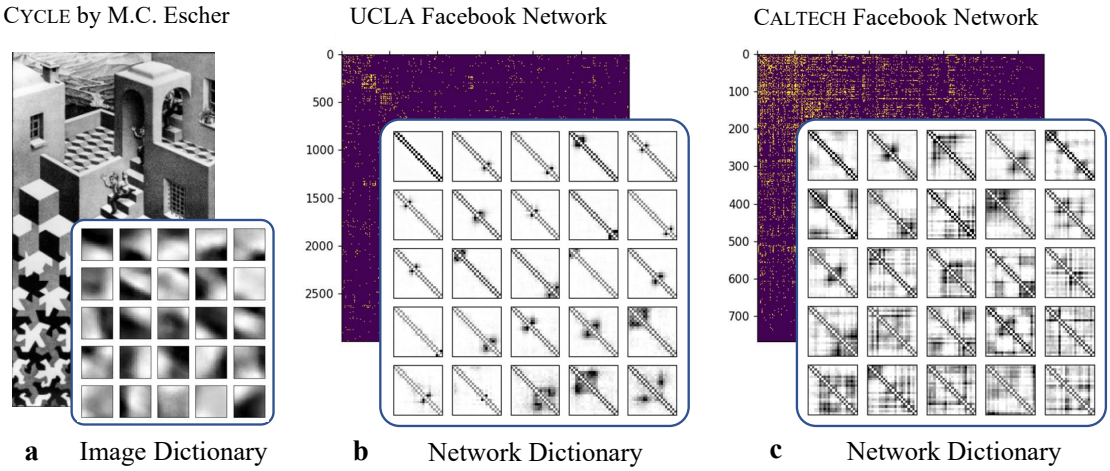


FIGURE 43. Illustration of matrix factorization. Each column of the data matrix is approximated by the linear combination of the columns of the dictionary matrix with coefficients given by the corresponding column of the code matrix. Figure excerpted from [LKVP21].

1.2. Matrix Factorization. *Matrix factorization* is one of the fundamental tools in dictionary learning problems. Given a large data matrix $\mathbf{X}$, can we find some small number of "dictionary vectors" so that we can represent each column of the data matrix as a linear combination of dictionary vectors? More precisely, given a data matrix $\mathbf{X} \in \mathbb{R}^{d \times n}$ and a rank parameter $r \in \mathbb{N}$, we wish to factorize X into the

product of $\mathbf{W} \in \mathbb{R}^{d\times r}$ and $\mathbf{H} \in \mathbb{R}^{r\times n}$ by solving the following optimization problem

$$\text{(Matrix Factorization)} \qquad \inf_{\mathbf{W}\in\mathbb{R}^{d\times r}, \mathbf{H}\in\mathbb{R}^{r\times n}} \|\mathbf{X} - \mathbf{W}\mathbf{H}\|_F^2. \tag{242}$$

Here $\mathbf{W}$ is called the *dictionary* and $\mathbf{H}$ is the *code* of data $\mathbf{X}$ using dictionary $\mathbf{W}$. A solution of such matrix factorization problem is illustrated in Figure 44.

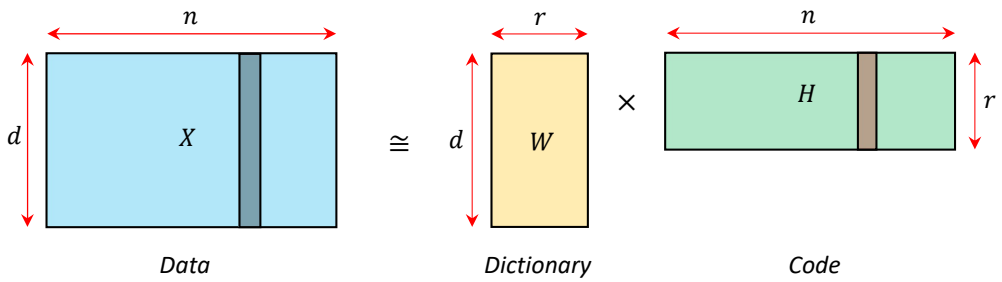


FIGURE 44. Illustration of matrix factorization. Each column of the data matrix is approximated by the linear combination of the columns of the dictionary matrix with coefficients given by the corresponding column of the code matrix. Figure excerpted from [LNB20].

Recall the method of least squared we studied for polynomial regression: (See also Exercise 1.2.1).

$$\hat{\mathbf{H}} = \operatorname*{arg\,min}_{\mathbf{H}\in\mathbb{R}^{r\times n}} \|\underbrace{\mathbf{X}}_{\text{given}} - \underbrace{\mathbf{W}}_{\text{given}}\underbrace{\mathbf{H}}_{\text{TBD}}\|_F^2 + \lambda\|\underbrace{\mathbf{H}}_{\text{TBD}}\|_F^2. \tag{243}$$

One can think of (243) as a matrix factorization problem where the first factor (dictionary) is known and we are only looking for the code $\mathbf{H}$. Becase of this reason, we also call (243) as a *coding* problem, which is also called a *sparse coding* problem if the regularizer λ is positive[1].

While the optimization problem (242) is an *unconstrained* problem in the sense that we are free to choose any dictionary $\mathbf{W} \in \mathbb{R}^{d\times r}$ and code $\mathbf{H} \in \mathbb{R}^{r\times n}$, We will be more interested in its *constrained* versions, in particular using nonnegativity constraints. Consider the following modifications:

$$\text{(Dictionary Learning)} \qquad \inf_{\mathbf{W}\in\mathbb{R}^{d\times r}, \mathbf{H}\in\mathbb{R}^{r\times n}_{\geq 0}} \|\mathbf{X} - \mathbf{W}\mathbf{H}\|_F^2, \tag{244}$$

$$\text{(NMF)} \qquad \inf_{\mathbf{W}\in\mathbb{R}^{d\times r}_{\geq 0}, \mathbf{H}\in\mathbb{R}^{r\times n}_{\geq 0}} \|\mathbf{X} - \mathbf{W}\mathbf{H}\|_F^2. \tag{245}$$

In (244), we imposed an additional constraint that the code $\mathbf{H} \in \mathbb{R}^{r\times n}$ should only take nonngative entries. This allows us to interpret the columns of dictionary $\mathbf{W}$ as 'features' that directly contribute to composing the columns of $\mathbf{X}$ (samples)[2]. We can go even further and also impose the same nonnegativity constraint on the dictioanry matrix $\mathbf{W} \in \mathbb{R}^{r\times n}$ as in (245). This problem is called *nonnegative matrix factorization* (NMF), which is an extremely popular technique in modern machine learning for its high interpretibility and mathematical simplicity.

1.3. Applications in Image processing. Before we discuss how to solve matrix factorization problems, we give some examples of matrix factorization in the context of image processing through image dictionary learning.

Suppose we are given an image, and we are intrested in the following *unsupervised problem* of learning 'essential features in all of its $k \times k$ patches'. In other words, we would like to know what important features are there that if we cut out a large number of $k \times k$ square patches from the original image.

[1] It is more effective to use an L_1-regularizer $\lambda\|\mathbf{H}\|_1$ for the purpose of sparse coding. This is also closely related to (Lagrangian version of) the popular algorithm of LASSO in statistics.

[2] A general formulation of dictionary learning may also impose a convex constraint $\mathscr{C} \subseteq \mathbb{R}^{d\times r}$ for dictionary $\mathbf{W}$ along with the nonnegativity constraint on the code $\mathbf{H}$.

Of course, still this problem is not precisely defined. Even though we can state a precise optimization problem for it, let's instead describe the algorithm first and then see what exactly we are solving.

As we have briefly mentioned in Section (1.1), for dictionary learning we first do patch sampling and then feature extraction. For patch sampling, we get a large number N (e.g., N =10K) of $k \times k$ (e.g., 21×21) square patches from the original image. We will vectorize each such patch as a k^2 dimensional column vector and form a data matrix $\mathbf{X}$ of shape $k^2 \times N$. We then use an algorithm[3] for NMF to get a dictionary matrix $\mathbf{W} \in \mathbb{R}^{k^2 \times r}$ and code matrix $\mathbf{H} \in \mathbb{R}^{r \times N}$ with $r = 25$. Each of the 25 columns of $\mathbf{W}$ are vectorized versions of the extracted features or 'basis shapes'. Reshaping these k^2 dimensional column vectors into $k \times k$ square form, we obtain the image dictionary of size 25, as shown in Figure 45.

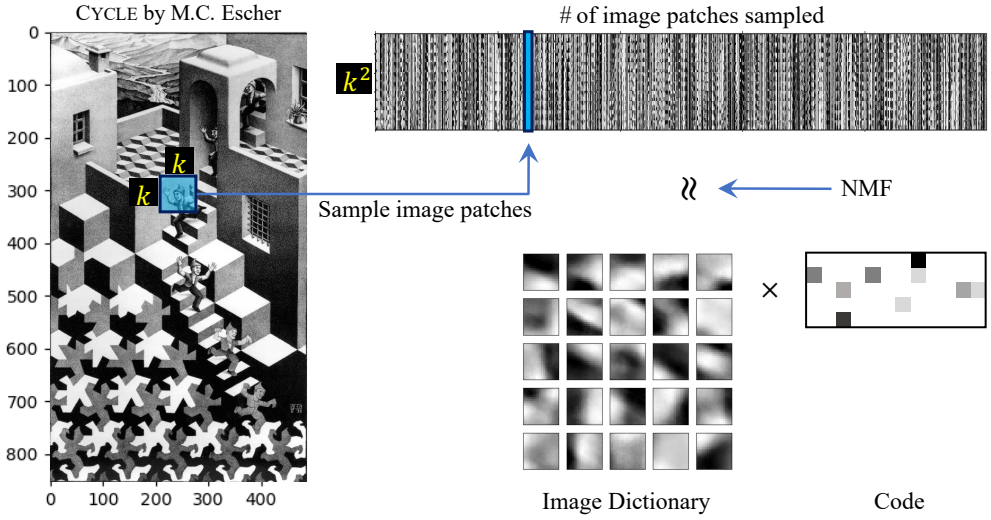


FIGURE 45. Examples of image dictionary learning by patch sampling and NMF.

After we learn a set of dictionary images from a given image, we can use them to reconstruct the original image. Moreover, if the given image is corrupted with some noise, then this reconstruction using learned image dictionary can be used for image denoising. Figure 46 gives an example of image dictionary learning and image denoising with noise separation. We omit details here.

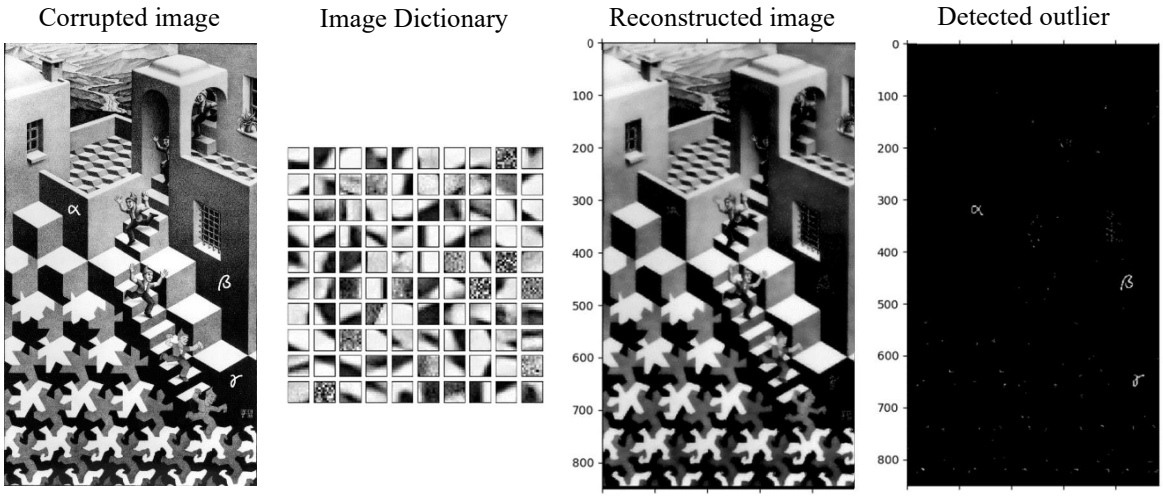


FIGURE 46. Examples of image dictionary learning by patch sampling and NMF. The learned image dictionary is used for denoising the original image corrupted with noises 'α', 'β', and 'γ'.

[3]We will discuss algorithms for solving matrix factorization problems later

2. Algorithms for Matrix Factorization

In this section, we discuss algorithms for solving matrix factorization problems. First, fix a data matrix $\mathbf{X} \in \mathbb{R}^{d\times n}$ and integer $r \geq 1$. Define the following loss function

$$\ell(\mathbf{W},\mathbf{H}) := \|\mathbf{X}-\mathbf{W}\mathbf{H}\|_F^2. \tag{246}$$

We would like to minimize the above loss function jointly in $\mathbf{W} \in \mathbb{R}^{d\times r}$ and $\mathbf{H} \in \mathbb{R}^{r\times n}$, possibly with convex constraints (e.g., nonnegativitity) depending on the situation.

Exercise 4.2.1. Show the following.

(i) Show that the function $\mathbf{H} \mapsto \|\mathbf{X}-\mathbf{W}\mathbf{H}\|_F^2$ is convex.

(ii) Show that the function $\mathbf{W} \mapsto \|\mathbf{X}-\mathbf{W}\mathbf{H}\|_F^2 = \|\mathbf{X}^T-\mathbf{H}^T\mathbf{W}^T\|_F^2$ is convex.

(iii) Show that the function $(\mathbf{W},\mathbf{H}) \mapsto \|\mathbf{X}-\mathbf{W}\mathbf{H}\|_F^2$ is non-convex.

According to Exercise 4.2.1, this is a non-convex optimization problem so it is difficult to find its global minima. However, if we fix one of the factors, then ℓ is convex with respect to the other factor (see Exercise 4.2.1), which is in fact a least squares problem (see Exercise 1.2.1). Hence one can iteratively fix one factor and optimize the other, each time solving a least squares problem. This is called *Alternating Least Squares* (ALS) algorithm for matrix factorizaiton, which we state in Algorithm 2 for NMF.

Algorithm 2 Alternating Least Squares for Matrix Factorization

Input: $\mathbf{X} \in \mathbb{R}^{d\times n}_{\geq 0}$ (data matrix); $r \geq 1$ (rank parameter); $[\mathbf{W}_0,\mathbf{H}_0] \in \mathbb{R}^{d\times r}_{\geq 0} \times \mathbb{R}^{r\times n}_{\geq 0}$ (initial estimate); N (number of iterations)
  for $n = 1,\dots,N$ **do**:
    $\mathbf{H}_n \leftarrow \arg\min_{\mathbf{H}\in\mathbb{R}^{r\times n}_{\geq 0}} \|\mathbf{X}-\mathbf{W}_{t-1}\mathbf{H}\|_F^2$
    $\mathbf{W}_n \leftarrow \arg\min_{\mathbf{W}\in\mathbb{R}^{d\times r}_{\geq 0}} \|\mathbf{X}-\mathbf{W}\mathbf{H}_n\|_F^2$
  end for
output: $[\mathbf{W}_N,\mathbf{H}_N]$.

The ALS algorithm in Algorithm 2 is a special case of *block coordinate descent* (BCD) algorithms, which is to iteratively optimize the objective function with respect to a block coordinate (group of individual coordinates). The general algorithm is stated in Algorithm 3. For more backgrounds and its convergence properties, see [Lyu20].

Algorithm 3 Block Coordinate Descent

Requirement: $\boldsymbol{\theta} = [\theta_1,\dots,\theta_m] \mapsto f(\theta_1,\dots,\theta_m)$ is convex in each block-coordinate θ_i, $i = 1,\dots,m$.
Input: $\boldsymbol{\theta}_0 = (\theta_0^{(1)},\cdots,\theta_0^{(m)}) \in \Theta^{(1)}\times\cdots\times\Theta^{(m)}$ (initial estimate); N (number of iterations)
  for $n = 1,\dots,N$ **do**:
    Update estimate $\boldsymbol{\theta}_n = [\theta_n^{(1)},\cdots,\theta_n^{(m)}]$ by
      For $i = 1,\cdots,m$ **do**:

$$\theta_n^{(i)} \in \arg\min_{\theta\in\Theta^{(i)}} f\left(\theta_n^{(1)},\cdots,\theta_n^{(i-1)},\theta,\theta_{n-1}^{(i+1)},\cdots,\theta_{n-1}^{(m)}\right); \tag{247}$$

      end for
  end for
output: $\boldsymbol{\theta}_N$

Exercise 4.2.2 (Nonnegative Coding with L_1 regularizer)**.** Suppose we have matrices $\mathbf{X} \in \mathbb{R}^{d\times n}$ and $\mathbf{W} \in \mathbb{R}^{d\times r}$. We seek to find a matrix $\hat{\mathbf{H}} \in \mathbb{R}^{r\times n}_{\geq 0}$ where

$$\hat{\mathbf{H}} = \arg\min_{\mathbf{H}\in\mathbb{R}^{r\times n}_{\geq 0}} \left(\ell(\mathbf{H}) := \|\mathbf{X}-\mathbf{W}\mathbf{H}\|_F^2 + \lambda_1\|\mathbf{H}\|_1 + \lambda_2\|\mathbf{H}\|_F^2\right). \tag{248}$$

Here $\lambda \geq 0$ is called the L_1-regularization parameter. (This is an instance of *constrained* quadratic optimization problem.) (c.f. The use of L_1 regularizer instead of L_2 as in Exercise 1.2.1 is more effective to increase the 'sparsity' of the solution, meaning that it will have less nonzero components.)

(i) Show that (use Exercise 1.2.2), denoting $\operatorname{sgn}(\mathbf{H}) = \mathbf{1}(\mathbf{H} \geq 0) - \mathbf{1}(\mathbf{H} < 0)$,

$$\frac{\partial}{\partial \mathbf{H}} \ell = 2\mathbf{W}^T(\mathbf{W}\mathbf{H} - \mathbf{X}) + \lambda_1 \operatorname{sgn}(\mathbf{H}) + 2\lambda_2 \mathbf{H}, \qquad \frac{\partial^2}{\partial \mathbf{H}^2} \ell = 2(\mathbf{W}^T\mathbf{W} + \lambda_2 \mathbf{I}). \tag{249}$$

(ii) From **(i)**, conclude that the quadratic function $\mathbf{H} \mapsto \ell(\mathbf{H})$ is convex, and and hence its every critical point is a local minimum. It follows that its every critical over the convex constraint set $\mathbb{R}^{r\times n}_{\geq 0}$ is a global minimum (but not necessarily unique). (See Ref)

Exercise 4.2.3. Give a psuedocode of a Block Coordinate Descent algorithm for the following matrix factorization problem

$$\inf_{\mathbf{W}\in\mathbb{R}^{d\times r}_{\geq 0},\, \mathbf{H}\in\mathbb{R}^{r\times n}_{\geq 0}} \|\mathbf{X} - \mathbf{W}\mathbf{H}\|_F^2 + a_0\|\mathbf{H}\|_1 + a_1\|\mathbf{W}\|_1 + a_2\|\mathbf{W}\|_F \tag{250}$$

The algorithm should be similar to the Alternating Least Squares (ALS) in Algorithm 2, but the subproblems of finding $\mathbf{H}_n$ and $\mathbf{W}_n$ should be more detailed by using gradient descent algorithms. (Remark: You may use the gradient given in Exercise 4.2.2. (Hint: The full algorithm is implemented in Jupyter Notebook))

3. Applications of Matrix Factorization

In this section, we apply matrix factorization (MF) to various image and text datasets for the purpose of dictionary learning. See Jupyter Notebook for the experiments given in this section.

3.1. MNIST images. We first apply MF to MNIST dataset to learn 25 dictionary images. There are total four ways to put nonnegativity constriants to $\mathbf{W}$ and $\mathbf{H}$. In the unconstrained case where $\mathbf{W}$ and $\mathbf{H}$ have no constraints, the basis images (see Figure 47 left) themselves can take negative values and they are supposed to span the original MNIST images by using any linear combination. Next, if we give nonnegativity constraint on the code $\mathbf{H}$, then the basis images (see Figure 47 middle) are still allowed to take negative pixel values but they can only additively combined to span original images. In this case, pixel values in dictionary images can still cancel each other when spanning the original image.

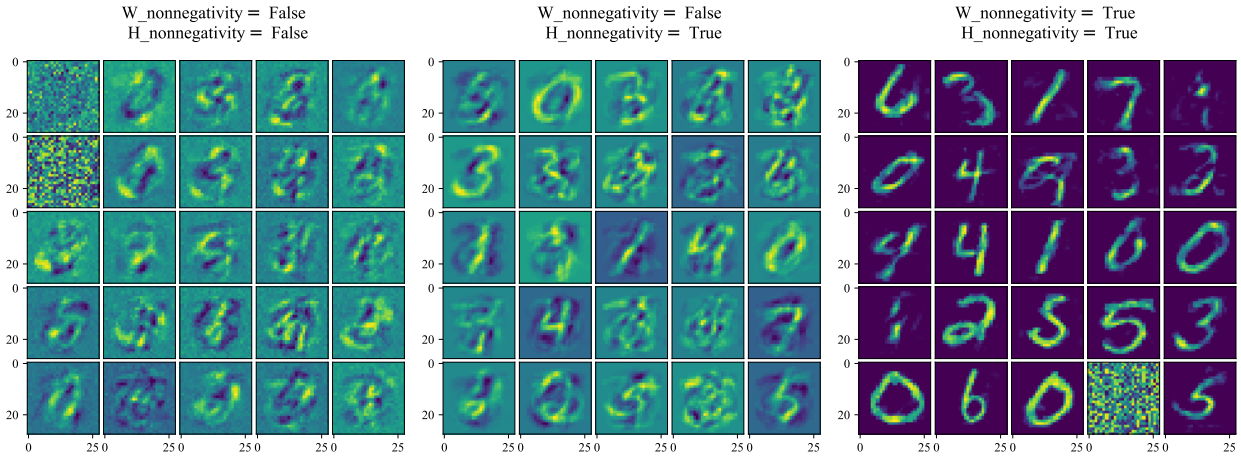


FIGURE 47. Examples of 25 dictionary images learned from MNIST dataset by Matrix Factorization with various nonnegativity constraints.

Lastly, if we constrain both $\mathbf{H}$ and $\mathbf{W}$ to be nonnegative, then there is absolutely no cancelation in any linear approximation of the original images by the basis images. Hence the basis images in this case

(see Figure 47) can only have positive pixel values that really contribute to approximating the original images. In other words, the features we see in the dictionary images are true features of the original images. This is the interpretibility of NMF that made it so popular in the machine learning and signal processing community [LS99].

In Figure 48, we show similar experiments on MNIST with random padding of thickness 20. We see similar patterns in dictionary images depending on the constraints as in Figure 47.

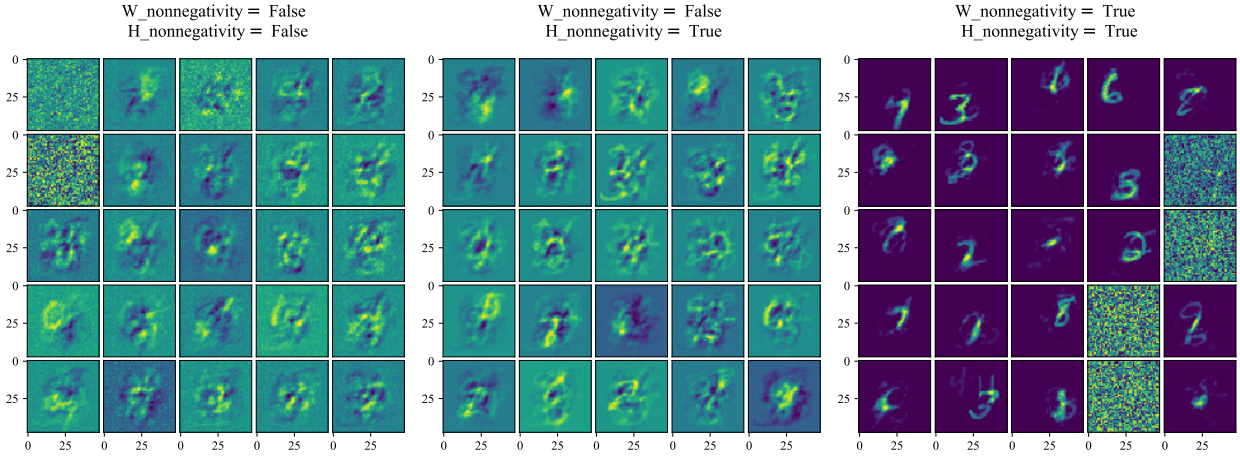


FIGURE 48. Examples of 25 dictionary images learned from MNIST dataset by Matrix Factorization with various nonnegativity constraints. Original 28×28 images are padded with zeros randomly so that they are 48×48 images.

3.2. Olivetti face images. The Olivetti face dataset contains a set of face images taken between April 1992 and April 1994 at AT&T Laboratories Cambridge. There are ten different images of each of 40 distinct subjects. For some subjects, the images were taken at different times, varying the lighting, facial expressions (open / closed eyes, smiling / not smiling) and facial details (glasses / no glasses). All the images were taken against a dark homogeneous background with the subjects in an upright, frontal position (with tolerance for some side movement) [SH94]. See Figure 49 for some sample images from this dataset.

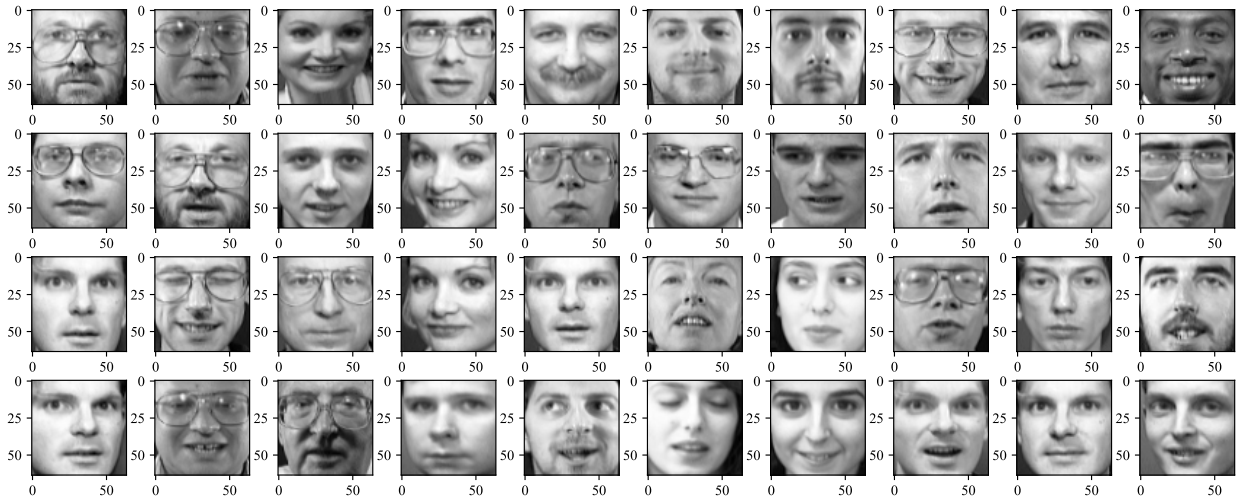


FIGURE 49. Sample images from the Olivetti face dataset

We perform a similar dictionary learning experiment for the Olivetti face dataset in Figure 50. We see similar patterns in dictionary images depending on the constraints as in Figure 47. Note that the "basis faces" learned by NMF (Figure 50 right) are not only the most interpretible, but also have the feature of *localization*. This ability of being able to learn localized features was first reported in [LS99] in 1999 by Lee and Seung. Later in 2004, Hoyer [Hoy04] further invesitgation on promoting such localized dictionary by using a particular sparseness regularizer.

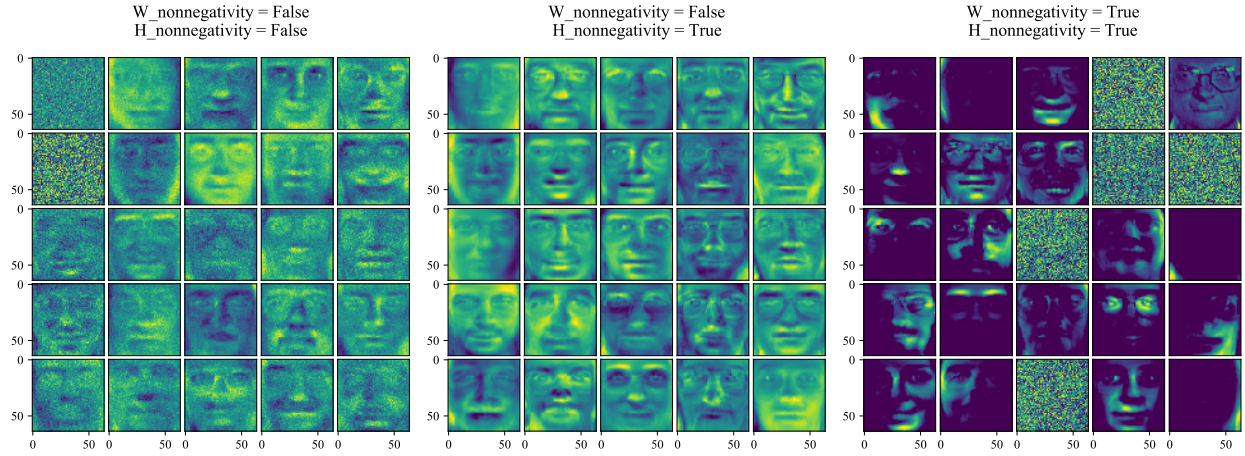


FIGURE 50. Examples of 25 dictionary images learned from the Olivetti face dataset by Matrix Factorization with various nonnegativity constraints.

Below in Figure 51, we discuss the effect of using regularizers on the dictionary matrix $\mathbf{W}$ in NMF. We use basic $L_1 + L_2$ regularizer for $\mathbf{W}$, which means that we are solving the following optimization problem in (250). The ALS algorithm for solving (250) is similar to Algorithm 2. In fact, the update for $\mathbf{H}$ is the same, and for updating $\mathbf{W}$, we need to solve the following coding problem:

$$\mathbf{W}_n \leftarrow \underset{\mathbf{W}\in\mathbb{R}^{d\times r}_{\geq 0}}{\arg\min} \|\mathbf{X} - \mathbf{W}\mathbf{H}_n\|_F^2 + a_1\|\mathbf{W}\|_1 + a_2\|\mathbf{W}\|_F^2. \tag{251}$$

Note that (251) is still a (constrained) convex optimization problem that can be solved iteratively by, e.g., gradient descent[4].

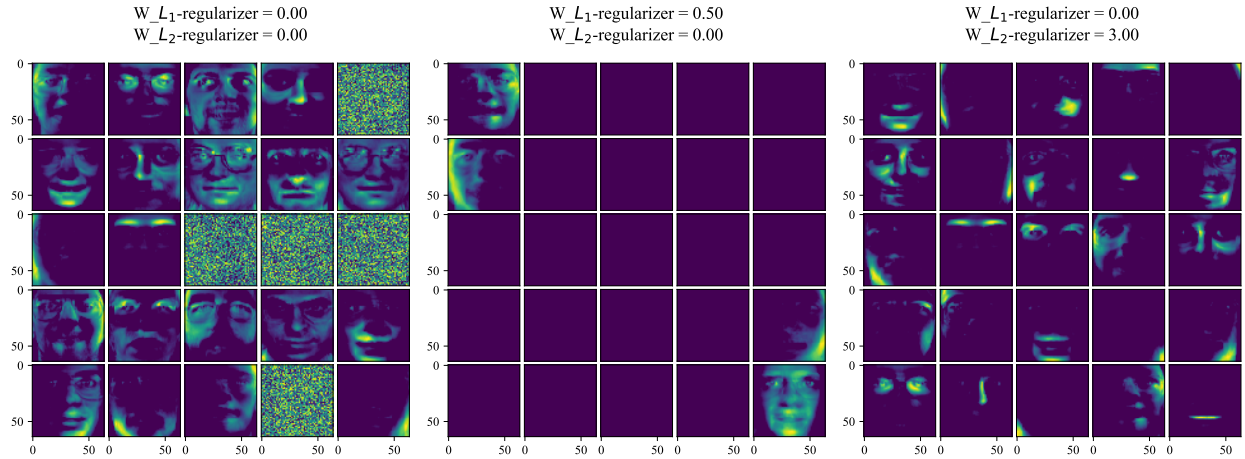


FIGURE 51. Examples of 25 dictionary images learned from the Olivetti face dataset by Matrix Factorization with various regularizers for the dictionary matrix.

[4]It is often the case that one does not have to spend too much iterations to solve sub-problems. In our implementation of ALS for MF in Jupyter Notebook, we used only up to 5 gradient descent itertaions for the sub-problems of coding $\mathbf{H}_n$ and $\mathbf{W}_n$.

In Figure 51, observe that using L_1-regularizer on $\mathbf{W}$ (see Figure 51 middle) makes most of its colums (i.e., vectorizations of basis images) to be zero, and its nonzero columns are not necessarily localized. On the other hand, using L_2-regularizer as in (Figure 51 right), we see the columns of $\mathbf{W}$ are nonzero but they are all localized. One often prefers to learn nonzero localized dictionary elements so that we get more simple and interpretible basis features; On the other hand, one often prefers to have the code matrix $\mathbf{H}$ to have a few nonzero coordinates so that we only need to use a few dictionary elements to approximate original vectors (e.g., for image reconstruction and denoising). The experiment in Figure 51 suggests that for this, we may use L_1 regularizer for the code $\mathbf{H}$ and L_2-regularizer for the dictionary $\mathbf{W}$.

Exercise 4.3.1. Give an explanation on the effect of L_1- and L_2-regularization on the dictionary matrix $\mathbf{W}$ as we see in Figure 51. Drawing contour plots of L_1- and L_2-norm of 2-dimensional vectors would be helpful. (Ref. [Bis06, Sec. 3.1.4]).

Exercise 4.3.2. Use the code in Jupyter Notebook that generates Figures 50 and 51 to determine the *largest* $r \in \mathbb{N}$ *such that every dictionary image properly converges to some non-random image*, where the nonnegativity constraints and regularizers are chosen as in the six cases in Figures 50 and 51 (estimate such r in each of the six cases.)

3.3. Topic modeling for 20Newsgroups. Next, we apply NMF to the 20Newsgroups dataset (see Section 8.1). Recall that each article is represented as a feature vector of word frequency using a fixed vocabulary (sklearn's standard vocab of size 45534) either by using bag-of-words or tf-idf vectorizer. We use 4483 articles from 10 categories (see (168)) combined. The input data matrix $\mathbf{X}$ has size 45534×4483 and we use NMF to factorize this as $\mathbf{WH}$, where $\mathbf{W} \in \mathbb{R}^{45534\times 10}_{\geq 0}$ and $\mathbf{H} \in \mathbb{R}^{10\times 45534}_{\geq 0}$.

In case of factorizing data matrix of vectorized images, the columns of $\mathbf{W}$ were dictionary images that form a basis for the images in the input. For factorizing vectorized texts here, the columns of $\mathbf{W}$ still form a basis for the texts in the input, but they can also be thought of as *topics* in the entire text courpus. Namely, they give groups of words that are used to (approximately) compose each article nonnegatively.



FIGURE 52. 10 topics learned from the 20Newsgroups data by NMF shown as wordclouds. Top 7 words in each topic are shown.

In Figure 52, we show ten topics learnd by NMF as wordclouds. We can indeed see that in each topic, words that belong to the same category appear together (e.g., 'jesus', 'bible', 'god' in topic 6 and 'os', 'pc', 'bit', 'mac' in topic 7). It is important to not, however, that we never used label information and these topics are learned in a purely unsupervised manner. NMF is one of the most widely used technique in topic modeling for text data[5].

[5]Other techniques include Latent Dirichlet Analysis and Nonnegative CP Tensor Decomposition.

We remark that using nonnegativity constraint both on the dictionary **W** and the code **H** are crucial. First, the columns of **W** needs to be vectors of nonnegative entries in order to be interpreted as word frequency vectors. Also, we need to prevent cancelation in the factorization by also making **H** nonnegative so that the words in each topic do share similar context. Indeed, Figure 53 shows the topics learned by a matrix factorization with nonnegative dictionary but unconstrained code. We can see that the words in each topic does not always share the same context when **H** is unconstrained.



FIGURE 53. 10 topics learned from the 20Newsgroups data by a matrix factorization with nonnegative dictionary and unconstrained code.Top 7 words in each topic are shown in wordclouds.

Exercise 4.3.3. The topics learned by NMF in Figures 52 and 53 uses tf-idf vectorizer of the documents. Use bag-of-words vectorizer instead and reproduce these figures. Can you see any difference? Use the code in Use the code in Jupyter Notebook that generates these plots.

4. Principal Component Analysis

In this section, we discuss another matrix-factorization-based classical dimension reduction technique called the 'principal component analysis'. Roughly speaking, the idea is to project n observed d-dimensional data vectors onto a lower r-dimensional space in a 'nice way'. We will discuss two equivalent such formulations of 'variance maximization' and 'error minimization'.

4.1. Maximum variance formulation of PCA. An important concept in the development of PCA is projection of a vector onto a linear subspace generated by a set of vectors. The details are provided in the following exercise.

Exercise 4.4.1 (Projection)**.** Let $\mathbf{x} \in \mathbb{R}^d$ and $C \subseteq \mathbb{R}^d$. The projection $\mathrm{Proj}_C(\mathbf{x})$ of $\mathbf{x}$ onto C is defined by

$$\mathrm{Proj}_C(\mathbf{x}) := \operatorname*{arg\,min}_{\mathbf{u} \in C \subseteq \mathbb{R}^d} \|\mathbf{x}-\mathbf{u}\|_F^2. \tag{252}$$

Namely, $\mathrm{Proj}_C(\mathbf{x})$ is the point in C that is closest to $\mathbf{x}$. It is defined as a constrianed quadratic minimization as above.

(i) Write $\|\mathbf{x}-\mathbf{u}\|_F^2 = \mathrm{tr}((\mathbf{x}-\mathbf{u})^T(\mathbf{x}-\mathbf{u})) = \mathrm{tr}(\mathbf{u}^T\mathbf{u}) - 2\mathrm{tr}(\mathbf{x}^T\mathbf{u}) + \mathrm{tr}(\mathbf{x}^T\mathbf{x})$. Show that

$$\frac{\partial}{\partial \mathbf{u}} \|\mathbf{x}-\mathbf{u}\|_F^2 = 2(\mathbf{u}-\mathbf{x}). \tag{253}$$

(ii) Assume C is the one-dimensional span $\langle \mathbf{v} \rangle := \{a\mathbf{v} \mid a \in \mathbb{R}\}$. Then

$$\text{Proj}_{\mathbf{v}}(\mathbf{x}) := \text{Proj}_{\langle \mathbf{v} \rangle}(\mathbf{x}) = \arg\min_{a \in \mathbb{R}} \|\mathbf{x} - a\mathbf{v}\|_F^2. \tag{254}$$

Use **(i)** and Lagrange multiplier or solve the above unconstrained optimization directly and show

$$\text{Proj}_{\mathbf{v}}(\mathbf{x}) = (\mathbf{x}^T\mathbf{v})\mathbf{v}. \tag{255}$$

(iii) Let $\mathbf{W} = [\mathbf{u}_1, \ldots, \mathbf{u}_r] \in \mathbb{R}^{d \times r}$ be such that its columns are orthonormal, i.e., $\mathbf{W}^T\mathbf{W} = \mathbf{I} \in \mathbb{R}^{r \times r}$. Show that

$$\text{Proj}_{\langle \mathbf{W} \rangle}(\mathbf{x}) := \text{Proj}_{\langle \mathbf{u}_1, \ldots, \mathbf{u}_r \rangle}(\mathbf{x}) = \sum_{i=1}^{r} (\mathbf{u}_i^T\mathbf{x})\mathbf{u}_i = \mathbf{W}(\mathbf{W}^T\mathbf{x}). \tag{256}$$

Next, we introduce the following notions:

Mean and Variance: Fix $\mathbf{X} = [\mathbf{x}_1, \ldots, \mathbf{x}_n] \in \mathbb{R}^{d \times n}$. We define its *mean* and *variance* as

$$\overline{\mathbf{X}} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{x}_i \in \mathbb{R}^d, \qquad \text{Var}(\mathbf{X}) = \frac{1}{n}\sum_{i=1}^{n} \|\mathbf{x}_i - \overline{\mathbf{X}}\|_F^2 \in \mathbb{R}. \tag{257}$$

Coriance: The *covariance matrix* $\mathbf{S} = \mathbf{S}(\mathbf{X}) \in \mathbb{R}^{d \times d}$ of $\mathbf{X} \in \mathbb{R}^{d \times n}$ is defined by

$$\mathbf{S} = \frac{1}{n}\sum_{i=1}^{n} (\mathbf{x}_i - \overline{\mathbf{X}})(\mathbf{x}_i - \overline{\mathbf{X}})^T. \tag{258}$$

Projection: Fix $\mathbf{X} = [\mathbf{x}_1, \ldots, \mathbf{x}_n] \in \mathbb{R}^{d \times n}$ and a dictionary matrix $\mathbf{W} \in \mathbb{R}^{d \times r}$ with orthonormal columns (i.e., $\mathbf{W}^T\mathbf{W} = \mathbf{I} \in \mathbb{R}^{r \times r}$). Define the *projection of* $\mathbf{X}$ *onto the subspace spanned by* $\mathbf{W}$ as

$$\text{Proj}_{\langle \mathbf{W} \rangle}(\mathbf{X}) := \left[\mathbf{W}\mathbf{W}^T\mathbf{x}_1, \ldots, \mathbf{W}\mathbf{W}^T\mathbf{x}_n\right] = \mathbf{W}(\mathbf{W}^T\mathbf{X}) \in \mathbb{R}^{d \times n}. \tag{259}$$

According to Exercise 4.4.1, the columns of $\text{Proj}_{\langle \mathbf{W} \rangle}(\mathbf{X})$ agree with the projection of the columns of $\mathbf{X}$ onto $\langle \mathbf{W} \rangle$.[6]

Projection inherently loses information by collapsing two distinct data points closer to each other. Hence if we were to find the best r-dimensional projection of the data matrix $\mathbf{X}$ that preserves as much information as possible, we should keep the variance of the n vectors after projection as large as possible. Moreover, looking for r-dimensional basis to project onto, we will require the basis vecrors to have unit norm. Hence we would like to seek a matrix $\mathbf{W} \in \mathbb{R}^{d \times r}$ of r basis vectors of unit norm such that the variance after projection is maximized:

(**PCA; maximum variance**)
$$\hat{\mathbf{W}} = \arg\max_{\substack{\mathbf{W} \in \mathbb{R}^{d \times r} \\ \text{rank}(\mathbf{W}) = r}} \text{Var}(\text{Proj}_{\langle \mathbf{W} \rangle}(\mathbf{X})), \tag{260}$$

where $\text{rank}(\mathbf{W})$ denotes the dimension of the linear subspace $\langle \mathbf{W} \rangle$ generated by the columns of $\mathbf{W}$[7].

In the following proposition, we show that finding the top r eigenvectors of the covariance matrix $\mathbf{S}$ solves the PCA problem in (260)[8]. These vectors are also called the *principal components* of $\mathbf{X}$, and the associated eivengalues are called *singular values*.

Proposition 4.4.2. *Let* $\mathbf{X} \in \mathbb{R}^{d \times n}$ *and* $\mathbf{S} = \mathbf{S}(\mathbf{X}) \in \mathbb{R}^{d \times d}$ *denote its covariance matrix. Fix* $r \in \{1, \ldots, n\}$. *Then a solution to the maximum variance PCA problem in* (265) *is given by* $\mathbf{W} \in \mathbb{R}^{d \times r}$ *where*

$$\textit{The columns of } \mathbf{W} \textit{ are the unit eigenvectors of } \mathbf{S} \textit{ with top } r \textit{ largest eigenvalues.} \tag{261}$$

Furthermore, the maximum variance $\text{Var}(\mathbf{W}^T\mathbf{X})$ *after projection acheived equals the sum of the top* r *largest eigenvalues of* $\mathbf{S}$.

[6] Note that $\mathbf{W}^T\mathbf{X} \in \mathbb{R}^{r \times d}$ gives the r-dimensional compressed version of $\mathbf{X}$, whose columns give coordinates of the columns of $\mathbf{X}$ after projection w.r.t. the orthonormal basis of the columns of $\mathbf{W}$.

[7] Note that $\text{rank}(\mathbf{W}) = \dim(\langle \mathbf{W} \rangle)$. So the columns of $\mathbf{W}$ are linearly independent iff $\text{rank}(\mathbf{W}) = r$

[8] But there could be other equivalent solutions.

PROOF. First, write $\mathbf{W} = [\mathbf{w}_1, \dots, \mathbf{w}_r]$ and we will choose an orthonormal basis $\mathbf{u}_1, \dots, \mathbf{u}_r$ for the linear subspace $\langle \mathbf{w}_1, \dots, \mathbf{w}_r \rangle$ spanned by the columns of $\mathbf{W}$ (e.g., by Gram-Schmidt). Then since $\langle \mathbf{w}_1, \dots, \mathbf{w}_r \rangle = \langle \mathbf{u}_1, \dots, \mathbf{u}_r \rangle$, we have

$$\mathrm{Proj}_{\langle \mathbf{W} \rangle}(\mathbf{X}) = \mathrm{Proj}_{\langle \mathbf{u}_1, \dots, \mathbf{u}_r \rangle}(\mathbf{X}). \tag{262}$$

Hence without loss of generality, we may assume that the columns of $\mathbf{W}$ are orthonormal. i.e., $\mathbf{W}^T\mathbf{W} = \mathbf{I} \in \mathbb{R}^{r \times r}$, in (260).

Since the columns of $\mathbf{W}$ are orthonormal, note that

$$\mathrm{Var}(\mathrm{Proj}_{\langle \mathbf{W} \rangle}(\mathbf{X})) = \mathrm{Var}(\mathbf{W}\mathbf{W}^T\mathbf{X}) = \frac{1}{n}\sum_{i=1}^{n} \|\mathbf{W}\mathbf{W}^T(\mathbf{x}_i - \overline{\mathbf{X}})\|_F^2 = \mathrm{tr}\left(\mathbf{W}\mathbf{W}^T\mathbf{S}\mathbf{W}\mathbf{W}^T\right), \tag{263}$$

where $\mathbf{S} = \mathbf{S}(\mathbf{X}) \in \mathbb{R}^{d \times d}$ denotes the covariance matrix of $\mathbf{X}$ defined at (258). By the cyclic property of trace (see Exercise 4.4.3) and since $\mathbf{W}^T\mathbf{W} = \mathbf{I}$,

$$\mathrm{tr}\left(\mathbf{W}\mathbf{W}^T\mathbf{S}\mathbf{W}\mathbf{W}^T\right) = \mathrm{tr}\left(\mathbf{W}^T\mathbf{S}\mathbf{W}(\mathbf{W}^T\mathbf{W})\right) = \mathrm{tr}\left(\mathbf{W}^T\mathbf{S}\mathbf{W}\right). \tag{264}$$

Thus (260) is equivalent to

$$\hat{\mathbf{W}} = \underset{\substack{\mathbf{W} \in \mathbb{R}^{d \times r} \\ \mathbf{W}^T\mathbf{W} = \mathbf{I}}}{\arg\max}\ \mathrm{tr}\left(\mathbf{W}^T\mathbf{S}\mathbf{W}\right). \tag{265}$$

We claim the following: [9]

If $\mathbf{W}$ *is a stationary point of* (265) *then we have an orthonormal* $\mathbf{U} \in \mathbb{R}^{r \times r}$ *s.t.*

(1) $\mathbf{U}^T\mathbf{W} \in \mathbb{R}^{d \times r}$ *is orthonormal; and*

(2) $\mathrm{tr}(\mathbf{W}^T\mathbf{S}\mathbf{W}) = \mathrm{tr}(\mathbf{U}\mathbf{W}^T\mathbf{S}\mathbf{W}\mathbf{U}^T)$*; and*

(3) $\mathbf{S}(\mathbf{W}\mathbf{U}^T) = (\mathbf{W}\mathbf{U}^T)\mathbf{D}$ *where* $\mathbf{D} \in \mathbb{R}^{r \times r}$ *is diagonal with* r *eigenvalues of* $\mathbf{S}$ *for its diagonal entries.*

In particular, this claim implies that if $\hat{\mathbf{W}} \in \mathbb{R}^{d \times r}$ is a solution to (265), then we can find an 'equivalent maximizer' $\mathbf{U}^T\hat{\mathbf{W}}$ (by (1) and (2)) with the additional property that the columns of $\mathbf{U}^T\hat{\mathbf{W}}$ are eigenvectors of $\mathbf{S}$ (by (3)). Notice that, by writing $\mathbf{D} = \mathrm{diag}(\lambda_1, \dots, \lambda_r)$ where λ_i's are eigenvalues of $\mathbf{S}$,

$$\begin{aligned} \mathrm{tr}\left((\mathbf{W}\mathbf{U}^T)^T\mathbf{S}(\mathbf{W}\mathbf{U}^T)\right) &= \mathrm{tr}\left((\mathbf{W}\mathbf{U}^T)^T(\mathbf{W}\mathbf{U}^T)\mathbf{D}\right) && (266)\\ &= \mathrm{tr}\left(\mathbf{U}\mathbf{W}^T\mathbf{W}\mathbf{U}^T\mathbf{D}\right) && (267)\\ &= \mathrm{tr}(\mathbf{D}) = \lambda_1 + \cdots + \lambda_r. && (268) \end{aligned}$$

Therefore, if we choose the columns of $\mathbf{U}^T\mathbf{W}$ to be an orthonormal set of r eigenvectors of $\mathbf{S}$ with top r eigenvalues of $\mathbf{S}$, then $\mathbf{U}^T\mathbf{W}$ has to be a solution to (265), hence proving the assertion.

It remains to show the claim above. Two essential ingrediemtns are multivariave Lagrange multipliers and spectral theorem for real symmetric matrices.

In order to find all stationary points of (265), we formulate its Lagrangian as

$$L(\mathbf{W}; \boldsymbol{\lambda}) := \mathrm{tr}\left(\mathbf{W}^T\mathbf{S}\mathbf{W}\right) + \sum_{j=1}^{r} \lambda_{jj}(1 - \mathrm{tr}(\mathbf{W}[:,j]^T\mathbf{W}[:,j])) + \sum_{\substack{1 \le i,j \le r \\ i \ne j}} \lambda_{ij}(0 - \mathrm{tr}(\mathbf{W}[:,i]^T\mathbf{W}[:,j])), \tag{269}$$

where $\boldsymbol{\lambda} = (\lambda_{ij}) \in \mathbb{R}^{r \times r}$ is a dual variable[10]. By the method of multivariate Lagrange multiplier, a matrix $\mathbf{W} \in \mathbb{R}^{d \times r}$ is a stationary point of (265) *only if* there exists $\boldsymbol{\lambda} \in \mathbb{R}^{r \times r}$ such that it is a stationary point of $L(\cdot; \boldsymbol{\lambda})$. So, differentiating the objective function above by the jth column $\mathbf{W}[:,j]$ of $\mathbf{W}$,

$$\begin{aligned} \frac{\partial}{\partial \mathbf{W}[:,j]} &= \mathbf{S}\mathbf{W}[:,j] - \sum_{1 \le i \le r} \lambda_{ij}\mathbf{W}[:,i] && (270)\\ &= \mathbf{S}\mathbf{W}[:,j] - \mathbf{W}[:,i]\boldsymbol{\lambda}[:,j]. && (271) \end{aligned}$$

[9] Simply speaking, it says that there exists a global maximizer $\hat{\mathbf{W}}_*$ of (265) such that its columns are eigenvectors of $\mathbf{S}$.

[10] Just think of it as a multi-dimensional Lagrange multiplier

Hence, in order for $\mathbf{W} \in \mathbb{R}^{d\times r}$ to be a stationary point of (265), there should exist a dual variable $\boldsymbol{\lambda} \in \mathbb{R}^{r\times r}$ such that

$$\mathbf{S}\mathbf{W} = \mathbf{W}\boldsymbol{\lambda}. \tag{272}$$

Using $\mathbf{W}^T\mathbf{W} = \mathbf{I} \in \mathbb{R}^{r\times r}$, this implies

$$\mathbf{W}^T\mathbf{S}\mathbf{W} = \boldsymbol{\lambda}. \tag{273}$$

Next, notice that $\mathbf{W}^T\mathbf{S}\mathbf{W} = \boldsymbol{\lambda}$ is a real symmetric matrix. So by a spectral theorem (see Exericse 4.4.4), there exists an orthonormal $\mathbf{U} \in \mathbb{R}^{r\times r}$[11] such that $\mathbf{W}^T\mathbf{S}\mathbf{W} = \mathbf{U}^T\mathbf{D}\mathbf{U}$, where $\mathbf{D}$ is a diagonal matrix whose diagonal entries are the eigenvalues of $\mathbf{W}^T\mathbf{S}\mathbf{W} \in \mathbb{R}^{r\times r}$. This shows

$$\mathbf{S}(\mathbf{W}\mathbf{U}^T) = (\mathbf{W}\mathbf{U}^T)\mathbf{D}, \quad \text{(claim (2))}. \tag{274}$$

Also, it is easy to see that $\mathbf{U}^T\mathbf{W} \in \mathbb{R}^{d\times r}$ is orthonormal, since $(\mathbf{U}^T\mathbf{W})^T(\mathbf{U}^T\mathbf{W}) = \mathbf{W}^T\mathbf{U}\mathbf{U}^T\mathbf{W} = \mathbf{W}^T\mathbf{W} = \mathbf{I} \in \mathbb{R}^{r\times r}$ (claim (1)). Lastly, observe that using $\mathbf{U}^T\mathbf{U} = \mathbf{I}$,

$$\operatorname{tr}(\mathbf{W}^T\mathbf{S}\mathbf{W}) = \operatorname{tr}(\mathbf{W}^T\mathbf{S}\mathbf{W}\mathbf{U}^T\mathbf{U}) = \operatorname{tr}(\mathbf{U}\mathbf{W}^T\mathbf{S}\mathbf{W}\mathbf{U}^T) \qquad \text{(claim (3))}. \tag{275}$$

This shows the claim, thereby finishing the proof of the assertion. □

Exercise 4.4.3 (Cyclic property of trace)**.** Show that $\operatorname{tr}(\mathbf{ABC}) = \operatorname{tr}(\mathbf{BCA})$.

The r principal components of $\mathbf{X}$ as stated in Proposition 4.4.2 are always orthogonal. This is implied by the spectral theorem for real symmetric matrices, see Exercise 4.4.4.

Exercise 4.4.4 (Spectral Theorem for Real Symmetric Matrices)**.** Let $\mathbf{A} \in \mathbb{R}^{n\times n}$ be a real symmetric matrix, i.e., $\mathbf{A}^T = \mathbf{A}$. Then the spectral theorem for real symmetric matrices states that the following hold:

1. $\mathbf{A}$ has n real eigenvalues (counting multiplicities);
2. There exists an orthonormal matrix $\mathbf{U} \in \mathbb{R}^{n\times n}$, $\mathbf{U}^T\mathbf{U} = \mathbf{I}$ and a diagonal matrix $\mathbf{D} \in \mathbb{R}^{n\times n}$ such that

$$\mathbf{A} = \mathbf{U}^T\mathbf{D}\mathbf{U}; \tag{276}$$

3. In **(ii)**, the orthonomal matrix $\mathbf{U}$ consists of eigenvectors of $\mathbf{A}$ and the diagonal entries of $\mathbf{D}$ are the eigenvalues of $\mathbf{A}$;

In this exercise, we will prove this result by an induction on n.

(i) Show that all of the eigenvalues of $\mathbf{A}$ are real.
(ii) Let $\mathbf{v}_1$ be an eigenvector of $\mathbf{A}$ and let λ_1 be its associated eigenvalue. By **(i)**, λ_1 is real. Extend $\mathbf{v}_1$ to an orthonormal basis $\mathscr{B} := \{\mathbf{v}_1, \mathbf{u}_2, \ldots, \mathbf{u}_n\}$ for $\mathbb{R}^n$ and let $\mathbf{Q} := [\mathbf{v}_1, \mathbf{u}_2, \ldots, \mathbf{u}_n] \in \mathbb{R}^{n\times n}$. Show that

$$\mathbf{Q}^T\mathbf{A}\mathbf{Q} = \left[\begin{array}{c|ccc} \lambda_1 & 0 & \ldots & 0 \\ \hline 0 & & & \\ \vdots & & \mathbf{A}' & \\ 0 & & & \end{array}\right], \tag{277}$$

where $\mathbf{A}' \in \mathbb{R}^{(n-1)\times(n-1)}$ is a real symmetric matrix.
(iii) Suppose there exists an orthonormal matrix $\mathbf{V} \in \mathbb{R}^{(n-1)\times(n-1)}$ and a diagonal matrix $\mathbf{D}' \in \mathbb{R}^{(n-1)\times(n-1)}$ such that $\mathbf{A}' = \mathbf{V}^T\mathbf{D}'\mathbf{V}$. Define

$$\overline{\mathbf{V}} = \left[\begin{array}{c|ccc} 1 & 0 & \ldots & 0 \\ \hline 0 & & & \\ \vdots & & \mathbf{V} & \\ 0 & & & \end{array}\right] \in \mathbb{R}^{n\times n}. \tag{278}$$

[11] $\mathbf{U}^T\mathbf{U} = \mathbf{U}\mathbf{U}^T = \mathbf{I}$ since $\mathbf{U}$ is a square matrix.

Then show that

$$\mathbf{A} = \mathbf{Q}\overline{\mathbf{V}}^T \left[\begin{array}{c|ccc} \lambda_1 & 0 & \dots & 0 \\ \hline 0 & & & \\ \vdots & & \mathbf{D}' & \\ 0 & & & \end{array}\right] \overline{\mathbf{V}}\mathbf{Q}^T. \tag{279}$$

Also show that $\mathbf{U} := \overline{\mathbf{V}}\mathbf{Q}^T \in \mathbb{R}^{n\times n}$ is orthonomal.

(iv) By using **(i)**-**(ii)**, prove that if $\mathbf{A} \in \mathbb{R}^{n\times n}$ is symmetric, show that there are orthogonal and diagonal matrices $\mathbf{U}, \mathbf{D} \in \mathbb{R}^{n\times n}$ such that $\mathbf{A} = \mathbf{U}^T\mathbf{D}\mathbf{U}$ and $\mathbf{D}$ consists of the eigenvalues of $\mathbf{A}$ as its diagonal entries. Also deduce that the columns of $\mathbf{U}$ are the eigenvectors of $\mathbf{A}$.

4.2. Minimum reconstruction error formulation of PCA. In this subsection, we give an alternative formulation of PCA by minimizing reconstruction error after projecting onto an r-dimensional orthonormal basis.

As before, let $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_n] \in \mathbb{R}^{d\times n}$ be a data matrix. We will center it by subtracting its mean $\overline{\mathbf{X}}$ so that $\overline{\mathbf{X}} = \mathbf{0}$ and $\mathbf{S} = n^{-1}\mathbf{X}\mathbf{X}^T$. Let $\mathbf{W} \in \mathbb{R}^{d\times r}$ be such that its columns are orthonormal, i.e., $\mathbf{W}^T\mathbf{W} = \mathbf{I} \in \mathbb{R}^{r\times r}$. The 'reconstruction error' of approximating $\mathbf{x}_i$ by the projection $\mathrm{Proj}_{\langle \mathbf{W}\rangle}(\mathbf{x}_i)$ is $\|\mathbf{x}_i - \mathrm{Proj}_{\langle \mathbf{W}\rangle}(\mathbf{x}_i)\|_F^2$. In the minimum reconstruction error formulation of PCA, we would like to choose $\mathbf{W}$ such that the total reconstruction error of all columns of $\mathbf{X}$ is minimized:

(**PCA; min. recons. error**)
$$\hat{\mathbf{W}} = \operatorname*{arg\,min}_{\substack{\mathbf{W}\in\mathbb{R}^{d\times r} \\ \mathrm{rank}(\mathbf{W})=r}} \frac{1}{n}\left\|\mathbf{X} - \mathrm{Proj}_{\langle \mathbf{W}\rangle}(\mathbf{X})\right\|_F^2. \tag{280}$$

In the following proposition, we show the above alternative formulation of PCA is equivalent to the maximum variance one in (260).

Proposition 4.4.5. *The minimum reconstruction error formulation of PCA* (280) *is equivalent to the maximum variance formulation of PCA* (260).

PROOF. As in the proof of Proposition 4.4.2, we may assume that $\mathbf{W} \in \mathbb{R}^{d\times r}$ is orthonormal, i.e., $\mathbf{W}^T\mathbf{W} = \mathbf{I} \in \mathbb{R}^{r\times r}$. We claim that for orthonormal $\mathbf{W}$,

$$\frac{1}{n}\left\|\mathbf{X} - \mathrm{Proj}_{\langle \mathbf{W}\rangle}(\mathbf{X})\right\|_F^2 = \mathrm{Var}(\mathbf{X}) - \mathrm{tr}\left(\mathbf{W}^T\mathbf{S}\mathbf{W}\right), \tag{281}$$

where $\mathbf{S} \in \mathbb{R}^{d\times d}$ denotes the covariance matrix of $\mathbf{X} \in \mathbb{R}^{d\times n}$. Then the assertion follows from (281) since then (280) is equivalent to

$$\operatorname*{arg\,min}_{\substack{\mathbf{W}\in\mathbb{R}^{d\times r} \\ \mathbf{W}^T\mathbf{W}=\mathbf{I}}} \left[\mathrm{Var}(\mathbf{X}) - \mathrm{tr}\left(\mathbf{W}^T\mathbf{S}\mathbf{W}\right)\right] = \operatorname*{arg\,max}_{\substack{\mathbf{W}\in\mathbb{R}^{d\times r} \\ \mathbf{W}^T\mathbf{W}=\mathbf{I}}} \mathrm{tr}\left(\mathbf{W}^T\mathbf{S}\mathbf{W}\right), \tag{282}$$

and the last is an equivalent formulation of the maximum variance PCA (260) as we saw in (265).

It remains to show the identity (281) for orthornomal $\mathbf{W} \in \mathbb{R}^{d\times r}$. To this end, we extend $\mathbf{W}$ to obtain a complete orthonormal basis (ONB) for $\mathbb{R}^d$. Namely, we can choose $\mathbf{W}^\perp \in \mathbb{R}^{d\times(d-r)}$ be such that the columns in their horizontal concatenation $\overline{\mathbf{W}} := [\mathbf{W}, \mathbf{W}^\perp] \in \mathbb{R}^{d\times d}$ form an orthonormal basis (ONB) for $\mathbb{R}^d$ (i.e., $\overline{\mathbf{W}}^T\overline{\mathbf{W}} = \mathbf{I} \in \mathbb{R}^{d\times r}$.) Fix a column $\mathbf{x}_i$ of $\mathbf{X}$. Note that since the columns of $\overline{\mathbf{W}}$ span the full space $\mathbb{R}^d$, we have

$$\mathbf{x}_i = \mathrm{Proj}_{\langle \overline{\mathbf{W}}\rangle}(\mathbf{x}_i) = \overline{\mathbf{W}}\,\overline{\mathbf{W}}^T\mathbf{x}_i. \tag{283}$$

On the other hand, note that $\overline{\mathbf{W}}[\mathbf{W}, \mathbf{O}]^T = [\mathbf{W}, \mathbf{W}^\perp][\mathbf{W}, \mathbf{O}]^T = \mathbf{W}\mathbf{W}^T$, so we have

$$\mathrm{Proj}_{\langle \mathbf{W}\rangle}(\mathbf{x}_i) = \mathbf{W}\mathbf{W}^T\mathbf{x}_i = \overline{\mathbf{W}}[\mathbf{W}, \mathbf{O}]^T\mathbf{x}_i. \tag{284}$$

Thus we get

$$\left\|\mathbf{x}_i - \mathrm{Proj}_{\langle \mathbf{W}\rangle}(\mathbf{x}_i))\right\|_F^2 = \left\|\overline{\mathbf{W}}\,\overline{\mathbf{W}}^T\mathbf{x}_i - \overline{\mathbf{W}}[\mathbf{W}, \mathbf{O}]^T\mathbf{x}_i\right\|_F^2 \tag{285}$$

$$= \left\| \overline{\mathbf{W}} \left([\mathbf{W}, \mathbf{W}^\perp]^T - [\mathbf{W}, \mathbf{O}]^T \right) \mathbf{x}_i \right\|_F^2 \tag{286}$$

$$= \left\| \left([\mathbf{W}, \mathbf{W}^\perp]^T - [\mathbf{W}, \mathbf{O}]^T \right) \mathbf{x}_i \right\|_F^2 \tag{287}$$

$$= \left\| (\mathbf{W}^\perp)^T \mathbf{x}_i \right\|_F^2, \tag{288}$$

where the third equality follows from the fact that $\overline{\mathbf{W}}$ is orthonormal.[12] Also note that $\left\| (\mathbf{W}^\perp)^T \mathbf{x}_i \right\|_F^2 = \left\| \mathbf{x}_i^T (\mathbf{W}^\perp) \right\|_F^2 = (\mathbf{W}^\perp) \mathbf{x}_i \mathbf{x}_i^T (\mathbf{W}^\perp)^T$. Hence

$$\frac{1}{n} \left\| \mathbf{X} - \text{Proj}_{\langle \mathbf{W} \rangle}(\mathbf{X}) \right\|_F^2 = \frac{1}{n} \sum_{i=1}^n (\mathbf{W}^\perp) \mathbf{x}_i \mathbf{x}_i^T (\mathbf{W}^\perp)^T = \text{tr}\left((\mathbf{W}^\perp) \mathbf{S} (\mathbf{W}^\perp)^T \right). \tag{289}$$

Lastly, note that $\text{tr}\left(\overline{\mathbf{W}}^T \mathbf{S} \overline{\mathbf{W}} \right) = \text{tr}\left(\mathbf{S} \overline{\mathbf{W}} \overline{\mathbf{W}}^T \right) = \text{tr}(\mathbf{S}) = \text{Var}(\mathbf{X})$. Hence we get

$$\text{Var}(\mathbf{X}) = \text{tr}\left(\overline{\mathbf{W}}^T \mathbf{S} \overline{\mathbf{W}} \right) = \text{tr}\left([\mathbf{W}, \mathbf{W}^\perp]^T \mathbf{S} [\mathbf{W}, \mathbf{W}^\perp] \right) \tag{290}$$

$$= \text{tr}\left(\mathbf{W}^T \mathbf{S} \mathbf{W} \right) + \text{tr}\left((\mathbf{W}^\perp)^T \mathbf{S} (\mathbf{W}^\perp) \right) + \text{tr}\left(\mathbf{W}^T \mathbf{S} (\mathbf{W}^\perp) \right) + \text{tr}\left((\mathbf{W}^\perp)^T \mathbf{S} \mathbf{W} \right) \tag{291}$$

$$= \text{tr}\left(\mathbf{W}^T \mathbf{S} \mathbf{W} \right) + \text{tr}\left((\mathbf{W}^\perp)^T \mathbf{S} (\mathbf{W}^\perp) \right) + \text{tr}\left(\mathbf{S} \underbrace{(\mathbf{W}^\perp \mathbf{W}^T)}_{\mathbf{O}} \right) + \text{tr}\left(\mathbf{S} \underbrace{\mathbf{W} (\mathbf{W}^\perp)^T}_{\mathbf{O}} \right) \tag{292}$$

$$= \text{tr}\left(\mathbf{W}^T \mathbf{S} \mathbf{W} \right) + \text{tr}\left((\mathbf{W}^\perp)^T \mathbf{S} (\mathbf{W}^\perp) \right). \tag{293}$$

Thus, the claimed identity (281) follows from combining the above and (289). This finishes the proof. □

5. Applications of PCA and Matrix Factorization

In this section, we apply PCA to various image and text datasets for the purpose of dictionary learning and compare with matrix factorization (MF). See Jupyter Notebook for the experiments given in this section. Here we use sklearn's PCA implementation sklearn.decomposition.PCA.

5.1. MNIST. We first apply PCA to the MNIST dataset to learn 24 dictionary images (principal components) and their singular values. These are shown in Figures 54. In Figure 54, it appears that the principal components with larger singular values captures the overall shape of the digits whereas thowe with smaller singular values gives finer details of the shape.

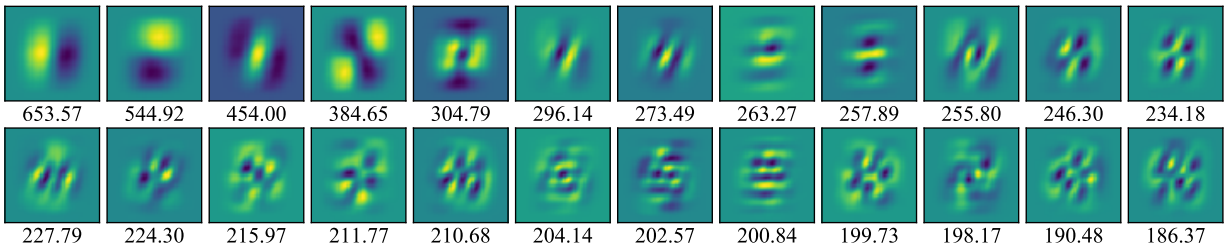


FIGURE 54. Examples of 24 dictionary images (principal components) learned from MNIST dataset by PCA. The numbers below each image are the corresponding singular values.

In Figure 55, we compare the MNIST image dictionary learned by PCA against those learned by MF. The numbers underneath the dictionary images for MF cases represent their 'total usage' computed by row sums of the code matrix $\mathbf{H}$. Namely, observe that from the factorization $\mathbf{X} \approx \mathbf{W}\mathbf{H} \in \mathbb{R}^{d \times n}$, $\mathbf{H}[i, j]$ equals the linear coefficient of $\mathbf{W}[:, i]$ in approximating the ith data point $\mathbf{X}[:, j]$. Hence we can define

$$\text{total usage of } \mathbf{W}[:, i] := \sum_{j=1}^n |\mathbf{H}[i, j]|. \tag{294}$$

[12] Suppose $\mathbf{A} \in \mathbb{R}^{d \times d}$ is orthonormal, i.e., $\mathbf{A}^T \mathbf{A} = \mathbf{I}$. Let $\mathbf{x} \in \mathbb{R}^d$. Then $\|\mathbf{A}\mathbf{x}\|_F^2 = (\mathbf{A}\mathbf{x})^T (\mathbf{A}\mathbf{x}) = \mathbf{x}^T \mathbf{A}^T \mathbf{A} \mathbf{x} = \mathbf{x}^T \mathbf{I} \mathbf{x} = \mathbf{x}^T \mathbf{x} = \|\mathbf{x}\|_F^2$.

Note that we take the absolute value of the code coefficient $\mathbf{H}[i, j]$ above in order to prevent cancelation in computing the total usage. Figure 56 shows similar experiments on MNIST with random padding of thickness 20. It is clear that NMF-learned dictionary are the easiest to interpret.

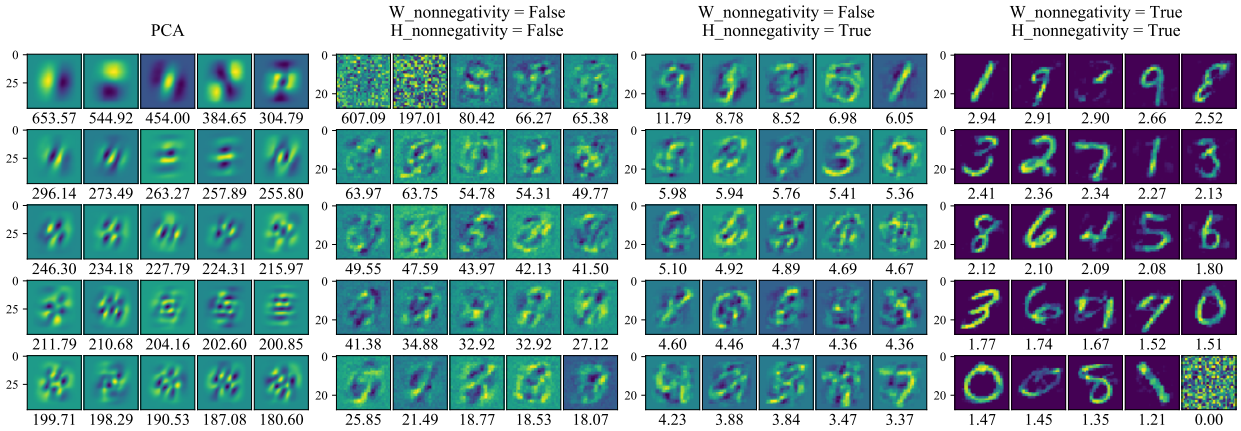


FIGURE 55. Examples of 25 dictionary images learned from MNIST dataset by Matrix Factorization with various nonnegativity constraints.

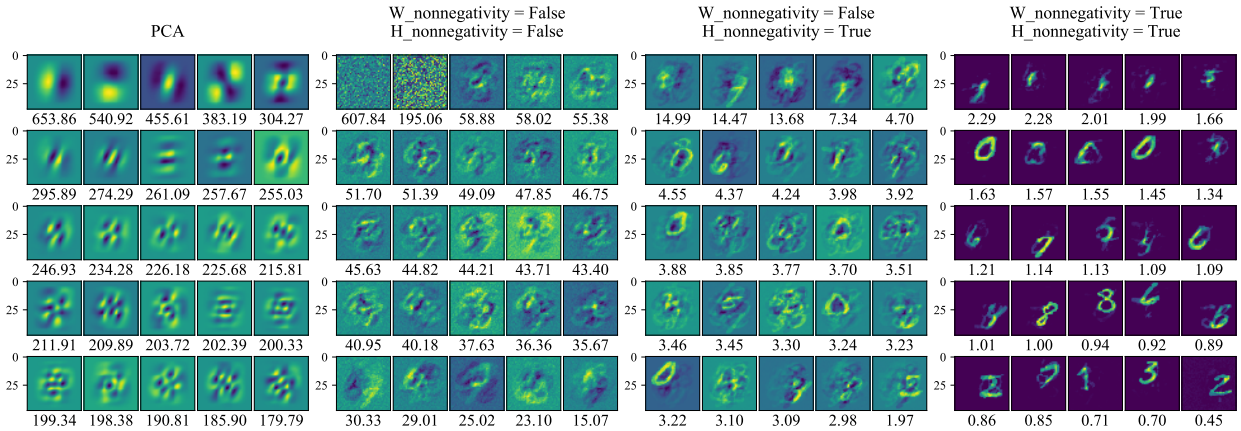


FIGURE 56. Examples of 25 dictionary images learned from MNIST dataset by Matrix Factorization with various nonnegativity constraints. Original 28×28 images are padded with zeros randomly so that they are 48×48 images.

5.2. Olivetti face images. We perform similar dictionary learning experiments for the Olivetti face dataset in Figure 57. Since the principal components may have negative entries, the PCA-learned dictioanry face images (a.k.a. 'eigenfaces') may have some shapes that are not part of any of the actual face images.

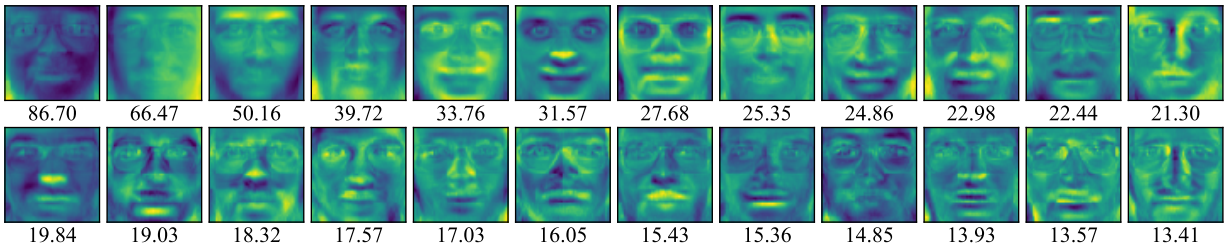


FIGURE 57. Examples of 24 dictionary images (principal components) learned from the Olivetti face dataset by PCA. The numbers below each image are the corresponding singular values.

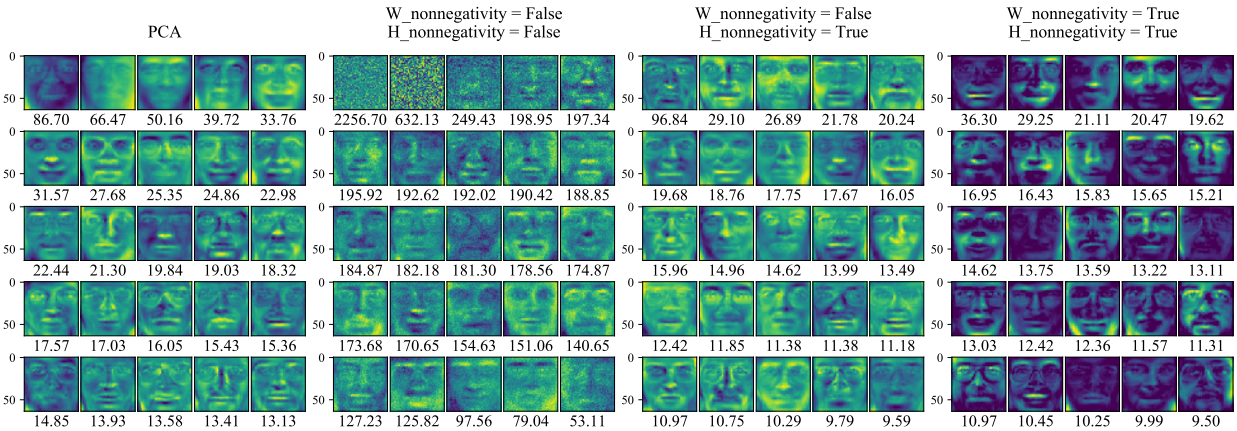


FIGURE 58. Examples of 25 dictionary images learned from the Olivetti face dataset by PCA and Matrix Factorization with various nonnegativity constraints.

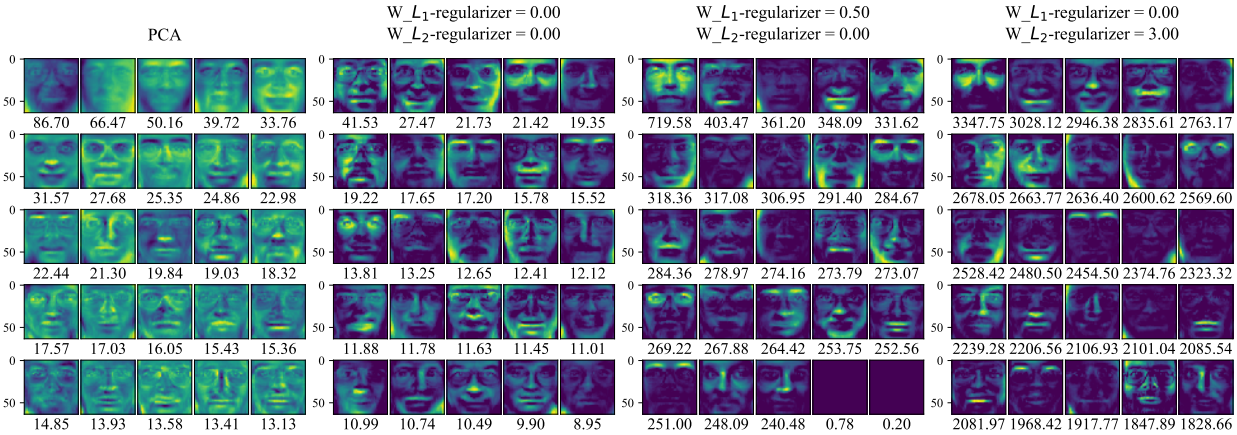


FIGURE 59. Examples of 25 dictionary images learned from the Olivetti face dataset by PCA and Matrix Factorization with various dictionary regularization.

6. Probabilistic PCA and EM algorithm

In this section, we will consider a probabilistic version of PCA and an iterative algorithm for solving the PCA optimization problem called the Expectation-Maximization (EM) algorithm. The discussion in this section is based on [Bis06, Sec. 9.3 and 12.2] and [Row98, TB99].

6.1. The EM algoirthm for PCA. The goal of this section is to derive the following *Expectation-Maximization* (EM) algorithm for PCA. In order to provide the full background and discussion, we need to introduce a probabistic version of PCA and formulate a maximum likelihood formulation of PCA. The EM algorithms form a general class of algorithms used to solve a complex maximum likelihood extimation problem in an efficient and iterative manner. We will discuss more details in the following section, and here we will state the algorithm and give some examples.

One might wonder what would be the point of using an iterative algorithm for PCA while we already have a closed-form solution (see Proposition 4.4.2). One of the main point here is the computational efficiency. Namely, suppose that our data matrix $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_n]$ has dimension $d \times n$, so each observation has dimsion d. Say we are interested in learning r principal components, where $r \ll d$ (e.g., $r = 25$ and $d = 10^5$). The closed form solution in Proposition 4.4.2 involves computing the full $d \times d$ covariance

matrix $\mathbf{S} = n^{-1}\sum_{i=1}^{n}(\mathbf{x}_i - \overline{\mathbf{X}})^T(\mathbf{x}_i - \overline{\mathbf{X}})$ and computing *all* of its eigenvector-eigenvalue pairs and keeping only the top r of them. However, computing the full $d \times d$ covariance matrix takes $O(nd^2)$ operations and computing its full eigendecomposition takes $O(d^3)$ operations. But since we are only interested in extracting top $r \ll d$ principal components, it would me much more efficient if we can avoid computing the full covariance matrix and still compute the top r principal components.

In Algorithm 4, we state the EM algorithm for PCA due to [Row98]. The computational cost of this algorithm is $O(rdn)$, which is much less compared to the computational cost $O((n+d)d^2)$ of the full PCA solution.

Algorithm 4 EM algorithm for PCA [Row98, TB99]

Input: $\mathbf{X} \in \mathbb{R}^{d\times n}$ (data matrix); $r \geq 1$ (rank parameter); $\mathbf{W}_0 \in \mathbb{R}^{d\times r}$ (initial estimate); N (number of iterations)
  for $n = 1, \ldots, N$ **do**:
    $\mathbf{H}_n \leftarrow (\mathbf{W}_{n-1}^T\mathbf{W}_{n-1})^{-1}\mathbf{W}_{n-1}^T\mathbf{X} \in \mathbb{R}^{r\times n}$ (E-step)
    $\mathbf{W}_n \leftarrow \mathbf{X}\mathbf{H}_n^T(\mathbf{H}_n\mathbf{H}_n^T)^{-1} \in \mathbb{R}^{d\times r}$ (M-step)
  end for
output: $\mathbf{W}_N$

Here are some important properties of Algorithm 4.

1. (*Convergence to stationary points*) $\mathbf{W}_n$ converges to a stationary point of the PCA optimization problem (260) (and also the equivalent formulation (265)), but not guaranteed to converge to a global optimum.
2. (*Stability of global optimum*) If the initial estimate $\mathbf{W}_0$ is close to a global optimum of the PCA problem $\mathbf{W}^*$, then $\mathbf{W}_n$ converges to $\mathbf{W}^*$.
3. (*Instability of sub-optimal solutions*) If the initial estimate $\mathbf{W}_0$ is close to a stationary point $\mathbf{W}_*$ of the PCA problem and if $\mathbf{W}_*$ is not a global optimum, then $\mathbf{W}_n$ does not converge to $\mathbf{W}_*$.

Item 1 is due to the general convergence property of EM algorithm for maximum likelihood estimation problems. We will discuss more details for this aspect in later subsections. On the other hand, the local stability properties of stationary points of the PCA problem in items 2 and 3 are due to [TB99]. These are desired properties for iterative algorithms for nonconvex optimization problems, but they are not sufficient to guarantee convergence to a globally optimal solution (e.g., principal components for the PCA problem)[13]. For this reason, it is a general wisdom for EM algorithms to chose a proper initialization or try multiple runs for different initializations.

6.2. Olivetti Face images. Before we delve into theoretical details of EM algorithms for PCA, we first discuss some applications of it to the Olivetti face images. In Figure 60, we show the 24 components in $\mathbf{W}_N$ learned by the EM-PCA in Algorithm 4 (100 trials each with random initialization $\mathbf{W}_0$ and 100 iterations).

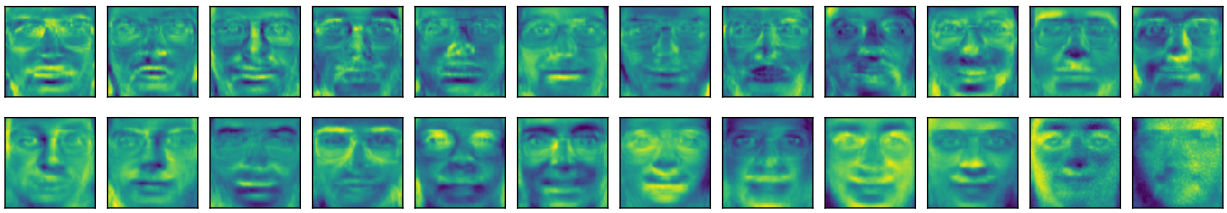


FIGURE 60. Examples of 24 dictionary images (principal components) learned from the Olivetti face dataset by the EM algorithm for PCA (Algorithm 4).

[13] In fact, the existence of a closed-form solution to PCA despite it being a non-convex problem is one of the reasons for the popularity of PCA.

The learned components are similar to the principal components in Figure 57, and if converged properly to a global optimum, these components should span a principal eigenspace of the covariance matrix $\mathbf{S}$ of the input data $\mathbf{X}$[14]. However, in our experiement, it turns out that the components in Figure 57 are almost eigenvectors of $\mathbf{S}$ but not quite. Indeed, in Figure 61, we see that the first column of $\mathbf{W}_N$ learned by the EM-PCA is almost an eigenvector of $\mathbf{S}$ but not as close as the exact principal compoenent. In order to see this, we plot the ratio of the ($64^2 = 4096$) coefficietns in $\mathbf{SW}[:,0]/\mathbf{W}[:,0]$. If $\mathbf{W}[:,0]$ is an eigenvector of $\mathbf{S}$, then these ratios should all equal to some constant λ, which is the eigenvalue of $\mathbf{W}[:,0]$. The standard deviation of these ratios are about 7.0567, while for the exact first principl component, the standard deviation is about 7.7190×10^{-5}.

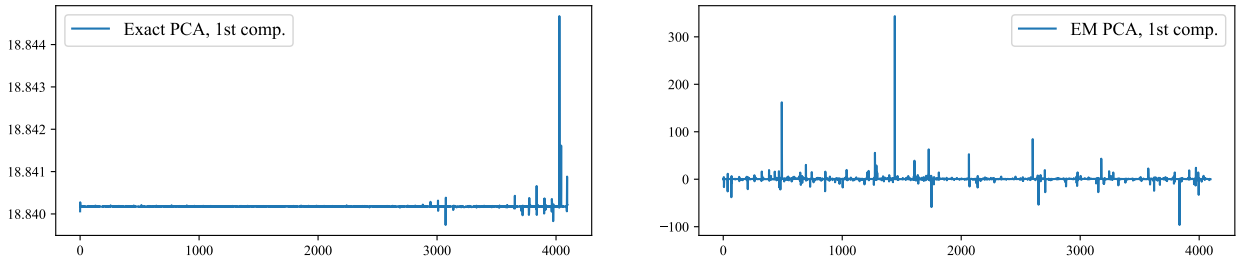


FIGURE 61. Plot of the ratio of coefficients $\mathbf{SW}[:,0]/\mathbf{W}[:,0]$ where $\mathbf{W}$ is either learned by the exact PCA or the EM PCA. $W[:,0]$ is an eigenvector of $\mathbf{S}$ if these ratios are constant.

6.3. The EM algorithm for MLE. The goal of the EM algorithm is to find maximum likelihood solutions for probabilistic models with latent variables. Suppose we have a probabilistic model for the pair $(\mathbf{x},\mathbf{h}) \in \mathbb{R}^p \times \mathbb{R}^r$ of random variables, given by a joint distribution $(x,h) \mapsto p(x,h\,|\,\boldsymbol{\theta})$, where $\boldsymbol{\theta}$ denotes the vector of all parameters to be learned. We will refer to $\mathbf{x}$ as the 'observed variable' and $\mathbf{h}$ as the 'latent variable'. Suppose we are given with iobservations $x_1,\dots,x_n$, which we will denote as a single data matrices $X := [x_1,\dots,x_n] \in \mathbb{R}^{p\times n}$. Each observed data x_i may be associated with latent variable h_i. We will denote the matrix of all latent variables as $H := [h_1,\dots,h_n] \in \mathbb{R}^{r\times n}$. When these are random, we will use the notations $\mathbf{X} := [\mathbf{x}_1,\dots,\mathbf{x}_n] \in \mathbb{R}^{p\times n}$ and $\mathbf{H} := [\mathbf{h}_1,\dots,\mathbf{h}_n] \in \mathbb{R}^{r\times n}$.

The goal is to find a parameter $\boldsymbol{\theta}$ that maximizes the likelihood of observing $\mathbf{X}$ under the model, which can be written as (by using iterated expectation; see Exercise A.3.3)

$$\mathbb{P}_{\boldsymbol{\theta}}(\mathbf{X} = X) = \mathbb{E}_{\mathbf{H}}\left[\mathbb{P}(\mathbf{X} = X\,|\,\mathbf{H},\boldsymbol{\theta})\right] = \sum_H p(X,H\,|\,\boldsymbol{\theta}). \tag{295}$$

As always in MLE problems, we assume that the columns of $\mathbf{X}$ and $\mathbf{H}$ are i.i.d., and in this case they follow the joint distribution $p(x,h\,|\,,\theta)$. We may still denote the joint distribution of $(\mathbf{X},\mathbf{H})$ by p.

Hence we seek to find the maximum likelihood parameter

$$\hat{\boldsymbol{\theta}} := \arg\max_{\boldsymbol{\theta}} \left(\log\left(\mathbb{E}_{\mathbf{H}}\left[\mathbb{P}(\mathbf{X} = X\,|\,\mathbf{H},\boldsymbol{\theta})\right]\right) = \log\left(\sum_H p(X,H\,|\,\boldsymbol{\theta})\right)\right). \tag{296}$$

The key difficulty in solving the above optimization problem is that the expectation $\mathbb{E}_{\mathbf{H}}$ over the latent variable $\mathbf{H}$ (i.e., a sum over H) is inside the log, which does not easily simplify and result in complicated analytical expression.

For the moment, suppose that an 'oracle' told us the unknown (random) latent variable $\mathbf{H}$ is in fact a particular value H. Then we do not have to average over $\mathbf{H}$ and simply need to solve

$$\arg\max_{\boldsymbol{\theta}} \left(\log\left(\mathbb{P}(\mathbf{X} = X\,|\,H,\boldsymbol{\theta})\right) = \log\big(p(X,H\,|\,\boldsymbol{\theta})\big)\right). \tag{297}$$

[14]Still in this case the columns $\mathbf{W}_N$ need not be orthonormal, but one may use Gram-Schmidt to find an ONB for the column span of $\mathbf{W}_N$ and corresponding eigenvalues can be found by multiplying each basis vector to $\mathbf{S}$.

However, in reality $\mathbf{H}$ is random so we may prefer to maximize the expectation $\mathbb{E}_{\mathbf{H}}[\log(\mathbb{P}(\mathbf{X}=X\,|\,\mathbf{H},\boldsymbol{\theta}))]$ instead of solving the above one-point problem. Now note that the distribution of $\mathbf{H}$ conditional on X is given by

$$\mathbf{H} \sim p(H\,|\,X,\boldsymbol{\theta}). \tag{298}$$

Even though X is known at this point, the parameter $\boldsymbol{\theta}$ is not. In order to proceed, we pretend $\boldsymbol{\theta}$ equals our current best estimate $\boldsymbol{\theta}^{\text{old}}$. Then we know the distribution of $\mathbf{H}$ given X and $\boldsymbol{\theta}=\boldsymbol{\theta}^{\text{old}}$, so a better approximation of the original problem (296) than (297) would be the following:

$$\boldsymbol{\theta}^{\text{new}} = \arg\max_{\boldsymbol{\theta}} \mathbb{E}_{\mathbf{H}}\left[\log(\mathbb{P}(\mathbf{X}=X\,|\,\mathbf{H},\boldsymbol{\theta}))\;\middle|\;X,\boldsymbol{\theta}^{\text{old}}\right] \tag{299}$$

$$= \arg\max_{\boldsymbol{\theta}} \left(\sum_{H} \log(\mathbb{P}(\mathbf{X}=X\,|\,H,\boldsymbol{\theta}))\; p(H\,|\,X,\boldsymbol{\theta}^{\text{old}})\right). \tag{300}$$

We can view (299) as an iterative algorithm that allows us to update the current best parameter $\boldsymbol{\theta}^{\text{old}}$ to a new one $\boldsymbol{\theta}^{\text{new}}$. We call this the *expectation-maximization* (EM) algorithm since we first take the expectation $\mathbb{E}_{\mathbf{H}}$ over the latent variable using the observed data X and the current best parameter $\boldsymbol{\theta}^{\text{old}}$, and then maximize the resulting function in $\boldsymbol{\theta}$ to get the next estimate $\boldsymbol{\theta}^{\text{new}}$.

Algorithm 5 EM algorithm for MLE with latent variable

1: **Input:** $\mathbf{X}\in\mathbb{R}^{d\times n}$ (data matrix); $r\geq 1$ (rank parameter); $\boldsymbol{\theta}_0\in\mathbb{R}^{d\times r}$ (initial estimate); N (number of iterations)
2: **for** $n=1,\ldots,N$ **do**:
3: $Q(\boldsymbol{\theta}\,|\,\boldsymbol{\theta}_{n-1}) \leftarrow \mathbb{E}_{\mathbf{H}}\left[\log(\mathbb{P}(\mathbf{X}=X\,|\,\mathbf{H},\boldsymbol{\theta}))\;\middle|\;X,\boldsymbol{\theta}_{n-1}\right]$ (E-step)
4: $\boldsymbol{\theta}_n \leftarrow \arg\max_{\boldsymbol{\theta}} Q(\boldsymbol{\theta}\,|\,\boldsymbol{\theta}_{n-1})$ (M-step)
5: **end for**
6: **output:** $\boldsymbol{\theta}_N$

Proposition 4.6.1 (Monotonicity of EM algorithm)**.** *Let $\boldsymbol{\theta}_n$ given by the EM algorithm (Algorithm 5). Then for all $k\geq 0$,*

$$p(X\,|\,\boldsymbol{\theta}_k) \leq p(X\,|\,\boldsymbol{\theta}_{k+1}). \tag{301}$$

In other words, EM algorithm improves (increases) the likelihood $p(X\,|\,\boldsymbol{\theta}_k)$ monotonimcally.

PROOF. We will show the following inequality holds for all $k\geq 1$:

$$\log p(X\,|\,\boldsymbol{\theta}) - p(X\,|\,\boldsymbol{\theta}_k) \geq Q(\boldsymbol{\theta}\,|\,\boldsymbol{\theta}_k) - Q(\boldsymbol{\theta}_k\,|\,\boldsymbol{\theta}_k), \tag{302}$$

where $Q(\boldsymbol{\theta}\,|\,\boldsymbol{\theta}_k) := \mathbb{E}_{\mathbf{H}}\left[\log\big(p(X\,|\,\mathbf{H},\boldsymbol{\theta})\big)\;\middle|\;X,\boldsymbol{\theta}_k\right]$. To see this, first note that $p(X,H\,|\,\boldsymbol{\theta}) = p(X\,|\,H,\boldsymbol{\theta})\cdot p(H\,|\,\boldsymbol{\theta})$. So we can write

$$\log p(X\,|\,\boldsymbol{\theta}) = \log p(X,H\,|\,\boldsymbol{\theta}) - \log p(H\,|\,X,\boldsymbol{\theta}) \tag{303}$$

Since the left hand side does not depend on H,

$$\log p(X\,|\,\boldsymbol{\theta}) = \mathbb{E}_{\mathbf{H}}\left[\log p(X\,|\,\boldsymbol{\theta})\;\middle|\;X,\boldsymbol{\theta}_k\right]. \tag{304}$$

$$= \underbrace{\mathbb{E}_{\mathbf{H}}\left[\log p(X,\mathbf{H}\,|\,\boldsymbol{\theta})\;\middle|\;X,\boldsymbol{\theta}_k\right]}_{Q(\boldsymbol{\theta}\,|\,\boldsymbol{\theta}_k)} + \underbrace{\mathbb{E}_{\mathbf{H}}\left[-\log p(\mathbf{H}\,|\,X,\boldsymbol{\theta})\;\middle|\;X,\boldsymbol{\theta}_k\right]}_{H(\boldsymbol{\theta}\,|\,\boldsymbol{\theta}_k)}. \tag{305}$$

The above equation holds for all $\boldsymbol{\theta}$, including $\boldsymbol{\theta}=\boldsymbol{\theta}_k$. So subtracting the two resulting equations,

$$\log p(X\,|\,\boldsymbol{\theta}) - \log p(X\,|\,\boldsymbol{\theta}_k) = Q(\boldsymbol{\theta}\,|\,\boldsymbol{\theta}_k) - Q(\boldsymbol{\theta}_k\,|\,\boldsymbol{\theta}_k) + [H(\boldsymbol{\theta}\,|\,\boldsymbol{\theta}_k) - H(\boldsymbol{\theta}_k\,|\,\boldsymbol{\theta}_k)]. \tag{306}$$

It then remains to show the last term in the bracket is nonnegative. Indeed,

$$H(\boldsymbol{\theta}\,|\,\boldsymbol{\theta}_k) - H(\boldsymbol{\theta}_k\,|\,\boldsymbol{\theta}_k) = \mathbb{E}_{\mathbf{H}}\left[-\log\frac{p(\mathbf{H}\,|\,X,\boldsymbol{\theta})}{p(\mathbf{H}\,|\,X,\boldsymbol{\theta}_k)}\,\middle|\,X,\boldsymbol{\theta}_k\right] \tag{307}$$

$$= D_{KL}\Big(p(\cdot\,|\,X,\boldsymbol{\theta})\,\Big\|\,p(\cdot\,|\,X,\boldsymbol{\theta}_k)\Big) \geq 0, \tag{308}$$

where the last equality uses the definition of KL divergence and the inequality is due to Gibb's inequality (see Exercise 4.6.3). This shows the assertion. □

Exercise 4.6.2 (log-sum inequality)**.** Let $a_1,\ldots,a_n \geq 0$ and $b_1,\ldots,b_n \geq 0$. Show the following inequality holds:

$$\sum_{i=1}^{n} a_i \log\frac{a_i}{b_i} \geq \left(\sum_{i=1}^{n} a_i\right)\log\frac{\sum_{i=1}^{n} a_i}{\sum_{i=1}^{n} b_i}. \tag{309}$$

Furthermore, show that the equality above holds if and only if a_i/b_i does not depend on $i = 1,\ldots,n$.

Exercise 4.6.3 (Gibb's inequality)**.** Let $\mathbf{p},\mathbf{q}$ denote probability distributions on the same probability space. Define the relative entropy (a.k.a. KullbackLeibler divergence) from $\mathbf{q}$ to $\mathbf{p}$ as

$$D_{KL}(\mathbf{p}\,\|\,\mathbf{q}) = \mathbb{E}_{X\sim\mathbf{p}}\left[-\log\big(\mathbf{q}(X)/\mathbf{p}(X)\big)\right]. \tag{310}$$

(i) Show that

$$\mathbb{E}_{X\sim\mathbf{p}}\left[-\log\big(\mathbf{q}(X)/\mathbf{p}(X)\big)\right] \geq -\log\mathbb{E}_{X\sim\mathbf{p}}\left[\big(\mathbf{q}(X)/\mathbf{p}(X)\big)\right] \tag{311}$$

$$= -\log\int q(X)\,dX = -\log 1 = 0. \tag{312}$$

(ii) Deduce Gibb's inequality from **(i)**: $D_{KL}(\mathbf{p}\,\|\,\mathbf{q}) \geq 0$, where the equality holds iff $\mathbf{p} = \mathbf{q}$.
(iii) Deduce the same result as in **(ii)** by using the log-sum inequality (see Exercise 4.6.2).

6.4. Probabilistic PCA. The *probabilistic PCA* [Row98, TB99] is the following *latent variable* model that explains a random observable $\mathbf{x} \in \mathbb{R}^p$ as a linear transform of some lower dimensional latent variable $\mathbf{h} \in \mathbb{R}^r$:

$$\mathbf{x} = \mathbf{W}\mathbf{h} + \boldsymbol{\mu} + \boldsymbol{\varepsilon}, \qquad \mathbf{h} \sim N(\mathbf{0},\mathbf{I}), \qquad \boldsymbol{\varepsilon} \sim N(\mathbf{0},\sigma^2\mathbf{I}), \tag{313}$$

where $\mathbf{h}$ and $\boldsymbol{\varepsilon}$ are independent. More explicitly, $\mathbf{x}$ is modeled as a Gaussian RV:

$$\mathbf{x} \sim N(\boldsymbol{\mu}, \mathbf{W}\mathbf{W}^T + \sigma^2\mathbf{I}). \tag{314}$$

Indeed, it is easy to see that $\mathbf{x}$ is a Gaussian RV, so we only need to compute its mean and covariance matrix as follow:

$$\mathbb{E}[\mathbf{x}] = \mathbb{E}[\mathbf{W}\mathbf{h} + \boldsymbol{\mu} + \boldsymbol{\varepsilon}] = \mathbb{E}[\mathbf{W}\mathbf{h}] + \boldsymbol{\mu} = \mathbf{W}\mathbb{E}[\mathbf{h}] + \boldsymbol{\mu} = \boldsymbol{\mu}, \tag{315}$$

$$\mathrm{Cov}(\mathbf{x}) = \mathbb{E}\left[(\mathbf{x}-\boldsymbol{\mu})(\mathbf{x}-\boldsymbol{\mu})^T\right] \tag{316}$$

$$= \mathbb{E}\left[(\mathbf{W}\mathbf{h} + \boldsymbol{\varepsilon})(\mathbf{W}\mathbf{h} + \boldsymbol{\varepsilon})^T\right] \tag{317}$$

$$= \mathbf{W}\mathbb{E}\left[\mathbf{h}\mathbf{h}^T\right]\mathbf{W}^T + \mathbb{E}[\boldsymbol{\varepsilon}]\mathbb{E}[\mathbf{h}^T]\mathbf{W}^T + \mathbf{W}]\mathbb{E}[\mathbf{h}]\mathbb{E}[\boldsymbol{\varepsilon}^T] = \mathbf{W}\mathbf{W}^T, \tag{318}$$

where for the covariance computation, we have used the fact that $\mathbb{E}[\mathbf{h}\mathbf{h}^T] = \mathrm{Cov}(\mathbf{h}) = \mathbf{I}$, the independence between $\mathbf{h}$ and $\boldsymbol{\varepsilon}$, and $\mathbb{E}[\boldsymbol{\varepsilon}] = \mathbf{0}$.

Denote $\widetilde{X} := X - [\boldsymbol{\mu},\ldots,\boldsymbol{\mu}]$ and $\mathbf{C} := \mathbf{W}\mathbf{W}^T + \sigma^2\mathbf{I}$. The log likelihood of observing $X = [\vec{x}_1,\ldots,\vec{x}_n]$ from (314) is

$$\ell(X\,|\,\boldsymbol{\mu},\mathbf{W},\sigma) = \sum_{i=1}^{n} \log(2\pi)^{-d/2}|\mathbf{C}|^{-1/2} - \frac{1}{2}(\vec{x}_i - \boldsymbol{\mu})^T\mathbf{C}^{-1}(\vec{x}_i - \boldsymbol{\mu}) \tag{319}$$

$$= -\frac{n}{2}\left(d\log(2\pi) + \log|\mathbf{C}| + \mathrm{tr}\left(\widetilde{X}\mathbf{C}^{-1}\widetilde{X}\right)\right). \tag{320}$$

From this, by differentiating $\boldsymbol{\mu}$ and setting it equal zero, we find the maximum likelihood estimate of $\boldsymbol{\mu}$ is

$$\hat{\boldsymbol{\mu}} = \overline{X} = \frac{1}{n}\sum_{i=1}^{n} \vec{x}_i. \tag{321}$$

On the other hand, the maximum likelihood estimates of $\mathbf{W}$ and σ may be found by using EM algorithm. In order to do this, we first need the conditional distribution of the hidden variable $\mathbf{h}$ given the data $\mathbf{x}$ as in the following exercise.

Exercise 4.6.4. Let $(\mathbf{x},\mathbf{h})$ denote the pair of observable and latent variables in the probabilistic PCA in (313). Show that the distribution of $\mathbf{h}$ given $\mathbf{x}$ is the following Gaussian distribution:

$$\mathbf{h}\,|\,\mathbf{x} \sim N\left(\mathbf{M}^{-1}\mathbf{W}^T(\mathbf{x}-\boldsymbol{\mu}), \sigma^{-2}\mathbf{M}^{-1}\right), \tag{322}$$

where $\mathbf{M} := \mathbf{W}^T\mathbf{W} + \sigma^2\mathbf{I} \in \mathbb{R}^{r\times r}$. In particular, the expected value of $\mathbf{h}$ given $\mathbf{x}$ is

$$\mathbb{E}[\mathbf{h}\,|\,\mathbf{x}] = \left(\mathbf{W}^T\mathbf{W} + \sigma^2\mathbf{I}\right)^{-1}\mathbf{W}^T(\mathbf{x}-\boldsymbol{\mu}). \tag{323}$$

(Hint: Use Bayes' Theorem and properties of multivariate Gaussian distribution, see [Bis06, Sec. 2.3.3].)

Now we are ready to derive the EM algorithm for probabilistic PCA.

Proposition 4.6.5 (EM algorithm for PPCA)**.** *Fix $X \in \mathbb{R}^{d\times n}$ and denote $\widetilde{X} = X - [\overline{X},\dots,\overline{X}]$ where $\overline{X} = \frac{1}{n}\sum_{i=1}^{n} X[:,i]$. The EM algorithm for PPCA (Algorithm 5) equals the following iterative algorithm:*

$$\begin{cases} \mathbb{E}[\mathbf{H}_k] & \leftarrow \left(\mathbf{W}_{k-1}^T\mathbf{W}_{k-1} + \sigma_{k-1}^2\mathbf{I}\right)\mathbf{W}_{k-1}^T\widetilde{X}, \\ \mathbb{E}[\mathbf{H}_k\mathbf{H}_k^T] & \leftarrow \sigma_{k-1}^2\left(\mathbf{W}_{k-1}^T\mathbf{W}_{k-1} + \sigma_{k-1}^2\mathbf{I}\right) + \mathbb{E}[\mathbf{H}_k]\mathbb{E}[\mathbf{H}_k]^T \\ \mathbf{W}_k & \leftarrow \widetilde{X}\mathbb{E}[\mathbf{H}_k^T](\mathbb{E}[\mathbf{H}_k\mathbf{H}_k^T])^{-1}, \\ \sigma_k^2 & \leftarrow n^{-1}\mathrm{tr}\left((\widetilde{X} - \mathbf{W}_k\mathbb{E}[\mathbf{H}_k])^T(\widetilde{X} - \mathbf{W}_k\mathbb{E}[\mathbf{H}_k])\right). \end{cases} \tag{324}$$

PROOF. (Ref: [TB99]) For the E-step, first we compute the conditional likelihood of observing the data $X = [\vec{x}_1,\dots,\vec{x}_n]$ given the latent variable $H = [\vec{h}_1,\dots,\vec{h}_n]$. Denote $\boldsymbol{\theta} = [\mathbf{W},\sigma]$ and suppose we are given $\boldsymbol{\theta}_{k-1} = [\mathbf{W}_{k-1},\sigma_{k-1}]$. Then

$$\ell(X\,|\,H,\boldsymbol{\theta}) := \sum_{i=1}^{n}\log\left(p(\vec{x}_i\,|\,\vec{h}_i,\boldsymbol{\theta})\right) \tag{325}$$

$$= n\log(\sigma^{-1}(2\pi)^{-d/2}) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}\|\vec{x}_i - \mathbf{W}\vec{h}_i - \boldsymbol{\mu}\|_F^2 \tag{326}$$

$$= n\log(\sigma^{-1}(2\pi)^{-d/2}) - \frac{1}{2\sigma^2}\mathrm{tr}\left((\widetilde{X} - \mathbf{W}H)^T(\widetilde{X} - \mathbf{W}H)\right) \tag{327}$$

$$= n\log(\sigma^{-1}(2\pi)^{-d/2}) - \frac{1}{2\sigma^2}\left[\mathrm{tr}(\widetilde{X}\widetilde{X}^T) - 2\mathrm{tr}(XH^T\mathbf{W}^T) + \mathrm{tr}(\mathbf{W}HH^T\mathbf{W}^T)\right]. \tag{328}$$

The E-step in Algorithm 5 amounts computing the following expectation:

$$Q(\boldsymbol{\theta}\,|\,\boldsymbol{\theta}_{k-1}) = \mathbb{E}_{\mathbf{H}}\left[\ell(X\,|\,\mathbf{H},\boldsymbol{\theta})\,\middle|\,X,\boldsymbol{\theta}_{k-1}\right] \tag{329}$$

$$= n\log(\sigma^{-1}(2\pi)^{-d/2}) - \frac{1}{2\sigma^2}\left[\mathrm{tr}(\widetilde{X}\widetilde{X}^T) - 2\mathrm{tr}(\widetilde{X}\mathbb{E}[\mathbf{H}]^T\mathbf{W}^T) + \mathrm{tr}(\mathbf{W}\mathbb{E}[\mathbf{H}\mathbf{H}^T]\mathbf{W}^T)\right], \tag{330}$$

where, using Exercise (4.6.4),

$$\mathbb{E}[\mathbf{H}] := \mathbb{E}_{\mathbf{H}}\left[\mathbf{H}\,\middle|\,X,\boldsymbol{\theta}_{k-1}\right] = \mathbf{M}^{-1}\mathbf{W}_n^T\widetilde{X} = \left(\mathbf{W}_{k-1}^T\mathbf{W}_{k-1} + \sigma_{k-1}^2\mathbf{I}\right)\mathbf{W}_{k-1}^T\widetilde{X} \tag{331}$$

$$\mathbb{E}[\mathbf{H}\mathbf{H}^T] := \mathbb{E}_{\mathbf{H}}\left[\mathbf{H}\mathbf{H}^T\,\middle|\,X,\boldsymbol{\theta}_{k-1}\right] = \mathrm{Cov}(\mathbf{H}) + \mathbb{E}[\mathbf{H}]\mathbb{E}[\mathbf{H}]^T = \sigma_{k-1}^2\left(\mathbf{W}_{k-1}^T\mathbf{W}_{k-1} + \sigma_{k-1}^2\mathbf{I}\right) + \mathbb{E}[\mathbf{H}]\mathbb{E}[\mathbf{H}]^T. \tag{332}$$

Now differentiating $Q(\boldsymbol{\theta}\,|\,\boldsymbol{\theta}_{k-1})$ above with respect to $\mathbf{W}$ and σ gives

$$\nabla_{\mathbf{W}} Q(\boldsymbol{\theta}\,|\,\boldsymbol{\theta}_{k-1}) = -\sigma^{-2}\left[\mathbf{W}\mathbb{E}[\mathbf{H}\mathbf{H}^T] - \widetilde{X}\mathbb{E}[\mathbf{H}]^T\right], \tag{333}$$

$$\frac{\partial}{\partial\sigma} Q(\boldsymbol{\theta}\,|\,\boldsymbol{\theta}_{k-1}) = -\frac{n}{\sigma} + \frac{1}{\sigma^3}\mathrm{tr}\left((\widetilde{X} - \mathbf{W}\mathbb{E}[\mathbf{H}])^T(\widetilde{X} - \mathbf{W}\mathbb{E}[\mathbf{H}])\right). \tag{334}$$

Setting these equal to zero, we get

$$\mathbf{W}_n = \widetilde{X}\mathbb{E}[\mathbf{H}]^T(\mathbb{E}[\mathbf{H}\mathbf{H}^T])^{-1}, \tag{335}$$

$$\sigma_n^2 = \frac{1}{n}\mathrm{tr}\left((\widetilde{X} - \mathbf{W}\mathbb{E}[\mathbf{H}])^T(\widetilde{X} - \mathbf{W}\mathbb{E}[\mathbf{H}])\right). \tag{336}$$

Denoting $\mathbb{E}[\mathbf{H}_n] = \mathbb{E}[\mathbf{H}]$ and $\mathbb{E}[\mathbf{H}_n\mathbf{H}_n^T] = \mathbb{E}[\mathbf{H}\mathbf{H}^T]$, we obtain the claimed iterative algorithm. □

Remark 4.6.6. Consider the special case of 'zero-noise limit' $\sigma \searrow 0$. Then taking $\sigma_k \searrow 0$ for each $k \geq 1$, the EM algorithm for PPCA in Proposition 4.6.5 reduces to

$$\begin{cases} \mathbf{H}_k & \leftarrow \left(\mathbf{W}_{n-1}^T\mathbf{W}_{n-1}\right)\mathbf{W}_{n-1}^T\widetilde{X}, \\ \mathbf{W}_k & \leftarrow \widetilde{X}\mathbf{H}_k^T(\mathbf{H}_k\mathbf{H}_k^T)^{-1}, \end{cases} \tag{337}$$

which is exactly Algorithm 4 we gave earlier.

CHAPTER 5

Markov chains

Say we would like to model the USD price of bitcoin. We could observe the actual price at every hour and record it by a sequence of real numbers $x_1, x_2, \cdots$. However, it is more interesting to build a 'model' that could predict the price of bitcoin at time t, or at least give some meaningful insight how the actual bitcoin price behaves over time. Since there are so many factors affecting the price at every time, it might be reasonable that the price at time t should be given by a certain RV, say X_t. Then our sequence of predictions would be a sequence of RVs, $(X_t)_{t\geq 0}$. This is an example of *stochastic processes*. Here 'process' means that we are not interest in just a single RV, that their sequence as a whole: 'stochastic' means that the way the RVs evolve in time might be random.

In this note, we will be studying a very important class of stochastic processes called *Markov chains*. The importance of Markov chains lies two places: 1) They are applicable for a wide range of physical, biological, social, and economical phenomena, and 2) the theory is well-established and we can actually compute and make predictions using the models.

1. Definition and examples

Roughly speaking, *Markov processes* are used to model temporally changing systems where future state only depends on the current state. For instance, if the price of bitcoin tomorrow depends only on its price today, then bitcoin price can be modeled as a Markov process. (Of course, the entire history of price often affects decisions of buyers/sellers so it may not be a realistic assumption.)

Even through Markov processes can be defined in vast generality, we concentrate on the simplest setting where the state and time are both discrete. Let $\Omega = \{1, 2, \cdots, m\}$ be a finite set, which we call the *state space*. Consider a sequence $(X_t)_{t\geq 0}$ of Ω-valued RVs, which we call a *chain*. We call the value of X_t the *state* of the chain at time t. In order to narrow down the way the chain $(X_t)_{t\geq 0}$ behaves, we introduce the following properties:

(i) (Markov property) The distribution of X_{t+1} given the history $X_0, X_1, \cdots, X_t$ depends only on X_t. That is, for any values of $j_0, j_1, \cdots, j_t, k \in \Omega$,

$$\mathbb{P}(X_{t+1} = k \,|\, X_t = j_t, X_{t-1} = j_{t-1}, \cdots, X_1 = j_1, X_0 = j_0) = \mathbb{P}(X_{t+1} = k \,|\, X_t = j_t). \tag{338}$$

(ii) (Time-homogeneity) The transition probabilities

$$p_{ij} = \mathbb{P}(X_{t+1} = j \,|\, X_t = i) \qquad i, j \in \Omega \tag{339}$$

do not depend on t.

When the chain $(X_t)_{t\geq 0}$ satisfies the above two properties, we say it is a (discrete-time and time-homogeneous) *Markov chain*. We define the *transition matrix* P to be the $m \times m$ matrix of transition probabilities:

$$P = (p_{ij})_{1\leq i,j\leq m} = \begin{bmatrix} p_{11} & p_{12} & \cdots & p_{1m} \\ p_{21} & p_{22} & \cdots & p_{2m} \\ \vdots & \vdots & & \vdots \\ p_{m1} & p_{m2} & \cdots & p_{mm} \end{bmatrix}. \tag{340}$$

Finally, since the state X_t of the chain is a RV, we represent its probability mass function (PMF) via a row vector

$$\mathbf{r}_t = [\mathbb{P}(X_t = 1), \mathbb{P}(X_t = 2), \cdots, \mathbb{P}(X_t = m)]. \tag{341}$$

Example 5.1.1. Let $\Omega = \{1,2\}$ and let $(X_t)_{t\geq 0}$ be a Markov chain on Ω with the following transition matrix

$$P = \begin{bmatrix} p_{11} & p_{12} \\ p_{21} & p_{22} \end{bmatrix}. \tag{342}$$

We can also represent this Markov chain pictorially as in Figure 65, which is called the 'state space diagram' of the chain $(X_t)_{t\geq 0}$.

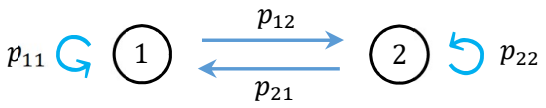


FIGURE 62. State space diagram of a 2-state Markov chain

For some concrete example, suppose

$$p_{11} = 0.2, \quad p_{12} = 0.8, \quad p_{21} = 0.6, \quad p_{22} = 0.4. \tag{343}$$

If the initial state of the chain X_0 is 1, then

$$\mathbb{P}(X_1 = 1) = \mathbb{P}(X_1 = 1 \,|\, X_0 = 1)\mathbb{P}(X_0 = 1) + \mathbb{P}(X_1 = 1 \,|\, X_0 = 2)\mathbb{P}(X_0 = 2) \tag{344}$$

$$= \mathbb{P}(X_1 = 1 \,|\, X_0 = 1) = p_{11} = 0.2 \tag{345}$$

and similarly,

$$\mathbb{P}(X_1 = 2) = \mathbb{P}(X_1 = 2 \,|\, X_0 = 1)\mathbb{P}(X_0 = 1) + \mathbb{P}(X_1 = 2 \,|\, X_0 = 2)\mathbb{P}(X_0 = 2) \tag{346}$$

$$= \mathbb{P}(X_1 = 2 \,|\, X_0 = 1) = p_{12} = 0.8. \tag{347}$$

Also we can compute the distribution of X_2. For example,

$$\mathbb{P}(X_2 = 1) = \mathbb{P}(X_2 = 1 \,|\, X_1 = 1)\mathbb{P}(X_1 = 1) + \mathbb{P}(X_2 = 1 \,|\, X_1 = 2)\mathbb{P}(X_1 = 2) \tag{348}$$

$$= p_{11}\mathbb{P}(X_1 = 1) + p_{21}\mathbb{P}(X_1 = 2) \tag{349}$$

$$= 0.2 \cdot 0.2 + 0.6 \cdot 0.8 = 0.04 + 0.48 = 0.52. \tag{350}$$

In general, the distribution of X_{t+1} can be computed from that of X_t via a simple linear algebra. Note that for $i = 1,2$,

$$\mathbb{P}(X_{t+1} = i) = \mathbb{P}(X_{t+1} = i \,|\, X_t = 1)\mathbb{P}(X_t = 1) + \mathbb{P}(X_{t+1} = i \,|\, X_t = 2)\mathbb{P}(X_t = 2) \tag{351}$$

$$= p_{1i}\mathbb{P}(X_t = 1) + p_{2i}\mathbb{P}(X_t = 2). \tag{352}$$

This can be written as

$$[\mathbb{P}(X_{t+1} = 2), \mathbb{P}(X_{t+1} = 2)] = [\mathbb{P}(X_{t+1} = 2), \mathbb{P}(X_{t+1} = 2)] \begin{bmatrix} p_{11} & p_{12} \\ p_{21} & p_{22} \end{bmatrix}. \tag{353}$$

That is, if we represent the distribution of X_t as a row vector, then the distribution of X_{t+1} is given by multiplying the transition matrix P to the left. ▲

Example 5.1.2 (Gambler's ruin)**.** Suppose a gambler has fortune of k dolors initially and starts gambling. At each time he wins or loses 1 dolor independently with probability p and $1-p$, respectively. The game ends when his fortune reaches either 0 or N dolors. What is the probability that he wins N dolors and goes home happy?

We use Markov chains to model his fortune after betting t times. Namely, let $\Omega = \{0,1,2,\cdots,N\}$ be the state space. Let $(X_t)_{t\geq 0}$ be a sequence of RVs where X_t is the gambler's fortune after betting t times. We first draw the state space diagram for $N = 4$ below: Next, we can write down its transition probabilities as

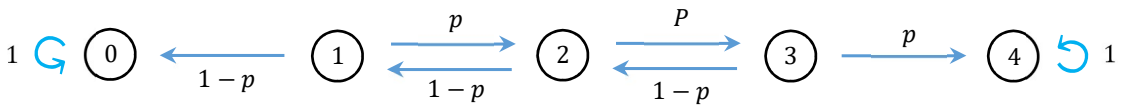


FIGURE 63. State space diagram of a 5-state gambler's chain

$$\begin{cases} \mathbb{P}(X_{t+1} = k+1 \,|\, X_t = k) = p & \forall 1 \le k < N \\ \mathbb{P}(X_{t+1} = k \,|\, X_t = k+1) = 1-p & \forall 1 \le k < N \\ \mathbb{P}(X_{t+1} = 0 \,|\, X_t = 0) = 1 \\ \mathbb{P}(X_{t+1} = N \,|\, X_t = N) = 1. \end{cases} \tag{354}$$

For example, the transition matrix P for $N = 5$ is given by

$$P = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 1-p & 0 & p & 0 & 0 & 0 \\ 0 & 1-p & 0 & p & 0 & 0 \\ 0 & 0 & 1-p & 0 & p & 0 \\ 0 & 0 & 0 & 1-p & 0 & p \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}. \tag{355}$$

We call the resulting Markov chain $(X_t)_{t\ge 0}$ the *gambler*'s chain. ▲

Example 5.1.3 (Ehrenfest Chain)**.** This chain is originated from the physics literature as a model for two cubical volumes of air connected by a thin tunnel. Suppose there are total N indistinguishable balls split into two "urns" A and B. At each step, we pick up one of the N balls uniformly at random, and move it to the other urn. Let X_t denote the number of balls in urn A after t steps. This is a Markov chain called the *Ehrenfest chain*. (See the state space diagram in Figure 64.)

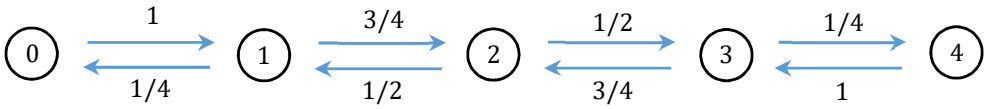


FIGURE 64. State space diagram of the Ehrenfest chain with 4 balls

It is easy to figure out the transition probabilities by considering different cases. If $X_t = k$, then urn B has $N-k$ balls at time t. If $0 < k < N$, then with probability k/N we move one ball from A to B and with probability $(N-k)/N$ we move one from B to A. If $k = 0$, then we must pick up a ball from urn B so $X_{t+1} = 1$ with probability 1. If $k = N$, then we must move one from A to B and $X_{t+1} = N-1$ with probability 1. Hence, the transition kernel is given by

$$\begin{cases} \mathbb{P}(X_{t+1} = k+1 \,|\, X_t = k) = (N-k)/N & \forall 0 \le k < N \\ \mathbb{P}(X_{t+1} = k-1 \,|\, X_t = k) = k/N & \forall 0 < k \le N \\ \mathbb{P}(X_{t+1} = 1 \,|\, X_t = 0) = 1 \\ \mathbb{P}(X_{t+1} = N-1 \,|\, X_t = N) = 1. \end{cases} \tag{356}$$

For example, the transition matrix P for $N = 5$ is given by

$$P = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 1/5 & 0 & 4/5 & 0 & 0 & 0 \\ 0 & 2/5 & 0 & 3/5 & 0 & 0 \\ 0 & 0 & 3/5 & 0 & 2/5 & 0 \\ 0 & 0 & 0 & 4/5 & 0 & 1/5 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}. \tag{357}$$

▲

Exercise 5.1.4. Repeat rolling two four sided dices with numbers 1,2,3, and 4 on them. Let Y_k be the some of the two dice at the kth roll. Let $S_n = Y_1 + Y_2 + \cdots + Y_n$ be the total of the first n rolls, and define $X_t = S_t \,(\text{mod} 6)$. Show that $(X_t)_{t\geq 0}$ is a Markov chain on the state space $\Omega = \{0,1,2,3,4,5\}$. Furthermore, identify its transition matrix.

We generalize our observation in Example 5.1.1 in the following exercise.

Exercise 5.1.5. Let $(X_t)_{t\geq 0}$ be a Markov chain on state space $\Omega = \{1,2,\cdots,m\}$ with transition matrix $P = (p_{ij})_{1\leq i,j\leq m}$. Let $\mathbf{r}_t = [\mathbb{P}(X_t = 1),\cdots,\mathbb{P}(X_t = m)]$ denote the row vector of the distribution of X_t.

(i) Show that for each $i \in \Omega$,

$$\mathbb{P}(X_{t+1} = i) = \sum_{j=1}^{m} p_{ji}\mathbb{P}(X_t = j). \tag{358}$$

(ii) Show that for each $t \geq 0$,

$$\mathbf{r}_{t+1} = \mathbf{r}_t P. \tag{359}$$

(iii) Show by induction that for each $t \geq 0$,

$$\mathbf{r}_t = \mathbf{r}_0 P^t. \tag{360}$$

Exercise 5.1.6. Let $(X_t)_{t\geq 0}$ be a Markov chain on state space $\Omega = \{1,2,\cdots,m\}$ with transition matrix $P = (p_{ij})_{1\leq i,j\leq m}$.

(i) (Multi-step transition prob.) Show that for each $x, y \in \Omega$ and $a, b \geq 1$,

$$\mathbb{P}\left(X_{a+b} = y \,|\, X_a = x\right) = P^b(x,y). \tag{361}$$

Hint: Use Exercise 5.1.5.

(ii) (The Chapman-Kolmogorov eq.) Show that for each $x, y \in \Omega$ and $n, m \geq 1$,

$$P^{n+m}(x,y) = \sum_{z\in\Omega} P^n(x,z)P^m(z,y). \tag{362}$$

While right-multiplication of P advances a given row vector of distribution one step forward in time, left-multiplication of P on a column vector computes the expectation of a given function with respect to the future distribution. This point is clarified in the following exercise.

Exercise 5.1.7. Let $(X_t)_{t\geq 0}$ be a Markov chain on a state space $\Omega = \{1,2,\cdots,m\}$ with transition matrix P. Let $f : \Omega \to \mathbb{R}$ be a function. Suppose that if the chain X_t has state x at time t, then we get a 'reward' of $f(x)$. Let $\mathbf{r}_t = [\mathbb{P}(X_t = 1),\cdots,\mathbb{P}(X_t = m)]$ be the distribution of X_t. Let $\mathbf{v} = [f(1), f(2),\cdots, f(m)]^T$ be the column vector representing the reward function f.

(i) Show that the expected reward at time t is given by

$$\mathbb{E}[f(X_t)] = \sum_{i=1}^{m} f(i)\mathbb{P}(X_t = i) = \mathbf{r}_t\mathbf{v}. \tag{363}$$

(ii) Use part (i) and Exercise 5.1.5 to show that

$$\mathbb{E}[f(X_t)] = \mathbf{r}_0 P^t \mathbf{v}. \tag{364}$$

(iii) The total reward up to time t is a RV given by $R_t = \sum_{k=0}^{t} f(X_t)$. Show that

$$\mathbb{E}[R_t] = \mathbf{r}_0(I + P + P^2 + \cdots + P^t)\mathbf{v}. \tag{365}$$

Exercise 5.1.8. Suppose that the probability it rains today is 0.4 if neither of the last two days was rainy, but 0.5 if at least one of the last two days was rainy. Let $\Omega = \{S, R\}$, where S=sunny and R=rainy. Let W_t be the weather of day t.

(i) Show that $(W_t)_{t\geq 0}$ is *not* a Markov chain.

(ii) Expand the state space into the set of pairs $\Sigma := \Omega^2$. For each $t \geq 0$, define $X_t = (W_{t-1}, W_t) \in \Sigma$. Show that $(X_t)_{t\geq 0}$ is a Markov chain on Σ. Identify its transition matrix.
(iii) What is the two-step transition matrix?
(iv) What is the probability that it will rain on Wednesday if it didn't rain on Sunday and Monday?

2. Stationary distribution and examples

2.1. Stationary distribution. Let $(X_t)_{t\geq 0}$ be a Markov chain on state space $\Omega = \{1, 2, \cdots, m\}$ with transition matrix P. If π is a distribution on Ω such that

$$\pi = \pi P, \tag{366}$$

then we say π is a *stationary distribution* of the Markov chain $(X_t)_{t\geq 0}$.

In Exercise 5.1.5, we observed that we can simply multiply the transition matrix P to a given row vector $\mathbf{r}_t$ of distribution on the state space Ω in order to get the next distribution $\mathbf{r}_{t+1}$. Hence if the initial distribution of the chain is π, then its distribution is invariant in time.

Example 5.2.1. Consider the 2-state Markov chain $(X_t)_{t\geq 0}$ with transition matrix (as in Exercise 5.1.1)

$$P = \begin{bmatrix} 0.2 & 0.8 \\ 0.6 & 0.4 \end{bmatrix}. \tag{367}$$

Then $\pi = [3/7, 4/7]$ is a stationary distribution of X_t. Indeed,

$$[3/7, 4/7] = [3/7, 4/7] \begin{bmatrix} 0.2 & 0.8 \\ 0.6 & 0.4 \end{bmatrix}. \tag{368}$$

Furthermore, this is the unique stationary distribution. To see this, let $\pi = [\pi_1, \pi_2]$ be a stationary distribution of X_t. Then $\pi = \pi P$ gives

$$0.2\pi_1 + 0.6\pi_2 = \pi_1 \tag{369}$$
$$0.8\pi_1 + 0.4\pi_2 = \pi_2. \tag{370}$$

These equations lead to

$$4\pi_1 = 3\pi_2. \tag{371}$$

Since π is a probability distribution, $\pi_1 + \pi_2 = 1$ so $\pi = [3/7, 4/7]$ is the only solution. This shows the uniqueness of the stationary distribution for X_t. ▲

A Markov chain may have multiple stationary distributions, as the following example illustrates.

Example 5.2.2. Let $(X_t)_{t\geq 0}$ be a 2-state Markov chain with transition matrix

$$P = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}. \tag{372}$$

Then any distribution $\pi = [p, 1-p]$ is a stationary distribution for the chain $(X_t)_{t\geq 0}$.

Stationary distributions are closely related with eigenvectors and eigenvalues of the transition matrix P. Namely, by taking transpose,

$$\pi^T = P^T \pi^T. \tag{373}$$

Hence, the transpose of any stationary distribution is an eigenvector of P^T associated with eigenvalue 1. More properties of stationary distribution in this line of thought are given by the following exercise.

Exercise 5.2.3. Given a matrix A, a row vector v, and a real number λ, we say v is a *left eigenvector* of A associated with *left eigenvalue* λ if

$$\mathbf{v}A = \lambda \mathbf{v}. \tag{374}$$

If $\mathbf{v}$ is a column vector and if $A\mathbf{v} = \lambda\mathbf{v}$, then we say v is a *(right) eigenvector* associated with *(right) eigenvalue* λ. Let $(X_t)_{t\geq 0}$ be a Markov chain on state space $\Omega = \{1, 2, \cdots, m\}$ with transition matrix $P = (p_{ij})_{1\leq i,j\leq m}$.

(i) Show that a distribution π on Ω is a stationary distribution for the chain $(X_t)_{t\geq 0}$ if and only if it is a left eigenvector of P associated with left eigenvalue 1.

(ii) Show that 1 is a right eigenvalue of P with right eigenvector $[1, 1, \cdots, 1]^T$.

(iii) Recall that a square matrix and its transpose have the same (right) eigenvalues and corresponding (right) eigenspaces have the same dimension. Show that the Markov chain $(X_t)_{t\geq 0}$ has a unique stationary distribution if any only if $[1, 1, \cdots, 1]^T$ spans the (right) eigenspace of P associated with (right) eigenvalue 1.

In the following exercise, we compute the stationary distribution of the so-called birth-death chain.

Exercise 5.2.4 (Birth-Death chain)**.** Let $\Omega = \{0, 1, 2, \cdots, N\}$ be the state space. Let $(X_t)_{t\geq 0}$ be a Markov chain on Ω with transition probabilities

$$\begin{cases} \mathbb{P}(X_{t+1} = k+1 \,|\, X_t = k) = p & \forall 0 \leq k < N \\ \mathbb{P}(X_{t+1} = k-1 \,|\, X_t = k) = 1-p & \forall 1 \leq k \leq N \\ \mathbb{P}(X_{t+1} = 0 \,|\, X_t = 0) = 1-p \\ \mathbb{P}(X_{t+1} = N \,|\, X_t = N) = p. \end{cases} \tag{375}$$

This is called a Birth-Death chain. Its state space diagram is as below.

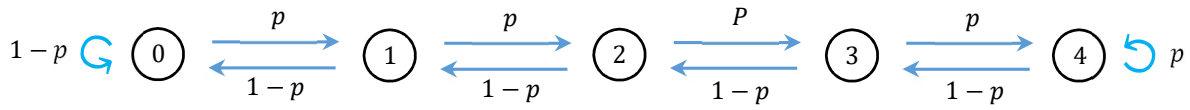


FIGURE 65. State space diagram of a 5-state Birth-Death chain

(i) Let $\pi = [\pi_0, \pi_1, \cdots, \pi_N]$ be a distribution on Ω. Show that π is a stationary distribution of the Birth-Death chain if and only if it satisfy the following 'balance equation'

$$p\pi_k = (1-p)\pi_{k+1} \qquad 0 \leq k < N. \tag{376}$$

(ii) Let $\rho = p/(1-p)$. From (ii), deduce that $\pi_k = \rho^k \pi_0$ for all $0 \leq k < N$.

(iii) Using the normalization condition $\pi_0 + \pi_1 + \cdots + \pi_N = 1$, show that $\pi_0 = 1/(1 + \rho + \rho^2 + \cdots + \rho^N)$. Conclude that

$$\pi_k = \frac{\rho^k}{1 + \rho + \rho^2 + \cdots + \rho^N} \qquad 0 \leq k \leq N. \tag{377}$$

Conclude that the Birth-Death chain has a unique stationary distribution given by (377), which becomes the uniform distribution on Ω when $p = 1/2$.

Exercise 5.2.5 (Success run chain)**.** Consider a Markov chain $(X_n)_{n\geq 0}$ on the state space $\Omega = \mathbb{Z}_{\geq 0}$ of non-negative integers with the following transition matrix P:

$$P(x, x+1) = p, \quad P(x, 0) = 1-p \qquad \text{for all } x \in \mathbb{Z}_{\geq 0}, \tag{378}$$

where $p \in [0, 1]$ is a fixed parameter. In words, one flips independent coins with success probability p; every time it comes up heads with probability p X_n increases by 1, and when it comes up tails, X_n resets back to 0.

(i) Let $\pi : \mathbb{Z}_{\geq 0} \to [0,\infty)$ be a function that satisfies $\pi P = \pi$. Show that the following recursion holds:

$$\pi(0) = (1-p)\sum_{k=0}^{\infty} \pi(k) \tag{379}$$

$$\pi(1) = p\pi(0) \tag{380}$$

$$\pi(2) = p\pi(1) \tag{381}$$

$$\vdots \tag{382}$$

Deduce that $\pi(k) = p^k \pi(0)$ for all $k \geq 1$.

(ii) Show that π is an stationary distribution on $\mathbb{Z}_{\geq 0}$ if and only if $\pi(k) = (1-p)p^k$ for $k \geq 0$. (Such π is called the Geometric(p) distribution that puts probability $p^k(1-p)$ on each $k \in \mathbb{Z}_{\geq 0}$. The number of success run of probability-p coin flips has this distribution.)

Next, we take a look at an example of an important class of Markov chains, which is called the random walk on graphs. This is the basis of many algorithms involving machine learning on networks (e.g., Google's PageRank).

Example 5.2.6 (Random walk on graphs)**.** We first introduce some notions in graph theory. A *graph* G consists of a pair (V,E) of sets of nodes V and edges $E \subseteq V^2$. A graph G can be concisely represented as a $|V| \times |V|$ matrix A_G, which is called the *adjacency matrix* of G. Namely, the (i,j) entry of A_G is defined by

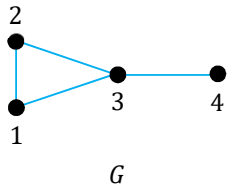
$$A_G(i,j) = \mathbf{1}(\text{nodes } i \text{ and } j \text{ are adjacent in } G). \tag{383}$$

We say G is *simple* if $(i,j) \in E$ implies $(j,i) \in E$ and $(i,i) \notin E$ for all $i \in V$. For a simple graph $G = (V,E)$, we say a node j is *adjacent* to i if $(i,j) \in E$. We denote $\deg_G(i)$ the number of neighbors of i in G, which we call the *degree* of i.

Consider we hop around the nodes of a given simple graph $G = (V,E)$: at each time, we jump from one node to one of the neighbors with equal probability. For instance, if we are currently at node 2 and if 2 is adjacent to 3, 5, and 6, then we jump to one of the three neighbors with probability $1/3$. The location of this jump process at time t can be described as a Markov chain. Namely, a Markov chain $(X_t)_{t\geq 0}$ on the node set V is called a *random walk on* G if

$$\mathbb{P}(X_{t+1} = j \,|\, X_t = i) = \frac{A_G(i,j)}{\deg_G(i)}. \tag{384}$$

Note that its transition matrix P is obtained by normalizing each row of the adjacency matrix A_G by the corresponding degree.



G

$$A_G = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}, \qquad P = \begin{bmatrix} 0 & 1/2 & 1/2 & 0 \\ 1/2 & 0 & 1/2 & 0 \\ 1/3 & 1/3 & 0 & 1/3 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

FIGURE 66. A 4-node simple graph G, its adjacency matrix A_G, and associated random walk transition matrix P

What is the stationary distribution of random walk on G? There could be many (see Exercise 5.7.1), but here is a typical one that always works. Define a probability distribution π on V by

$$\pi(i) = \frac{\deg_G(i)}{\sum_{j\in V} \deg_G(i)} = \frac{\deg_G(i)}{2|E|}. \tag{385}$$

Namely, the probability that $X_t = i$ is proportional to the degree of node i. Then π is a stationary distribution for P. Indeed,

$$\sum_{i\in V} \pi(j)P(j,i) = \sum_{i\in V} \frac{\deg_G(i)}{2|E|}\frac{A_G(i,j)}{\deg_G(i)} \tag{386}$$

$$= \frac{1}{2|E|}\sum_{i\in V} A_G(i,j) = \frac{\deg_G(j)}{2|E|} = \pi(j). \tag{387}$$

Hence if we perform a random walk on facebook network, then we are about ten times more likely to visit a person of 100 friends than to visit a person of 10 friends. ▲

Next, we will discuss two classes of Markov chains for which it is very easy to compute stationary distributions.

2.2. Reversible chains. A probability distribution μ on Ω is *reversible* for transition matrix $P : \Omega^2 \to [0,1]$ if the following *detailed balance equation* is satisfied:

$$\mu(x)\,P(x,y) = \mu(y)\,P(y,x) \qquad \text{for all } x,y \in \Omega. \tag{388}$$

Below we observe that reversiblity implies stationarity.

Proposition 5.2.7. *Let P be a transition matrix on state space Ω. If a probability distribution μ on Ω is reversible for P, then it is also stationary for P.*

PROOF. Note that

$$\big[\mu P\big](x) = \sum_{y\in\Omega} \mu(y)P(y,x) \overset{\text{reversibility}}{=} \sum_{y\in\Omega} \mu(x)P(x,y) = \mu(x)\sum_{y\in\Omega} P(x,y) = \mu(x). \tag{389}$$

□

Example 5.2.8 (Mass transport)**.** In order to give some concrete intuition, consider the following thought experiment. Imagine Ω is the note set of some graph, and $\mu(x)$ represents the mass of sand at node x for each $x \in \Omega$. Overnight, a truck runs around the graph and transport $P(x,y)$ proportion of sand $\mu(x)$ at node x to node y. Hence

$$(\text{Amount of mass transported from } x \text{ to } y) = \mu(x)P(x,y). \tag{390}$$

Let μ' denote the new distribution of sands on Ω. Observe that

$$(\text{Total amount of mass transported from } x) = \sum_{y\in\Omega} \mu(x)P(x,y) \tag{391}$$

$$= \mu(x)\sum_{y\in\Omega} P(x,y) = \mu(x). \tag{392}$$

Hence the new sand distribution μ' is given by

$$\mu'(x) = (\text{Total amount of mass transported into } x) = \sum_{y\in\Omega} \mu(y)P(y,x). \tag{393}$$

In other words, $\mu' = \mu P$. From this, we see that

$$\mu = \mu' \iff \mu = \mu P \iff \mu \text{ is stationary for } \mathbb{P}. \tag{394}$$

On the other hand, the detailed balance equation (388) can be interpreted as

$$(\text{mass sent from } x \text{ to } y) = (\text{mass sent from } y \text{ to } x). \tag{395}$$

From this, it is obvious why a reversible distribution is also stationary. ▲

A *reversible Markov chain* is a Markov chain whose initial distribution is reversible. Namely, let $(X_n)_{n\geq 0}$ denote a Markov chain on state space Ω with transition matrix P. Let the initial state X_0 be distributed as a reversible distribution μ. By Proposition 5.2.7, X_n is a stationary process, meaning that the distribution of X_n does not change in n. However, we can say more than just stationarity. Namely, one cannot distinguish if X_n is run *backward in time*. More precisely,

$$\mathbb{P}_\mu(X_0 = x_0, X_1 = x_1, \ldots, X_n = x_n) = \mathbb{P}_\mu(X_n = x_0, X_{n-1} = x_1, \ldots, X_0 = x_n) \tag{396}$$

for all $n \geq 1$ and states $x_0, \ldots, x_n$.

In practice, reversibility is easier to check than stationarity. So showing reveribility could be handy to stationarity.

Example 5.2.9 (SSRW on graphs is reversible)**.** Let $(X_n)_{n\geq 0}$ be a simple symmetric random walk $G = (V, E)$. Let the initial state X_0 be distribued as the distribution $\mu(x) = \frac{\deg(x)}{2|E|}$. From Example 5.2.6, we already know that μ is a stationary distribution. However, the detailed balance equation is also easily verified:

$$\mu(x)\,P(x,y) = \frac{\deg(x)}{2|E|}\frac{A_G(x,y)}{\deg(x)} = \frac{\deg(y)}{2|E|}\frac{A_G(y,x)}{\deg(y)} = \mu(y)\,P(y,x). \tag{397}$$

Hence X_n is not just stationary, but also reversible. Imagine X_n follows its trajectory backwards in time. The above detailed balance equation makes it so that the backward random walk still follows the same rule: Choose a neighbor uniformly at random and move. ▲

2.3. Doubly stochastic chains. Let P be the transition matrix of a Markov chain $(X_n)_{n\geq 0}$ on a state space Ω. Recall that the row sums of P are always 1, since each row gives the transition probabilities from a fixed state to all other possible states. We say the Markov chain $(X_n)_{n\geq 0}$ (as well as P) is *doubly stochastic* if the column sums of P are also always 1.

Definition 5.2.10. A matrix $P : \Omega \times \Omega \to [0, \infty)$ is *doubly stochastic* if its row and column sums are all one, that is, for all $x, y \in \Omega$,

$$\sum_{z\in\Omega} P(x,z) = \sum_{z\in\Omega} P(z,y) = 1. \tag{398}$$

For doubly stochastic Markov chains, the uniform distribution on Ω (which makes sense if Ω is finite) is always a stationary distribution.

Proposition 5.2.11. *Let $(X_n)_{n\geq 0}$ be a doubly stochastic Markov chain on a finite state space Ω. Then $\pi = \text{Uniform}(\Omega)$ is a stationary distribution for the chain.*

PROOF. Let P denote the transition matrix of the chain $(X_n)_{n\geq 0}$. Note that $\pi(x) = 1/|\Omega|$ for all $x \in \Omega$. Then, viewing π as a row vector, for each $x \in \Omega$,

$$(\pi P)(x) = \sum_{z\in\Omega} \pi(z)P(z,x) = \frac{1}{|\Omega|}\sum_{z\in\Omega} P(z,x) = \frac{1}{|\Omega|} = \pi(x). \tag{399}$$

Hence π is stationary for P. □

Example 5.2.12 (Random Walk on a ring)**.** Let $(X_n)_{n\geq 0}$ denote a random walk on the integer ring $\mathbb{Z}/m\mathbb{Z}$, which consists of integers $0, \ldots, m-1$ and we view $m \equiv 0$ modulo m. The chain moves by 1 clockwise with probability p and 1 counterclockwise with probability $1-p$. Then it is easy to verify the corresponding transition matrix is doubly stochastic. Hence by Proposition 5.2.11, the uniform distribution is a stationary distribution. This is quite resonable since the process is invariant under 'rotation'. ▲

Exercise 5.2.13. Suppose a Markov chain has the uniform distribution as a stationary distribution. Does it have to be a doubly stochastic chain?

3. Markov property to the infinite future

The Markov property (338) states that, knowing the state X_n at time n, the distribution of the immediate future X_{n+1} does not depend on the past $X_0,\dots,X_{n-1}$. How about one step ahead, X_{n+2}? As one can guess, the distribution of X_{n+2} also does not depend on the past $X_0,\dots,X_{n-1}$ given the present. As usual, the argument is by conditioning on the 'missing step' X_{n+1} and applying the markov property (338):

$$\mathbb{P}(X_{n+2}=j_{n+2}\,|\,X_n=j_n,\dots,X_0=j_0) \tag{400}$$
$$=\sum_{j_{n+1}}\mathbb{P}(X_{n+2}=j_{n+2}\,|\,X_{n+1}=j_{n+1},X_n=j_n,\dots,X_0=j_0)\mathbb{P}(X_{n+1}=j_{n+1}\,|\,X_n=j_n,\dots,X_0=j_0) \tag{401}$$
$$=\sum_{j_{n+1}}\mathbb{P}(X_{n+2}=j_{n+2}\,|\,X_{n+1}=j_{n+1})\mathbb{P}(X_{n+1}=j_{n+1}\,|\,X_n=j_n) \tag{402}$$
$$=\sum_{j_{n+1}}\mathbb{P}(X_{n+2}=j_{n+2},\,X_{n+1}=j_{n+1}\,|\,X_n=j_n) \tag{403}$$
$$=\mathbb{P}(X_{n+2}=j_{n+2}\,|\,X_n=j_n). \tag{404}$$

Note that the second equality applies the Markov property to each conditional probabilites to forget the past information. Using a similar argument, one can show that any distant future, X_{n+k} for $k\geq 1$, does not depend on the past $X_0,\dots,X_{n-1}$ given X_n. In fact, this is ture for 'infinite future' as well, meaning the entire process $(X_{n+k})_{k\geq 1}$. Here we give a formal statement.

Proposition 5.3.1. *Let $(X_n)_{n\geq 0}$ be a Markov chain on a finite state space Ω with transition matrix P. Then*

$$\mathbb{P}\left((X_{n+1},X_{n+2},\dots)\in U\,|\,X_n=x,\,(X_0,\dots,X_{n-1})\in B\right)=\mathbb{P}\left((X_{n+1},X_{n+2},\dots)\in U\,|\,X_n=x\right). \tag{405}$$

PROOF. Omitted. □

Example 5.3.2 (Absorption in Gambler's ruin)**.** Recall the Gambler's ruin Markov chain X_n, which is on the state space $\Omega=\{0,1,\dots,N\}$ and trasition matrix P with $P(i,i+1)=p$ for $i\in\{0,\dots,N-1\}$, $P(i-1,i)=1-p$ for $i\in\{1,\dots,N\}$, $P(0,0)=1P(N,N)=1$. We will assume the initial fortune is $X_0=x$. We will compute the probability of winning, that is, the probability that the Gambler's fortune X_n absorbs into N rather than to 0. As a function of x, denote

$$h(x)=\mathbb{P}_x(\text{state } N \text{ is reached before } 0) \tag{406}$$
$$:=\mathbb{P}(\text{state } N \text{ is reached before } 0\,|\,X_0=x). \tag{407}$$

The goal is to solve for $h(x)$. Note that it has the following bounary values

$$h(0)=0,\qquad h(N)=1. \tag{408}$$

If we start at $x\in\{1,\dots,N-1\}$, then the Markov chain goes to $x+1$ with probability p and renews as a Gambler's ruin chain with initial state $x+1$; with probability $q:=1-p$, it goes to $x-1$ and renews there. Hence it should be evident that

$$h(x)=ph(x+1)+qh(x)\qquad \text{for } 0<x<N. \tag{409}$$

Noting that $p+q=1$, (410) can be re-written as

$$ph(x)+qh(x)=ph(x+1)+qh(x)\qquad \text{for } 0<x<N. \tag{410}$$

This gives

$$h(x+1)-h(x)=\frac{q}{p}(h(x)-h(x-1))\qquad \text{for } 0<x<N, \tag{411}$$

which is a multiplicative recursion for the differences $h(x+1)-h(x)$. It follows that

$$h(x+1)-h(x)=(q/p)^x h(1)\qquad \text{for } 0<x<N. \tag{412}$$

Summing over x and using one of the boundary condition $h(0) = 0$, we get

$$h(x+1) = h(1)\sum_{k=0}^{x}(q/p)^x \qquad \text{for } 0 < x < N \tag{413}$$

$$= \begin{cases} h(1)\frac{1-(q/p)^{x+1}}{1-(q/p)} & \text{if } p \neq q \\ h(1)(x+1) & \text{if } p = q = 1/2. \end{cases} \tag{414}$$

So it only remains to figure out $h(1)$. For this, we use the remaining boundary condition $h(N) = 1$ by setting $x = N-1$:

$$1 = h(N) = \begin{cases} h(1)\frac{1-(q/p)^{N}}{1-(q/p)} & \text{if } p \neq q \\ h(1)N & \text{if } p = q = 1/2. \end{cases} \tag{415}$$

It follows that $h(1) = \frac{1-(q/p)}{1-(q/p)^N}$ for $p \neq q$ and $h(1) = 1/N$ for $p = q = 1/2$. Thus, we conclude that

$$h(x) = \begin{cases} \frac{1-(q/p)^{x}}{1-(q/p)^N} & \text{if } p \neq q \\ x/N & \text{if } p = q = 1/2. \end{cases} \tag{416}$$

For instance, say we start the gamble with $x = 50$ dollor and win the whole game once we accumulate $N = 100$ dollors before hitting 0 dollor. If each game is fair ($p = q = 1/2$), the probabilty of winning the game is $h(50) = 50/100 = 0.5$. If it is a slightly unfavorable game with $p = 49/100$ and $q = 51/100$, then the probability of winning is $h(1) = \frac{1-(51/49)^{50}}{1-(51/49)^{100}} \approx 0.11917$. So, just make each game slightly unfavorable and your casino is on business! ▲

4. Strong Markov Property

In this section, we will discuss an extension of the Markov property stopped at random times. Namely, we would like to restart a Markov chain $(X_n)_{n\geq 1}$ at a (possibly random) time T and would like to say the future process $(X_n)_{n>T}$ is independent of the past $(X_n)_{0\leq n<T}$ given the present state X_T. Can we do this for any random time T? The short answer is no. For an illustration, let $(X_n)_{n\geq 0}$ be the gambler's ruin chain in Example 5.1.2 with initial fortune $X_0 = 1$ and cap $N = 100$. Consider three (possibly) random times:

1. $T = T_1 = 50$;
2. $T = T_2 =$ first time to reach \$50;
3. $T = T_3 =$ first time to reach \$50 that i immediately followed by \$51.

We will consider the probability of jumping from 50 to 51 from time T to $T+1$. If we can really restart the Markov chain at time T, this transition probability should all be p, the probability of winning one round of gamble. This is indeed the case for $T = T_1 = 50$ by using time-homogenity. On the other hand, it fails to be true for $T = T_3$, since $X_{T_3+1} = 51$ with probability 1 by the definition of T_3. In other words, the Markov chain is 'forced' to transition to 51 at time T_3, not following its transition probabilities. Roughly speaking, this change of behavior happends becuase *knowning the value of T_3 contains some information about the chain at future times*. Finally, how about $T = T_2$? At first glance, knowing the value of T_2 should not exert any information on how it would behave in the following steps, since the value of T_2 is completely determined by the history of the chain at prior times $n \leq T_2$. This leads us to the definition of 'stopping times', which we define for a general stochastic process and not necessarily only for Markov chains.

Definition 5.4.1 (Stopping time)**.** Let $(X_n)_{n\geq 0}$ be a stochastic process defined on a probability space $(\Omega, \mathscr{F}, \mathbb{P})$. Let T be a random variable taking valus from $\mathbb{Z}_{\geq 0} \cup \{\infty\}$. Then T is a *stopping time* (for the

process $(X_n)_{n\geq 0}$) if for each nonnegative integer n, there is a subset $C_n \subseteq \Omega^{n+1}$ such that the following equality of events holds:

$$\{T = n\} = \{(X_0, X_1, \ldots, X_n) \in C_n\}. \tag{417}$$

In words, T is a stopping time for $(X_n)_{n\geq 0}$ if one can determine if $T = n$ by checking a certain condition for the history $X_0, X_1, \ldots, X_n$, without using any information about the future states $X_{n+1}, X_{n+2}, \ldots$. In this sense, the random time T_3 defined above is not a stopping time, but T_2 is, since we can write

$$\{T_2 = n\} = \{X_0 \neq 50,\, X_1 \neq 50,\, \ldots, X_{n-1} \neq 50,\, X_n = 50\}. \tag{418}$$

Example 5.4.2 (First hitting time is a stopping time)**.** Let $(X_n)_{n\geq 0}$ be a stochastic process defined on a probability space $(\Omega, \mathscr{F}, \mathbb{P})$. Let $A \subseteq \Omega$ be a subset of the sample space. We will define τ_A to be the 'first hitting time' of A, which is the first time that the process X_n enters into states in A:

$$\tau_A := \inf\{n \geq 0 \,|\, X_n \in A\}. \tag{419}$$

Here 'inf' stands for infimum, which agrees with the minimum if the set is nonempty, and conventionally taken to be ∞ if the set is empty. For example, $\tau_A = \infty$ means the process X_n never enters the states in A. In order to see that τ_A is indeed a stopping time, we simply mimic what we did in (418): For each $n \geq 0$,

$$\{\tau_A = n\} = \{X_0 \neq A,\, X_1 \neq A,\, \ldots, X_{n-1} \neq A,\, X_n \in A\}. \tag{420}$$

Similarly, define T_A to be the first time after time zero to hit some state in A:

$$T_A := \inf\{n \geq 1 \,|\, X_n \in A\}. \tag{421}$$

As before, T_A is also a stopping time since, for each $n \geq 0$,

$$\{T_A = n\} = \{X_1 \neq A,\, X_1 \neq A,\, \ldots, X_{n-1} \neq A,\, X_n \in A\}. \tag{422}$$

Hence, in general, first hitting times are stopping times. ▲.

Exercise 5.4.3. Let $(X_n)_{n\geq 0}$ be a stochastic process defined on a probability space $(\Omega, \mathscr{F}, \mathbb{P})$. Let $A \subseteq \Omega$ be a subset of the sample space. For each $k \geq 1$, let $T_A^{(k)}$ denote the kth time that the process X_n visits some state in A. That is,

$$T_A^{(m)} = \begin{cases} \inf\{n > T_A^{(m-1)} : X_n \in A\} & \text{if } T_A^{(m-1)} < \infty \\ \infty & \text{otherwise.} \end{cases} \tag{423}$$

Show that $T_A^{(k)}$ is a stopping time for all $k \geq 1$.

Now that we have introduced stopping times, we can state the strong Markov property of Markov chains, which states that one can restart Markov chains at stopping times.

Proposition 5.4.4 (Strong Markov property)**.** *Let $(X_n)_{n\geq 0}$ be a Markov chain on a finite state space Ω and let T denote a stopping time. Let A be any event determined by $(X_0, X_1, \ldots, X_T)$. Then for any states $x, u_1, \ldots, u_n \in \Omega$,*

$$\mathbb{P}(X_{T+1} = u_1, \ldots, X_{T+m} = u_m \,|\, T < \infty,\, A,\, X_T = x) = \mathbb{P}(X_1 = u_1, \ldots, X_m = u_m \,|\, X_0 = x). \tag{424}$$

PROOF. We will condition on the value of T and apply the usual Markov property. For instance, suppose $T = n$. Then

$$\begin{aligned} \mathbb{P}(X_{T+1} = u_1, \ldots, X_{T+m} = u_m \,|\, T = n,\, A,\, X_T = x) &= \mathbb{P}(X_{n+1} = u_1, \ldots, X_{n+m} = u_m \,|\, A,\, X_n = x) && (425) \\ &= \mathbb{P}(X_{n+1} = u_1, \ldots, X_{n+m} = u_m \,|\, X_n = x) && (426) \\ &= \mathbb{P}(X_1 = u_1, \ldots, X_m = u_m \,|\, X_0 = x), && (427) \end{aligned}$$

where the last equality uses time-homogeneity of the Markov chain and the second equality uses the Marko property. (Note that, A is determined by the history $X_0, \ldots, X_n$. Hence we can decompose the event $A \cap \{X_n = x\}$ by $A' \cap \{X_n = x\}$, where A' is an event determined by $X_0, \ldots, X_{n-1}$. Then we can forget A' in the conditioning by the Markov property.)

Now, in generall, we do not know the value of T. Hence we will use iterated expectation for probability (Exercise A.3.3) to condition on the value of T and then average over all values of T:

$$\mathbb{P}(X_{T+1} = u_1, \dots, X_{T+m} = u_m \,|\, T < \infty,\, A,\, X_T = x) \tag{428}$$
$$= \mathbb{E}_T\left[\mathbb{P}(X_{T+1} = u_1, \dots, X_{T+m} = u_m \,|\, T,\, A,\, X_T = x)\right] \tag{429}$$
$$= \mathbb{E}_T\left[\mathbb{P}(X_1 = u_1, \dots, X_m = u_m \,|\, X_0 = x)\right] \tag{430}$$
$$= \mathbb{P}(X_1 = u_1, \dots, X_m = u_m \,|\, X_0 = x). \tag{431}$$

Here, the third equailty follows from the observation we made in the first paragraph: If we know the value of $T < \infty$, the assertion holds. □

Example 5.4.5. Let $(X_n)_{n\geq 0}$ be the gambler's ruin chain in Example 5.1.2 with initial fortune $X_0 = 1$ and cap $N = 100$. Let $T = T_2$ denote the first hitting time of \$50. We have seen that it is a stopping time. So by the strong Markov property,

$$\mathbb{P}(X_{T+1} = 51 \,|\, X_T = 50) = \mathbb{P}(X_1 = 51 \,|\, X_0 = 50) = p. \tag{432}$$

▲.

5. Recurrence and Transience

We now have the sufficient tools to study the behavior of Markov chains. Throughout this section, we fix a Markov chain $(X_n)_{n\geq 0}$ on finite state space Ω with transition matrix P. Here are some quick highlights.

Elementary behavior of Markov chains:

(i) For each state $x \in \Omega$, the Markov chain X_n either visits x infinitely often with probability one (we say x is 'recurrent') or only finitely many times with probability one (we say x is 'transient').

(ii) Starting from any state, the Markov chain X_n must enter the set of recurrent states eventually. Once entering a recurrent state, it will never go back to any transient states.

(iii) The set of recurrent states is further decomposed into 'communicating classes'. The Markov chain eventually enters exactly one communicating class and stays there forever, visiting every states in that class infinitley often.

5.1. Definition of recurrence/transitions and equivalent conditions. There are multiple ways to define recurrence/transience of states under a Markov chain: using 1) return probabilities; 2) infinitely many returns; 3) expected number of returns. Our first approach is via return probabilities. Namely, for each state $x \in \Omega$, define

$$T_x = T_x^{(1)} := \inf\{n \geq 1 : X_n = x\}, \tag{433}$$

which is the first time $n \geq 1$ at which the chain X_n hits the state x. In Example 5.4.2, we have seen that T_x is a stopping time for each $x \in \Omega$. For states $x, y \in \Omega$, let

$$\rho_{xy} = \mathbb{P}(T_y < \infty \,|\, X_0 = x), \tag{434}$$

which is the probability that the chain X_n eventually visits y starting from x.

Definition 5.5.1. Let $(X_n)_{n\geq 0}$ be a Markov chain on finite state space Ω. Each state $x \in \Omega$ is said to be *recurrent* if $\rho_{xx} = 1$ and *transient* if $\rho_{xx} < 1$.

Exercise 5.5.2. Show the following implications:

$$\rho_{xx} > 0 \qquad \Longleftrightarrow \qquad P^m(x,x) > 0 \text{ for some } m \geq 1. \tag{435}$$

Hint: Use

$$\sup_{n\geq 1} \mathbb{P}\left(X_n = x \,\middle|\, X_0 = x\right) \leq \mathbb{P}\left(\bigcup_{n=1}^{\infty} \{X_n = x\} \,\middle|\, X_0 = x\right) \leq \sum_{n=1}^{\infty} \mathbb{P}\left(X_n = x \,\middle|\, X_0 = x\right). \tag{436}$$

Proposition 5.5.3. *Fix states $x, y \in \Omega$ and let $T_y^{(m)}$ denote the time of mth visit to y, as defined in Exercise 5.4.3. Then for each $m \geq 1$,*

$$\mathbb{P}(T_y^{(m)} < \infty \,|\, X_0 = x) = \rho_{xy}\rho_{yy}^{m-1}. \tag{437}$$

PROOF. We first note that the assertion for $m = 1$ follows from the definition of ρ_{xy}:

$$\mathbb{P}(T_y^{(1)} < \infty \,|\, X_0 = x) = \rho_{xy}. \tag{438}$$

Next, suppose the assertion holds for $m = k$. We deduce the assertino for $m = k+1$ by using the strong Markov property. Namely, note that since $T_y^{(k+1)} > T_y^{(k)}$, $T_y^{(k+1)} < \infty$ implies $T_y^{(k)} < \infty$. Also, note that $X_{T_y^{(k)}} = y$ by definition. Hence,

$$\begin{aligned}
\mathbb{P}(T_y^{(k+1)} < \infty \,|\, X_0 = x) &= \mathbb{P}(T_y^{(m+1)} < \infty, T_y^{(k)} < \infty \,|\, X_0 = x) && (439)\\
&= \mathbb{P}(T_y^{(m+1)} < \infty \,|\, T_y^{(k)} < \infty, X_0 = x)\,\mathbb{P}(T_y^{(k)} < \infty \,|\, X_0 = x) && (440)\\
&= \mathbb{P}(T_y^{(m+1)} < \infty \,|\, X_{T_y^{(k)}} = y, T_y^{(k)} < \infty, X_0 = x)\,\mathbb{P}(T_y^{(k)} < \infty \,|\, X_0 = x) && (441)\\
&= \mathbb{P}(T_y^{(1)} < \infty \,|\, X_0 = y)\,\mathbb{P}(T_y^{(k)} < \infty \,|\, X_0 = x) && (442)\\
&= \rho_{yy}(\rho_{xy}\rho_{yy}^{k-1}) = \rho_{xy}\rho_{yy}^{k}, && (443)
\end{aligned}$$

where the last two equalities use the strong Markov property and the induction hypothesis, respectively. □

Theorem 5.5.4. *The following hold:*

(i) *$x \in \Omega$ is recurrent if and only if $\mathbb{P}(T_x^{(m)} < \infty$ for all $m \geq 1 \,|\, X_0 = x) = 1$.*
(ii) *$x \in \Omega$ is transient if and only if $\mathbb{P}(T_x^{(m)} < \infty$ for all $m \geq 1 \,|\, X_0 = x) = 0$.*

PROOF. Denote the probability in the assertion as p^*. Note that for each $m \geq 1$, by monotonicity of events,

$$p^* = \mathbb{P}(T_x^{(m)} < \infty \text{ for all } m \geq 1 \,|\, X_0 = x) \leq \mathbb{P}(T_x^{(m)} < \infty \,|\, X_0 = x) = \rho_{xx}^m. \tag{444}$$

Hence if $\rho_{xx} < 1$, $\rho_{xx}^m \to 0$ as $m \to \infty$ so the $p^* = 0$. Hence $p^* = 0$ if x is transient.

On the other hand, suppose x is recurrent. Then by subadditivity (**??**),

$$\mathbb{P}\left(\bigcup_{m=1}^{\infty} \{T_x^{(m)} = \infty\} \,\middle|\, X_0 = x \right) \leq \sum_{m=1}^{\infty} \mathbb{P}\left(T_x^{(m)} = \infty \,|\, X_0 = x\right) = 0. \tag{445}$$

Hence the probability on the left hand side equals zero. By taking the complement,

$$1 = 1 - \mathbb{P}\left(\bigcup_{m=1}^{\infty} \{T_x^{(m)} = \infty\} \,\middle|\, X_0 = x \right) = \mathbb{P}\left(\bigcap_{m=1}^{\infty} \{T_x^{(m)} < \infty\} \,\middle|\, X_0 = x \right) = p^*. \tag{446}$$

This shows $p^* = 1$ if x is recurrent.

Finally, the converse implications are automatic, since each state $x \in \Omega$ is either recurrent or transient, mutually exclusive, by definition. (Note that this also implies p^* is either 0 or 1. In other words, with probability one, either x is visited infinitely often or only finitely many times.) □

Next, consider the following counting variable

$$N_y := \sum_{n=1}^{\infty} \mathbf{1}(X_n = y) = \sum_{m=1}^{\infty} \mathbf{1}(T_y^{(m)} < \infty), \tag{447}$$

which equals the total number of visits to state $y \in \Omega$ by the Markov chain. By using Fubini's theorem, one can write

$$\mathbb{E}[N_y \,|\, X_0 = x] = \mathbb{E}\left[\sum_{m=1}^{\infty} \mathbf{1}(T_y^{(m)} < \infty) \,\middle|\, X_0 = x \right] = \sum_{m=1}^{\infty} \mathbb{E}\left[\mathbf{1}(T_y^{(m)} < \infty) \,\middle|\, X_0 = x \right] \tag{448}$$

$$= \sum_{m=1}^{\infty} \mathbb{P}\left(T_y^{(m)} < \infty \,|\, X_0 = x\right) \tag{449}$$

$$= \sum_{m=1}^{\infty} \rho_{xy} \rho_{yy}^{m-1}. \tag{450}$$

Note that the last expression equals zero if $\rho_{xy} = 0$, infinity if $\rho_{xy} > 0$ and $\rho_{yy} = 1$, and a converging geometric series if $\rho_{xy} > 0$ and $\rho_{yy} < 1$. Hence we obtain the following result:

Proposition 5.5.5. *The following hold:*

$$\mathbb{E}[N_y \,|\, X_0 = x] = \begin{cases} 0 & \text{if } \rho_{xy} = 0 \\ \infty & \text{if } \rho_{xy} > 0 \text{ and } y \text{ is recurrent} \\ \frac{\rho_{xy}}{1-\rho_{yy}} & \text{if } \rho_{xy} > 0 \text{ and } y \text{ is transient.} \end{cases} \tag{451}$$

Now we can give an alternative characterization of the recurrence of a state in terms of the sum of multi-step transition probabilities.

Theorem 5.5.6. *For each $x, y \in \Omega$, it holds that*

$$\mathbb{E}[N_y \,|\, X_0 = x] = \sum_{n=1}^{\infty} P^n(x, y). \tag{452}$$

Furthermore, a state $x \in \Omega$ is recurrent if and only if $\mathbb{E}[N_x \,|\, X_0 = x] = \sum_{m=1}^{\infty} P^m(x, x) = \infty$.

PROOF. Recall that $N_y = \sum_{n=1}^{\infty} \mathbf{1}(X_n = y)$. Hence

$$\mathbb{E}[N_y \,|\, X_0 = x] = \mathbb{E}\left[\sum_{n=1}^{\infty} \mathbf{1}(X_n = y) \,\middle|\, X_0 = x\right] = \sum_{n=1}^{\infty} \mathbb{E}\left[\mathbf{1}(X_n = y) \,\middle|\, X_0 = x\right] \tag{453}$$

$$= \sum_{n=1}^{\infty} \mathbb{P}\left(X_n = y \,\middle|\, X_0 = x\right) = \sum_{n=1}^{\infty} P^n(x, y), \tag{454}$$

verifying the first assertion. But Proposition 5.5.5 implies that x is recurrent if and only if $\mathbb{E}[N_x \,|\, X_0 = x] = \infty$. Hence the assertion follows. □

5.2. Recurrence and transience of random walks on $\mathbb{Z}^d$. In this subsection, we study recurrence and transience of simple random walks on the integer lattice $\mathbb{Z}^d$ in d-dimensions. As a set, $\mathbb{Z}^d$ consists of the d tuple of integer coordinates: $\mathbb{Z}^d = \{(a_1, \ldots, a_d) \,|\, a_1, \ldots, a_d \in \mathbb{Z}\}$. We will view it as a graph by giving 'nearest neighbor edges'. Namely, if we have two points $\mathbf{x}, \mathbf{y} \in \mathbb{Z}^d$, then we connect $\mathbf{x}$ and $\mathbf{y}$ by an edge if and only if they differ by 1 at exactly a single coordiante. For example, in $\mathbb{Z}^2$, $(1,2)$ is adjacent to $(2,1)$ but not adjacent to $(2,3)$. As a graph, $\mathbb{Z}^2$ looks like a square grid in $\mathbb{R}^2$. Similarly, $\mathbb{Z}^3$ would be the 3-dimensional square lattice consisting of the grid points (a, b, c) for $a, b, c \in \mathbb{Z}$.

A random ralk on any graph is simple if it only moves along edges and do not hop multiple edges, and *symmetric* if at any node, there is an equal chance to pick any neighbor to move to[1]. Hence a simple symmetric random walk (SSRW) on $\mathbb{Z}$ moves either up or down with equal probability; SSRW on $\mathbb{Z}^2$ chooses one of the 4 directions with equal probability and moves along one edge; SRW on $\mathbb{Z}^d$ has $2d$ directions to choose from at each iteration with equal probability.

The goal of the following serious of examples is to show the following Polya's random walk theorem[2]:

$$\textit{SSRW on } \mathbb{Z}^d \textit{ is recurrent if } d \leq 2 \textit{ and transient for } d \geq 3. \tag{455}$$

In Example 5.5.7, we have seen that SRW on $\mathbb{Z}$ is recurrent. For higher dimensions, we will compute the sum of return probabilities as we did before.

[1] In this lecture note, we assume all random walks on graphs are simple.

[2] Kakutani at UCLA colloquium said: "A drunk man will eventually find his way home, but a drunk bird may get lost forever"

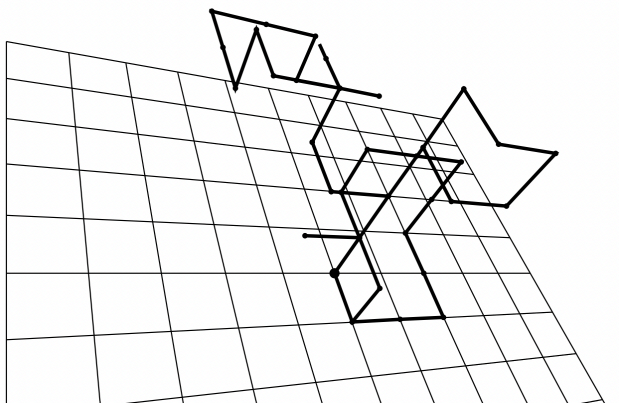


FIGURE 67. A sample path of RW on $\mathbb{Z}^3$. Figure excerpted from http://www.cmat.edu.uy/~lessa/resource/randomwalknotes.pdf

Example 5.5.7 (Recurrence and transience of simple random walk on $\mathbb{Z}$)**.** Let $(X_n)_{n\geq 0}$ denote a simple random walk on the 1-dimensional integer lattice $\mathbb{Z}$. Namely, it is a Markov chain with state space $\mathbb{Z}$ and transition matrix P such that $P(x,x+1)=p$ and $P(x,x-1)=1-p$ for all $x\in\mathbb{Z}$. Here $p\in[0,1]$ is a parameter that governs the bias in one-step transition of the walker. Simply put, the walker moves up by 1 with probability p and down by 1 with probability $1-p$. Suppose $X_0=0$, meaning that the walkter is originally at the origin.

The question we ask is as follows: Is the origin transient or recurrent? The answer depends on p. If $p>1/2$, then the walker has a positive bias, so it would be natural to guess the origin is transient – it will drift away toward the infinity and will never get back to the origin after some time. Likewise, if $p<1/2$, it will drift away toward negative infinity and the origin should be transient. Lastly, if the walker is symmetric, $p=1/2$, maybe the origin is recurrent. We will justify these guesses using Theorem 5.5.6.

The key observation here is the following combinatorial fact: In order to return to the origin, the walker must take an even number of steps with equal number of up-steps and down-steps. It follows that

$$\mathbb{P}(X_{2n}=0\,|\,X_0=0)=\binom{2n}{n}p^n(1-p)^n=\frac{(2n)!}{n!n!}(p(1-p))^n. \tag{456}$$

We will need to understand how $n!$ grows as n grows. We will apply Stirling's approximation, which states that

$$n!\sim(n/e)^n\sqrt{2\pi n}, \tag{457}$$

where $a_n\sim b_n$ means $a_n/b_n\to 1$ as $n\to\infty$. Using this, one can show that

$$\mathbb{P}(X_{2n}=0\,|\,X_0=0)\sim\frac{(2n/e)^{2n}\sqrt{4\pi n}}{(n/e)^{2n}(2\pi n)}(p(1-p))^n=\frac{1}{\sqrt{\pi n}}(4p(1-p))^n. \tag{458}$$

Now consider the symmetric case $p=1/2$. Then $\mathbb{P}(X_{2n}=0)\sim 1/\sqrt{\pi n}$. This implies that for some constant $c>0$, $\mathbb{P}(X_{2n}=0)\geq c/\sqrt{\pi n}$ for all $n\geq 1$. Now in order to see the recurrence of the origin, we compute the sum of all return probabilities:

$$\sum_{n=0}^{\infty}\mathbb{P}(X_n=0\,|\,X_0=0)=\sum_{n=0}^{\infty}\mathbb{P}(X_{2n}=0\,|\,X_0=0)\geq\sum_{n=0}^{\infty}\frac{c}{\sqrt{\pi n}}=\infty, \tag{459}$$

where the first equality uses the fact that the walker cannot return to the origin in odd number of steps, and the last equality uses the fact that $\sum_{n=0}^{\infty}1/\sqrt{n}=\infty$. Hence by Theorem 5.5.6, we deduce that the origin is recurrent if $p=1/2$. By Theorem 5.5.4 this means that the walker returns to the origin infinitely

many times with probability one. By Proposition 5.5.5, this also means that the expected number of visits to the origin is infinite.

On the other hand, suppose $p \neq 1/2$. Recall that the function $p \mapsto p(1-p)$ attains its global maximum of $1/4$ at $p = 1/2$. Hence $0 \leq 4p(1-p) < 1$ if $p \in [0,1]$ and $p \neq 1/2$. In this case, for some constant $c > 0$, $\mathbb{P}(X_{2n} = 0 \,|\, X_0 = 0) \leq c\rho^n$ for all $n \geq 1$, where $\rho = 4p(1-p) \in (0,1)$. Hence

$$\sum_{n=0}^{\infty} \mathbb{P}(X_n = 0 \,|\, X_0 = 0) = \sum_{n=0}^{\infty} \mathbb{P}(X_{2n} = 0 \,|\, X_0 = 0) \leq \sum_{n=0}^{\infty} c\rho^n < \infty. \tag{460}$$

So by Theorem 5.5.6, we deduce that the origin is transient if $p \neq 1/2$. By Theorem 5.5.4 this means that the walker returns to the origin only finitely many times with probability one. By Proposition 5.5.5, this also means that the expected number of visits to the origin is finite. ▲

Example 5.5.8 (SSRW on $\mathbb{Z}^2$ is recurrent ("A drunk man eventually finds way home"))**.** Let $\mathbf{X}_n$ denote a SSRW on $\mathbb{Z}^2$, which chooses one of the 4 steps $[1,0]$, $[-1,0]$, $[0,1]$, $[0,-1]$ independently with equal probability at each move. It would be easy to analyze the return probability of $\mathbf{X}_n$ if we can treat its two coordinates as independent random walks on $\mathbb{Z}$, since we have already analyze one-dimensional random walks. However, the two coordinates of $\mathbf{X}_n$ are *not* independent. For example, if its first coordinate increased or decreased, then we know that its second coordinate did not change.

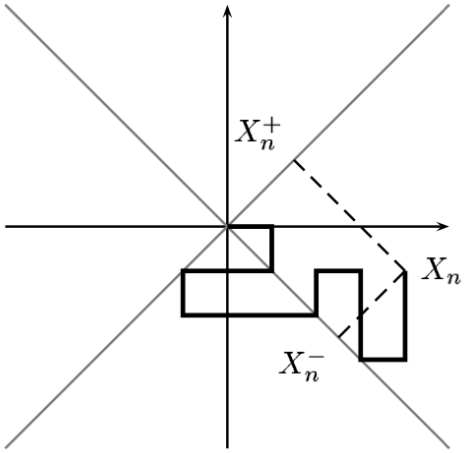


FIGURE 68. A sample path of RW on $\mathbb{Z}^2$. Projecting it on $y = \pm x$ shows that it has the same distribution as a pair of independent random walks on $\mathbb{Z}$. Figure excerpted from http://www.cmat.edu.uy/~lessa/resource/randomwalknotes.pdf

In order to handle this issue, we are going to compare $\mathbf{X}_n$ with the following auxiliary process $\mathbf{Y}_n = [Y_n^{(1)}, Y_n^{(2)}]$, where $Y_n^{(1)}$ and $Y_n^{(2)}$ are independent SSRW on $\mathbb{Z}$. So each $Y_n^{(i)}$ changes by ± 1 with equal probability, independently for each $i = 1,2$:

$$\mathbb{P}(Y_{n+1}^{(i)} - Y_n^{(i)} = 1) = \mathbb{P}(Y_{n+1}^{(i)} - Y_n^{(i)} = -1) = 1/2 \quad \text{for } i = 1,2. \tag{461}$$

Now, since $Y_n^{(1)}$ and $Y_n^{(2)}$ change by ± 1 at the same time, the vector $\mathbf{Y}_n = [Y_n^{(1)}, Y_n^{(2)}]$ on $\mathbb{Z}^2$ changes by $[1,1]$, $[-1,1]$, $[1,-1]$, $[-1,-1]$ with equal probability. This is exactly the original SSRW $\mathbf{X}_n$ rotated by 45 degrees and scaled by $\sqrt{2}$, so that it lives on the 'checkerboard lattice' $\{(a,b) \,|\, a,b \in \mathbb{Z}, a+b \text{ is even}\} \subseteq \mathbb{Z}^2$. From this observation, it follows that the probability of $\mathbf{Y}_n$ returning to the origin is the same as the probability that $\mathbf{X}_n$ returning to the origin. This gives

$$\mathbb{P}(\mathbf{X}_n = [0,0] \,|\, \mathbf{X}_0 = [0,0]) = \mathbb{P}(\mathbf{Y}_n = [0,0] \,|\, \mathbf{Y}_0 = [0,0]) \tag{462}$$

$$= \mathbb{P}\left(Y_n^{(1)} = 0,\, Y_n^{(2)} = 0 \,|\, Y_0^{(1)} = 0,\, Y_0^{(2)} = 0\right) \tag{463}$$

$$= \mathbb{P}\left(Y_n^{(1)} = 0 \,|\, Y_0^{(1)} = 0\right) \mathbb{P}\left(Y_n^{(2)} = 0 \,|\, Y_0^{(2)} = 0\right) \tag{464}$$

$$= \mathbb{P}\left(Y_n^{(1)} = 0 \,|\, Y_0^{(1)} = 0\right)^2. \tag{465}$$

where the last equality is due to the independence between $Y_n^{(1)}$ and $Y_n^{(2)}$. By using the analysis in Example 5.5.7, we get

$$\mathbb{P}\left(\mathbf{X}_{2n} = [0,0] \,|\, \mathbf{X}_0 = [0,0]\right) \sim \frac{1}{\pi n}. \tag{466}$$

To conclude, note that $\mathbf{X}_n$ can only return to the origin at even number of steps, and the above asymptotic yields that for some constant $c > 0$, $\mathbb{P}\left(\mathbf{X}_{2n} = [0,0] \,|\, \mathbf{X}_0 = [0,0]\right) \geq c/(\pi n)$ for all $n \geq 1$. Hence

$$\sum_{n=0}^{\infty} \mathbb{P}\left(\mathbf{X}_n = 0 \,|\, \mathbf{X}_0 = 0\right) = \sum_{n=0}^{\infty} \mathbb{P}\left(\mathbf{X}_{2n} = 0 \,|\, \mathbf{X}_0 = 0\right) \geq \sum_{n=0}^{\infty} \frac{c}{\pi n} = \infty. \tag{467}$$

Therefore $\mathbf{X}_n$ is recurrent by Theorem 5.5.6. ▲

Example 5.5.9 (SSRW on $\mathbb{Z}^d$ for $d \geq 3$ is transient ("A drunk bird never finds way home"))**.** Let $\mathbf{X}_n$ denote a SSRW on $\mathbb{Z}^d$ with $d \geq 3$, which chooses one of the $2d$ steps $[\pm 1, 0, \ldots, 0]$, $[0, \pm 1, 0, \ldots, 0]$, … $[0, \ldots, 0, \pm 1]$ independently with equal probability at each move. We will first apply a similar argument as in Example 5.5.8 and try to show that $\mathbf{X}_n$ is transient. This simple argument almost works, but we will see why it fails.

As we did in the two dimensional case, consider a process $\mathbf{Y}_n = [Y_n^{(1)}, \ldots, Y_n^{(d)}]$ on $\mathbb{Z}^d$ where $Y_n^{(i)}$'s are independent SSRWs on $\mathbb{Z}$. So each $Y_n^{(i)}$ changes by ± 1 with equal probability, independently for each $i = 1, \ldots, d$. As before, note that

$$\mathbb{P}\left(\mathbf{X}_n = [0, \ldots, 0] \,|\, \mathbf{X}_0 = [0, \ldots, 0]\right) \text{“} = \text{”} \mathbb{P}\left(\mathbf{Y}_n = [0, \ldots, 0] \,|\, \mathbf{Y}_0 = [0, \ldots, 0]\right) \tag{468}$$

$$= \mathbb{P}\left(Y_n^{(1)} = 0, \ldots, Y_n^{(d)} = 0 \,|\, Y_0^{(1)} = 0, \ldots Y_n^{(d)} = 0\right) \tag{469}$$

$$= \mathbb{P}\left(Y_n^{(1)} = 0 \,|\, Y_0^{(1)} = 0\right)^d. \tag{470}$$

Then by using the analysis in Example 5.5.7,

$$\mathbb{P}\left(\mathbf{X}_{2n} = \mathbf{0} \,|\, \mathbf{X}_0 = \mathbf{0}\right) \sim \left(\frac{1}{\sqrt{\pi n}}\right)^d. \tag{471}$$

The above probabilities are not summable if $d \geq 3$, as $\sum_{n=1}^{\infty} n^{-d/2} = \infty$ for $d \geq 3$. Hence by Theorem 5.5.6, conclude that $\mathbf{X}_n$ is transient.

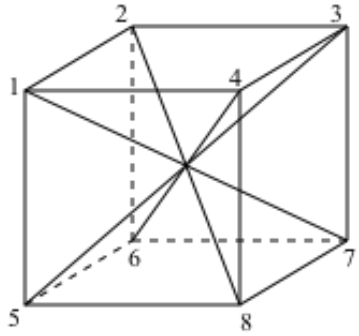


FIGURE 69. A triple of independent SSRWs on $\mathbb{Z}$ has 8 possible moves $\{-1, 1\}^3$

The reason that the above argument fails is that the quoted equality in (469) is not true. For example, set $d = 3$. Then there are 6 steps that $\mathbf{X}_n$ takes: $[\pm 1, 0, 0,]$, $[0, \pm 1, 0]$, and $[0, 0, \pm 1]$. However, $\mathbf{Y}_n$ can

take $2^3 = 8$ distinct steps in the integer lattice $\mathbb{Z}^3$ (see Figure 69). For a correct argument, we rely on a combinatorial argument we employed in Example 5.5.7, which we will demonstrate in the case of $d = 3$.

Let $d = 3$. Note that the walk can only return to the origin in even number of steps. Let a, b, c denote the number of $[1,0,0]$, $[0,1,0]$, $[0,0,1]$-steps that $\mathbf{X}_n$ takes in the first $2n$ steps, starting from the origin. In order for $\mathbf{X}_{2n} = \mathbf{0}$, we need to have $a+b+c = n$ and the number of $[-1,0,0]$, $[0,-1,0]$, $[0,0,-1]$-steps must be a, b, c, respectively. Hence

$$\mathbb{P}(\mathbf{X}_n = \mathbf{0} \,|\, \mathbf{X}_0 = \mathbf{0}) = \sum_{\substack{0 \le a,b,c \\ a+b+c=n}} \frac{(2n)!}{(a!b!c!)^2} \left(\frac{1}{6}\right)^{2n} \tag{472}$$

$$= \binom{2n}{n} \left(\frac{1}{2}\right)^{2n} \sum_{\substack{0 \le a,b,c \\ a+b+c=n}} \left(\frac{n!}{(a!b!c!)}\right)^2 \left(\frac{1}{3}\right)^{2n}, \tag{473}$$

where we have used the identity $\frac{(2n)!}{(a!b!c!)^2} = \binom{2n}{n}\left(\frac{n!}{(a!b!c!)}\right)^2$. Now note that

$$1 = \sum_{\substack{0 \le a,b,c \\ a+b+c=n}} \frac{n!}{(a!b!c!)} \left(\frac{1}{3}\right)^n, \tag{474}$$

since the right hand side equals the total probability of putting n balls into three bins uniformly at random. Also, observe that

$$\frac{n!}{(a!b!c!)} \le \frac{n!}{\lfloor n/3 \rfloor! \lfloor n/3 \rfloor! \lfloor n/3 \rfloor!} \tag{475}$$

for all $a, b, c \ge 0$ with $a+b+c = n$ and $\lfloor n/3 \rfloor$ is the largest smaller integer than $n/3$. In words, one has the most amount of arrangements of n balls into three bins if the number of balls in each bins are as equal as possible. Using this,

$$\mathbb{P}(\mathbf{X}_{2n} = \mathbf{0}) = \binom{2n}{n} \left(\frac{1}{2}\right)^{2n} \sum_{\substack{0 \le a,b,c \\ a+b+c=n}} \left(\frac{n!}{(a!b!c!)}\right)^2 \left(\frac{1}{3}\right)^{2n} \tag{476}$$

$$\le \binom{2n}{n} \left(\frac{1}{2}\right)^{2n} \frac{n!}{\lfloor n/3 \rfloor! \lfloor n/3 \rfloor! \lfloor n/3 \rfloor!} \left(\frac{1}{3}\right)^n \tag{477}$$

$$\sim \frac{1}{\sqrt{\pi n}} \frac{(n/e)^n \sqrt{2\pi n}}{(n/3e)^n \sqrt{2\pi n/3}^3} \left(\frac{1}{3}\right)^n = O(n^{-3/2}). \tag{478}$$

From this, one can deduce that $\sum_{n=1}^{\infty} \mathbb{P}(\mathbf{X}_n = \mathbf{0} \,|\, \mathbf{X}_0 = \mathbf{0}) \le \sum_{n=1}^{\infty} c n^{-3/2} < \infty$. Thus, by Theorem 5.5.6, conclude that $\mathbf{X}_n$ is transient. ▲

6. Existence and unieuqness of stationary distribution

Does a Markov chain always have a stationary distribution? If yes, when there is a unique stationary distribution? In Exercise 5.2.3 (only for finite state spaces) we have exploited the relation between stationary distribution and eigenvectors associated with eigenvalue 1 to study existence and uniqueness of stationary distribution. Namely, the all-one column vector is a right-eigenvector associated with eigenvalue 1 of a given transition matrix P. Hence its transpose P^T has an eigenvector v associated with eigenvalue 1. That is, $P^T v = v$. Then taking transpose, we get $v^T P = v^T$, so $\pi = v^T/D$, where D is the sum of all entries in v, is a stationary distribution of P. For the uniqueness, see part (iii) of Exercise 5.2.3.

However, the linear algebra approach do not provide us a useful intuition for the behavior of the Markov chain itself. In this section, we give an alternative and constructive argument to show that every Markov chain has a stationary distribution. This is valuable since we can write down what the stationary distribution is, whereas the linear algebra argument doesn't give this information.

For each $x, y \in \Omega$, let $V_n(x,y)$ denote the number of visits to y in the first n steps given that $X_0 = x$,

$$V_n(x,y) := \sum_{k=1}^{n} \mathbf{1}(X_k = y \,|\, X_0 = x). \tag{479}$$

We will see that with reasonable assumptions, a Markov chain will admit a unique stationary distribution given by

$$\pi(y) := \lim_{t\to\infty} \frac{1}{n}\mathbb{E}[V_n(x,y)], \tag{480}$$

which is the expected proportion of times spending at y starting at x and it turns out that it does not depend on x. The main result in this section, Theorem 5.6.4, states that when the Markov chain satisfies certain reasonable conditions, (503) is indeed a stationary distribution of the Markov chain. However, it is not a priori clear that the limit in (503) exists. Instead of taking the limiting frequency of visits to y, we will instead construct a measure by only counting visits to y during a single 'excursion' from x to x, which occurs during the time interval $[0, T_x]$. As we will see later, such a construction will give a valid probability distribution if and only if $\mathbb{E}[T_x \,|\, X_0 = x]$ is finite. For a further discussion, we introduce the following notion.

Definition 5.6.1. Let $(X_n)_{n\geq 0}$ be a Markov chain on a state space Ω. We say a recurrent state $x \in \Omega$ is *positive recurrent* if $\mathbb{E}[T_x \,|\, X_0 = x] < \infty$ and *null recurrent* if $\mathbb{E}[T_x \,|\, X_0 = x] = \infty$.

Proposition 5.6.2 (Poisitive recurrence is a class property)**.** *Let x, y be two communicating states. If one is positive recurrent, then so is the other .*

Example 5.6.3 (SRW on $\mathbb{Z}$ is null recurrent)**.** In Example 5.5.7, we have seen that SRW on $\mathbb{Z}$ is recurrent, which is equivalent to $\mathbb{P}(T_x < \infty \,|\, X_0 = x) = 1$. However, this does not necessarily mean that we have finite expected return time, $\mathbb{E}[T_x \,|\, X_0] = 0$. This is indeed the case for SRW on $\mathbb{Z}$, which turns out ot be null recurrent. For this, we remark the following asymptotic of 'persistence probability' of SRW on $\mathbb{Z}$:

$$\mathbb{P}(X_1 \geq 1,\, X_2 \geq 1, \ldots, X_n \geq 1 \,|\, X_0 = 0) \sim \sqrt{\frac{2}{\pi n}}. \tag{481}$$

(For reference, see, e.g., [**?**].) It follows that, by the tail-sum formula for the expectation of nonnegative RVs,

$$\mathbb{E}[T_0 \,|\, X_0 = 0] = \sum_{n=1}^{\infty} \mathbb{P}(T_0 \geq n) \sim \sum_{n=1}^{\infty} \sqrt{\frac{2}{\pi n}} = \infty. \tag{482}$$

Thus the origin is null recurrent. By symmetry, all other states are also null recurrent. ▲

The full theorem regarding the existence and uniqueness of the stationary distribution is the following:

Theorem 5.6.4 (Existence and uniqueness of stationary distribution)**.** *Assume P is irreducible. Then P has a stationary distribution if and only if all states are positive recurrent. In this case the stationary distribution π is unique and its values satisfy*

$$\pi(x) = \frac{1}{\mathbb{E}[T_x \,|\, X_0 = x]} \qquad \forall\ x \in \Omega. \tag{483}$$

Corollary 5.6.5. *Assume Ω is finite and P is irreducible. Then P has a unique stationary distribution π given by* (483)*.*

PROOF. By Theorem 5.6.4, we only need to verify if all states in Ω are positive recurrent. This is done in Exercise 5.6.6. □

Exercise 5.6.6 (Finite irreducible implies positive recurrence)**.** Let $(X_n)_{n\geq 0}$ be an irreducible Markov chain on a finite state space Ω with transition matrix P. Let $X_0 = x \in \Omega$. For any $y \in \Omega$ and $t \geq 0$, define the *first hitting time* of y by

$$T_y = \min\{k \geq 1 \,|\, X_k = y\}. \tag{484}$$

We will show that, for all $x, y \in \Omega$,

$$\mathbb{E}[T_y \,|\, X_0 = x] < \infty. \tag{485}$$

That is, the expected hitting time of $(X_n)_{n\geq 0}$ to any state starting from any state is finite.

(i) Recall that by the irreducibility and Exercise 5.5.2, for any $z, y \in \Omega$, $P^r(z, y) > 0$ for some $r \geq 1$ (r may depend on z and y). Let

$$d = \max_{z,y\in\Omega} \min\{r \geq 1 \,|\, P^r(z, y) > 0\}. \tag{486}$$

Show that there exists some constant $\delta > 0$ such that

$$\min_{y,z\in\Omega} \mathbb{P}(X_n \text{ visits } y \text{ some time during } [0, d] \,|\, X_0 = z) \geq \delta > 0. \tag{487}$$

(ii) Show that for any $k \geq 1$, (You may use strong Markov property)

$$\mathbb{P}(T_y > kd) \leq (1-\delta)\mathbb{P}(T_y > (k-1)d). \tag{488}$$

Use induction to conclude that

$$\mathbb{P}(T_y > kd) \leq (1-\delta)^k. \tag{489}$$

(iii) Noting that $\mathbb{P}(T_y \geq t)$ is a non-increasing function in t, show that

$$\mathbb{E}[T_y] = \sum_{t=1}^{\infty} \mathbb{P}(T_y \geq t) \leq d \sum_{k=0}^{\infty} \mathbb{P}(T_y \geq kd) \leq d \sum_{k=0}^{\infty} (1-\delta)^k = d/\delta < \infty. \tag{490}$$

Conclude that all states in an finite irreducible Markov chain are positive recurrent.

7. Uniqueness of stationary distribution

In establishing Theorem 5.6.4, we first consider the uniqueness of stationary distribution and when it should hold. Before we state our main result, we first take a look at random walk on disconnected graphs and that it must have multiple stationary distribution.

Exercise 5.7.1 (Random walk on disconnected graphs)**.** Let $G = (V, E)$ be a graph with two disjoint components G_1 and G_2. Let $(X_n)_{n\geq 0}$ be a random walk on G and denote by P its transition matrix.

(i) Let P_1 and P_2 be the transition matrices for random walks on G_1 and G_2, respectively. Let $V_1 = \{1, 2, \cdots, n\}$ and $V_2 = \{n+1, n+2, \cdots, n+m\}$ be the set of nodes in G_1 and G_2. Then show that P is of the following block diagonal form

$$P = \begin{bmatrix} P_1 & O \\ O & P_2 \end{bmatrix}. \tag{491}$$

(ii) Let $\pi_1 = [\pi_1^{(1)}, \cdots, \pi_1^{(n)}]$ and $\pi_2 = [\pi_2^{(1)}, \cdots, \pi_2^{m}]$ be any stationary distributions for P_1 and P_2. Let

$$\bar{\pi}_1 = [\pi_1^{(1)}, \cdots \pi_1^{(n)}, 0 \cdots, 0] \tag{492}$$

$$\bar{\pi}_2 = [0, \cdots, 0, \pi_2^{(1)}, \cdots, \pi_2^{m}] \tag{493}$$

be distributions on $V = V_1 \cup V_2 = \{1, 2, \cdots n, n+1, \cdots, n+m\}$. Show that for any $\lambda \in [0, 1]$, $\pi := \lambda\bar{\pi}_1 + (1-\lambda)\bar{\pi}_2$ is a stationary distribution for P.

(iii) Recall that random walk on arbitrary graph has a stationary distribution, where probability of each node is proportional to its degree. Conclude that the random walk $(X_n)_{n\geq 0}$ on G has infinitely many stationary distribution.

A graph G is said to be *connected* if there exists a 'path' between any given two nodes. A *path* in G is a sequence of distinct nodes $v_0, v_1, \cdots, v_k$ such that consecutive nodes are adjacent in G. In Exercise 5.7.1, we have seen that the random walk on a disconnected graph has infinitely many stationary distribution. How about random walk on connected graphs? We will see that they have a unique stationary distribution, which must be the typical one that we know.

Proposition 5.7.2 (RW on connected graphs)**.** *Let $G = (V, E)$ be a connected graph. Then the random walk on G has a unique stationary distribution π, which is given by*

$$\pi(i) = \frac{\deg_G(i)}{2|E|}. \tag{494}$$

PROOF. Let P denote the transition matrix of random walk on G. Let $V = \{1, 2, \cdots, N\}$ and let E_1 be the column eigenspace of P associated with eigenvalue 1. That is,

$$E_1 = \{\mathbf{v} \in \mathbb{R}^N \mid P\mathbf{v} = \mathbf{v}\}. \tag{495}$$

Note that $[1, 1, \cdots, 1]^T \in E_1$, so E_1 has dimension ≥ 1. According to Exercise 5.2.3, it suffice to show that E_1 has dimension 1. This is equivalent to say that E_1 is spanned by $[1, \cdots, 1]^T$.

The argument uses the 'maximum principle' for 'harmonic functions'. Let $\mathbf{x} \in E_1$ be arbitrary. We will view $\mathbf{x}$ as a function on the node set V. So we want to show that $\mathbf{x}$ is a constant function on V. Since V is finite, there is some node k where $\mathbf{x}$ attains its global maximum. Suppose node k has some neighbor j such that $\mathbf{x}(j) < \mathbf{x}(k)$. Then from the eigenvector condition $P\mathbf{x} = \mathbf{x}$,

$$\begin{aligned}
\mathbf{x}(k) &= \sum_{i=1}^{N} P(k, i)\mathbf{x}(i) && (496)\\
&= P(k, j)\mathbf{x}(j) + \sum_{i \neq j} P(k, i)\mathbf{x}(i) && (497)\\
&< P(k, j)\mathbf{x}(k) + \sum_{i \neq j} P(k, i)\mathbf{x}(i) && (498)\\
&\leq P(k, j)\mathbf{x}(k) + \sum_{i \neq j} P(k, i)\mathbf{x}(k) = \mathbf{x}(k), && (499)
\end{aligned}$$

which is a contradiction. This implies that $\mathbf{x}$ also attains its global maximum. Apply the similar argument to the neighbors of k. Repeating the same argument, we see that $\mathbf{x}$ must be constant on a 'connected component' of G containing k. But since G is connected, it follows that $\mathbf{x}$ is constant on V. This shows the assertion. □

If we carefully examine the proof of above result, we find that we haven't really used the value of entries of the transition matrix P. Rather, we only used an abstract property following from the connectivity of G: We can reach any node from any other node. We extract this as a general property of Markov chains.

Definition 5.7.3 (Irreducibility)**.** Let P be the transition matrix of a Markov chain $(X_n)_{n \geq 0}$ on a finite state space Ω. We say the chain (or P) is *irreducible* if for any $i, j \in \Omega$,

$$\mathbb{P}(X_k = j \mid X_0 = i) > 0 \tag{500}$$

for some integer $k \geq 0$.

In words, a Markov chain is irreducible if every state is 'accessible' from any other state.

Exercise 5.7.4 (RW on connected graphs is irreducible)**.** Let $G = (V, E)$ be a connected graph. Let $(X_n)_{n \geq 0}$ be a random walk on G and denote by P its transition matrix. Let $v_0, v_1, \cdots, v_k$ such that consecutive nodes are adjacent in G. Show that

$$\begin{aligned}
P^k(v_0, v_k) = \mathbb{P}(X_k = v_k \mid X_0 = v_0) &\geq P(v_0, v_1)P(v_1, p_2) \cdots P(v_{k-1}, v_k) && (501)\\
&= \frac{1}{\deg_G(v_0)} \frac{1}{\deg_G(v_1)} \cdots \frac{1}{\deg_G(v_k)} > 0. && (502)
\end{aligned}$$

Conclude that random walks on connected graphs are irreducible.

Now we state the general uniqueness theorem of stationary distribution.

Lemma 5.7.5 (Uniqueness of stationary distribution)**.** *Let $(X_n)_{n\geq 0}$ be an irreducible Markov chain. Then it has at most a unique stationary distribution.*

PROOF. Exercise. Mimic the proof of Proposition 5.7.2. □

7.1. Construction of stationary distribution (Optional*). The gist of the to construct of a stationary distribution using the visit counts $V_n(x,y)$ is given in the following proposition. It constructs a stationary distribution for general Markov chains with a positive recurrent state.

Lemma 5.7.6. *Consider a Markov chain $(X_n)_{n\geq 0}$ on Ω with transition matrix P. Fix a recurrent state $x\in\Omega$ and for each $y\in\Omega$, define*

$$\lambda_x(y) := \mathbb{E}\left[V_{T_x}(x,y)\right], \tag{503}$$

which is the expected number of visits to y during $[0,T_x]$ (a single excursion from x to x).

(i) *For each $x\in\Omega$, $\lambda_x(x)=1$ and $\sum_{y\in\Omega}\lambda_x(y)=\mathbb{E}[T_x\,|\,X_0=x]$.*

(ii) *For each $x\in\Omega$, $\lambda_x=\lambda_x P$.*

(iii) *If x is positive recurrent, then $\pi_x := \frac{1}{\mathbb{E}[T_x\,|\,X_0=x]}\lambda_x$ is a stationary distribution for P.*

PROOF. To show **(i)**, note that $N_{T_x}(x,x)=1$ so $\lambda_x(x)=1$. Also,

$$\sum_{y\in\Omega}\lambda_x(y)=\sum_{y\in\Omega}\mathbb{E}\left[V_{T_x}(x,y)\right]=\sum_{y\in\Omega}\mathbb{E}\left[\sum_{n=1}^{T_x}\mathbf{1}(X_n=y)\,\middle|\,X_0=x\right] \tag{504}$$

$$=\mathbb{E}\left[\sum_{y\in\Omega}\sum_{n=1}^{T_x}\mathbf{1}(X_n=y)\,\middle|\,X_0=x\right] \tag{505}$$

$$=\mathbb{E}\left[\sum_{n=1}^{T_x}\underbrace{\sum_{y\in\Omega}\mathbf{1}(X_n=y)}_{=1}\,\middle|\,X_0=x\right]=\mathbb{E}\left[\sum_{n=1}^{T_x}1\,\middle|\,X_0=x\right]=\mathbb{E}[T_x\,|\,X_0=x], \tag{506}$$

where the third equality uses Fubini's theorem for nonnegative summands.

To show **(ii)**, our goal is to verify for each $y\in\Omega$,

$$\sum_{z\in\Omega}\lambda_x(z)P(z,y)=\lambda_x(y). \tag{507}$$

Indeed, first observe that

$$\mathbb{E}\left[\sum_{n=1}^{T_x}\mathbf{1}(X_n=z)\,\middle|\,X_0=x\right]=\mathbb{E}\left[\sum_{n=1}^{\infty}\mathbf{1}(X_n=z,\,T_x\geq n)\,\middle|\,X_0=x\right] \tag{508}$$

$$=\sum_{n=1}^{\infty}\mathbb{P}\left(X_n=z,\,T_x\geq n\,\middle|\,X_0=x\right) \tag{509}$$

$$=\sum_{n=1}^{\infty}\mathbb{P}\left(X_1\neq x,\ldots,X_{n-1}\neq x,\,X_n=z\,\middle|\,X_0=x\right), \tag{510}$$

where for the first equality, we have converted a random number of summation into an infinite sum with additioanl indicator. Next, by time-homogeneity and Markov property, we observe that

$$\mathbb{P}\left(X_1\neq x,\ldots,X_{n-1}\neq x,\,X_n=z\,\middle|\,X_0=x\right)P(z,y) \tag{511}$$

$$=\mathbb{P}\left(X_1\neq x,\ldots,X_{n-1}\neq x,\,X_n=z\,\middle|\,X_0=x\right)\mathbb{P}(X_{n+1}=y\,|\,X_n=z) \tag{512}$$

$$= \mathbb{P}\left(X_1 \neq x, \ldots, X_{n-1} \neq x, X_n = z \,\middle|\, X_0 = x\right) \mathbb{P}(X_{n+1} = y \,|\, X_1 \neq x, \ldots, X_{n-1} \neq x, X_n = z) \tag{513}$$

$$= \mathbb{P}\left(X_1 \neq x, \ldots, X_{n-1} \neq x, X_n = z, X_{n+1} = y \,\middle|\, X_0 = x\right). \tag{514}$$

Hence, noting that $\lambda_x(x) = 1$ from Lemma 5.7.6,

$$\sum_{z \in \Omega} \lambda_x(z) P(z, y) = P(x, y) + \sum_{z \in \Omega \setminus \{x\}} \mathbb{E}\left[\sum_{n=1}^{T_x} \mathbf{1}(X_n = z) \,\middle|\, X_0 = x\right] P(z, y) \tag{515}$$

$$= P(x, y) + \sum_{z \in \Omega \setminus \{x\}} \sum_{n=1}^{\infty} \mathbb{P}\left(X_1 \neq x, \ldots, X_{n-1} \neq x, X_n = z, X_{n+1} = y \,\middle|\, X_0 = x\right) \tag{516}$$

$$= P(x, y) + \sum_{n=1}^{\infty} \sum_{z \in \Omega \setminus \{x\}} \mathbb{P}\left(X_1 \neq x, \ldots, X_{n-1} \neq x, X_n = z, X_{n+1} = y \,\middle|\, X_0 = x\right) \tag{517}$$

$$= P(x, y) + \sum_{n=1}^{\infty} \mathbb{P}\left(X_1 \neq x, \ldots, X_{n-1} \neq x, X_n \neq x, X_{n+1} = y \,\middle|\, X_0 = x\right) \tag{518}$$

$$= \sum_{n=1}^{\infty} \mathbb{P}\left(X_1 \neq x, \ldots, X_{n-1} \neq x, X_n \neq y \,\middle|\, X_0 = x\right) \tag{519}$$

$$= \mathbb{E}\left[\sum_{n=1}^{T_x} \mathbf{1}(X_n = z) \,\middle|\, X_0 = x\right] = \lambda_x(y). \tag{520}$$

This shows **(ii)**.

Lastly, if x is positive recurrent, then one can normalize λ_x by the total sum $\mathbb{E}[T_x \,|\, X_0 = x] < \infty$ (see **(i)**) to obtain a probability distribution π_x that is invariant under right-multiplication of P. This shows **(iii)**. □

The following result shows the (the more important) half of Theorem 5.6.4.

Theorem 5.7.7. *Consider an irreducible Markov chain $(X_n)_{n \geq 0}$ on Ω with transition matrix P. Suppose all states are positive recurrent. Then there is a unique stationary distribution π for P given by* (483).

PROOF. According to Lemma 5.7.6, for each $x \in \Omega$, we have stationary distribution π_x defined in Lemma 5.7.6 **(iii)**. But by Lemma 5.7.5, there is a unique stationary distribution for P, which we will denote as π. Then for each $x \in \Omega$, it holds that

$$\pi(x) = \pi_x(x) = \frac{\lambda_x(x)}{\mathbb{E}[T_x \,|\, X_0 = x]} = \frac{1}{\mathbb{E}[T_x \,|\, X_0 = x]}. \tag{521}$$

This shows the assertion. □

In the following proposition, we make a simple observation that any stationary distribution will take value zero on transient states.

Proposition 5.7.8 (Stationary distribution is supported on recurrent states)**.** *Let a transition matrix P on Ω has an stationary distribution π. Then for each $x \in \Omega$, $\pi(x) > 0$ implies that x is recurrent.*

PROOF. Suppose $\pi(x) > 0$. Note that $\pi = \pi P^n$ for all $n \geq 1$. Hence

$$\infty = \sum_{n=1}^{\infty} \pi(x) = \sum_{n=1}^{\infty} \sum_{z \in \Omega} \pi(z) P^n(z, x) = \sum_{z \in \Omega} \sum_{n=1}^{\infty} \pi(z) P^n(z, x) \tag{522}$$

$$= \sum_{z \in \Omega} \pi(z) \sum_{n=1}^{\infty} P^n(z, x) \tag{523}$$

$$= \sum_{z \in \Omega} \pi(z) \mathbb{E}[N_x \,|\, X_0 = z], \tag{524}$$

where we have used Fubini's theorem to switch the order of double summation of nonnegative terms and the last equality uses (452). This implies that there exists some $z \in \Omega$ such that $\pi(z) > 0$ and $\mathbb{E}[N_x \,|\, X_0 = z] = \infty$. But by Proposition 5.5.5 this implies that x is recurrent (as well as $\rho_{zx} > 0$), as desired. □

PROOF OF THEOREM 5.6.4. According to Theorem 5.7.7, it suffices to show that, assuming irreducibility and the existence of stationary distribution π, all states are positive recurrent. Indeed, since π is a probability distribution, $\pi(x) > 0$ for some $x \in \Omega$, so by Proposition 5.7.8, x is recurrent. But since P is irreducible, it follows that all states are recurrent. Next, we need the following lemma:

Lemma: For irreducible and recurrent Markov chains, any two invariant measures for P is a constant multiple of each other. That is, if $\lambda = \lambda P$ and $\mu = \mu P$, then $\lambda = c\mu$ for some $c > 0$.

For the proof of the above lemma, see [**?** , Thm. 2.67]. Now, by Lemma 5.7.6, λ_x is an invariant measure for P for each $x \in \Omega$. In particular, we have $\lambda_x = c_x \pi$ for some $c_x > 0$. Then

$$\mathbb{E}[T_x \,|\, X_0 = x] = \sum_{y \in \Omega} \lambda_x(y) = \sum_{y \in \Omega} c_x \pi(y) = c_x < \infty, \tag{525}$$

so x is positive recurrent. Since this holds for all x, we conclude that all states are positive recurrent. □

PROOF OF PROPOSITION 5.6.2. Suppose two states $x, y \in \Omega$ communicate and x is positive recurrent. By restricting onto the communicating class that contains x and y, we may assume that the Markov chain is irreducible and recurrent. By Lemma 5.7.6, there is a statioanry distribution, say, π_x. Then by a similar argument as in the the last part of the proof of Theorem 5.6.4, we have $\lambda_y = c_x \pi_x$ and this implies that y is also positive recurrent. □

8. Limit theorems for Markov chains

In this section, we establish a Markov chain version of the strong law of large numbers. Namely, consider the sequence of RVs $f(X_0), f(X_1), \ldots$ where $(X_n)_{n \geq 0}$ is a Markov chain and $f : \Omega \to \mathbb{R}$ is a function. Can a version of law of large number hold for the random increments $f(X_0), f(X_1), \ldots$? Namely, does the average of such increments converge to 'the' expected value? These are certainly not i.i.d. since they are driven by the underlying Markov chain X_n. However, if the chain starts from a positive recurrent state, say x, then one can group those increments according to the excursions of the chain. By the strong Markov property such excursions are i.i.d., so we can use the usual laws of large numbers for i.i.d. increments. This is the basic idea behind the proof of the following result, which states that "*time-average of a function converges to a space-average given by the stationary distribution*".

Theorem 5.8.1 (SLLN for finite Markov chains)**.** *Let $(X_n)_{n \geq 0}$ be an irreducible Markov chain on finite state space Ω with transition matrix P with arbitrary initial distribution. Let $f : \Omega \to \mathbb{R}$ denote a function. Then almost surely,*

$$\lim_{n \to \infty} \frac{1}{n} \sum_{k=1}^{n} f(X_k) = \mathbb{E}_{X \sim \pi}\big[f(X)\big] = \sum_{x \in \Omega} f(x)\pi(x), \tag{526}$$

where π denotes the unique stationary distribution for P.

PROOF. Since Ω is finite and P is irreducible, all states are positive recurrent (see Exercise 5.6.6) and there exists a unique stationary distribution π (see Theorem 5.6.4). Let $V_x(n) = \sum_{k=1}^{n} \mathbf{1}(X_k = x)$ denote the number of visits to x in the first n steps of the Markov chain. The key step in converting time-average into space-average is the following observation: for each $x \in \Omega$, $f(x)$ is added up by $V_x(n)$ times:

$$\sum_{k=1}^{n} f(X_k) = \sum_{k=1}^{n} f(X_k) \sum_{x \in \Omega} \mathbf{1}(X_k = x) = \sum_{x \in \Omega} \sum_{k=1}^{n} f(X_k)\mathbf{1}(X_k = x) \tag{527}$$

$$= \sum_{x \in \Omega} \sum_{k=1}^{n} f(x)\mathbf{1}(X_k = x) \tag{528}$$

$$= \sum_{x\in\Omega} f(x) \sum_{k=1}^{n} \mathbf{1}(X_k = x) \tag{529}$$

$$= \sum_{x\in\Omega} f(x) V_x(n). \tag{530}$$

Now dividing by n and taking the limit $n \to \infty$, we get

$$\lim_{n\to\infty} \sum_{k=1}^{n} f(X_k) = \lim_{n\to\infty} \sum_{x\in\Omega} f(x) \frac{V_x(n)}{n} = \sum_{x\in\Omega} f(x) \lim_{n\to\infty} \frac{V_x(n)}{n} = \sum_{x\in\Omega} f(x)\pi(x), \tag{531}$$

where the second equality of exchanging limit and summation uses the fact that Ω is finite (it would be non-trivial if Ω is infinite) and the last equality uses Exercise 5.8.2. □

Exercise 5.8.2 (Convergence of empirical distribution)**.** Let $(X_t)_{t\geq 0}$ be an irreducible Markov chain on Ω with a unique stationary distribution π. For each $x \in \Omega$, define the following quantities

$$T_x^{(k)} := \text{time of the } k\text{th visit to } x \tag{532}$$

$$V_x(n) := \sum_{k=1}^{n} \mathbf{1}(X_k = x) = \text{number of visits to } x \text{ during } [0, n]. \tag{533}$$

Here X_0 follows an arbitrary initial distribution μ on Ω.

(i) Let $\tau_k = T_x^{(k)} - T_x^{(k-1)}$ for $k \geq 2$. Show that τ_k's are i.i.d. and $0 < \mathbb{E}[\tau_k] < \infty$. (For the i.i.d. part, use strong Markov property. For the finite expectation part, see Exercise 5.6.6 for the finite case and Theorem 5.6.4 for the general case.)

(ii) Use Markov property and strong law of large numbers to show

$$\mathbb{P}\left(\lim_{n\to\infty} \frac{\tau_2 + \cdots + \tau_n}{n} = \mathbb{E}[\tau_2]\right) = 1. \tag{534}$$

Furthermore, noting that $T_x^{(n)} = T_x^{(n)} + \tau_2 + \cdots + \tau_n$, conclude that

$$\mathbb{P}\left(\lim_{n\to\infty} \frac{T_n}{n} = \mathbb{E}[\tau_2]\right) = 1. \tag{535}$$

(iii) Let $V_x(t) = V(t)$ be the number of visits to x up to time t. Using the fact that $\mathbb{E}[\tau_k] < \infty$, show that $V(t) \to \infty$ as $t \to \infty$ a.s. Also, noting that $T_x^{(V_x(n))} \leq n < T_x^{(V_x(n)+1)}$, use **(ii)** to deduce

$$\mathbb{P}\left(\lim_{n\to\infty} \frac{V_x(n)}{n} = \frac{1}{\mathbb{E}[\tau_2]}\right) = 1. \tag{536}$$

Furthermore, use Theorem 5.6.4 to deduce that

$$\mathbb{P}\left(\lim_{t\to\infty} \frac{V_x(t)}{t} = \pi(x)\right) = 1. \tag{537}$$

Example 5.8.3. Let $(X_n)_{n\geq 0}$ be a Markov chain on $\Omega = \{0, 1\}$ with transition matrix

$$P = \begin{bmatrix} 1/4 & 3/4 \\ 2/3 & 1/3 \end{bmatrix}. \tag{538}$$

Given a function $f : \Omega \to \mathbb{R}$, we would like to find the limit

$$\lim_{n\to\infty} \frac{1}{n} \sum_{k=1}^{n} f(X_k). \tag{539}$$

To this end, first note that P is irreducible. Since Ω is finite, it has a unique stationary distribution. More explictly, one can solve $\pi = \pi P$ for a probability distribution π on Ω and see that π must be $[8/17, 9/17]$. Then by Theorem 5.8.1, almost surely,

$$\lim_{n\to\infty} \frac{1}{n} \sum_{k=1}^{n} f(X_k) = f(0)\pi(0) + f(1)\pi(1) = \frac{8}{17} f(0) + \frac{9}{17} f(1). \tag{540}$$

Note that this holds regardless of the initial distribution of the Markov chain. ▲

The Markov chain SLLN stated in Theorem 5.8.1 holds for finite irreducible chains. When the state space is countably infinite, the analogous result continues to hold if we need to augment irreduciblity with positive recurrence:

Theorem 5.8.4 (SLLN for countable Markov chains)**.** *Let $(X_n)_{n\geq 0}$ be an irreducible and positive recurrent Markov chain on state space Ω with transition matrix P unique statioanry distribution π. Let $f:\Omega\to\mathbb{R}$ denote a function that is integerable w.r.t. π, that is, $\sum_{x\in\Omega}|f(x)|\pi(x)<\infty$. Then almost surely, for arbitrary initial distribution,*

$$\lim_{n\to\infty}\frac{1}{n}\sum_{k=1}^{n}f(X_k)=\mathbb{E}_{X\sim\pi}\left[f(X)\right]=\sum_{x\in\Omega}f(x)\pi(x). \tag{541}$$

PROOF. Omitted. See [**?** , Thm 2.94]. □

Example 5.8.5 (Success run)**.** Consider the success run chain in Exercise 5.2.5: It is a Markov chain $(X_n)_{n\geq 0}$ on the state space $\Omega=\mathbb{Z}_{\geq 0}$ of nonnegative integers with the following transition matrix P:

$$P(x,x+1)=p,\quad P(x,0)=1-p\qquad\text{for all } x\in\mathbb{Z}_{\geq 0}, \tag{542}$$

where $p\in[0,1]$ is a fixed parameter. We have shown that it has a unique stationary distribution π with $\pi(k)=(1-p)p^k$ for $k=1,2,\ldots$. Hence for a function $f:\mathbb{Z}_{\geq 0}\to\mathbb{R}$, by Theorem 5.8.4, we have

$$\lim_{n\to\infty}\frac{1}{n}\sum_{k=1}^{n}f(X_k)=\sum_{k=1}^{\infty}f(k)(1-p)p^k\qquad\text{a.s.} \tag{543}$$

We take the following two special examples:

(a) Suppose one gets a dollor at each success, but only when the success run has length at least two. What is the long-term rate rewards, relative to the number of trials? In this case we set

$$f(k)=\mathbf{1}(k\geq 3). \tag{544}$$

Then the long-term rate of reward is

$$\lim_{n\to\infty}\frac{1}{n}\sum_{k=1}^{n}f(X_k)=\sum_{k=3}^{\infty}(1-p)p^k=p^3\qquad\text{a.s.} \tag{545}$$

(b) What is the average length of success run in the long run? In this case we take $f(k)=k$ for $k\geq 1$ and

$$\lim_{n\to\infty}\frac{1}{n}\sum_{k=1}^{n}f(X_k)=\sum_{k=1}^{\infty}k(1-p)p^k=\frac{p}{1-p}\qquad\text{a.s.} \tag{546}$$

▲

9. Convergence to the stationary distribution

Let $(X_n)_{n\geq 0}$ be an irreducible Markov chain on a finite state space Ω. By Theorem 5.6.4, we know that the chain has unique stationary distribution π. Hence if we denote by π_t the distribution of X_t, then we should expect that π_t 'converges' to to π as $t\to\infty$ in some sense. We will prove a precise version of this claim in this section.

We start with an example, where we show the convergence of a 2-state chain using a diagonalization of its transition matrix.

Exercise 5.9.1. Let $(X_n)_{n\geq 0}$ be a Markov chain on $\Omega=\{1,2\}$ with the following transition matrix

$$P=\begin{bmatrix}0.2 & 0.8\\ 0.6 & 0.4\end{bmatrix}. \tag{547}$$

(i) Show that P admits the following diagonalization

$$P=\begin{bmatrix}1 & -4/3\\ 1 & 1\end{bmatrix}\begin{bmatrix}1 & 0\\ 0 & -2/5\end{bmatrix}\begin{bmatrix}1 & -4/3\\ 1 & 1\end{bmatrix}^{-1}. \tag{548}$$

(ii) Show that P^t admits the following diagonalization

$$P^t = \begin{bmatrix} 1 & -4/3 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & (-2/5)^t \end{bmatrix} \begin{bmatrix} 1 & -4/3 \\ 1 & 1 \end{bmatrix}^{-1}. \tag{549}$$

(iii) Let $\mathbf{r}_t$ denote the row vector of distribution of X_t. Show that

$$\mathbf{r}_t = \mathbf{r}_0 \begin{bmatrix} 1 & -4/3 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & (-2/5)^t \end{bmatrix} \begin{bmatrix} 1 & -4/3 \\ 1 & 1 \end{bmatrix}^{-1}. \tag{550}$$

Also show that

$$\lim_{t\to\infty} \mathbf{r}_t = \mathbf{r}_0 \begin{bmatrix} 3/7 & 4/7 \\ 3/7 & 4/7 \end{bmatrix} = [3/7, 4/7]. \tag{551}$$

(iv) Show that $\pi = [3/7, 4/7]$ is the unique stationary distribution for P. Conclude that regardless of the initial distribution $\mathbf{r}_0$, the distribution of the Markov chain $(X_n)_{n\geq 0}$ converges to $[3/7, 4/7]$.

Next, we observe that an irreducible MC may not always converge to the stationary distribution. The key issue there is the notion of 'periodicity'.

Example 5.9.2. Let $(X_n)_{n\geq 0}$ be a 2-state MC on state space $\Omega = \{0, 1\}$ with transition matrix

$$P = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}. \tag{552}$$

Namely, the chain deterministically alternates between the two states. Note that it is irreducible and has a unique stationary distribution

$$\pi = [1/2, 1/2]. \tag{553}$$

Let π_t be the distribution of X_n, where the initial distribution is given by $\pi_0 = [1, 0]$. Then we have

$$\pi_1 = [0, 1] \tag{554}$$
$$\pi_2 = [1, 0] \tag{555}$$
$$\pi_3 = [0, 1] \tag{556}$$
$$\pi_4 = [1, 0], \tag{557}$$

and so on. Hence the distributions π_n do not converge to the stationary distribution π. ▲

Example 5.9.3 (RW on torus)**.** Let $\mathbb{Z}_n$ be the set of integers modulo n. Let $G = (V, E)$ be a graph where $V = \mathbb{Z}_n \times \mathbb{Z}_n$ and two nodes $u = (u_1, u_2)$ and $v = (v_1, v_2)$ are adjacent if and only if

$$|u_1 - u_2| + |v_1 - v_2| = 1. \tag{558}$$

Such a graph G is called the $n \times n$ *torus* and we write $G = \mathbb{Z}_n \times \mathbb{Z}_n$. Intuitively, it is obtained from the $n \times n$ square grid by adding boundary edges to wrap around (see Figure 70 left).

Now let $(X_t)_{t\geq 0}$ be a random walk on G. Since G is connected, X_t is irreducible. Since all nodes in G have degree 4, the uniform distribution on $\mathbb{Z}_n \times \mathbb{Z}_n$, which we denote by π, is the unique stationary distribution of X_t. Let π_t denote the distribution of X_t.

For instance, consider $G = \mathbb{Z}_6 \times \mathbb{Z}_6$. As illustrated in Figure 70 below, observe that if X_0 is one of the red nodes (where sum of coordinates is even), then X_t is at a red node for any t = even and at a black node (where sum of coordinates is odd) at t = odd. Hence, π_t is supported only on the 'even' nodes for even times and on the 'odd' nodes for the odd times. Hence π_t does not converge in any sense to the uniform distribution π. ▲

The key issue in the 2-periodicity of RW on $G = \mathbb{Z}_6 \times \mathbb{Z}_6$ is that it takes even number of steps to return to any given node. Generalizing this observation, we introduce the following notion of periodicity.

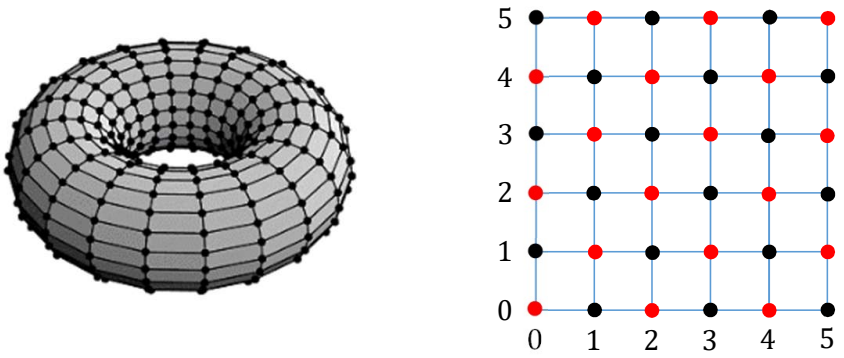


FIGURE 70. (Left) Torus graph (Figure excerpted from [LP17]). (Right) RW on torus $G = \mathbb{Z}_6 \times \mathbb{Z}_6$ has period 2.

Definition 5.9.4. Let P be the transition matrix of a Markov chain $(X_t)_{t\geq 0}$ on a finite state space Ω. For each state $x \in \Omega$, let $\mathcal{T}(x) = \{t \geq 1 \,|\, P^t(x,x) > 0\}$ be the set of times when it is possible for the chain to return to starting state x. We define the *period* of x by the greatest common divisor of $\mathcal{T}(x)$. We say the chain X_t is *aperiodic* if all states have period 1.

Example 5.9.5. Let $\mathcal{T}(x) = \{4,6,8,10,\cdots\}$. Then the period of x is 2, even through it is not possible to go from x to x in 2 steps. If $\mathcal{T}(x) = \{4,6,8,10,\cdots\} \cup \{3,6,9,12,\cdots\}$, then the period of x is 1. This means the return times to x is irregular. For the RW on $G = \mathbb{Z}_6 \times \mathbb{Z}_6$ in Example 5.9.3, all nodes have period 2. ▲

Exercise 5.9.6 (Aperiodicity of RW on graphs)**.** Let $(X_t)_{t\geq 0}$ be a random walk on a connected graph $G = (V,E)$.

(i) Show that all nodes have the same period.
(ii) If G contains an odd cycle C (e.g., triangle), show that all nodes in C have period 1.
(iii) Show that X_t is aperiodic if and only if G contains an odd cycle.
(iv)* Show that X_t is periodic if and only if G is *bipartite*. (A graph G is bipartite if there exists a partition $V = A \cup B$ of nodes such that all edges are between A and B.)

Remark 5.9.7. If $(X_t)_{t\geq 0}$ is an irreducible Markov chain on a finite state space Ω, then all states $x \in \Omega$ must have the same period. The argument is similar to that for Exercise 5.9.6 (i).

We now state the convergence theorem for random walks on graphs.

Theorem 5.9.8 (Convergence of RW on graphs)**.** *Let $(X_t)_{t\geq 0}$ be a random walk on a connected graph $G = (V,E)$ with an odd cycle. Let π denote the unique stationary distribution π. Then for each $x, y \in V$,*

$$\lim_{t\to\infty} \mathbb{P}(X_t = y \,|\, X_0 = x) = \pi(y). \tag{559}$$

PROOF. Since G contains an odd cycle, random walk on G is aperiodic according to Exercise 5.9.6. Let $(Y_t)_{t\geq 0}$ be another RW on G. We evolve X_t and Y_t simultaneously independently until they meet, after which we evolve them in union. Namely, let τ be the 'meeting time' of these two walkers:

$$\tau = \min\{t \geq 0 \,|\, X_t = Y_t\}. \tag{560}$$

Then we let $X_t = Y_t$ for all $t \geq \tau$. Still, if we disguise one random walk, then the other behaves exactly as it should do. (This is a 'coupling' between the two random walks)

Now suppose $X_0 = x$ for some $x \in V$ and Y_0 is drawn from the stationary distribution π. In particular, the distribution of Y_t is π for all t. Let π_t denote the distribution of X_t. Then for any $y \in V$,

$$|\pi_t(y) - \pi(y)| = \left|\mathbb{P}(X_t = y) - \mathbb{P}(Y_t = y)\right| \tag{561}$$

$$= \left|\mathbb{P}(X_t = y, Y_t = y) + \mathbb{P}(X_t = y, Y_t \neq y) - \mathbb{P}(X_t = y, Y_t = y) - \mathbb{P}(X_t \neq y, Y_t = y)\right| \tag{562}$$

$$\leq \mathbb{P}(X_t = y,\, Y_t \neq y) + \mathbb{P}(X_t \neq y,\, Y_t = y) \tag{563}$$
$$\leq \mathbb{P}(X_t \neq Y_t) \tag{564}$$
$$= \mathbb{P}(\tau > t). \tag{565}$$

Hence if suffices to show that $\mathbb{P}(\tau > t) \to 0$ as $t \to \infty$, as this will yield $|\pi_t(y) - \pi(y)| \to 0$ as $t \to \infty$. This follows from the fact that $\mathbb{P}(\tau < \infty) = 1$, that is, the two independent random walks on G eventually meet with probability 1 (see Exercise 5.9.11). □

The following exercise shows that if a RW on G is irreducible and aperiodic, then it is possible to reach any node from any other node in a fixed number of steps.

Exercise 5.9.9. Let $(X_t)_{t\geq 0}$ be a RW on a connected graph $G = (V, E)$. Let P denote its transition matrix. Suppose G contains an odd cycle, so that the walk is irreducible and aperiodic. For each $x \in V$, let $\mathcal{T}(x)$ denote the set of all possible return times to the state x.

(i) For any $x \in V$, show that $a, b \in \mathcal{T}(x)$ implies $a + b \in \mathcal{T}(x)$.

(ii) For any $x \in V$, show that $\mathcal{T}(x)$ contains 2 and some odd integer b.

(iii) For each $x \in V$, let b_x be the smallest odd integer contained in $\mathcal{T}(x)$. Show that $m \in \mathcal{T}(x)$ whenever $m \geq b_x$.

(iv) Let $b^* = \max_{x\in V} b_x$. Show that $m \in \mathcal{T}(x)$ for all $x \in V$ whenever $m \geq b^*$.

(v) Let $r = |V| + b^*$. Show that $P^r(x, y) > 0$ for all $x, y \in V$.

With a little more work, one can also show a similar statement for general irreducible and aperiodic Markov chains.

Exercise 5.9.10. Let P be the transition matrix of a Markov chain $(X_t)_{t\geq 0}$ on a finite state space Ω. Show that (i) implies (ii):

(i) P is irreducible and aperiodic.

(ii) There exists an integer $r \geq 0$ such that every entry of P^r is positive.

Furthermore, show that (ii) implies (i) if X_t is a RW on some graph $G = (V, E)$.

(Hint for (i)⇒(ii): You may use the fact that if a subset $\mathcal{T}$ of positive integers $\mathbb{N}$ is closed under addition with $\gcd(\mathcal{T}) = 1$, then it must contain all but finitely many integers. Proof is similar to Exercise 5.9.9, but a bit more number-theoretic. See [LP17, Lem. 1.27].)

Exercise 5.9.11. Let (X_t) and (Y_t) be independent random walks on a connected graph $G = (V, E)$. Suppose that G contains an odd cycle. Let P be the transition matrix of the random walk on G.

(i) Let $t \geq 0$ be arbitrary. Use Exercise 5.9.9 to deduce that for some integer $r \geq 1$, we have

$$\mathbb{P}(X_{t+r} = Y_{t+r} \,|\, X_t = x, Y_t = y) = \sum_{z\in V} \mathbb{P}(X_{t+r} = z = Y_{t+r} \,|\, X_t = x, Y_t = y) \tag{566}$$
$$= \sum_{z\in V} \mathbb{P}(X_{t+r} = z \,|\, X_t = x)\mathbb{P}(Y_{t+r} = z \,|\, Y_t = y) \tag{567}$$
$$= \sum_{z\in V} P^r(x, z)P^r(y, z) > 0. \tag{568}$$

(ii) Let $r \geq 1$ be as in (i) and let

$$\delta = \min_{x,y\in V} \sum_{z\in V} P^r(x, z)P^r(y, z) > 0. \tag{569}$$

Use (i) and Markov property that in every r steps, X_t and Y_t meet with probability $\geq \delta > 0$.

(iii) Let τ be the first time that X_t and Y_t meet. From (ii) and Markov property, deduce that

$$\mathbb{P}(\tau \geq kr) = \mathbb{P}(\tau \geq r)\mathbb{P}(X_t \text{ and } Y_t \text{ never meet during } [r, kr) \,|\, X_{r-1} \neq Y_{r-1}) \tag{570}$$
$$\leq (1 - \delta)\mathbb{P}(X_t \text{ and } Y_t \text{ never meet during } [r, kr) \,|\, X_{r-1} \neq Y_{r-1}). \tag{571}$$

By an induction on k, conclude that

$$\mathbb{P}(\tau \geq kr) \leq (1 - \delta)^k \to 0 \quad \text{as } k \to \infty. \tag{572}$$

The general convergence theorem is stated below.

Theorem 5.9.12 (Convergence Thm)**.** *Let $(X_t)_{t\geq 0}$ be an irreducible aperiodic Markov chain on a finite state space Ω. Let π denote the unique stationary distribution of X_t. Then for each $x, y \in V$,*

$$\lim_{t\to\infty} \mathbb{P}(X_t = y \,|\, X_0 = x) = \pi(y). \tag{573}$$

PROOF. As always, there is a linear algebra proof to this result (see, e.g., [LP17, Thm. 4.9]) Mimic the argument for Theorem 5.9.8 for a coupling based proof. □

10. Markov chain Monte Carlo

So far, we were given a Markov chain $(X_t)_{t\geq 0}$ on a finite state space Ω and studied existence and uniqueness of its stationary distribution and convergence to it. In this section, we will consider the reverse problem. Namely, given a distribution π on a sample space Ω, can we construct a Markov chain $(X_t)_{t\geq 0}$ such that π is a stationary distribution? If in addition the chain is irreducible an aperiodic, then by the convergence theorem (Theorem 5.9.12), we know that the distribution π_t of X_t converges to π. Hence if we run the chain for long enough, the state of the chain is asymptotically distributed as π. In other words, we can sample a random element of Ω according to the prescribed distribution π by emulating it through a suitable Markov chain. This method of sampling is called *Markov chain Monte Carlo* (MCMC).

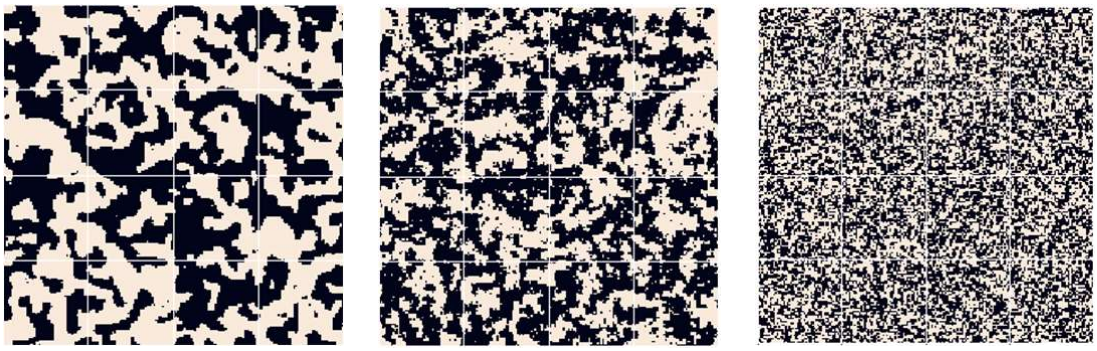


FIGURE 71. MCMC simulation of Ising model on 200 by 200 torus at temperature $T = 1$ (left), 2 (middle), and 5 (right).

Example 5.10.1 (Uniform distribution on regular graphs)**.** Let $G = (V, E)$ be a connected *regular* graph, meaning that all nodes have the same degree. Let μ be the uniform distribution on the node set V. How can we sample a random node according to μ? If we have a list of all nodes, then we can label them from 1 to $N = |V|$, choose a random number between 1 and N, and find corresponding node. But often times, we do not have the full list of nodes, especially when we want to sample a random node from a social network. Given only local information (set of neighbors for each given node), can we still sample a uniform random node from G?

One answer is given by random walk. Indeed, random walks on graphs are defined by only using local information of the underlying graph: Choose a random neighbor and move there. Moreover, since G is connected, there is a unique stationary distribution π for the walk, which is given by

$$\pi(x) = \frac{\deg_G(x)}{2|E|}. \tag{574}$$

Since G is regular, any two nodes have the same degree, so $\pi(x) = \pi(y)$ for all $x, y \in V$. This means π equals the uniform distribution μ on V. Hence the sampling algorithm we propose is as follows:

(∗) *Run a random walk on G for $t \gg 1$ steps, and take the random node that the walk sits on.*

However, there is a possible issue of convergence. Namely, if the graph G does not contain any odd cycle, then random walk on G is periodic (see Exercise 5.9.6), so we are not guaranteed to have convergence. We can overcome this by using a lazy random walk instead, which is introduced in Exercise 5.10.2. We know that the lazy RW on G is irreducible, aperiodic, and has the same set of stationary distribution as the ordinary RW on G. Hence we can apply the sampling algorithm (∗) above for lazy random walk on G to sample a uniform random node in G. ▲

Exercise 5.10.2 (Lazy RW on graphs)**.** Let $G = (V, E)$ be a graph. Let $(X_t)_{t\geq 0}$ be a Markov chain on the node set V with transition probabilities

$$\mathbb{P}(X_{t+1} = j \mid X_t = i) = \begin{cases} 1/2 & \text{if } j = i \\ 1/(2\deg_G(i)) & \text{if } j \text{ is adjacent to } i \\ 0 & \text{otherwise.} \end{cases} \tag{575}$$

This chain is called the *lazy random walk* on G. In words, the usual random walker on G now flips a fair coin to decide whether it stays at the same node or make a move to one of its neighbors.

(i) Show that for any connected graph G, the lazy random walk on G is irreducible and aperiodic.
(ii) Let P be the transition matrix for the usual random walk on G. Show that the following matrix

$$Q = \frac{1}{2}(P + I) \tag{576}$$

is the transition matrix for the lazy random walk on G.
(iii) For any distribution π on V, show that $\pi Q = \pi$ if and only if $\pi P = \pi$. Conclude that the usual and lazy random walks on G have the same set of stationary distribution.

Example 5.10.3 (Finding local minima)**.** Let $G = (V, E)$ be a connected graph and let $f : V \to [0, \infty)$ be a 'cost' function. The objective is to find a node $x^* \in V$ such that f takes global minimum at x^*. This problem has a lot of application in machine learning, for example. Note that if the domain V is very large, then an exhaustive search is too expensive to use.

Here is simple form of the popular algorithm of *stochastic gradient descent*, which lies at the heart of most of the important machine learning algorithms.

(i) Initialize the first guess $X_0 = x_0 \in V$.
(ii) Suppose $X_t = x \in V$ is chosen. Let

$$D_t = \{y \text{ a neighbor of } x \text{ or } x \text{ itself} \mid f(y) \leq f(x)\}. \tag{577}$$

Define X_{t+1} to be a uniform random node from D_t.
(iii) The algorithm terminates if it finds a local minima.

In words, at each step we move to a random neighbor which could possibly decrease the current value of f. It is easy to see that one always converges to a local minima, which may not be a global minimum. In a very complex machine learning task (e.g., training a deep neural network), this is often good enough. Is this a Markov chain? Irreducible? Aperiodic? Stationary distribution? ▲

There is a neat solution to finding global minimum. The idea is to allow that we go uphill with a small probability.

Example 5.10.4 (Finding global minimum)**.** Let $G = (V, E)$ be a connected regular graph and let $f : V \to [0, \infty)$ be a cost function. Let

$$V^* = \left\{ x \in V \mid f(x) = \min_{y \in V} f(y) \right\} \tag{578}$$

be the set of all nodes where f attains global minimum.

Fix a parameter $\lambda \in (0,1]$, and define a probability distribution π_λ on V by

$$\pi_\lambda(x) = \frac{\lambda^{f(x)}}{Z_\lambda}, \tag{579}$$

where $Z_\lambda = \sum_{x\in V} \lambda^{f(x)}$ is the normalizing constant. Since $\pi_\lambda(x)$ is decreasing in $f(x)$, it favors nodes x for which $f(x)$ is small.

Let $(X_t)_{t\geq 0}$ be Markov chain on V, whose transition is defined as follows. If $X_t = x$, then let y be a uniform random neighbor of x. If $f(y) \leq f(x)$, then move to y; If $f(y) > f(x)$, then move to y with probability $\lambda^{f(y)-f(x)}$ and stay at x with probability $1-\lambda^{f(y)-f(x)}$. We analyze this MC below:

(i) (Irreducibility) Since G is connected and we are allowing any move (either downhill or uphill) we can go from one node to any other in some number of steps. Hence the chain $(X_t)_{t\geq 0}$ is irreducible.

(ii) (Aperiodicity) By (i) and Remark 5.9.7, all nodes have the same period. Moreover, let $x \in V^*$ be an arbitrary node where f takes global minimum. Then all return times are possible, so x has period 1. Hence all nodes have period 1, so the chain is aperiodic.

(iii) (Stationarity) Here we show that π_λ is a stationary distribution of the chain. To do this, we first need to write down the transition matrix P. Namely, if we let $A_G(y,z)$ denote the indicator that y and z are adjacent, then

$$P(x,y) = \begin{cases} \frac{A_G(x,y)}{\deg_G(x)} \min(1, \lambda^{f(y)-f(x)}) & \text{if } x \neq y \\ 1 - \sum_{y\neq x} P(x,y) & \text{if } y = x. \end{cases} \tag{580}$$

To show $\pi_\lambda P = \pi_\lambda$, it suffices to show for any $y \in V$ that

$$\sum_{z\in V} \pi_\lambda(z) P(z,y) = \pi_\lambda(y). \tag{581}$$

Note that

$$\sum_{z\in V} \pi_\lambda(z)P(z,y) = \pi_\lambda(y)P(y,y) + \sum_{z\neq y} \pi_\lambda(z)P(z,y) \tag{582}$$

$$= \pi_\lambda(y) - \sum_{z\neq y} \pi_\lambda(y)P(y,z) + \sum_{z\neq y} \pi_\lambda(z)P(z,y). \tag{583}$$

Hence it suffices to show that

$$\pi_\lambda(y)P(y,z) = \pi_\lambda(z)P(z,y) \tag{584}$$

for any $z \neq y$. Indeed, considering the two cases $f(z) \leq f(y)$ and $f(z) > f(y)$, we have

$$\pi_\lambda(y)P(y,z) = \frac{\lambda^{f(y)}}{Z_\lambda} \frac{A_G(y,z)}{\deg_G(y)} \min(1, \lambda^{f(z)-f(y)}) = \frac{1}{Z_\lambda} \frac{A_G(z,y)}{\deg_G(z)} \lambda^{\max(f(y),f(z))}, \tag{585}$$

$$\pi_\lambda(z)P(z,y) = \frac{\lambda^{f(z)}}{Z_\lambda} \frac{A_G(z,y)}{\deg_G(z)} \min(1, \lambda^{f(y)-f(z)}) = \frac{1}{Z_\lambda} \frac{A_G(y,z)}{\deg_G(y)} \lambda^{\max(f(y),f(z))}. \tag{586}$$

Now since $A_G(z,y) = A_G(y,z)$ and we are assuming G is a regular graph, this yields (584), as desired. Hence π_λ is a stationary distribution for the chain $(X_t)_{t\geq 0}$.

(iv) (Convergence) By (i), (iii), Theorem 5.6.4, we see that π_λ is the unique stationary distribution for the chain X_t. Furthermore, by (i)-(iii) and Theorem 5.9.12, we conclude that the distribution of X_t converges to π_λ.

(v) (Global minimum) Let $f_* = \min_{x\in V} f(x)$ be the global minimum of f. Note that by definition of V^*, we have $f(x) = f_*$ for any $x \in V^*$. Then observe that

$$\lim_{\lambda\to 0} \pi_\lambda(x) = \lim_{\lambda\to 0} \frac{\lambda^{f(x)}}{\sum_{y\in V} \lambda^{f(y)}} = \lim_{\lambda\to 0} \frac{\lambda^{f(x)}/\lambda^{f_*}}{\sum_{y\in V} \lambda^{f(y)}/\lambda^{f_*}} \tag{587}$$

$$= \lim_{\lambda\to 0} \frac{\lambda^{f(x)-f_*}}{|V^*| + \sum_{y\in V\setminus V^*} \lambda^{f(y)-f_*}} = \frac{\mathbf{1}(x\in V^*)}{|V^*|}. \tag{588}$$

Thus for λ very small, π_λ is approximately the uniform distribution on the set of all nodes V^* where f attains global minimum. ▲

Exercise 5.10.5 (Detailed Balance equation)**.** Let P be a transition matrix of a Markov chain on state space Ω. Suppose π is a probability distribution on Ω that satisfies the following *detailed balance equation*

$$\pi(x)P(x,y) = \pi(y)P(y,x) \qquad \text{for all } x, y \in \Omega. \tag{589}$$

(i) Show that for all $x \in \Omega$,

$$\sum_{y\in\Omega} \pi(y)P(y,x) = \sum_{y\in\Omega} \pi(x)P(x,y) = \pi(x)\sum_{y\in\Omega} P(x,y) = \pi(x). \tag{590}$$

(ii) Conclude that π is a stationary distribution for P, that is, $\pi P = \pi$.

Exercise 5.10.6 (Metropolis-Hastings algorithm)**.** Let P be a transition matrix of a Markov chain on state space $\Omega = \{1, 2, \cdots, m\}$. Let π be a probability distribution on Ω, which is not necessarily a stationary distribution for P. Our goal is to design a Markov chain on Ω that has π as a stationary distribution. Below we will derive the famous *Metropolis-Hastings algorithm* for MCMC sampling.

Fix a $m \times m$ matrix A of entries from $[0, 1]$. Consider a Markov chain $(X_t)_{t\geq 0}$ on Ω that uses additional rejection step on top of the transition matrix P as follows:

(`Generation`) Suppose the current location $X_t = a$. Generate a candidate $b \in \Omega$ from the conditional distribution $P(a, \cdot)$.

(`Rejection`) Flip an independent coin with success probability $A(a, b)$. Upon success, accept the proposed move and set $X_{t+1} = b$; Otherwise, reject the move and set $X_{t+1} = a$.

Here, the (a, b) entry $A(a, b)$ is called the *acceptance probability* of the move $a \to b$.

(i) Let Q denote the transition matrix of the chain $(X_t)_{t\geq 0}$ defined above. Show that

$$Q(a,b) = \begin{cases} P(a,b)A(a,b) & \text{if } b \neq a \\ 1 - \sum_{c:c\neq a} P(a,c)A(a,c) & \text{if } b = a. \end{cases} \tag{591}$$

(ii) Show that $\pi(x)Q(x,y) = \pi(y)Q(y,x)$ $\forall x, y \in \Omega$, $x \neq y$ implies $\pi Q = \pi$. Deduce that if

$$\pi(x)P(x,y)A(x,y) = \pi(y)P(y,x)A(y,x) \qquad \forall x, y \in \Omega,\ x \neq y, \tag{592}$$

then π is a stationary distribution for $(X_t)_{t\geq 0}$.

(iii) Since we also want the Markov chain to converge fast, we want to choose the acceptance probability $A(a, b) \in [0, 1]$ as large as possible for each $a, b \in \Omega$. Show that the following choice (denoting $\alpha \wedge \beta := \min(\alpha, \beta)$)

$$A(x,y) = \frac{\pi(y)P(y,x)}{\pi(x)P(x,y)} \wedge 1 \qquad \forall x, y \in \Omega,\ x \neq y \tag{593}$$

satisfies the condition in (ii) and each $A(x, y)$ is maximized for all $x \neq y$.

(iv) Let $(Y_t)_{t\geq 0}$ be a random walk on the 5-wheel graph $G = (V, E)$ as shown in Figure 72. Show that $\pi =$

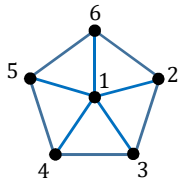


FIGURE 72. 5-wheel graph G

$\left[\frac{5}{20}, \frac{3}{20}, \frac{3}{20}, \frac{3}{20}, \frac{3}{20}, \frac{3}{20}\right]$ is the unique stationary distribution of Y_t. Apply the Metropolis-Hastings

algorithm derived in (i)-(iii) above to modify Y_t to obtain a new Markov chain X_t on V that converges to Uniform(V) in distribution.

Bibliography

[Bis06] Christopher M Bishop, *Pattern recognition and machine learning*, springer, 2006.

[HK70] Arthur E Hoerl and Robert W Kennard, *Ridge regression: Biased estimation for nonorthogonal problems*, Technometrics **12** (1970), no. 1, 55–67.

[Hoy04] Patrik O Hoyer, *Non-negative matrix factorization with sparseness constraints.*, Journal of machine learning research **5** (2004), no. 9.

[LKVP21] Hanbaek Lyu, Yacoub H Kureh, Joshua Vendrow, and Mason A Porter, *Learning low-rank latent mesoscale structures in networks*, arXiv preprint arXiv:2102.06984 (2021).

[LNB20] Hanbaek Lyu, Deanna Needell, and Laura Balzano, *Online matrix factorization for markovian data and applications to network dictionary learning*, Journal of Machine Learning Research **21** (2020), no. 251, 1–49.

[LP17] David A Levin and Yuval Peres, *Markov chains and mixing times*, vol. 107, American Mathematical Soc., 2017.

[LS99] Daniel D. Lee and H. Sebastian Seung, *Learning the parts of objects by non-negative matrix factorization*, Nature **401** (1999), no. 6755, 788.

[LXT+17] Hao Li, Zheng Xu, Gavin Taylor, Christoph Studer, and Tom Goldstein, *Visualizing the loss landscape of neural nets*, arXiv preprint arXiv:1712.09913 (2017).

[Lyu20] Hanbaek Lyu, *Convergence of block coordinate descent with diminishing radius for nonconvex optimization*, arXiv preprint arXiv:2012.03503 (2020).

[RHW86] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams, *Learning representations by back-propagating errors*, nature **323** (1986), no. 6088, 533–536.

[Row98] Sam Roweis, *Em algorithms for pca and spca*, Advances in neural information processing systems (1998), 626–632.

[SH94] Ferdinando S Samaria and Andy C Harter, *Parameterisation of a stochastic model for human face identification*, Proceedings of 1994 IEEE workshop on applications of computer vision, IEEE, 1994, pp. 138–142.

[SSY18] Tao Sun, Yuejiao Sun, and Wotao Yin, *On markov chain gradient descent*, arXiv preprint arXiv:1809.04216 (2018).

[TB99] Michael E Tipping and Christopher M Bishop, *Probabilistic principal component analysis*, Journal of the Royal Statistical Society: Series B (Statistical Methodology) **61** (1999), no. 3, 611–622.

[Tib96] Robert Tibshirani, *Regression shrinkage and selection via the lasso*, Journal of the Royal Statistical Society: Series B (Methodological) **58** (1996), no. 1, 267–288.

[ZL12] Songfeng Zheng and Weixiang Liu, *Functional gradient ascent for probit regression*, Pattern recognition **45** (2012), no. 12, 4428–4437.

APPENDIX A

Review of basic probability theory

1. Probability measure and probability space

Many things in life are uncertain. Can we 'measure' and compare such uncertainty so that it helps us to make a more informed decision? Probability theory provides a systematic way of doing so.

We begin with idealizing our situation. Let Ω be a finite set, called *sample space*. This is the collection of all possible outcomes that we can observe (think of six sides of a die). We are going to perform some experiment on Ω, and the outcome could be any subset E of Ω, which we call an *event*. Let us denote the collection of all events $E \subseteq \Omega$ by 2^Ω. A *probability measure* on Ω is a function $\mathbb{P} : 2^\Omega \to [0,1]$ such that for each event $E \subseteq \Omega$, it assigns a number $\mathbb{P}(E) \in [0,1]$ and satisfies the following properties:

(i) $\mathbb{P}(\emptyset) = 0$ and $\mathbb{P}(\Omega) = 1$.
(ii) If two events $E_1, E_2 \subseteq \Omega$ are disjoint, then $\mathbb{P}(E_1 \cup E_2) = \mathbb{P}(E_1) + \mathbb{P}(E_1)$.

In words, $\mathbb{P}(E)$ is our quantization of how likely it is that the event E occurs out of our experiment.

Exercise A.1.1. Let $\mathbb{P}$ be a probability measure on sample space Ω. Show the following.

(i) Let $E = \{x_1, x_2, \cdots x_k\} \subseteq \Omega$ be an event. Then $\mathbb{P}(E) = \sum_{i=1}^k \mathbb{P}(\{x_i\})$.
(ii) $\sum_{x\in\Omega} \mathbb{P}(\{x\}) = 1$.

If $\mathbb{P}$ is a probability measure on sample space Ω, we call the pair $(\Omega, \mathbb{P})$ a *probability space*. This is our idealized world where we can precisely measure the uncertainty of all possible events. Of course, there could be many (in fact, infinitely many) different probability measures on the same sample space.

Exercise A.1.2 (coin flip)**.** Let $\Omega = \{H, T\}$ be a sample space. Fix a parameter $p \in [0,1]$, and define a function $\mathbb{P}_p : 2^\Omega \to [0,1]$ by $\mathbb{P}_p(\emptyset) = 0$, $\mathbb{P}_p(\{H\}) = p$, $\mathbb{P}_p(\{T\}) = 1-p$, $\mathbb{P}_p(\{H,T\}) = 1$. Verify that $\mathbb{P}_p$ is a probability measure on Ω for each value of p.

A typical way of constructing a probability measure is to specify how likely it is to see each individual element in Ω. Namely, let $f : \Omega \to [0,1]$ be a function that sums up to 1, i.e., $\sum_{x\in\Omega} f(x) = 1$. Define a function $\mathbb{P} : 2^\Omega \to [0,1]$ by

$$\mathbb{P}(E) = \sum_{\omega\in E} f(x). \tag{594}$$

Then this is a probability measure on Ω, and f is called a *probability distribution* on Ω. For instance, the probability distribution on $\{H,T\}$ we used to define $\mathbb{P}_p$ in Exercise A.1.2 is $f(H) = p$ and $f(T) = 1-p$.

Example A.1.3 (Uniform probability measure)**.** Let $\Omega = \{1, 2, \cdots, m\}$ be a sample space and let $\mathbb{P}$ be the *uniform probability measure* on Ω, that is,

$$\mathbb{P}(\{x\}) = 1/m \qquad \forall x \in \Omega. \tag{595}$$

Then for the event $A = \{1,2,3\}$, we have

$$\mathbb{P}(A) = \mathbb{P}(\{1\} \cup \{2\} \cup \{3\}) \tag{596}$$
$$= \mathbb{P}(\{1\}) + \mathbb{P}(\{2\}) + \mathbb{P}(\{3\}) \tag{597}$$
$$= \frac{1}{m} + \frac{1}{m} + \frac{1}{m} = \frac{3}{m} \tag{598}$$

Likewise, if $A \subseteq \Omega$ is any event and if we let $|A|$ denote the size (number of elements) of A, then

$$\mathbb{P}(A) = \frac{|A|}{m}. \tag{599}$$

For example, let $\Omega = \{1,2,3,4,5,6\}^2$ be the sample space of a roll of two fair dice. Let A be the event that the sum of two dice is 5. Then

$$A = \{(1,4),(2,3),(3,2),(4,1)\}, \tag{600}$$

so $|A| = 4$. Hence $\mathbb{P}(A) = 4/36 = 1/9$. ▲

2. Random variables

Given a finite probability space $(\Omega, \mathbb{P})$, a (discrete) *random variable* (RV) is any real-valued function $X : \Omega \to \mathbb{R}$. We can think of it as the outcome of some experiment on Ω (e.g., height of a randomly selected friend). We often forget the original probability space and specify a RV by is *probability mass function* (PMF) $f_X : \mathbb{R} \to [0,1]$,

$$f_X(x) = \mathbb{P}(X = x) = \mathbb{P}(\{\omega \in \Omega \mid X(\omega) = x\}). \tag{601}$$

Namely, $\mathbb{P}(X = x)$ is the likelihood that the RV X takes value x.

Example A.2.1. Say you win \$1 if a fair coin lands heads and lose \$1 if lands tails. We can set up our probability space $(\Omega, \mathbb{P})$ by $\Omega = \{H, T\}$ and $\mathbb{P}$ = uniform probability measure on Ω. The RV $X : \Omega \to \mathbb{R}$ for this game is $X(H) = 1$ and $X(T) = -1$. The PMF of X is given by $f_X(1) = \mathbb{P}(X = 1) = \mathbb{P}(\{H\}) = 1/2$ and likewise $f_X(-1) = 1/2$.

Exercise A.2.2. Let $(\Omega, \mathbb{P})$ be a probability space and $X : \Omega \to \mathbb{R}$ be a RV. Let f_X be the PMF of X, that is, $f_X(x) = \mathbb{P}(X = x)$ for all x. Show that f_X adds up to 1, that is,

$$\sum_x f_X(x) = 1, \tag{602}$$

where the summation runs over all numerical values x that X can take.

There are two useful statistics of a RV to summarize its two most important properties: It's average and uncertainty. First, if one has to guess the value of a RV X, what would be the best choice? It is the *expectation* (or mean) of X, defined as below:

$$\mathbb{E}[X] = \sum_x x\,\mathbb{P}(X = x). \tag{603}$$

Exercise A.2.3. For any RV X and real numbers $a, b \in \mathbb{R}$, show that

$$\mathbb{E}[aX + b] = a\mathbb{E}[X] + b. \tag{604}$$

On the other hand, say you play two different games where in the first game, you win or lose \$1 depending on a fair coin flip, and in the second game, you win or lose \$10. In both games, your expected winning is 0. But the two games are different in how much the outcome fluctuates around the mean. This notion if the fluctuation is captured by the following quantity called *variance*:

$$\mathrm{Var}(X) = \mathbb{E}[(X - \mathbb{E}(X))^2]. \tag{605}$$

Namely, it is the expected squared difference between X and its expectation $\mathbb{E}(X)$.

Exercise A.2.4. Let X be a RV. Show that $\mathrm{Var}(X) = \mathbb{E}[X^2] - \mathbb{E}[X]^2$.

Here are some of the simplest and yet most important RVs.

Exercise A.2.5 (Bernoulli RV)**.** A RV X is a *Bernoulli* variable with (success) probability $p \in [0,1]$ if it takes value 1 with probability p and 0 with probability $1 - p$. In this case we write $X \sim \mathrm{Bernoulli}(p)$. Show that $\mathbb{E}(X) = p$ and $\mathrm{Var}(X) = p(1-p)$.

Exercise A.2.6 (Indicator variables)**.** Let $(\Omega, \mathbb{P})$ be a probability space and let $E \subseteq \Omega$ be an event. The *indicator variable* of the event E, which is denoted by $\mathbf{1}_E$, is the RV such that $\mathbf{1}_E(\omega) = 1$ if $\omega \in E$ and $\mathbf{1}_E(\omega) = 0$ if $\omega \in E^c$. Show that $\mathbf{1}_E$ is a Bernoulli variable with success probability $p = \mathbb{P}(E)$.

Example A.2.7 (Uniform RV)**.** Let $\Omega = \{x_1, x_2, \cdots x_N\}$ be a finite sample space of real numbers. A RV X is a *uniform* variable on Ω, denoted as $X \sim \text{Uniform}(\Omega)$, if it takes each of the element in Ω with equal probability. That is,

$$\mathbb{P}(X = x_i) = \frac{1}{n} = \frac{1}{|\Omega|} \qquad \text{for all } 1 \le i \le n. \tag{606}$$

Let $X \sim \text{Uniform}(\Omega)$ as above. Then

$$\mathbb{E}[X] = \sum_{i=1}^{n} x_i \cdot \frac{1}{n} = \frac{1}{n} \sum_{i=1}^{n} x_i. \tag{607}$$

Also,

$$\text{Var}(X) = \sum_{i=1}^{n} (x_i - \mathbb{E}[X])^2 \cdot \frac{1}{n} = \frac{1}{n} \sum_{i=1}^{n} (x_i - \mathbb{E}[X])^2. \tag{608}$$

The following exercise ties the expectation and the variance of a RV into a problem of finding a point estimator that minimizes the mean squared error.

Exercise A.2.8 (Variance as minimum MSE)**.** Let X be a RV. Let $\hat{x} \in \mathbb{R}$ be a number, which we consider as a 'guess' (or 'estimator' in Statistics) of X. Let $\mathbb{E}[(X - \hat{x})^2]$ be the *mean squared error* (MSE) of this estimation.

(i) Show that

$$\mathbb{E}[(X - \hat{x})^2] = \mathbb{E}[X^2] - 2\hat{x}\mathbb{E}[X] + \hat{x}^2 \tag{609}$$

$$= (\hat{x} - \mathbb{E}[X])^2 + \mathbb{E}[X^2] - \mathbb{E}[X]^2 \tag{610}$$

$$= (\hat{x} - \mathbb{E}[X])^2 + \text{Var}(X). \tag{611}$$

(ii) Conclude that the MSE is minimized when $\hat{x} = \mathbb{E}[X]$ and the global minimum is $\text{Var}(X)$. In this sense, $\mathbb{E}[X]$ is the 'best guess' for X and $\text{Var}(X)$ is the corresponding MSE.

To define a discrete RV, it was enough to specify its PMF. For a continuous RV, *probability distribution function* (PDF) plays an analogous role of PMF. We also need to replace summation $\sum$ with an integral $\int dx$. Namely, X is a *continuous RV* if there is a function $f_X : \mathbb{R} \to [0, \infty)$ such that for any interval $[a, b]$, the probability that X takes a value from an interval $(a, b]$ is given by integrating f_X over the interval $(a, b]$:

$$\mathbb{P}(X \in (a, b]) = \int_a^b f_X(x)\, dx. \tag{612}$$

The *cumulative distribution function* (CDF) of a RV X (either discrete or continuous), denoted by F_X, is defined by

$$F_X(x) = \mathbb{P}(X \le x). \tag{613}$$

By definition of PDF, we get

$$F_X(x) = \int_{-\infty}^{x} f_X(t)\, dt. \tag{614}$$

Conversely, PDFs can be obtained by differentiating corresponding CDFs.

Exercise A.2.9. Let X be a continuous RV with PDF f_X. Let a be a continuity point of f_X, that is, f_X is continuous at a. Show that $F_X(x)$ is differentiable at $x = a$ and

$$\frac{dF_X}{dx}\Big|_{x=a} = f_X(a). \tag{615}$$

The expectation of a continuous RV X with pdf f_X is defined by

$$\mathbb{E}(X) = \int_{-\infty}^{\infty} x f_X(x)\, dx, \tag{616}$$

and its variance $\mathrm{Var}(X)$ is defined by the same formula (605).

3. Conditional and iterated expectations

Let X, Y be discrete RVs. Recall that the expectation $\mathbb{E}(X)$ is the 'best guess' on the value of X when we do not have any prior knowledge on X. But suppose we have observed that some possibly related RV Y takes value y. What should be our best guess on X, leveraging this added information? This is called the *conditional expectation of X given $Y = y$*, which is defined by

$$\mathbb{E}[X|Y = y] = \sum_x x\, \mathbb{P}(X = x|Y = y). \tag{617}$$

This best guess on X given $Y = y$, of course, depends on y. So it is a function in y. Now if we do not know what value Y might take, then we omit y and $\mathbb{E}[X|Y]$ becomes a RV, which is called the *conditional expectation of X given Y*.

Proposition A.3.1 (Iterated expectation)**.** *Let X, Y be RVs. Then $\mathbb{E}[X] = \mathbb{E}[\mathbb{E}[X|Y]]$.*

PROOF. We only show the assertion for the case when both X and Y are discrete, and the general case can be argued similarly.

We are going to write the iterated expectation $\mathbb{E}[\mathbb{E}[X|Y]]$ as a double sum and swap the order of summation (Fubini's theorem, as always).

$$\begin{aligned}
\mathbb{E}[\mathbb{E}[X|Y]] &= \sum_y \mathbb{E}[X|Y = y]\mathbb{P}(Y = y) && (618)\\
&= \sum_y \left(\sum_x x\, \mathbb{P}(X = x|Y = y)\right)\mathbb{P}(Y = y) && (619)\\
&= \sum_y \sum_x x\, \mathbb{P}(X = x|Y = y)\mathbb{P}(Y = y) && (620)\\
&= \sum_y \sum_x x\, \mathbb{P}(X = x,\, Y = y) && (621)\\
&= \sum_x \sum_y x\, \mathbb{P}(Y = y|X = x)\mathbb{P}(X = x) && (622)\\
&= \sum_x x \left(\sum_y \mathbb{P}(Y = y|X = x)\right)\mathbb{P}(X = x) && (623)\\
&= \sum_x x\, \mathbb{P}(X = x) = \mathbb{E}(X). && (624)
\end{aligned}$$

□

Remark A.3.2. Here is an intuitive reason why the iterated expectation works. Suppose you want to make the best guess $\mathbb{E}(X)$. Pretending you know Y, you can improve your guess to be $E(X\,|\,Y)$. Then you admit that you didn't know anything about Y and averaged over all values of Y. The result is $\mathbb{E}[\mathbb{E}[X\,|\,Y]]$, and this should be the same best guess on X when we don't know anything about Y.

All our discussions above hold for continuous RVs as well: We simply replace the sum by integral and PMF by PDF. To summarize how we compute the iterated expectations when we condition on discrete and continuous RV:

$$\mathbb{E}[\mathbb{E}[X\,|\,Y]] = \begin{cases} \sum_y \mathbb{E}[X\,|\,Y = y]\mathbb{P}(Y = y) & \text{if } Y \text{ is discrete} \\ \int_{-\infty}^{\infty} \mathbb{E}[X\,|\,Y = y] f_Y(y)\, dy & \text{if } Y \text{ is continuous.} \end{cases} \tag{625}$$

Exercise A.3.3 (Iterated expectation for probability)**.** Let X, Y be RVs.

(i) For any $x \in \mathbb{R}$, show that $\mathbb{P}(X \le x) = \mathbb{E}[\mathbf{1}(X \le x)]$.

(ii) By using iterated expectation, show that

$$\mathbb{P}(X \le x) = \mathbb{E}[\mathbb{P}(X \le x \mid Y)], \tag{626}$$

where the expectation is taken over for all possible values of Y.

Example A.3.4. Let $Y \sim \text{Uniform}([0,1])$ and $X \sim \text{Binomial}(n, Y)$. Then $X|Y = y \sim \text{Binomial}(n, y)$ so $\mathbb{E}[X|Y = y] = ny$. Hence

$$\mathbb{E}[X] = \int_0^1 \mathbb{E}[X \mid Y = y] f_Y(y)\, dy = \int_0^1 ny\, dy = n/2. \tag{627}$$

▲

Example A.3.5. Let $X_1 \sim \text{Exp}(\lambda_1)$ and $X_2 \sim \text{Exp}(\lambda_2)$ be independent exponential RVs. We will show that

$$\mathbb{P}(X_1 < X_2) = \frac{\lambda_1}{\lambda_1 + \lambda_2} \tag{628}$$

using the iterated expectation. Using iterated expectation for probability,

$$\mathbb{P}(X_1 < X_2) = \int_0^\infty \mathbb{P}(X_1 < X_2 \mid X_1 = x_1) \lambda_1 e^{-\lambda_1 x_1}\, dx_1 \tag{629}$$

$$= \int_0^\infty \mathbb{P}(X_2 > x_1) \lambda_1 e^{-\lambda_1 x_1}\, dx_1 \tag{630}$$

$$= \lambda_1 \int_0^\infty e^{-\lambda_2 x_1} e^{-\lambda_1 x_1}\, dx_1 \tag{631}$$

$$= \lambda_1 \int_0^\infty e^{-(\lambda_1+\lambda_2) x_1}\, dx_1 = \frac{\lambda_1}{\lambda_1 + \lambda_2}. \tag{632}$$

▲

4. Conditional variance

As we have defined conditional expectation, we may define the variance of a RV Y given that another RV X takes a particular value. Recall that the (unconditioned) variance of Y is defined by

$$\text{Var}(Y) = \mathbb{E}[(Y - \mathbb{E}(Y))^2]. \tag{633}$$

Note that there are two places where we take expectation. Given X, we should improve both expectations so the *conditional variance of Y given X is defined by*

$$\text{Var}(Y \mid X) = \mathbb{E}[(Y - \mathbb{E}[Y \mid X])^2 \mid X]. \tag{634}$$

Proposition A.4.1. *Let X and Y be RVs. Then*

$$\text{Var}(Y \mid X) = \mathbb{E}[Y^2 \mid X] - \mathbb{E}[Y \mid X]^2. \tag{635}$$

PROOF. Using linearity of conditional expectation and the fact that $\mathbb{E}[Y \mid X]$ is not random given X,

$$\text{Var}(Y \mid X) = \mathbb{E}[Y^2 - 2Y\mathbb{E}[Y \mid X] + \mathbb{E}[Y \mid X]^2 \mid Y] \tag{636}$$

$$= \mathbb{E}[Y^2 \mid X] - \mathbb{E}[2Y\mathbb{E}[Y \mid X] \mid X] + \mathbb{E}[\mathbb{E}[Y \mid X]^2 \mid X] \tag{637}$$

$$= \mathbb{E}[Y^2 \mid X] - \mathbb{E}[Y \mid X]\mathbb{E}[2Y \mid X] + \mathbb{E}[Y \mid X]^2 \mathbb{E}[1 \mid X] \tag{638}$$

$$= \mathbb{E}[Y^2 \mid X] - 2\mathbb{E}[Y \mid X]^2 + \mathbb{E}[Y \mid X]^2 \tag{639}$$

$$= \mathbb{E}[Y^2 \mid X] - \mathbb{E}[Y \mid X]^2. \tag{640}$$

□

The following exercise explains in what sense the conditional expectation $\mathbb{E}[Y \mid X]$ is the best guess on Y given X, and that the minimum possible mean squared error is exactly the conditional variance $\mathrm{Var}(Y \mid X)$.

Exercise A.4.2. Let X, Y be RVs. For any function $g : \mathbb{R} \to \mathbb{R}$, consider $g(X)$ as an estimator of Y. Let $\mathbb{E}_Y[(Y - g(X))^2 \mid X]$ be the *mean squared error*.

(i) Show that

$$\mathbb{E}_Y[(Y - g(X))^2 \mid X] = \mathbb{E}_Y[Y^2 \mid X] - 2g(X)\mathbb{E}_Y[Y \mid X] + g(X)^2 \tag{641}$$

$$= (g(X) - \mathbb{E}_Y(Y \mid X))^2 + \mathbb{E}_Y[Y^2 \mid X] - \mathbb{E}_Y[Y \mid X]^2 \tag{642}$$

$$= (g(X) - \mathbb{E}_Y(Y \mid X))^2 + \mathrm{Var}(Y \mid X). \tag{643}$$

(ii) Conclude that the mean squared error is minimized when $g(X) = \mathbb{E}_Y[Y \mid X]$ and the global minimum is $\mathrm{Var}(Y \mid X)$.

Next, we study how we can decompose the variance of Y by conditioning on X.

Proposition A.4.3 (Law of total variance)**.** *Let X and Y be RVs. Then*

$$\mathrm{Var}(Y) = \mathbb{E}_X[\mathrm{Var}(Y \mid X)] + \mathrm{Var}(\mathbb{E}[Y \mid X]). \tag{644}$$

PROOF. Using previous result, iterated expectation, and linearity of expectation, we have

$$\begin{aligned}
\mathrm{Var}(Y) &= \mathbb{E}(Y^2) - (\mathbb{E}(Y))^2 && (645)\\
&= \mathbb{E}_X(\mathbb{E}(Y^2|X)) - (\mathbb{E}_X(\mathbb{E}(Y|X)))^2 && (646)\\
&= \mathbb{E}_X(\mathrm{Var}(Y|X) + (\mathbb{E}(Y|X))^2) - (\mathbb{E}_X(\mathbb{E}(Y|X)))^2 && (647)\\
&= \mathbb{E}_X(\mathrm{Var}(Y|X)) + \left[\mathbb{E}_X(\mathbb{E}(Y|X))^2) - (\mathbb{E}_X(\mathbb{E}(Y|X)))^2\right] && (648)\\
&= \mathbb{E}_X(\mathrm{Var}(Y|X)) + \mathrm{Var}(\mathbb{E}(Y|X)). && (649)
\end{aligned}$$

□

Here is a handwavy explanation of why the above is true. Given Y, we should measure the fluctuation of $Y \mid X$ from the conditional expectation $\mathbb{E}[Y \mid X]$, and this is measured as $\mathrm{Var}(Y \mid X)$. Since we don't know Y, we average over all Y, giving $\mathbb{E}[\mathrm{Var}(Y \mid X)]$. But the reference point $\mathbb{E}[Y \mid X]$ itself varies with Y, so we should also measure its own fluctuation by $\mathrm{Var}(\mathbb{E}[Y \mid X])$. These fluctuations add up nicely like Pythagorian theorem because $\mathbb{E}[Y \mid X]$ is an optimal estimator so that these two fluctuations are 'orthogonal'.

Example A.4.4. Let $X \sim \mathrm{Uniform}([0,1])$ and $Y \sim \mathrm{Binomial}(n, X)$. Since $Y \mid X = x \sim \mathrm{Binomial}(n, x)$, we have $\mathbb{E}[Y \mid X = x] = nx$ and $\mathrm{Var}(Y \mid X = x) = nx(1-x)$. Also, since $X \sim \mathrm{Uniform}([0,1])$, we have

$$\mathrm{Var}(\mathbb{E}[Y \mid X]) = \mathrm{Var}(nX) = \frac{n^2}{12}. \tag{650}$$

So by iterated expectation, we get

$$\mathbb{E}[Y] = \mathbb{E}_X[\mathbb{E}[Y \mid X]] = \int_0^1 nx\,dx = \frac{n}{2}. \tag{651}$$

On the other hand, by law of total variance,

$$\mathrm{Var}(Y) = \mathbb{E}[\mathrm{Var}(Y \mid X)] + \mathrm{Var}(\mathbb{E}[Y \mid X]) \tag{652}$$

$$= \int_0^1 nx(1-x)\,dx + \mathrm{Var}(nX) \tag{653}$$

$$= n\left[\frac{x^2}{2} - \frac{x^3}{3}\right]_0^1 + \frac{n^2}{12} = \frac{n^2}{12} + \frac{n}{6}. \tag{654}$$

▲

5. Joint probability distributions

Let X and Y be discrete RVs defined on sample spaces Ω_1 and Ω_2, which we think of the outcome of two random experiments. We can think of the pair (X, Y) of the outcomes as a single observable that describes the two experiments jointly. This object is called a *random vector*, being a vector of random variables. Namely, (X, Y) is a vector-valued function $\Omega_1 \times \Omega_2 \to \mathbb{R}^2$ defined on the joint sample space $\Omega_1 \times \Omega_2$. As for the PDF of a random variable, which is the probability that a RV takes a particular value, we can also ask the probability that the random vector (X, Y) takes a particular vector (x, y). This is called the *joint PMF* of X and Y:

$$f_{X,Y}(x, y) = \mathbb{P}((X, Y) = (x, y)) = \mathbb{P}(X = x,\ Y = y). \tag{655}$$

Given a joint PMF of X and Y, we can determine the PMFs of X and Y simply by 'integrating out' the other variable. Namely, note that for any $x \in \mathbb{R}$,

$$\mathbb{P}(X = x) = \sum_{y \in Y[\Omega_2]} \mathbb{P}(X = x,\ Y = y) = \sum_{y \in Y[\Omega_2]} f_{X,Y}(x, y), \tag{656}$$

where $Y[\Omega_2]$ denotes the set of all possible values of Y. Hence, we sum $f_{X,Y}(x, y)$ over all y to get $f_X(x)$; likewise, we sum $f_{X,Y}(x, y)$ over all x to get $f_Y(y)$. We also call the PMFs of each X and Y the *marginal PMFs* of $f_{X,Y}$.

Proposition A.5.1. *Let X, Y be discrete RVs. Then they are independent if and only if there are functions f and g such that*

$$\mathbb{P}(X = x,\ Y = y) = f(x)g(y) \tag{657}$$

for all possible values (x, y) for (X, Y).

PROOF. If X and Y are independent, then

$$\mathbb{P}(X = x,\ Y = y) = \mathbb{P}(X = x)\mathbb{P}(Y = y) = f_X(x) f_Y(y). \tag{658}$$

Hence the conclusion holds. Conversely, suppose (657) holds for some functions f and g. Then

$$\mathbb{P}(X = x) = \sum_y \mathbb{P}(X = x,\ Y = y) = \sum_y f(x)g(y) = f(x) \sum_y g(y). \tag{659}$$

Since sum of the probabilities $\mathbb{P}(X = x)$ is 1, we have

$$\left(\sum_x f(x)\right)\left(\sum_y g(y)\right) = 1. \tag{660}$$

Hence if we define $\bar{f}(x) = f(x)/\sum_x f(x)$ and $\bar{g}(y) = \sum_y g(y)$, then

$$\mathbb{P}(X = x) = f(x) \sum_y g(y) = \frac{f(x) \sum_y g(y)}{\left(\sum_x f(x)\right)\left(\sum_y g(y)\right)} = \bar{f}(x) \sum_y \bar{g}(y) = \bar{f}(x). \tag{661}$$

Similarly, we have $\mathbb{P}(Y = y) = \bar{g}(y)$. This yields

$$\mathbb{P}(X = x,\ Y = y) = f(x)g(y) = \frac{f(x)g(y)}{\left(\sum_x f(x)\right)\left(\sum_y g(y)\right)} = \bar{f}(x)\bar{g}(y) = \mathbb{P}(X = x)\mathbb{P}(Y = y). \tag{662}$$

Since (x, y) was arbitrary, this shows X and Y are independent. □

Let X and Y be continuous RVs defined on sample spaces Ω_1 and Ω_2. We say X and Y are *jointly continuous* if the random vector (X, Y) has a function $f_{X,Y} : \mathbb{R}^2 \to [0, \infty)$ such that for each subset $\mathscr{A} \subseteq \mathbb{R}^2$,

$$\mathbb{P}((X, Y) \in \mathscr{A}) = \int \int_{(x,y) \in \mathscr{A}} f_{X,Y}(x, y)\, dx\, dy. \tag{663}$$

For instance, if $\mathscr{A}$ is a rectangle of the form $\mathscr{A} = [a, b] \times [c, d]$, then by Fubini's theorem, (663) reads

$$\mathbb{P}(a \le X \le b,\ c \le Y \le d) = \int_a^b \int_c^d f_{X,Y}(x, y)\, dx\, dy. \tag{664}$$

Such a function $f_{X,Y}$ is called the *joint PDF* of X and Y.

As for the joint PMFs, we can determine the PDF of X and Y simply by 'integrating out' the other variable. We also call the PMFs of each X and Y the *marginal PMFs* of $f_{X,Y}$.

Proposition A.5.2. *Let X and Y be continuous RVs with joint PDF $f_{X,Y}$. Then for any $x, y \in \mathbb{R}$, we have*

$$f_X(x) = \int_{-\infty}^{\infty} f_{X,Y}(x,y)\,dy, \qquad f_Y(y) = \int_{-\infty}^{\infty} f_{X,Y}(x,y)\,dx. \tag{665}$$

PROOF. Note that for any interval $[a,b] \subseteq \mathbb{R}$, by definition of joint PDF and Fubini's theorem,

$$\mathbb{P}(a \le X \le b) = \int_a^b \int_{-\infty}^{\infty} f_{X,Y}(x,y)\,dx\,dy \tag{666}$$

$$= \int_a^b \left(\int_{-\infty}^{\infty} f_{X,Y}(x,y)\,dy \right) dx. \tag{667}$$

Then by the definition of PDF of a continuous RV, it follows that the PDF f_X of X is given the the first equation in (665). A Similar argument applies to f_Y as well. □

Example A.5.3. Let X and Y be continuous RVs with joint PDF given by

$$f_{X,Y}(x,y) = \lambda^2 e^{-\lambda(x+y)} \mathbf{1}(x,y \ge 0). \tag{668}$$

Then for each $x \in \mathbb{R}$,

$$f_X(x) = \int_{-\infty}^{\infty} \lambda^2 e^{-\lambda(x+y)} \mathbf{1}(x,y \ge 0)\,dy \tag{669}$$

$$= \lambda e^{-\lambda x} \mathbf{1}(x \ge 0) \int_{-\infty}^{\infty} \lambda e^{-\lambda y} \mathbf{1}(y \ge 0)\,dy \tag{670}$$

$$= \lambda e^{-x} \mathbf{1}(x \ge 0). \tag{671}$$

Note that for the last equality, we have recognized the function $y \mapsto \lambda e^{-\lambda y}\mathbf{1}(y \ge 0)$ as the PDF of $\mathrm{Exp}(\lambda)$ RV. This shows

$$f_X(x) = \lambda e^{-\lambda x} \mathbf{1}(x \ge 0). \tag{672}$$

Hence $X \sim \mathrm{Exp}(\lambda)$. Similarly, we obtain $Y \sim \mathrm{Exp}(\lambda)$. ▲

6. Definition and Examples of MLE

Maximum Likelihood Estimation (MLE) is a classical notion in statistics, which has a prominent use in the context of modern machine learning. A core problem in statistics is to infer an unknown RV X from its sample values $x_1, \cdots, x_n$. In many cases, we can narrow down our inference problem by imposing a probabilistic (statistical) model for X. Namely, say we know X is a Bernoulli RV with unknown success probability p. We then only need to estimate the parameter p using our sample, not the entire distribution of X. MLE gives a systematic approach to this parameter estimation problem. Below we give the pipeline of MLE.

Pipeline of MLE.
Input: An unknown RV X with distribution $f_{X;\theta}$, parameterized by θ in a parameter space Ω. Also have sample values $x_1, x_2, \cdots, x_n$.
Objective: Obtain an estimation $\hat{\theta}$ of θ using the sample.
Method: Choose $\hat{\theta} \in \Omega$ so that it maximizes the following *likelihood function*

$$L(x_1, \cdots, x_n; \theta) = \prod_{i=1}^{n} f_{X,\theta}(x_i). \tag{673}$$

Here the RV $\hat{\theta}(X_1,\cdots,X_n)$ is called the *maximum likelihood estimator* of θ, and its observed value $\hat{\theta}(x_1,\cdots,x_n)$ is called the *maximum likelihood estimate* of θ.

What is the reasoning behind maximizing the likelihood function as above? The underlying assumption is that, *you are seeing the sample values $x_1,\cdots,x_n$ because it was the most likely to see them!* Namely, suppose we have designed the sampling procedure well-enough so that we get i.i.d. samples $X_1,\cdots,X_n$ each with distribution $f_{X;\theta}$. Then we have

$$L(x_1,\cdots,x_n;\theta)=\mathbb{P}(X_1=x_1,\cdots,X_n=x_n;\theta). \tag{674}$$

Namely, the likelihood function on the LHS is the probability of observing a sequence of specific sample values $x_1,\cdots,x_n$ under the i.i.d. assumption and assuming the parameter value θ. Hence, MLE chooses $\hat{\theta}$ to be the value of the parameter in Ω under which the probability of obtaining the current sample is maximized.

Example A.6.1 (MLE for Bernoulli RVs)**.** Suppose $X\sim$ Bernoulli(p) for some unknown $p\in[0,1]$. Suppose we have sample values $x_1,\cdots,x_n$ obtained from an i.i.d. sampling for X. Denote $\bar{x}=n^{-1}\sum_{i=1}^n x_i$. We will show that the MLE for p is $\bar{X}$, that is,

$$\hat{p}=\bar{X}. \tag{675}$$

Let $X_1,\cdots,X_n$ be i.i.d. with distribution Bernoulli(p). Note that

$$\mathbb{P}(X_i=x_i;p)=\begin{cases}p & \text{if } x_i=1\\ 1-p & \text{if } x_i=0\end{cases} \tag{676}$$

$$=p^{x_i}(1-p)^{1-x_i}. \tag{677}$$

Hence the likelihood function is given by

$$L(x_1,\cdots,x_n;p)=\prod_{i=1}^n\mathbb{P}(X_i=x_i;p) \tag{678}$$

$$=\prod_{i=1}^n p^{x_i}(1-p)^{1-x_i} \tag{679}$$

$$=p^{\sum_{i=1}^n x_i}(1-p)^{n-\sum_{i=1}^n x_i}. \tag{680}$$

In order to maximize the likelihood function, it suffices to maximize its logarithm [1]. The *log likelihood function* is given by

$$l(x_1,\cdots,x_n;p):=\log L(x_1,\cdots,x_n;p)=n\bar{x}\log p+(n-n\bar{x})\log(1-p). \tag{681}$$

To maximize the log likelihood function, we take partial derivative in p:

$$\frac{\partial l(x_1,\cdots,x_n;p)}{\partial p}=\frac{n\bar{x}}{p}-\frac{n-n\bar{x}}{1-p}. \tag{682}$$

Setting this equal to zero, we obtain

$$\frac{\bar{x}}{p}=\frac{1-\bar{x}}{1-p}. \tag{683}$$

Rearranging, we obtain $p=\bar{x}$. Hence we have (675) as desired. ▲

Example A.6.2 (MLE for Exponential RVs)**.** Suppose $X\sim\text{Exp}(\lambda)$ for some unknown $\lambda>0$. We have sample values $x_1,\cdots,x_n$ obtained from an i.i.d. sampling for X. Denote $\bar{x}=n^{-1}\sum_{i=1}^n x_i$. We will show that the MLE for λ is $1/\bar{X}$, that is,

$$1/\hat{\lambda}=\bar{X}. \tag{684}$$

[1] since log is a strictly increasing function.

Let $X_1, \cdots, X_n$ be i.i.d. with distribution $\text{Exp}(\lambda)$, which have the following PDF

$$f_X(x;\lambda) = \lambda e^{-\lambda x}\mathbf{1}(x \geq 0). \tag{685}$$

Noting that $x_1, \cdots, x_n \geq 0$, it follows that the likelihood function is given by

$$L(x_1, \cdots, x_n; \lambda) = \prod_{i=1}^{n} f_X(x_i;\lambda) \tag{686}$$

$$= \lambda^n \prod_{i=1}^{n} e^{-\lambda x_i} = \lambda^n e^{-\lambda \sum_{i=1}^n \bar{x}} = \lambda^n e^{-\lambda n \bar{x}}. \tag{687}$$

So the log likelihood function is given by

$$l(x_1, \cdots, x_n; \lambda) = n \log \lambda - \lambda n \bar{x}. \tag{688}$$

To maximize the log likelihood function, we take partial derivative in λ:

$$\frac{\partial l(x_1, \cdots, x_n; \lambda)}{\partial \lambda} = \frac{n}{\lambda} - n\bar{x}. \tag{689}$$

Setting this equal to zero, we obtain $1/\lambda = \bar{x}$. Hence $1/\hat{\lambda} = \bar{X}$, as desired. ▲

Example A.6.3 (MLE for Normal RVs)**.** Suppose $X \sim N(\mu, \sigma^2)$ for some unknown $\mu \in \mathbb{R}$ and $\sigma \geq 0$. Let $X_1, \cdots, X_n$ be i.i.d. with distribution $N(\mu, \sigma^2)$. Denote the sample mean and variance of the empirical distribution by

$$\bar{X} = n^{-1} \sum_{i=1}^{n} X_i, \qquad V = n^{-1} \sum_{i=1}^{n} (X_i - \bar{X})^2. \tag{690}$$

We will show that the MLE for μ and σ^2 are given by sample mean and variance of the empirical distribution:

$$\hat{\mu} = \bar{X}, \qquad \hat{(\sigma^2)} = V. \tag{691}$$

Recall that $N(\mu, \sigma^2)$ has the following PDF

$$f_X(x;\mu,\sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right). \tag{692}$$

It follows that the likelihood function is given by

$$L(x_1, \cdots, x_n; \mu, \sigma^2) = \prod_{i=1}^{n} f_X(x_i;\mu,\sigma^2) \tag{693}$$

$$= (2\pi\sigma^2)^{-n/2} \prod_{i=1}^{n} \exp\left(-\frac{(x_i-\mu)^2}{2\sigma^2}\right) \tag{694}$$

$$= (2\pi\sigma^2)^{-n/2} \exp\left(-\frac{1}{2\sigma^2} \sum_{i=1}^{n} (x_i-\mu)^2\right). \tag{695}$$

So the log likelihood function is given by

$$l(x_1, \cdots, x_n; \mu, \sigma^2) = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^{n} (x_i-\mu)^2. \tag{696}$$

To maximize the log likelihood function, we take partial derivatives in μ and σ^2:

$$\frac{\partial l(x_1, \cdots, x_n; \mu, \sigma^2)}{\partial \mu} = \frac{1}{\sigma^2} \sum_{i=1}^{n} (x_i - \mu) = \frac{n}{\sigma^2} \left(\bar{x} - \mu\right), \tag{697}$$

$$\frac{\partial l(x_1, \cdots, x_n; \mu, \sigma^2)}{\partial \sigma^2} = -\frac{n}{2} \frac{1}{\sigma^2} + \frac{1}{2\sigma^4} \sum_{i=1}^{n} (x_i - \mu)^2. \tag{698}$$

Setting these equations to be zero, we obtain

$$\begin{cases} \bar{x} = \mu \\ -n + \frac{1}{\sigma^2}\sum_{i=1}^{n}(x_i-\mu)^2 = 0. \end{cases} \tag{699}$$

Using the first equation for the second, we obtain

$$\sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i-\bar{x})^2. \tag{700}$$

This shows (691). ▲

7. Bayesian Inference

In this section, we discuss another widely used method of parameter estimation, called the *Bayesian estimation*. Just like in MLE, we would like to estimate an unknown parameter θ in a distribution $f_{X;\theta}$ from analyzing an i.i.d. sample $X_1, \cdots, X_n$ drawn from that distribution. One of the key aspects in Bayesian estimation is to quantify our prior knowledge on the unknown parameter θ as yet another probabilistic model, which we call as *belief*. After observing a new *data* $\{X_1 = x_1, \cdots, X_n = x_n\}$, our current best belief on θ (*prior distribution*) will be updated to a new belief (posterior distribution) according to Bayes' Theorem. Since we are making some additional modeling assumptions on θ, we will be able to draw more quantitative inference on θ using Bayesian estimation.

One of the early bottlenecks in Bayesian estimation was the computational overhead in computing posterior distributions. Due to the recent explosion in our computational capacity, coupled with advanced sampling techniques such as MCMC, Bayesian methods have become one of the indispensable tools in modern statistics and machine learning.

We begin by recalling the following basic observation, from which Bayes' theorem can be easily derived:

$$\mathbb{P}(B|A)\mathbb{P}(A) = \mathbb{P}(A\cap B) = \mathbb{P}(A|B)\mathbb{P}(B). \tag{701}$$

In the case of the events involving continuous random variables taking a certain value, we interpret the above probabilities as probability densities. The following is a random variable version of Bayes' Theorem discussed in the previous subsection.

Theorem A.7.1 (Bayes' Theorem)**.** *Let X and Θ be random variables. Then*

$$\overbrace{\mathbb{P}(\Theta=\theta\,|\,X=x)}^{\text{posterior}} = \frac{\mathbb{P}(X=x\,|\,\Theta=\theta)\mathbb{P}(\Theta=\theta)}{\mathbb{P}(X=x)} \propto \overbrace{\mathbb{P}(X=x\,|\,\Theta=\theta)}^{\text{likelihood}}\overbrace{\mathbb{P}(\Theta=\theta)}^{\text{prior}} \tag{702}$$

PROOF. Follows from (701). □

Remark A.7.2. Depending on whether X and Θ are discrete or continuous, we interpret the probabilities in the above theorem as PDF or PMF, accordingly.

Bayesian inference is usually carried out in the following procedure.

Bayesian inference:

(i) To explain an observable $\mathbf{x}$, we choose a *probabilistic model* $p(x\,|\,\theta)$, which is a probability distribution on the possible values x of $\mathbf{x}$, depending on a parameter θ.

(ii) Choose a probability distribution $\pi(\theta)$, called the *prior distribution*, that expresses our beliefs about a parameter θ to the best of our current knowledge.

(iii) After observing data $\mathscr{D} = \{x_1, \cdots, x_n\}$, we update our beliefs and compute the *posterior distribution* $p(\theta\,|\,\mathscr{D})$ according to Bayes' Theorem.

When we generate data $\mathscr{D} = \{x_1, \cdots, x_n\}$, we assume that each x_i is obtained by an independent sample X_i of the observable $\mathbf{x}$ from the true distribution of $\mathbf{x}$. According to Bayes' Theorem, we can compute the posterior distribution as

$$\mathbb{P}(\Theta = \theta \,|\, X_1 = x_1, \cdots, X_n = x_n) = \frac{\mathbb{P}(X_1 = x_1, \cdots, X_n = x_n \,|\, \Theta = \theta)\mathbb{P}(\Theta = \theta)}{\mathbb{P}(X_1 = x_1, \cdots, X_n = x_n)}, \tag{703}$$

or in a more compact form,

$$p(\theta \,|\, x_1, \cdots, x_n) = \frac{p(x_1, \cdots, x_n \,|\, \theta)\pi(\theta)}{p(x_1, \cdots, x_n)}. \tag{704}$$

By a slight abuse of notation, we use lowercase p to denote either PMF or PDF, depending on the context.

The conditional probability $p(x_1, \cdots, x_n \,|\, \theta)$ is called the *likelyhood function*, which is the probability of obtaining independent data sample $\mathscr{D} = \{x_1, \cdots, x_n\}$ according to our probability model $p(x \,|\, \theta)$ assuming model parameter $\Theta = \theta$. By the independence between samples, the likelihood function factors into the product of marginal likelihood of each sample:

$$p(x_1, \cdots, x_n \,|\, \theta) = \mathbb{P}(X_1 = x_1, \cdots, X_n = x_n \,|\, \Theta = \theta) \tag{705}$$

$$= \prod_{i=1}^{n} \mathbb{P}(X_i = x_i \,|\, \Theta = \theta) = \prod_{i=1}^{n} p(x_i \,|\, \theta). \tag{706}$$

On the other hand, the joint probability $\mathbb{P}(X_1 = x_1, \cdots, X_n = x_n)$ of obtaining the data sample $\mathscr{D} = \{x_1, \cdots, x_n\}$ from our probability model can be computed by conditioning on the values of model parameter Θ:

$$\mathbb{P}(X_1 = x_1, \cdots, X_n = x_n) = \mathbb{E}\left[\mathbb{P}(X_1 = x_1, \cdots, X_n = x_n \,|\, \Theta) \,\middle|\, \Theta\right] \tag{707}$$

$$= \int_{-\infty}^{\infty} p(x_1, \cdots, x_n \,|\, \theta)\pi(\theta)\, d\theta. \tag{708}$$

What's so important about Bayes' theorem is its interpretation as a means of *inference*, which is one of the fundamental tools in modern machine learning.

Example A.7.3. Suppose Bob has a coin with an unknown probability of heads, which we denote by Θ. This is called the *parameter*. Suppose Alice knows that Bob has one of the three kinds of coins A, B, and C, with the probability of heads being $p_A = 0.2$, $p_B = 0.5$, and $p_C = 0.8$, respectively. This piece of information is called the *model*. Since Alice has no information, she initially assumes that Bob has one of the three coins equally likely. Namely, she assumes the uniform distribution over the sample space $\Omega = \{0.2, 0.5, 0.8\}$. This knowledge is called *prior*.

Now Bob flips his coin 10 times and got 7 heads, and reports this information, which we call `Data`, to Alice. Now that Alice has more information, she needs to update her *prior* to *posterior*, which is the probability distribution on Ω that best explains the `Data`. Namely, it is the conditional probability distribution $\mathbb{P}(\Theta = \theta \,|\, \texttt{Data})$.

First, using our prior, we compute $\mathbb{P}(\texttt{Data})$, the probability of seeing this particular data at hand. This can be done by partitioning:

$$\mathbb{P}(\texttt{Data}) = \mathbb{P}(\texttt{Data} \,|\, \Theta = 0.2)\mathbb{P}(\Theta = 0.2) + \mathbb{P}(\texttt{Data} \,|\, \Theta = 0.5)\mathbb{P}(\Theta = 0.5) \tag{709}$$

$$+ \mathbb{P}(\texttt{Data} \,|\, \Theta = 0.8)\mathbb{P}(\Theta = 0.8) \tag{710}$$

$$= \mathbb{P}(7/10 \text{ heads} \,|\, \Theta = 0.2)\frac{1}{3} + \mathbb{P}(7/10 \text{ heads} \,|\, \Theta = 0.5)\frac{1}{3} + \mathbb{P}(7/10 \text{ heads} \,|\, \Theta = 0.8)\frac{1}{3} \tag{711}$$

$$= \frac{1}{3}\left(\binom{10}{7}(0.2)^7(0.8)^3 + \binom{10}{7}(0.5)^7(0.5)^3 + \binom{10}{7}(0.8)^7(0.2)^3\right) \approx 0.1064, \tag{712}$$

where $\binom{10}{7} = 120$ is the number of ways to choose 7 out of 10 objects.

Second, we reformulate the first equality of Theorem A.7.1 as

$$\mathbb{P}(\Theta \,|\, \texttt{Data}) = \frac{\mathbb{P}(\texttt{Data}\,|\,\Theta)\mathbb{P}(\Theta)}{\mathbb{P}(\texttt{Data})}. \tag{713}$$

Hence we can compute the posterior distribution by

$$\mathbb{P}(\Theta = 0.2 \,|\, \texttt{Data}) = \frac{\mathbb{P}(\texttt{Data}\,|\,\Theta = 0.2)\mathbb{P}(\Theta = 0.2)}{\mathbb{P}(\texttt{Data})} = \frac{\binom{10}{7}(0.2)^7(0.8)^3\frac{1}{3}}{0.1064} \approx 0.0025 \tag{714}$$

$$\mathbb{P}(\Theta = 0.5 \,|\, \texttt{Data}) = \frac{\mathbb{P}(\texttt{Data}\,|\,\Theta = 0.5)\mathbb{P}(\Theta = 0.5)}{\mathbb{P}(\texttt{Data})} = \frac{\binom{10}{7}(0.5)^7(0.5)^3\frac{1}{3}}{0.1064} \approx 0.3670 \tag{715}$$

$$\mathbb{P}(\Theta = 0.8 \,|\, \texttt{Data}) = \frac{\mathbb{P}(\texttt{Data}\,|\,\Theta = 0.8)\mathbb{P}(\Theta = 0.8)}{\mathbb{P}(\texttt{Data})} = \frac{\binom{10}{7}(0.8)^7(0.2)^3\frac{1}{3}}{0.1064} \approx 0.6305. \tag{716}$$

Note that according to the posterior distribution, $\Theta = 0.8$ is the most likely value, which is natural given that we have 7 heads out of 10 flips. However, our knowledge is always incomplete so our posterior knowledge is still a probability distribution on the sample space.

What if Bob flips his coin another 10 times and reports only 3 heads to Alice? Then she will have to use her current prior $\pi = [0.0025, 0.3060, 0.6305]$ (which was obtained as the posterior in the previous round) to compute yet another posterior using the new data. This will be likely to give a higher weight to $\Theta = 0.2$. ▲

Exercise A.7.4. Suppose we have a prior distribution $\pi = [0.0025, 0.3680, 0.6305]$ on the sample space $\Omega = \{0.2, 0.5, 0.8\}$ for the inference problem of unknown parameter Θ. Suppose we are given the data that ten independent flips of probability Θ coin come up heads twice. Compute the posterior distribution using this data and Bayesian inference.

Exercise A.7.5. A test for pancreatic cancer is assumed to be correct 95% of the time: if a person has cancer, the test results in positive with a probability 0.95, and if the person does not have cancer, then the test results in negative with probability 0.95. From recent medical research, it is known that only 5% of the population has pancreatic cancer. Given that the person just tested positive, what is the probability of having cancer?

Example A.7.6 (Bernoulli model and uniform prior)**.** Bob has a coin with unknown probability Θ of heads. Alice has no information whatsoever, so her prior distribution π for Θ is the uniform distribution Uniform($[0,1]$). Bob flips his coin independently n times, and let $X_1, \cdots, X_n$ be the outcome, X_i's are i.i.d. Bernoulli(Θ) variables. Let x_i be the observed value of X_i, and let $s_n = x_1 + \cdots + x_n$ be the number of heads in the n flips. Given data $\mathscr{D} = \{x_1, \cdots, x_n\}$, Alice wants to compute her posterior distribution on Θ.

The the likelyhood function is given by

$$p(x_1, \cdots, x_n \,|\, \theta) = \prod_{i=1}^{n} \theta^{x_i}(1-\theta)^{1-x_i} \tag{717}$$

$$= \theta^{s_n}(1-\theta)^{n-s_n}, \tag{718}$$

where we denote $s_n = x_1 + \cdots + x_n$. Since $\pi \equiv 1$, the posterior distribution is given by

$$p(\theta \,|\, x_1, \cdots, x_n) = \frac{\theta^{s_n}(1-\theta)^{n-s_n}}{p(x_1, \cdots, x_n)}, \tag{719}$$

Note that

$$p(x_1, \cdots, x_n) = \int_0^1 \theta^{s_n}(1-\theta)^{n-s_n}\, d\theta = \frac{1}{\binom{n}{s_n}(n+1)}, \tag{720}$$

where the last equality follows from Exercise A.7.8. Hence

$$p(\theta \,|\, x_1, \cdots, x_n) = \binom{n}{s_n}(n+1)\theta^{s_n}(1-\theta)^{n-s_n} \tag{721}$$

$$= \frac{(n+1)!}{s_n!(n-s_n)!}\theta^{(s_n+1)-1}(1-\theta)^{(n-s_n+1)-1}. \tag{722}$$

Hence, according to Exercise A.7.9, we can write

$$\Theta \,|\, x_1, \cdots, x_n \sim \text{Beta}(s_n+1, n-s_n+1). \tag{723}$$

▲

Exercise A.7.7. Bob has a coin with unknown probability Θ of heads. Alice has the following Beta prior (See Exercise A.7.9 for the definition of Beta distribution):

$$\pi = \text{Beta}(\alpha, \beta). \tag{724}$$

Suppose that Bob gives Alice the data $\mathscr{D}_n = \{x_1, \cdots, x_n\}$, which is the outcome of n independent coin flips. Denote $s_n = x_1 + \cdots + x_n$. Show that Alice's posterior distribution is $\text{Beta}(\alpha + s_n, \beta + n - s_n)$. Namely,

$$\Theta \,|\, \mathscr{D}_n \sim \text{Beta}(\alpha + s_n, \beta + n - s_n). \tag{725}$$

Exercise A.7.8. Let $Y \sim \text{Uniform}([0,1])$ and $X \sim \text{Binomial}(n, Y)$ be independent RVs.

(i) Use iterated expectation for probability to write

$$\mathbb{P}(X = k) = \binom{n}{k} \int_0^1 y^k (1-y)^{n-k}\, dy. \tag{726}$$

(ii) Write $A_{n,k} = \int_0^1 y^k (1-y)^{n-k}\, dy$. Use integration by parts and show that

$$A_{n,k} = \frac{k}{n-k+1} A_{n,k-1}. \tag{727}$$

for all $1 \le k \le n$. Conclude that for all $0 \le k \le n$,

$$A_{n,k} = \frac{1}{\binom{n}{k}} \frac{1}{n+1}. \tag{728}$$

(iii) Conclude that $X \sim \text{Uniform}(\{0, 1, \cdots, n\})$.

Exercise A.7.9 (Beta distribution)**.** A random varible X taking values from $[0,1]$ has Beta distribution of parameters α and β, which we denote by $\text{Beta}(\alpha, \beta)$, if it has PDF

$$f_X(x) = \frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)} x^{\alpha-1}(1-x)^{\beta-1}, \tag{729}$$

where $\Gamma(z)$ is the Euler Gamma function defined by

$$\Gamma(z) = \int_0^\infty x^{z-1} e^{-x}\, dx. \tag{730}$$

(i) Use integration by parts to show the following recursion

$$\Gamma(z+1) = z\Gamma(z). \tag{731}$$

Deduce that $\Gamma(n) = (n-1)!$ for all integers $n \ge 1$.

(ii) Let $X \sim \text{Beta}(k+1, n-k+1)$. Use (i) to show that

$$f_X(x) = \frac{n!(n+1)}{k!(n-k)!} x^k (1-x)^{n-k} = \frac{x^k(1-x)^{n-k}}{1/\binom{n}{k}(n+1)}. \tag{732}$$

Use Exercise A.7.8 to verify that the above function is indeed a PDF (i.e., it integrates to 1).

(iii)* Show that if $X \sim \text{Beta}(\alpha, \beta)$, then

$$\mathbb{E}[X] = \frac{\alpha}{\alpha+\beta}, \quad \text{Var}(X) = \frac{\alpha\beta}{(\alpha+\beta)^2(\alpha+\beta+1)}. \tag{733}$$

Example A.7.10 (Bayesian estimator)**.** Recall Exercise A.7.7, where we want to infer an unknown parameter θ in Bernoulli(θ), starting from the beta prior $\pi = \text{Beta}(\alpha, \beta)$. After observing a data $\mathscr{D} = (x_1, \cdots, x_n)$ consisting of $s_n = \sum_{i=1}^n x_i$ successes, our posterior distribution is Beta distribution with parameters $s_n + 1$ and $\beta + n - s_n$. In other words,

$$\Theta | \mathscr{D} \sim \text{Beta}(\alpha + s_n, \beta + n - s_n). \tag{734}$$

Now suppose we want to get a point estimator $\hat{\theta}$ for the unknown parameter from our posterior distribution. Any choice of the estimator will result in some kind of error, so we would like our estimator to be minimizing an error function of choice. A standard choice is the mean squared error (MSE), which in our case would be under conditioning on the observed data. That is, we want our estimator $\hat{\theta}$ to be such that

$$\hat{\theta} = \text{argmin}\, \mathbb{E}[(\Theta - \hat{\theta})^2 \,|\, \mathscr{D}]. \tag{735}$$

According to Exercise A.4.2, the above MSE will be minimized when $\hat{\theta} = \mathbb{E}[\Theta \,|\, \mathscr{D}]$, and the minimum MSE is the conditional variance $\text{Var}(\Theta \,|\, \mathscr{D})$. Noting the mean of beta distribution in Exercise A.7.9, we obtain the following Bayesian estimator

$$\hat{\theta} = \mathbb{E}[\Theta \,|\, \mathscr{D}] = \mathbb{E}[\text{Beta}(\alpha + s_n, \beta + n - s_n)] = \frac{\alpha + s_n}{\alpha + \beta + n}. \tag{736}$$

▲